

Introduction to Graph Theory

**A comprehensive guide combining mathematical rigor,
visualization, algorithmic implementations, and interview
practices**

Jiho Lee

University of California, San Diego

Math 154: Graph Theory

April 26, 2026

Contents

I	Foundations	1
1	Basic Classes of Graphs	1
1.1	Cycles	1
1.2	Paths	1
1.3	Degrees and Neighborhoods	1
1.4	Examples of Graphs and Their Vertex Degrees	2
1.4.1	Complete Graph K_n	2
1.4.2	Cycle Graph C_n	3
1.4.3	Path Graph P_n	3
1.4.4	Star Graph S_n	4
1.4.5	Wheel Graph W_n	4
1.4.6	Complete Bipartite Graph $K_{m,n}$	5
1.4.7	Regular Graph	6
1.4.8	Hypercube Graph Q_n	6
1.4.9	Petersen Graph	7
1.5	Mathematical Properties Related to Degrees	7
1.5.1	Handshaking Lemma	7
1.5.2	Degree Sequence	8
1.5.3	Average Degree	8
1.5.4	Maximum and Minimum Degree	8
1.5.5	Isolated and Pendant Vertices	9
1.6	Subgraphs	10
1.7	Walks and Paths in Graphs	11
1.7.1	More Examples of Walks, Paths, and Components	13
1.8	Chapter 1 Exercises	15
1.8.1	Easy Questions	15
1.8.2	Intermediate Questions	15
1.8.3	Hard Questions	16
1.8.4	Questions with Diagrams	16
2	Eulerian Graphs	18
2.1	Trails and Tours	18
2.2	Eulerian Digraphs and DeBruijn Sequences	25
2.2.1	Eulerian Digraphs	25
2.2.2	De Bruijn Sequences	25
2.3	De Bruijn Graphs	31
2.4	Chapter 2 Exercises	34
2.4.1	Easy Problems: Eulerian Graphs	34
2.4.2	Medium Problems: Eulerian Graphs	34
2.4.3	Hard Problems: Eulerian Graphs	34
2.4.4	Easy Problems: De Bruijn Sequences	34
2.4.5	Medium Problems: De Bruijn Sequences	35
2.4.6	Hard Problems: De Bruijn Sequences	35
2.4.7	Easy Problems: De Bruijn Graphs	35
2.4.8	Medium Problems: De Bruijn Graphs	35
2.4.9	Hard Problems: De Bruijn Graphs	35

3	Hamiltonian Graphs	36
3.1	Hamiltonian Walks, Paths, Cycles	36
3.2	Dirac's Theorem	39
3.3	Visual Illustration of Dirac's Theorem Proof	40
3.4	Alternative Form of Dirac's Theorem	41
3.5	Examples of Dirac's Theorem	41
3.6	Related Results and Extensions	45
	3.6.1 Ore's Theorem	45
	3.6.2 Chvátal's Theorem	46
3.7	Traveling Salesman Problem	48
	3.7.1 Approximation Algorithms	49
3.8	Chapter 3 Exercises	53
	3.8.1 Easy Problems	53
	3.8.2 Medium Problems	53
	3.8.3 Hard Problems	53
	3.8.4 Diagram Problems	54
4	Long Cycles	55
4.1	Long Cycles and Hamiltonian	55
4.2	Algorithm for Finding Long Cycles	58
4.3	Chapter 4 Exercises	59
	4.3.1 Easy Problems	59
	4.3.2 Medium Problems	59
	4.3.3 Hard Problems	59
	4.3.4 Diagram Problems	60
5	Trees and Forests	61
5.1	Trees, Forests, and Bridges	61
5.2	Characterization of Trees	63
5.3	Breadth-First Search and Depth-First Search Algorithms	66
	5.3.1 BFS Algorithm and BFS Tree	67
	5.3.2 DFS Algorithm and DFS Tree	70
	5.3.3 Comparison of BFS and DFS Traversals	72
5.4	Prim's Algorithm	74
5.5	Kruskal's Algorithm	78
	5.5.1 Key Similarities	81
5.6	Dijkstra's Algorithm	83
5.7	Floyd-Warshall Algorithm	87
5.8	Bellman-Ford Algorithm	90
	5.8.1 Algorithm Characteristics	93
5.9	Bipartiteness	95
5.10	Chapter 5 Exercises	98
	5.10.1 Easy Questions	98
	5.10.2 Medium Questions	98
	5.10.3 Hard Questions	98
	5.10.4 Diagram Questions	99
	5.10.5 Visual Questions	99
6	Structure of Connected Graphs	103
6.1	Blocks, Cut Vertices, and Connectedness	103
6.2	Menger's Theorem	108
6.3	Structure of Connected Graphs Exercises	117
	6.3.1 Easy Questions	117
	6.3.2 Medium Questions	117

6.3.3	Hard Questions	117
6.3.4	Diagram Questions	118
6.3.5	Additional Questions	118
7	Matchings and Factors	120
7.1	Independent Sets and Covers	122
7.2	Hall's Theorem and Various Matching Algorithms	131
7.3	Matchings and Factors Exercises	149
7.3.1	Easy Questions	149
7.3.2	Medium Questions	149
7.3.3	Hard Questions	149
7.3.4	Diagram Questions	150
8	Vertex and Edge-Coloring	152
8.1	Basic Definitions	152
8.2	König's Theorem on Edge-Coloring of Bipartite Graphs	152
8.2.1	Background and Motivation	152
8.2.2	Formal Statement	152
8.2.3	Implications and Significance	153
8.2.4	Equivalent Formulations	153
8.2.5	Historical Context	154
8.3	Vizing's Theorem	154
8.3.1	Formal Statement	154
8.3.2	Key Lemmas Supporting Vizing's Theorem	155
8.3.3	Implications and Applications	155
8.3.4	Historical Context	156
8.3.5	Examples of Vizing's and König's Theorem	156
8.4	Shannon's Edge Coloring Theorem	161
8.4.1	Explanation of Shannon's Bound	161
8.4.2	Proof of Shannon's Edge Coloring Theorem	161
8.5	Degenerate Graphs and Their Coloring Properties	168
8.6	Degeneracy Relationships for Specific Graph Classes	173
8.7	Extensions and Applications	174
8.8	Algorithms for d -Degenerate Graphs	175
8.9	Introduction to Scheduling Problems	176
8.9.1	Degenerate Graphs in Scheduling Problems	177
8.9.2	Axillary Algorithms for Scheduling Problems	181
8.10	Vertex and Edge-Coloring Exercises	202
8.10.1	Easy Questions	202
8.10.2	Medium Questions	202
8.10.3	Hard Questions	202
8.10.4	Diagram Questions	203
9	Planar Graphs	204
9.1	Definition and Basic Properties	204
9.2	Faces of Planar Graphs	204
9.3	Fundamental Theorems	204
9.4	Applications: Proving Non-Planarity	205
9.5	Introduction and Formal Definitions of Art Gallery Problems	207
9.6	Construction and Visualization	207
9.7	Triangulation	207
9.8	Three-Coloring	208
9.9	The Complete Guard Placement Algorithm	208
9.10	Orthogonal Polygons	209

9.11	Mobile Guards	209
9.12	Monotone Polygons	209
9.13	Fortress Problem	210
9.14	Prison Yard Problem	210
9.15	Open Problems	210
9.16	Planar Graphs and Art Gallery Problems	211
9.16.1	Easy Questions	211
9.16.2	Medium Questions	211
9.16.3	Hard Questions	211
9.16.4	Construction and Diagram Questions	212
9.16.5	Proof-Based Questions	212
9.16.6	Advanced Applications	212
10	Network Flows and the Max-Flow Min-Cut Theorem	213
10.1	Directed Graphs and Capacities	213
10.2	Flows and the Value of a Flow	213
10.3	Cuts and the Capacity of a Cut	214
10.4	Flow Value Equals Net Flow Across Any Cut	214
10.5	Residual Graphs and Augmenting Paths	214
10.6	The Max-Flow Min-Cut Theorem	215
10.7	The Ford–Fulkerson Algorithm	216
10.8	Applications	216
10.8.1	Menger’s Theorem	216
10.8.2	König’s Theorem Revisited	216
10.9	Worked Example	217
10.10	Exercises	217
10.10.1	Easy Questions	217
10.10.2	Medium Questions	217
10.10.3	Hard Questions	218
10.10.4	Diagram Questions	218
11	Advanced Topics in Graph Theory	218
11.1	Extremal Graph Theory	218
11.1.1	The Turán Number and the Turán Graph	218
11.1.2	Mantel’s Theorem: Three Proofs	218
11.1.3	Turán’s Theorem	219
11.1.4	Stability: The Erdős–Simonovits Theorem	220
11.1.5	The Erdős–Stone–Simonovits Theorem	220
11.1.6	The Bipartite Case: Kővári–Sós–Turán	221
11.1.7	The Zarankiewicz Problem	221
11.1.8	Graphs of High Girth: The Moore Bound	221
11.1.9	Supersaturation	222
11.1.10	Exercises	222
11.2	Ramsey Theory	222
11.2.1	The Classical Ramsey Numbers	223
11.2.2	Probabilistic Lower Bounds	224
11.2.3	Ramsey Numbers for Graphs	224
11.2.4	Hypergraph Ramsey Numbers	224
11.2.5	Schur’s Theorem and van der Waerden	225
11.2.6	Erdős–Szekeres: Happy Ending	225
11.2.7	Canonical Ramsey and Induced Ramsey	225
11.2.8	Exercises	225
11.3	The Probabilistic Method	226
11.3.1	First Moment: Linearity of Expectation	226

11.3.2	The Alteration (Deletion) Method	227
11.3.3	Second Moment and Paley–Zygmund	227
11.3.4	The Lovász Local Lemma	227
11.3.5	Concentration: Azuma and Talagrand	228
11.3.6	Dependent Random Choice	228
11.3.7	The Entropy Method	228
11.3.8	Exercises	229
11.4	Spectral Graph Theory	229
11.4.1	Matrices of a Graph	230
11.4.2	Walk Counting and Basic Identities	230
11.4.3	The Min-Max Principle and Rayleigh Quotient	230
11.4.4	Perron–Frobenius	231
11.4.5	The Matrix-Tree Theorem	231
11.4.6	Algebraic Connectivity and Cheeger’s Inequality	231
11.4.7	Expander Graphs	232
11.4.8	Random Walks and Mixing	232
11.4.9	Hoffman’s Bound and Chromatic Number	232
11.4.10	Strongly Regular Graphs	233
11.4.11	Exercises	233
11.5	Random Graphs	234
11.5.1	Random Graph Models	234
11.5.2	Subgraph Thresholds	234
11.5.3	Degree Distribution	235
11.5.4	Threshold for Connectivity	235
11.5.5	The Giant Component Phase Transition	235
11.5.6	Chromatic Number of $G(n, p)$	235
11.5.7	Hamiltonicity	236
11.5.8	Random Regular Graphs	236
11.5.9	Preferential Attachment and Scale-Free Models	236
11.5.10	Small Subgraph Central Limit Theorem	236
11.5.11	Exercises	236
11.6	Szemerédi’s Regularity Lemma	237
11.6.1	ε -Regular Pairs	237
11.6.2	The Regularity Lemma	237
11.6.3	The Counting Lemma	238
11.6.4	The Removal Lemma	238
11.6.5	From Triangle Removal to Roth	238
11.6.6	The Blow-up Lemma	239
11.6.7	Graphons and the Limit Theory	239
11.6.8	Property Testing	239
11.6.9	Exercises	239
11.7	Graph Minors and Treewidth	240
11.7.1	Minors and Topological Minors	240
11.7.2	Tree Decompositions and Treewidth	240
11.7.3	Duality: Brambles and Cops–Robbers	241
11.7.4	Courcelle’s Theorem and Fixed-Parameter Tractability	241
11.7.5	Excluded Minors and Sparsity	241
11.7.6	The Graph Minor Structure Theorem	242
11.7.7	The Graph Minor Theorem	242
11.7.8	The Disjoint Paths Problem	242
11.7.9	Exercises	242
11.8	Comprehensive Chapter Exercises	243

12 Algebraic Graph Theory: Auxiliary Topics	244
12.1 The Matchings Polynomial and the Heilmann–Lieb Theorem	244
12.1.1 Definition and Basic Recurrences	244
12.1.2 The Path-Tree Construction	245
12.1.3 The Heilmann–Lieb Theorem	245
12.1.4 Special Cases and Classical Polynomials	245
12.1.5 Exercises	245
12.2 Equitable Partitions and Quotient Interlacing	246
12.2.1 Definition and Basic Theory	246
12.2.2 Cauchy Interlacing for Quotients	246
12.2.3 Walk-Regularity and Distance Partitions	247
12.2.4 Exercises	247
12.3 Distance-Regular Graphs and Association Schemes	247
12.3.1 Distance-Regular Graphs	248
12.3.2 Association Schemes	248
12.3.3 The Hamming and Johnson Schemes	249
12.3.4 Exercises	249
12.4 Cayley Graphs and Vertex-Transitive Graphs	249
12.4.1 Definitions and First Examples	250
12.4.2 Spectrum from Characters	250
12.4.3 Non-Abelian Cayley Graphs	250
12.4.4 Vertex-Transitive Graphs and Cayley-ness	250
12.4.5 Exercises	251
12.5 The Tutte Polynomial and Graphic Matroids	251
12.5.1 Deletion, Contraction, and the Tutte Polynomial	251
12.5.2 Specializations: Chromatic, Flow, Reliability	252
12.5.3 Whitney’s Broken-Circuit Theorem	252
12.5.4 Graphic Matroids and Cycle/Bond Duality	252
12.5.5 Connections: Stanley’s Chromatic Symmetric Function	253
12.5.6 Exercises	253
12.6 Further Reading and Pointers	253

This page is intentionally left blank.

Part I

Foundations

1 Basic Classes of Graphs

1.1 Cycles

Definition 1. A cycle of length n is the graph $C_n = (V, E)$ where:

$$V = \{v_0, v_1, \dots, v_{n-1}\}$$
$$E = \{\{v_i, v_{i+1 \bmod n}\} : 0 \leq i < n\}$$

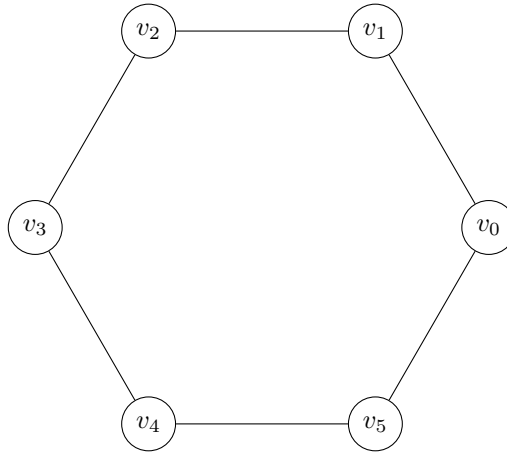


Figure 1: A cycle graph C_6 with 6 vertices.

1.2 Paths

Definition 2. A path of length k is the graph $P_k = (V, E)$ where:

$$V = \{v_0, v_1, \dots, v_k\}$$
$$E = \{\{v_i, v_{i+1}\} : 0 \leq i < k\}$$

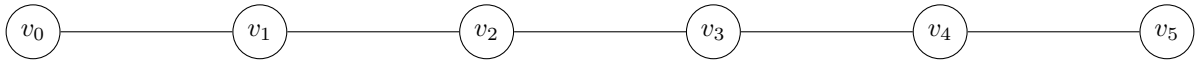


Figure 2: A path graph P_5 with 6 vertices.

1.3 Degrees and Neighborhoods

Definition 3. If G is a graph and $v \in V(G)$, then:

- The neighborhood of v is $N_G(v) = \{w \in V(G) : \{v, w\} \in E(G)\}$
- The degree of v is $d(v) = d_G(v) = |N_G(v)|$

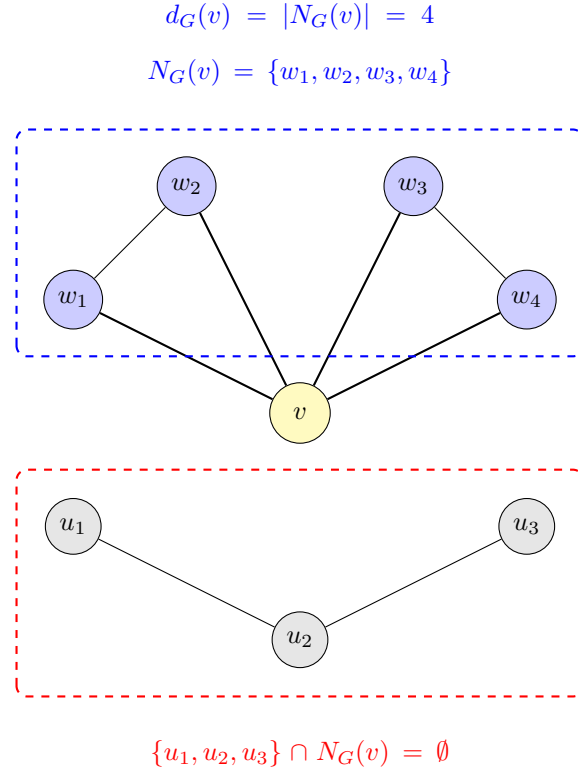


Figure 3: Example of neighborhood and degree of a vertex in a graph.

Definition 4 (Incidence). In graph theory, incidence refers to the relationship between vertices and edges:

- A vertex v and an edge e are *incident* with each other if v is an endpoint of e .
- If $e = \{u, v\}$, then:
 - Edge e is incident with vertices u and v
 - Vertex u is incident with edge e
 - Vertex v is incident with edge e
- The incidence relation forms the fundamental connection between vertices and edges in a graph.

1.4 Examples of Graphs and Their Vertex Degrees

1.4.1 Complete Graph K_n

In a complete graph with n vertices, every vertex is connected to all other vertices.

- For any vertex v in K_n : $\deg(v) = n - 1$
- Example: In K_5 , every vertex has $\deg(v) = 4$
- Total edges: $|E| = \frac{n(n-1)}{2} = \frac{5 \cdot 4}{2} = 10$

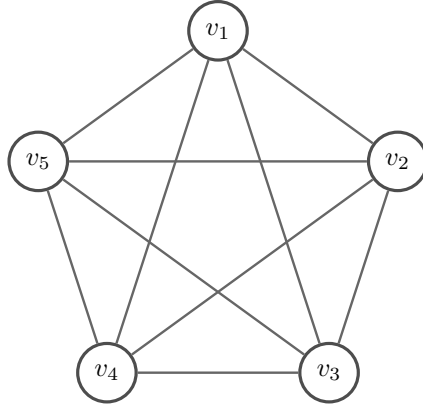
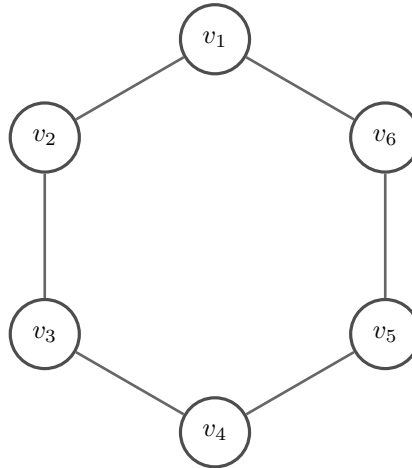


Figure 4: In this complete graph K_5 with 5 vertices, each vertex v_i has degree 4. Every vertex is connected to all other vertices in the graph. Total number of edges = $\frac{n(n-1)}{2} = \frac{5 \cdot 4}{2} = 10$

1.4.2 Cycle Graph C_n

A cycle graph with n vertices forms a closed loop.

- For any vertex v in C_n : $\deg(v) = 2$
- Example: In a hexagon C_6 , every vertex has $\deg(v) = 2$
- Total edges: $|E| = n = 6$



$$\forall i \in \{1, 2, 3, 4, 5, 6\} : \deg(v_i) = 2 \quad \text{and} \quad |E| = |V| = 6$$

Figure 4: In this cycle graph C_6 with 6 vertices, each vertex v_i has degree 2. Every vertex is connected to exactly two other vertices in the graph. Total number of edges = $n = 6$

1.4.3 Path Graph P_n

A path graph with n vertices forms a single path.

- For endpoints: $\deg(v) = 1$

- For internal vertices: $\deg(v) = 2$
- Example: In P_7 , the degree sequence is $(1, 2, 2, 2, 2, 2, 1)$
- Total edges: $|E| = n - 1 = 6$



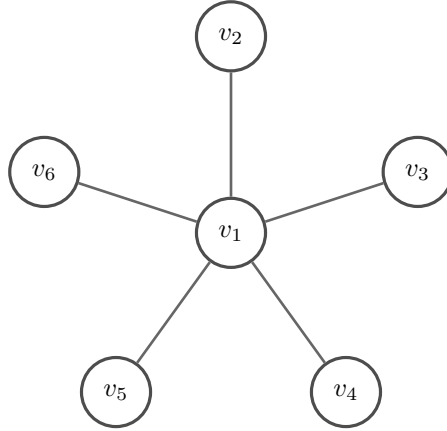
$$\deg(v_1) = \deg(v_7) = 1 \quad \text{and} \quad \deg(v_i) = 2 \quad \forall i \in \{2, 3, 4, 5, 6\}$$

Figure 5: In this path graph P_7 with 7 vertices, the endpoint vertices v_1 and v_7 have degree 1, while all internal vertices have degree 2. Total number of edges $= n - 1 = 6$

1.4.4 Star Graph S_n

A star graph has one central vertex connected to $n - 1$ leaf vertices.

- For central vertex c : $\deg(c) = n - 1$
- For any leaf vertex v : $\deg(v) = 1$
- Example: In S_6 , the central vertex has $\deg(c) = 5$, and the 5 leaf vertices each have $\deg(v) = 1$
- Total edges: $|E| = n - 1 = 5$



$$\deg(v_1) = 5 \quad \text{and} \quad \deg(v_i) = 1 \quad \forall i \in \{2, 3, 4, 5, 6\}$$

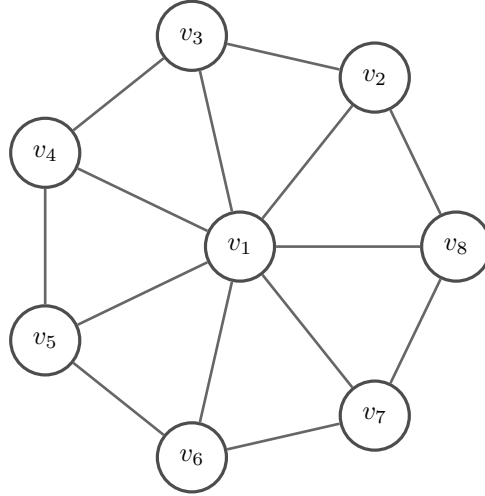
Figure 6: In this star graph S_6 with 6 vertices, the central vertex v_1 has degree 5, while all leaf vertices v_2 through v_6 have degree 1. Total number of edges $= n - 1 = 5$

1.4.5 Wheel Graph W_n

A wheel graph is formed by connecting a single vertex to all vertices of a cycle.

- For central vertex c : $\deg(c) = n - 1$

- For vertices on the cycle: $\deg(v) = 3$
- Example: In W_8 , the central vertex has $\deg(c) = 7$, and the 7 outer vertices each have $\deg(v) = 3$
- Total edges: $|E| = 2(n - 1) = 14$



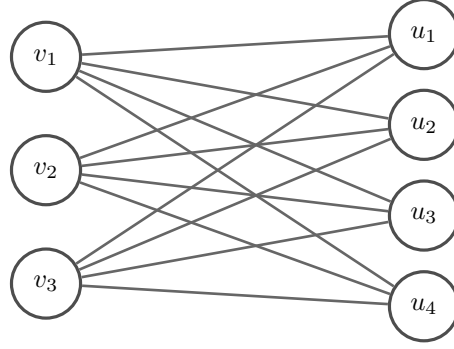
$$\deg(v_1) = 7 \quad \text{and} \quad \deg(v_i) = 3 \quad \forall i \in \{2, 3, 4, 5, 6, 7, 8\}$$

Figure 7: In this wheel graph W_8 with 8 vertices, the central vertex v_1 has degree 7, while all cycle vertices v_2 through v_8 have degree 3. Total number of edges $= 2(n - 1) = 14$

1.4.6 Complete Bipartite Graph $K_{m,n}$

A complete bipartite graph has two sets of vertices (with m and n vertices) where every vertex in the first set is connected to every vertex in the second set.

- For vertices in the first set: $\deg(v) = n$
- For vertices in the second set: $\deg(v) = m$
- Example: In $K_{3,4}$, the 3 vertices in the first set each have $\deg(v) = 4$, and the 4 vertices in the second set each have $\deg(v) = 3$
- Total edges: $|E| = m \cdot n = 12$



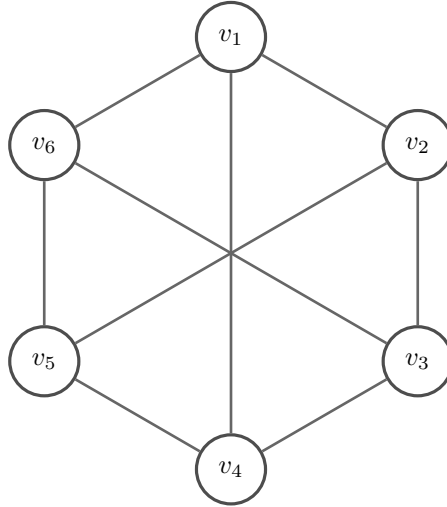
$$\deg(v_i) = 4 \quad \forall i \in \{1, 2, 3\} \quad \text{and} \quad \deg(u_j) = 3 \quad \forall j \in \{1, 2, 3, 4\}$$

Figure 8: In this complete bipartite graph $K_{3,4}$ with 7 vertices, each vertex v_i in the first set has degree 4, while each vertex u_j in the second set has degree 3. Total number of edges = $m \cdot n = 3 \cdot 4 = 12$

1.4.7 Regular Graph

In a k -regular graph, every vertex has the same degree k .

- For any vertex v : $\deg(v) = k$
- Example: In a 3-regular graph (cubic graph), every vertex has $\deg(v) = 3$
- Total edges in a k -regular graph with n vertices: $|E| = \frac{kn}{2}$



$$\deg(v_i) = 3 \quad \forall i \in \{1, 2, 3, 4, 5, 6\}$$

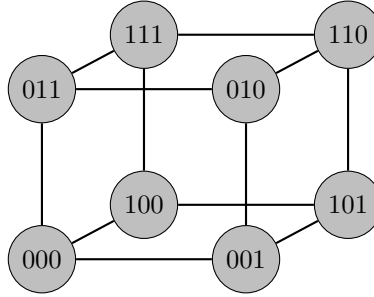
Figure 9: In this 3-regular graph with 6 vertices, each vertex v_i has exactly 3 incident edges. Total number of edges = $\frac{kn}{2} = \frac{3 \cdot 6}{2} = 9$

1.4.8 Hypercube Graph Q_n

In an n -dimensional hypercube, every vertex has degree n .

- For any vertex v in Q_n : $\deg(v) = n$

- Example: In Q_3 (3-dimensional cube), every vertex has $\deg(v) = 3$
- Total vertices: $|V| = 2^n$
- Total edges: $|E| = n \cdot 2^{n-1}$



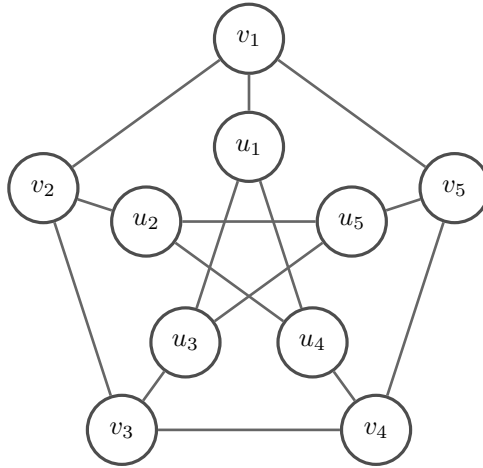
$$\deg(v) = 3 \quad \forall v \in V(Q_3)$$

Figure 10: Hypercube graph Q_3 with vertices labeled by binary strings. Each vertex has degree 3, with connections to neighbors that differ in exactly one bit position.

1.4.9 Petersen Graph

The Petersen graph is a specific 3-regular graph with 10 vertices.

- For any vertex v : $\deg(v) = 3$
- Total edges: $|E| = 15$



$$\deg(v_i) = \deg(u_i) = 3 \quad \forall i \in \{1, 2, 3, 4, 5\}$$

Figure 11: Petersen graph with 10 vertices. Each vertex has degree 3, with a total of 15 edges. This is a famous non-Hamiltonian 3-regular graph.

1.5 Mathematical Properties Related to Degrees

1.5.1 Handshaking Lemma

The sum of all degrees equals twice the number of edges:

$$\sum_{v \in V} \deg(v) = 2|E| \tag{1}$$

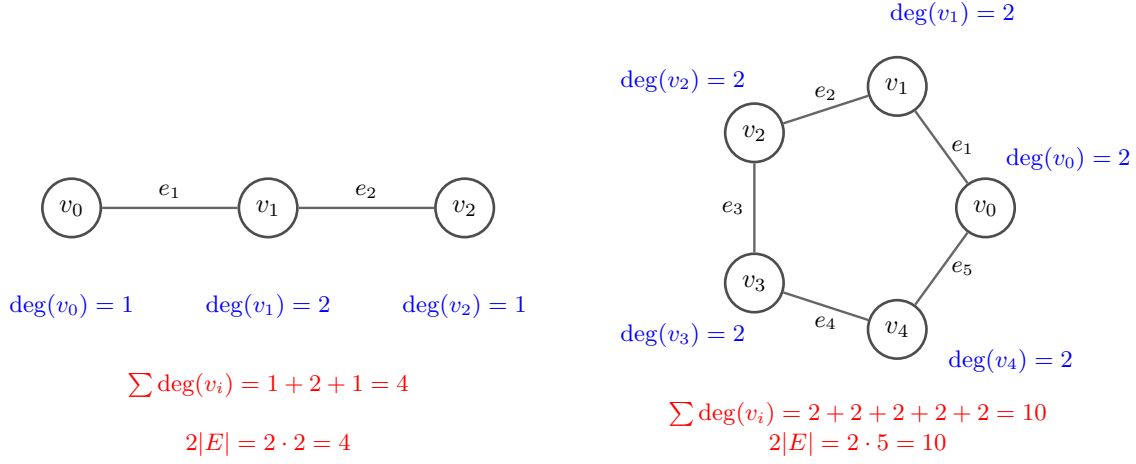


Figure 12 and 13: Illustration of the Handshaking Lemma. Left: A path graph with 3 vertices and 2 edges. Right: A cycle graph with 5 vertices and 5 edges. In both cases, the sum of vertex degrees equals twice the number of edges.

Example 1. Consider a graph with vertex set $V = \{1, 2, 3, 4\}$ and edge set $E = \{\{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{3, 4\}\}$.

$$\begin{aligned}
 \sum_{v \in V} d(v) &= d(1) + d(2) + d(3) + d(4) \\
 &= 3 + 2 + 3 + 2 \\
 &= 10
 \end{aligned}$$

And $2|E| = 2 \cdot 5 = 10$.

1.5.2 Degree Sequence

The non-increasing sequence of vertex degrees.

- Example: A graph with degree sequence $(4, 3, 3, 2, 2, 1, 1)$ has 7 vertices with varying connectivity
- Valid degree sequences must sum to an even number

1.5.3 Average Degree

The average degree of vertices in a graph $G = (V, E)$ is:

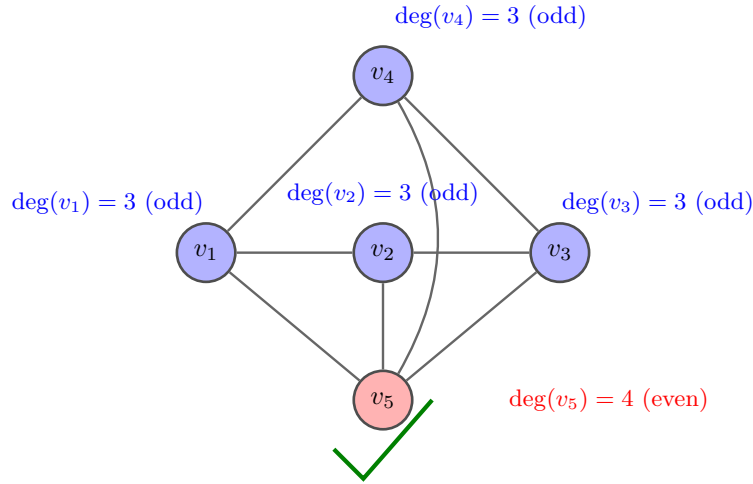
$$\text{avg}(\deg) = \frac{\sum_{v \in V} \deg(v)}{|V|} = \frac{2|E|}{|V|} \quad (2)$$

1.5.4 Maximum and Minimum Degree

$$\Delta(G) = \max\{\deg(v) \mid v \in V\} \quad (3)$$

$$\delta(G) = \min\{\deg(v) \mid v \in V\} \quad (4)$$

Corollary 1. In any graph, there are an even number of vertices of odd degree.

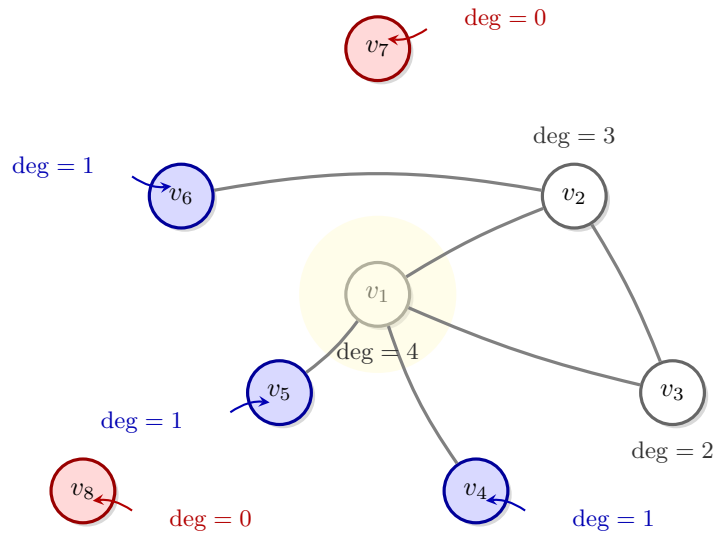


There are 4 odd-degree vertices (even number)

Figure 14: A graph illustrating the handshaking lemma's corollary that any graph must contain an even number of odd-degree vertices. This example has 4 vertices with odd degree (shown in blue) and 1 vertex with even degree (shown in red).

1.5.5 Isolated and Pendant Vertices

- Isolated vertex: $\deg(v) = 0$
- Pendant vertex (leaf): $\deg(v) = 1$
- Example: A tree with n vertices has exactly $n - 2$ vertices of degree ≥ 2 and at least 2 leaves



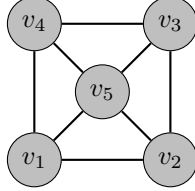
- **Isolated vertex:** $\deg(v) = 0$
- **Pendant vertex:** $\deg(v) = 1$
- **Regular vertex:** $\deg(v) \geq 2$

Figure 15: Illustration of special vertex types based on degree. The center vertex v_1 (highlighted) connects to both regular vertices and pendant vertices, while isolated vertices remain disconnected from the graph.

1.6 Subgraphs

Definition 5. A graph F is a subgraph of graph G if $V(F) \subseteq V(G)$ and $E(F) \subseteq E(G)$, written $F \subseteq G$.

Original Graph G



Subgraph $F \subseteq G$

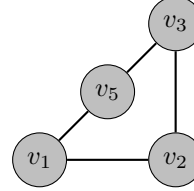
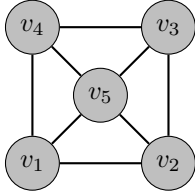


Figure 16: Illustration of a subgraph relationship. The left diagram shows the original graph G with 5 vertices and 8 edges. The right diagram shows a subgraph $F \subseteq G$ where vertex v_4 has been removed, along with its incident edges. Additionally, some edges between the remaining vertices (v_2-v_5 and v_3-v_4) have been removed, demonstrating that a subgraph maintains subset relationships for both vertex and edge sets.

Definition 6. Graph F is a spanning subgraph of G if $V(F) = V(G)$.

Original Graph G



Spanning Subgraph F of G

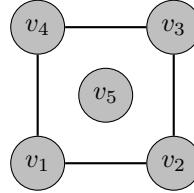
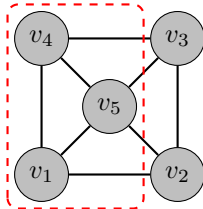


Figure 17: Illustration of a spanning subgraph. The original graph G (left) has 5 vertices and 8 edges, including connections from central vertex v_5 to all other vertices. The spanning subgraph F (right) contains all vertices of G but only a subset of edges, specifically removing all connections to the central vertex v_5 . A spanning subgraph maintains the same vertex set as the original graph ($V(F) = V(G)$) but may have fewer edges ($E(F) \subseteq E(G)$).

Definition 7. If $X \subseteq V(G)$, the subgraph induced by X is:

$$G[X] = (X, \{\{u, v\} \in E(G) : u, v \in X\})$$

Original Graph G



$$X = \{v_1, v_4, v_5\}$$

Induced Subgraph $G[X]$

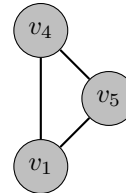


Figure 18: Illustration of an induced subgraph. The original graph G (left) has a subset of vertices $X = \{v_1, v_4, v_5\}$ highlighted by a red dashed rectangle. The induced subgraph $G[X]$ (right) contains exactly these vertices along with all edges from the original graph that connect vertices within this set. Unlike a general subgraph, an induced subgraph must include every edge from the original graph whose endpoints are both in the selected vertex subset.

1.7 Walks and Paths in Graphs

Definition 8. A walk in a graph is a sequence of vertices and edges:

$$v_0, \{v_0, v_1\}, v_1, \{v_1, v_2\}, v_2, \dots, \{v_{k-1}, v_k\}, v_k$$

often abbreviated as $v_0, v_1, v_2, \dots, v_k$. The length of a walk is the number of edges in that walk.

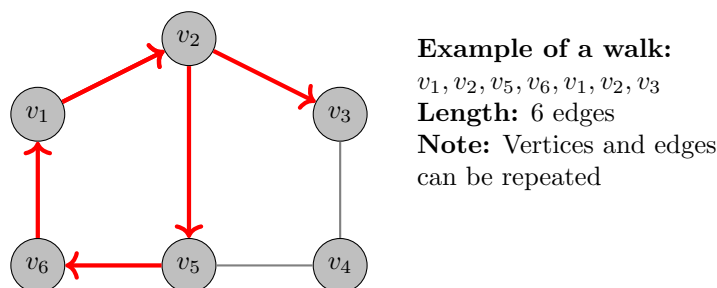


Figure 19: Illustration of a walk in graph theory. A walk is a sequence of vertices and edges where each consecutive pair of vertices in the sequence is connected by an edge. This example shows a walk of length 6 (red arrows) from v_1 to v_3 that passes through the sequence $v_1, v_2, v_5, v_6, v_1, v_2, v_3$. Unlike paths or trails, walks can repeat both vertices and edges. This walk repeats vertex v_1 and v_2 , and also repeats the edge between them.

Definition 9. A path of length k is a walk:

$$v_0, v_1, v_2, \dots, v_k$$

where all vertices are distinct.

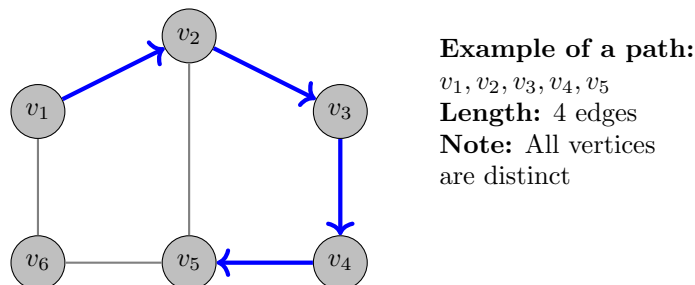


Figure 20: Illustration of a path in graph theory. A path is a walk where no vertex is repeated, ensuring each vertex appears at most once in the sequence. This example shows a path of length 4 (blue arrows) from v_1 to v_5 through the sequence v_1, v_2, v_3, v_4, v_5 . Paths are fundamental structures in graph theory and are used to define important concepts such as connectivity, distance, and shortest paths between vertices. Unlike walks, the no-repetition constraint makes paths more restrictive but also more useful for many graph algorithms.

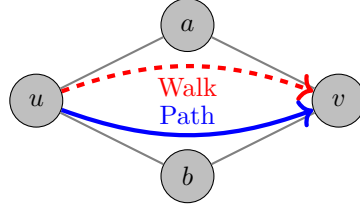
Lemma 1. A graph is connected if and only if every pair of vertices are the end of some walk.

Definition 10 (Connected Graph). A graph $G = (V, E)$ is said to be *connected* if for every pair of distinct vertices $u, v \in V$, there exists a path in G from u to v . In other words, a graph is connected if and only if there is at least one path between any two vertices in the graph.

Definition 11 (Disconnected Graph). A graph $G = (V, E)$ is said to be *disconnected* if there exists at least one pair of distinct vertices $u, v \in V$ such that there is no path in G from u to v . Equivalently, a graph is disconnected if and only if its vertex set can be partitioned into two nonempty subsets such that there is no edge connecting any vertex in the first subset to any vertex in the second subset.

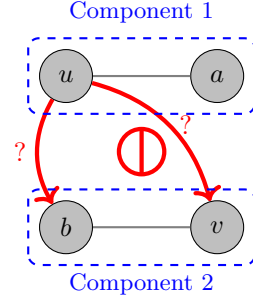
Definition 12 (Connected Component). A *connected component* of a graph $G = (V, E)$ is a maximal connected subgraph of G . That is, it is a connected subgraph $H = (V_H, E_H)$ of G such that no larger connected subgraph of G contains H as a proper subgraph. A connected graph has exactly one connected component, while a disconnected graph has two or more connected components.

Connected Graph



Every pair of vertices
can be connected by a path

Disconnected Graph



Some pairs of vertices
cannot be connected by any path

Figure 21: Comparison of connected and disconnected graphs. In the connected graph (left), there exists at least one path between any pair of vertices, as shown by the example paths between vertices u and v . The red dashed line represents one possible walk, while the blue solid line represents a path. In a disconnected graph (right), there exist pairs of vertices that cannot be connected by any path, as indicated by the crossed-out paths. The disconnected graph has two connected components (outlined in blue), which are maximal connected subgraphs.

Proof. Suppose vertices $u, v \in V(G)$ are the ends of a walk:

$$u = v_0, v_1, \dots, v_k = v$$

Take a shortest walk from u to v . We can cut out any loops in the walk. If some v_i is visited more than once, say:

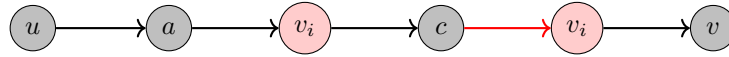
$$u \rightarrow v_i \rightarrow \dots \rightarrow v_i \rightarrow v$$

Then we can use the segment of the walk from u to v_i followed by the segment from v_i to v :

$$u \rightarrow v_i \rightarrow v$$

which is shorter, a contradiction. So a shortest walk from u to v is a path. □

Converting a Walk to a Path



Original walk with loop



Shortened walk (path) by removing the loop

Figure 22: Illustration of the process of converting a walk to a path by removing loops. In the original walk (top), vertex v_i appears twice, creating a loop in the walk. By removing the portion of the walk between the two occurrences of v_i (marked in red), we obtain a shorter walk from u to v that doesn't repeat any vertices (bottom), thus forming a path. This technique is fundamental for proving that if there exists a walk between two vertices in a graph, then there must also exist a path between them.

1.7.1 More Examples of Walks, Paths, and Components

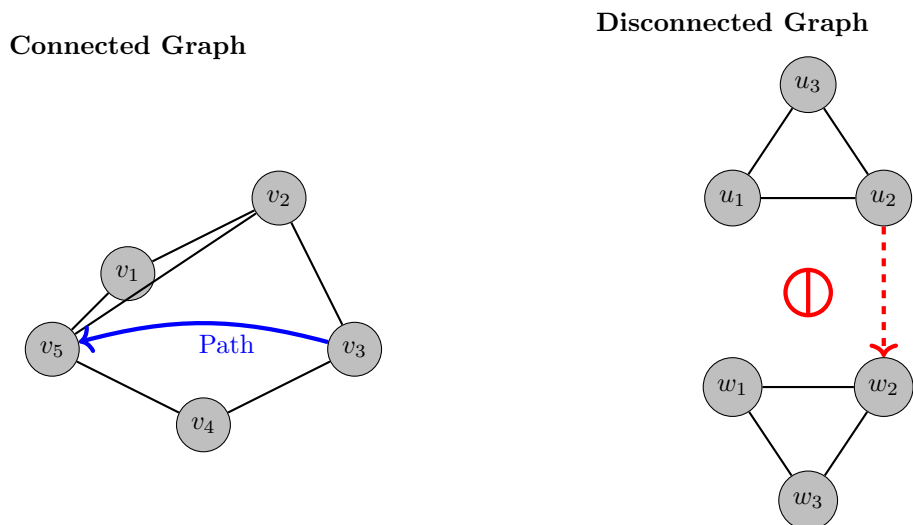


Figure 23: Comparison of connected and disconnected graphs. The connected graph (left) has a single component where a path exists between any pair of vertices, demonstrated by the blue path from v_3 to v_5 . The disconnected graph (right) contains two triangular structures that have no paths between them, as shown by the crossed-out red line. This fundamental distinction affects many graph properties, including traversability, algorithmic complexity, and network reliability.

Two separated components Three separated components Isolated vertices

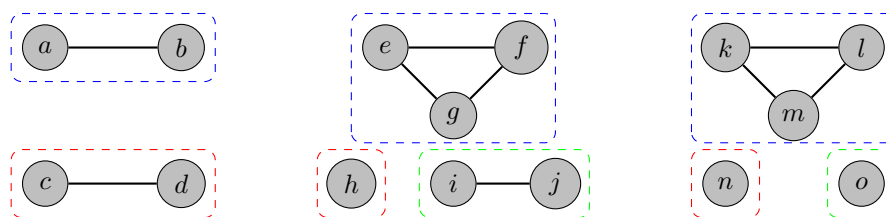


Figure 24: Various forms of disconnected graphs. Left: A disconnected graph with two separate edge components, where each component is a single edge. Middle: A graph with three distinct components - a triangular component, a single isolated vertex, and a path component with two vertices. Right: A graph with a triangular component and two isolated vertices. Each connected component (highlighted by dashed colored rectangles) forms a maximal connected subgraph. In graph theory, isolated vertices are considered separate connected components.

Graph Components

Disconnected Graph with 3 Components

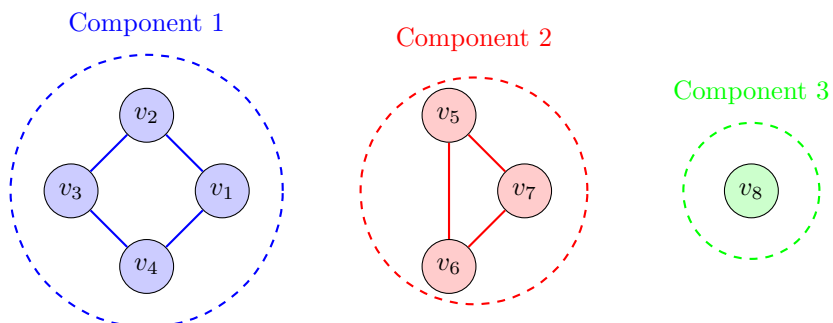
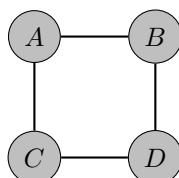


Figure 25: Illustration of a disconnected graph with three distinct components. Component 1 (blue) is a cycle graph with 4 vertices. Component 2 (red) is a triangle with 3 vertices. Component 3 (green) consists of a single isolated vertex. Each component is a maximal connected subgraph, meaning that there exists a path between any two vertices within the same component, but no path between vertices from different components. The number of components in a graph is an important invariant that affects connectivity, traversability, and many graph algorithms.

1.8 Chapter 1 Exercises

1.8.1 Easy Questions

1. Define the degree of a vertex in a graph and calculate the sum of all vertex degrees in the cycle graph C_5 .
2. Draw all possible non-isomorphic simple graphs with exactly 3 vertices.
3. Consider the following graph:

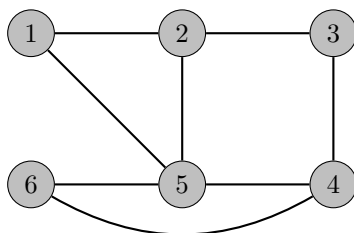


List all paths from vertex A to vertex D .

4. What is the difference between a walk and a path in a graph? Give an example of a walk that is not a path.
5. For each of the following graphs, determine if it is connected or disconnected:
 - A star graph S_6
 - A path graph P_4
 - A graph with vertices $\{a, b, c, d\}$ and edges $\{\{a, b\}, \{c, d\}\}$

1.8.2 Intermediate Questions

6. Draw a graph with degree sequence $(3, 3, 2, 2, 2, 0)$. Is there more than one non-isomorphic graph with this degree sequence?
7. Prove that a connected graph with n vertices must have at least $n - 1$ edges.
8. Consider a graph G with vertex set $V = \{1, 2, 3, 4, 5, 6\}$ and edges $\{1, 2\}, \{1, 5\}, \{2, 3\}, \{2, 5\}, \{3, 4\}, \{4, 5\}, \{4, 6\}, \{5, 6\}$.



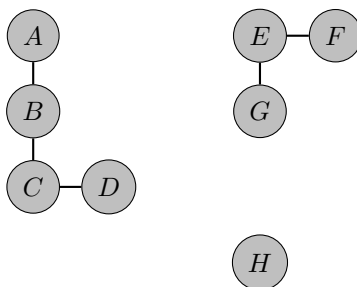
- Find an induced subgraph with vertex set $\{1, 2, 5, 6\}$
 - Find a spanning subgraph with exactly 6 edges
9. The Petersen graph is shown in your textbook (Figure 11). Find the longest path in this graph. Prove that your path is indeed the longest possible.
 10. Show that any graph with at least two vertices must have at least two vertices with the same degree.

1.8.3 Hard Questions

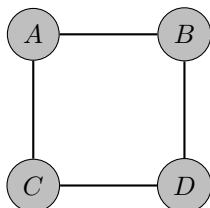
11. Prove that if G is a simple graph with n vertices and more than $\frac{(n-1)(n-2)}{2}$ edges, then G must be connected.
12. Let G be a graph with n vertices. If each vertex has degree at least $\frac{n-1}{2}$, prove that G is connected.
13. A graph is called bipartite if its vertices can be divided into two disjoint sets such that every edge connects a vertex in the first set to one in the second set.
 - Prove that a graph is bipartite if and only if it contains no cycles of odd length.
 - Which of the following graphs are bipartite: C_4 , C_5 , $K_{3,3}$, the Petersen graph?
14. Design a graph that that:
 - Has exactly 8 vertices
 - Is connected
 - Has exactly 3 vertices of degree 3, and the rest of degree 2
 - Contains exactly one cycle of length 4 and one cycle of length 5
15. Consider the hypercube graph Q_4 .
 - How many vertices and edges does Q_4 have?
 - What is the degree of each vertex?
 - Devise an algorithm to determine if there exists a path of length exactly k between any two vertices in Q_4 .

1.8.4 Questions with Diagrams

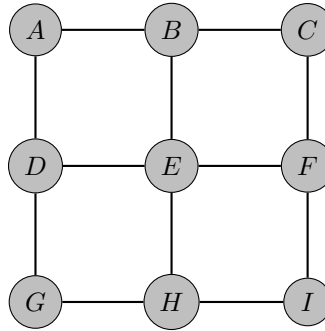
16. For the graph below, identify:
 - All isolated vertices
 - All pendant vertices
 - The degree sequence
 - The number of connected components



17. Draw all possible spanning trees of the following graph: (Hint 1: Refer to Definition 33 in Chapter 5-2, pg 63)



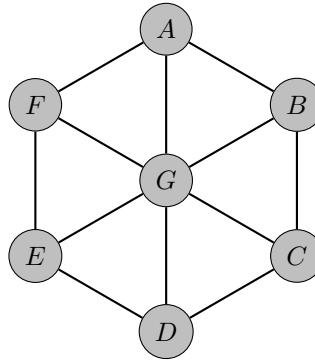
18. Consider the graph G shown below:



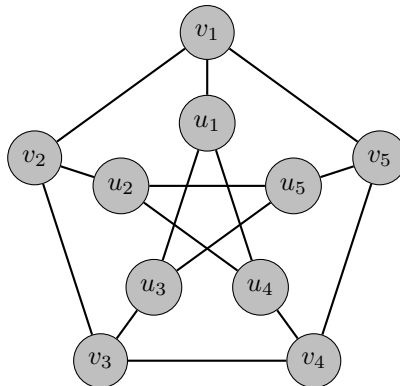
- Draw an induced subgraph $G[\{A, B, D, E\}]$
- Find a subgraph that is not an induced subgraph
- Draw a spanning subgraph that is a tree
- What is the maximum number of edges that can be removed while keeping the graph connected?

19. In the wheel graph W_6 (shown below), find:

- The shortest path from any outer vertex to the vertex diametrically opposite on the cycle
- The number of distinct cycles of length 3
- The number of distinct cycles of length 4



20. Given the Petersen graph:



- Prove that it does not contain a cycle of length 3
- Find all cycles of length 5
- Show that the graph is not bipartite (Hint : Chapter 5-9)

2 Eulerian Graphs

2.1 Trails and Tours

Definition 13. A trail in a graph is a sequence of vertices:

$$v_1, v_2, \dots, v_n$$

such that $\{v_i, v_{i+1}\} \in E(G)$ and no edge appears twice. Unlike a path, a trail may visit the same vertex multiple times, but it cannot use the same edge more than once.

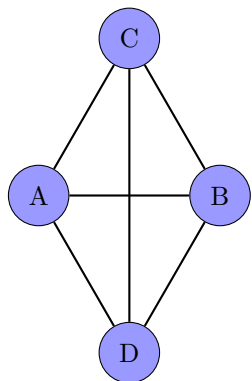
Definition 14. A tour is a trail $v_1, v_2, \dots, v_n$ with $v_n = v_1$. That is, a tour is a closed trail that starts and ends at the same vertex. A tour is also sometimes called a circuit.

Definition 15. An Eulerian trail is a trail that traverses every edge of the graph exactly once.

Definition 16. An Eulerian tour (or Eulerian circuit) is a tour that traverses every edge of the graph exactly once.

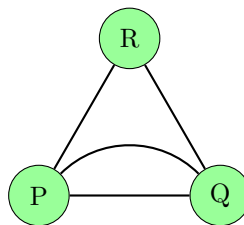
Theorem 1. Let G be a connected graph (or multigraph), and let $u, v \in V(G)$. Then:

1. The graph has an Eulerian tour if and only if all vertices have even degree.
2. The graph has an Eulerian trail from u to v if and only if u and v have odd degree and all other vertices have even degree.



Complete graph K_4

Eulerian tour exists



Graph with 2 odd vertices

Eulerian trail exists, but no tour

Figure 26: Example graphs illustrating Eulerian concepts. Left: Complete graph K_4 where all vertices have degree 3 (odd). Although it doesn't have an Eulerian tour, it does have an Eulerian trail. Right: A graph with exactly two odd-degree vertices (P and Q) that has an Eulerian trail but no Eulerian tour.

Proof of (1). ($\Rightarrow$) Suppose there is an Eulerian tour $C = v_1, v_2, \dots, v_m, v_1$. We prove all vertices have even degree by induction on $|E(G)|$.

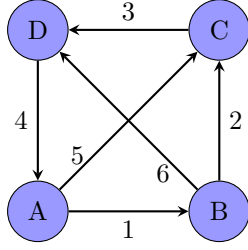
Base case: $|E(G)| = 1$. The Eulerian tour is v_1, v_1 , so $d(v_1) = 2$.

Suppose the strong induction statement is true for all graphs with fewer than m edges. Let $C = v_1, v_2, \dots, v_m, v_1$ be an Eulerian tour in G with m edges. Let $j = \min\{i > 1 : v_i = v_j \text{ for some } j < i\}$.

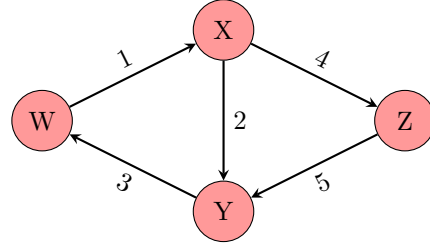
If $j = m + 1$, then the entire tour is a simple cycle with m edges, so every vertex has degree 2.

If $j < m + 1$:

- Remove all edges $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{j-1}, v_j\}$



Eulerian tour: A-B-C-D-A-C-B-D-A
All vertices have degree 4 (even)



Eulerian trail: W-X-Y-W-X-Z-Y
W and Y have degree 3 (odd)
X and Z have degree 2 and 4 (even)

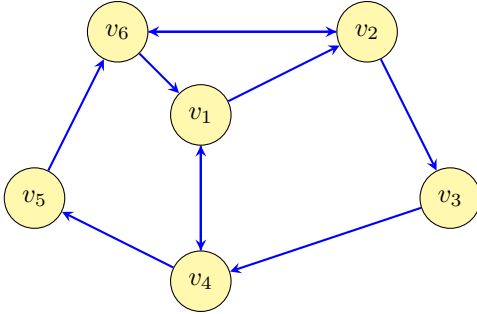
Figure 27: Explicit examples of Eulerian tours and trails. Left: A graph where every vertex has even degree (4), with a complete Eulerian tour shown by numbered edges. The tour visits every edge exactly once and returns to the starting vertex A. Right: A graph with exactly two odd-degree vertices (W and Y), showing an Eulerian trail that traverses every edge exactly once but doesn't form a closed circuit.

By induction, the remaining edges form an Eulerian tour $v_j, v_{j+1}, \dots, v_m, v_1$, so all vertices in this subgraph have even degree. Also, $C' = v_1, v_2, \dots, v_j$ is also an Eulerian tour in its subgraph, so all those vertices have even degree. Therefore, all vertices in the original graph have even degree.

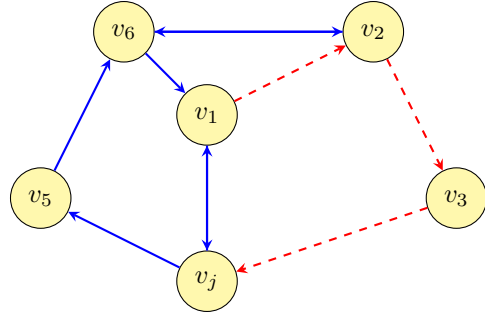
($\Leftarrow$) Let $C = v_1, v_2, \dots, v_n$ be a longest trail in the graph.

Case 1: Suppose $v_n \neq v_1$. Since v_n has even degree, we know an edge $\{v_n, v_{n+1}\}$ not in the trail exists. Now $C' = v_1, v_2, \dots, v_n, v_{n+1}$ is a longer trail, a contradiction.

Case 2: $v_n = v_1$. Since G is connected, all edges must be used by the trail. Otherwise, relabel the trail as $v'_1, v'_2, \dots, v'_{n-1}, v'_n = v'_1$. We can add $\{v'_i, v'_{n+1}\}$ to create a longer trail, contradiction. Therefore, the graph has an Eulerian tour. $\square$



Eulerian tour: each vertex is
visited an even number of times

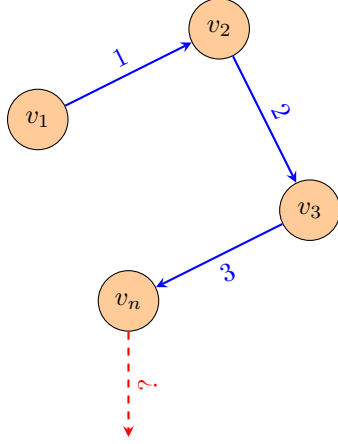


Induction step: Decompose into
smaller Eulerian tours

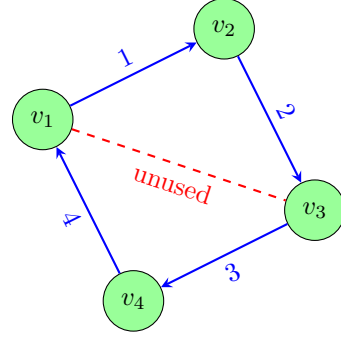
Figure 28: Visualization of the forward direction proof - showing why Eulerian tours imply even degrees. Left: A graph with an Eulerian tour (blue arrows) where each vertex must be visited and departed from an equal number of times, giving each vertex an even degree. Right: The induction step showing how an Eulerian tour can be decomposed into smaller Eulerian subcircuits.

Proof of (2). ($\Rightarrow$) Suppose G has an Eulerian trail from u to v with $u \neq v$. Add an edge e between u and v . The resulting graph $G' = G \cup \{e\}$ has an Eulerian tour. By part (1), all vertices in G' have even degree.

In the original graph G , the degrees of u and v are both reduced by 1, making them odd, while all other vertices maintain their even degree.



Case 1: $v_n \neq v_1$ and $d(v_n)$ is even
 $\Rightarrow$ Unused edge exists (contradiction)

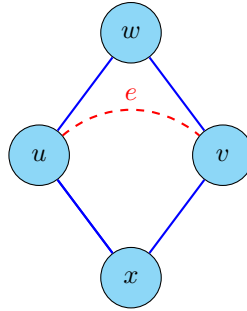


Case 2: $v_n = v_1$ but not all edges used
 $\Rightarrow$ Can extend trail (contradiction)

Figure 29: Visualization of the reverse direction proof - showing why even degrees imply Eulerian tours. Left: If the longest trail doesn't end at its starting point and the final vertex has even degree, an unused edge must exist (contradiction). Right: If the trail forms a tour but doesn't use all edges, the graph has another edge that could extend the tour (contradiction).

($\Leftarrow$) Suppose u and v have odd degree, and all other vertices have even degree. Add an edge e between u and v . In the resulting graph $G' = G \cup \{e\}$, all vertices now have even degree.

By part (1), G' has an Eulerian tour. This tour must use the edge e at some point. Remove e from the tour, and we obtain an Eulerian trail in G from u to v . $\square$



Proof of (2): Add edge e between u and v
This creates a graph where all vertices have even degree
 $\Rightarrow$ It has an Eulerian tour by part (1)

Figure 30: Visualization of the proof for part (2) - Converting between Eulerian trails and tours. By adding a new edge e (red dashed line) between the two odd-degree vertices u and v , we transform the graph into one where every vertex has even degree. This creates a graph with an Eulerian tour, and removing edge e gives us the original Eulerian trail.

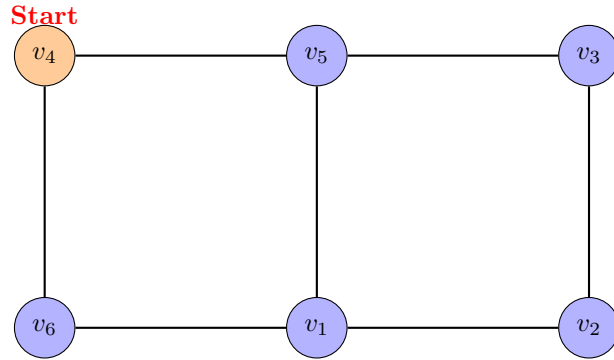


Figure 31: Graph for applying Fleury's Algorithm. This figure displays the initial connected graph with six vertices labeled v_1 through v_6 arranged in a rectangular pattern. Vertex v_4 is highlighted in orange as the starting point, indicated by the red "Start" label. The graph has seven edges connecting the vertices in a way that allows for an Eulerian trail.

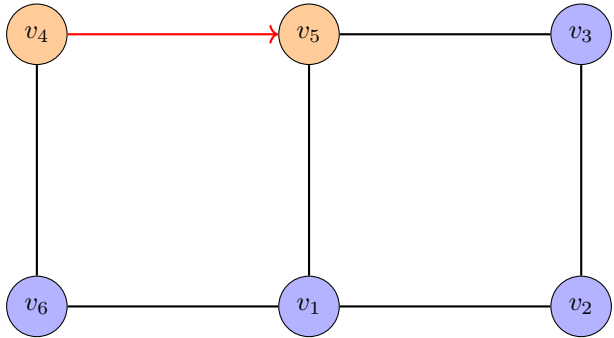


Figure 32: Step 1: Choose edge $\{v_4, v_5\}$. This figure shows the first step of Fleury's Algorithm. Vertices v_4 and v_5 are both highlighted in orange, with a red directed arrow indicating the traversal from v_4 to v_5 . This represents the first edge chosen in the Eulerian trail.

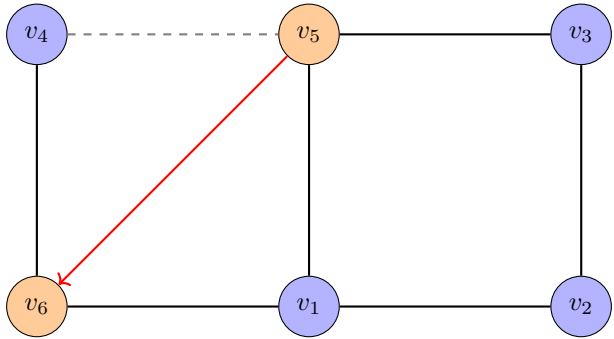


Figure 33: Step 2: Choose edge $\{v_5, v_6\}$. Vertices v_5 and v_6 are highlighted in orange. The previously traversed edge (from v_4 to v_5) is shown as a dashed gray line. A red directed arrow connects v_5 to v_6 , showing the second edge chosen in the algorithm, following the rule of selecting edges that don't disconnect the graph.

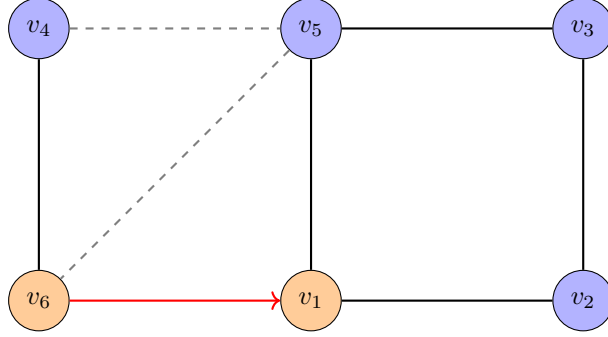


Figure 34: Step 3: Choose edge $\{v_6, v_1\}$. Vertices v_6 and v_1 are highlighted in orange. Previously traversed edges (v_4-v_5 and v_5-v_6) are shown as dashed gray lines. A red directed arrow connects v_6 to v_1 , representing the current edge being traversed in the Eulerian trail.

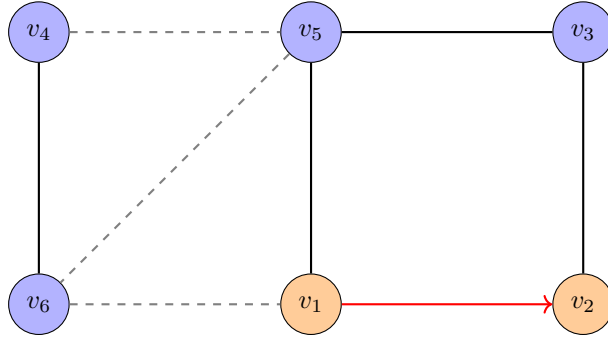


Figure 35: Step 4: Choose edge $\{v_1, v_2\}$. Vertices v_1 and v_2 are highlighted in orange. Three edges have now been traversed and appear as dashed gray lines. The current traversal from v_1 to v_2 is shown with a red directed arrow, continuing to follow the principle of choosing edges that maintain connectivity when possible.

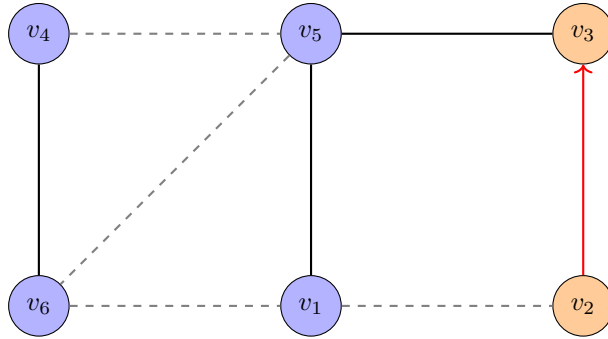


Figure 36: Step 5: Choose edge $\{v_2, v_3\}$. Vertices v_2 and v_3 are highlighted in orange. Four previously traversed edges are shown as dashed gray lines. The current edge being traversed from v_2 to v_3 is indicated by a red directed arrow.

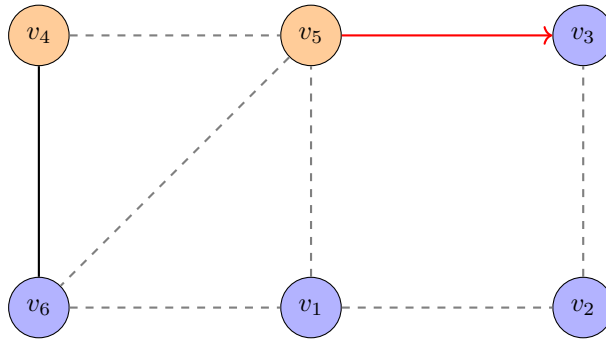


Figure 37: Step 6: Choose edge $\{v_5, v_3\}$. Vertices v_4 and v_5 are highlighted in orange. Most edges have been traversed (shown as dashed gray lines). The current traversal is from v_5 to v_3 , indicated by a red directed arrow. Note that the algorithm now traverses the remaining edges to complete the Eulerian trail.

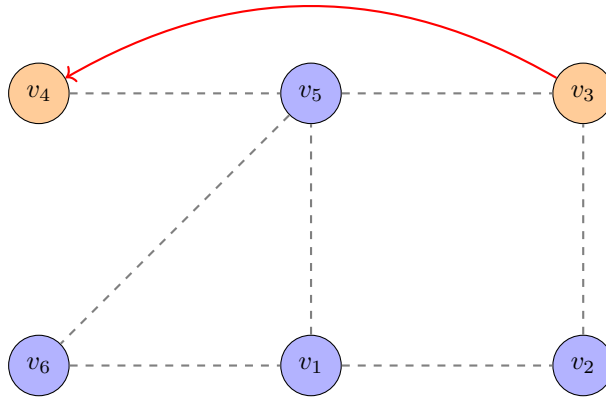


Figure 38: Step 7: Choose final edge $\{v_3, v_4\}$ to complete the tour. Vertices v_4 and v_3 are highlighted in orange with a red directed arrow from v_3 to v_4 . This represents the completion of the Eulerian trail as the algorithm returns to the starting vertex after having traversed all edges in the graph exactly once.

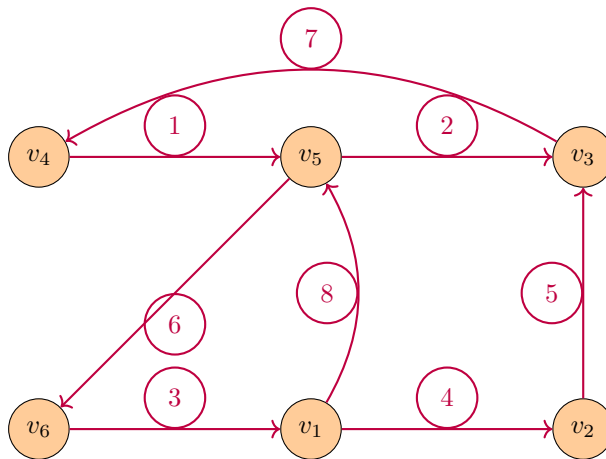


Figure 39: Complete Eulerian tour using Fleury's Algorithm: $v_4 \rightarrow v_5 \rightarrow v_3 \rightarrow v_2 \rightarrow v_1 \rightarrow v_6 \rightarrow v_4 \rightarrow v_5 \rightarrow v_3$. All vertices are highlighted in orange, and all edges are shown as purple directed arrows with numbers indicating the order of traversal (1-8). This figure displays the entire Eulerian path found by the algorithm, showing how every edge in the graph is traversed exactly once.

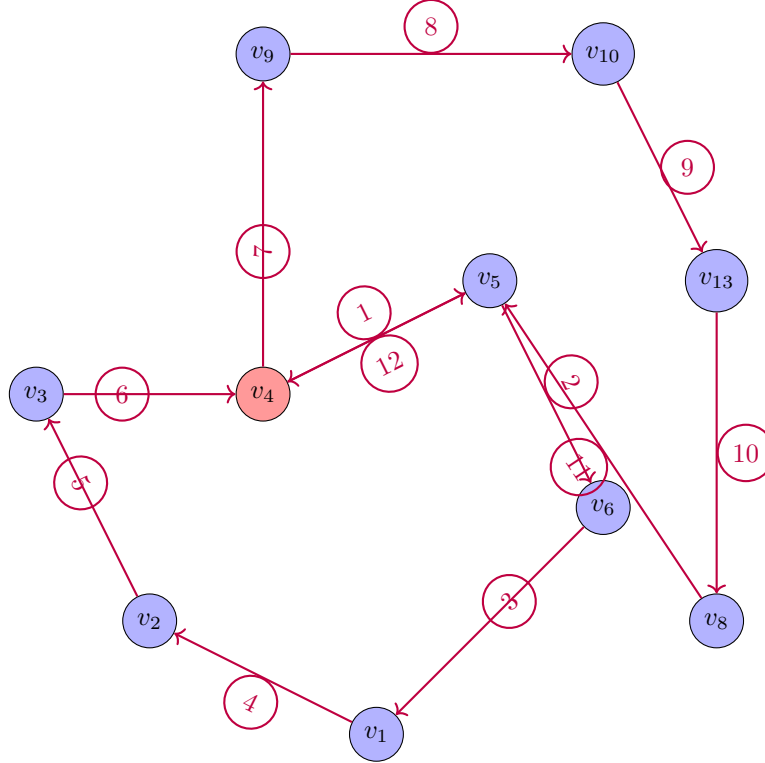


Figure 40: Example Eulerian tour: $v_4 \rightarrow v_5 \rightarrow v_6 \rightarrow v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow v_4 \rightarrow v_9 \rightarrow v_{10} \rightarrow v_{13} \rightarrow v_8 \rightarrow v_5 \rightarrow v_4$. This figure shows a more complex example of an Eulerian tour on a larger graph with 10 vertices. The starting vertex v_4 is highlighted in red, and the complete tour is shown with purple directed arrows numbered 1-12. This demonstrates how Fleury's Algorithm scales to larger graphs while still finding a valid Eulerian path that traverses each edge exactly once before returning to the starting vertex.

Algorithm 1 Fleury's Algorithm for finding an Eulerian path or circuit

```

1: procedure FLEURY( $G(V, E)$ ,  $start$ )
2:   Let  $currentVertex \leftarrow start$ 
3:   Initialize empty list  $path$ 
4:   while  $E$  is not empty do
5:     if degree of  $currentVertex$  is 1 then
6:       Let  $nextEdge$  be the only edge incident to  $currentVertex$ 
7:     else
8:       Let  $nextEdge$  be any edge incident to  $currentVertex$  such that removing  $nextEdge$  doesn't
       disconnect the graph
9:     end if
10:    Let  $nextVertex$  be the vertex at the other end of  $nextEdge$ 
11:    Add  $currentVertex$  to  $path$ 
12:    Remove  $nextEdge$  from  $E$ 
13:     $currentVertex \leftarrow nextVertex$ 
14:  end while
15:  Add  $currentVertex$  to  $path$  ▷ Add the final vertex
16:  return  $path$ 
17: end procedure

```

2.2 Eulerian Digraphs and DeBruijn Sequences

2.2.1 Eulerian Digraphs

Definition 17 (Eulerian Directed Graphs). A directed graph (digraph) is Eulerian if there exists a closed walk that traverses each edge exactly once. The following properties characterize Eulerian digraphs:

1. A digraph is Eulerian if and only if it is connected and the in-degree equals the out-degree for every vertex.
2. A digraph has an Eulerian path (but not necessarily a cycle) if and only if:
 - At most one vertex has $(\text{out-degree}) - (\text{in-degree}) = 1$
 - At most one vertex has $(\text{in-degree}) - (\text{out-degree}) = 1$
 - For all other vertices, in-degree equals out-degree

For a directed graph $G = (V, E)$, define:

$$d^+(v) = \text{out-degree of vertex } v \quad (5)$$

$$d^-(v) = \text{in-degree of vertex } v \quad (6)$$

Then G is Eulerian if and only if:

1. G is connected (or more precisely, the underlying undirected graph of G is connected)
2. $\forall v \in V, d^+(v) = d^-(v)$

2.2.2 De Bruijn Sequences

Definition 18 (De Bruijn Sequence). A De Bruijn sequence $B(k, n)$ is a cyclic sequence of length k^n over an alphabet A of size k such that each possible string of length n appears exactly once as a contiguous substring.

Properties 1 (De Bruijn Sequences).

1. The length of a De Bruijn sequence $B(k, n)$ is k^n .
2. Every string of length n over the alphabet appears exactly once as a substring.
3. A De Bruijn sequence $B(k, n)$ can be viewed as a Hamiltonian path in the De Bruijn graph $G(k, n-1)$.
4. The number of distinct De Bruijn sequences $B(k, n)$ is given by:

$$\frac{(k!)^{k^{n-1}}}{k^n} \quad (7)$$

5. More specifically, the number of binary De Bruijn sequences $B(2, n)$ is:

$$2^{2^{n-1} - n} \quad (8)$$

Properties 2 (Connection to Eulerian Digraphs). *De Bruijn sequences have a fundamental connection to Eulerian digraphs:*

1. A De Bruijn graph $G(k, n-1)$ is a directed graph with k^{n-1} vertices, each labeled with a string of length $n-1$.
2. There is an edge from vertex u to vertex v if the last $n-2$ symbols of u match the first $n-2$ symbols of v .

3. For every vertex in $G(k, n-1)$, the in-degree equals the out-degree (both equal to k).
4. Therefore, $G(k, n-1)$ is an Eulerian digraph.
5. A De Bruijn sequence $B(k, n)$ corresponds to an Eulerian circuit in $G(k, n-1)$.

Properties 3 (Mathematical Structure). The adjacency matrix A of the De Bruijn graph $G(k, n)$ has the following properties:

1. A is a $k^n \times k^n$ matrix.
2. $A_{i,j} = 1$ if the last $n-1$ symbols of vertex i match the first $n-1$ symbols of vertex j .
3. $A_{i,j} = 0$ otherwise.
4. The number of Eulerian circuits (and thus De Bruijn sequences) can be computed using the BEST theorem:

$$\text{Number of Eulerian circuits} = t_w \prod_{v \in V} (d^+(v) - 1)! \quad (9)$$

where t_w is the number of oriented spanning trees (refer to def 33) rooted at vertex w .

Properties 4 (Combinatorial Aspects). The combinatorial aspects of De Bruijn Sequence $B(k, n)$ has the following properties

1. A De Bruijn sequence can be constructed from a Eulerian circuit in the De Bruijn graph $G(k, n-1)$.
2. Every De Bruijn sequence $B(k, n)$ contains k^n distinct substrings of length n .
3. The De Bruijn sequence has a feedback function f that generates the next bit:

$$a_{i+n} = f(a_i, a_{i+1}, \dots, a_{i+n-1}) \quad (10)$$

Theorem 2 (Generating Function). For a binary De Bruijn sequence, the generating function can be expressed as:

$$G(x) = \sum_{i=0}^{2^n-1} a_i x^i \mod (x^{2^n} - 1) \quad (11)$$

where a_i represents the i -th element in the sequence.

Theorem 3 (LFSR Representation). A binary De Bruijn sequence can be generated using a Linear Feedback Shift Register with the characteristic polynomial:

$$p(x) = x^n + c_{n-1}x^{n-1} + \dots + c_1x + c_0 \quad (12)$$

The recurrence relation is:

$$a_{i+n} = c_0 a_i \oplus c_1 a_{i+1} \oplus \dots \oplus c_{n-1} a_{i+n-1} \quad (13)$$

where $\oplus$ represents the XOR operation.

Theorem 4 (Counting Formula). The number of distinct De Bruijn sequences $B(k, n)$ is:

$$N(k, n) = \frac{k!^{k^{n-1}}}{k^n} \quad (14)$$

For the binary case $B(2, n)$, the number of distinct sequences is:

$$N(2, n) = 2^{2^{n-1}-n} \quad (15)$$

Example 2 (Small Values). The number of distinct De Bruijn sequences for small parameters:

$$N(2, 2) = 1 \quad (16)$$

$$N(2, 3) = 2 \quad (17)$$

$$N(2, 4) = 16 \quad (18)$$

$$N(2, 5) = 2048 \quad (19)$$

$$N(3, 2) = 2 \quad (20)$$

$$N(3, 3) = 48 \quad (21)$$

$$N(4, 2) = 24 \quad (22)$$

Theorem 5 (FKM Algorithm). The FKM (Fredricksen, Kessler, Maiorana) algorithm constructs a binary De Bruijn sequence $B(2, n)$ by generating all necklaces of length n , ordering them lexicographically, replacing each with its aperiodic prefixes, and concatenating the last bits of these prefixes in order.

Algorithm 2 FKM Algorithm for $B(2, n)$

Require: Integer $n \geq 2$

Ensure: Binary De Bruijn sequence $B(2, n)$

```

1:  $S \leftarrow$  empty sequence
2: function GENERATENECKLACES( $n$ )
3:    $N \leftarrow \emptyset$ ;  $\text{seen} \leftarrow \emptyset$ 
4:   for all binary strings  $b$  of length  $n$  in lexicographic order do
5:     if  $b \notin \text{seen}$  then
6:       necklace  $\leftarrow$  smallest rotation of  $b$ 
7:        $N \leftarrow N \cup \{\text{necklace}\}$ ;  $\text{seen} \leftarrow \text{seen} \cup \{\text{all rotations of } b\}$ 
8:     end if
9:   end for
10:  return  $N$ 
11: end function
12: function GETAPERIODICPREFIXES(necklace)
13:   $\text{prefixes} \leftarrow \emptyset$ ;  $p \leftarrow$  period of necklace
14:  for  $i \leftarrow 1$  to  $p$  do
15:     $\text{prefixes} \leftarrow \text{prefixes} \cup \{\text{first } i \text{ bits of necklace}\}$ 
16:  end for
17:  return  $\text{prefixes}$ 
18: end function
19:  $\text{necklaces} \leftarrow \text{Sort}(\text{GenerateNecklaces}(n))$ 
20: for each necklace in  $\text{necklaces}$  do
21:   for each prefix in  $\text{GetAperiodicPrefixes}(\text{necklace})$  do
22:      $S \leftarrow S \cdot \text{last bit of prefix}$ 
23:   end for
24: end for
25: return  $S$ 
```

Remark 1. The FKM algorithm can also be implemented using Lyndon words:

Theorem 6 (Prefer-1 Algorithm). The Prefer-1 algorithm generates a binary De Bruijn sequence $B(2, n)$ by starting with n zeros and repeatedly appending either a 1 (preferred) or a 0, choosing whichever creates a new n -tuple that hasn't appeared before in the sequence.

Theorem 7 (Correctness). The Prefer-1 algorithm correctly generates a binary De Bruijn sequence of order n with length 2^n .

Algorithm 3 Lyndon-based FKM Algorithm

Require: Integer $n \geq 2$ **Ensure:** Binary De Bruijn sequence $B(2, n)$

```
1: function ISLYNDONWORD( $w$ )
2:   for  $i \leftarrow 1$  to  $|w| - 1$  do
3:     if  $w = \text{rotation of } w \text{ by } i$  or  $w > \text{rotation of } w \text{ by } i$  then
4:       return false
5:     end if
6:   end for
7:   return true
8: end function
9: function PERIOD( $w$ )
10:  for  $i \leftarrow 1$  to  $|w|$  do
11:    if  $w = \text{rotation of } w \text{ by } i$  then return  $i$ 
12:    end if
13:  end for
14:  return  $|w|$ 
15: end function
16:  $S \leftarrow$  empty sequence
17: for each binary string  $w$  of length  $n$  in lexicographic order do
18:   if IsLyndonWord( $w$ ) then
19:      $period \leftarrow \text{Period}(w)$ 
20:     for  $i \leftarrow 1$  to  $period$  do
21:        $S \leftarrow S \cdot \text{last bit of first } i \text{ bits of } w$ 
22:     end for
23:   end if
24: end for
25: return  $S$ 
```

Algorithm 4 Prefer-1 Algorithm for $B(2, n)$

Require: Integer $n \geq 2$ **Ensure:** Binary De Bruijn sequence $B(2, n)$

```
1:  $R \leftarrow$   $n$ -bit register initialized to all zeros
2:  $S \leftarrow$  sequence of  $n$  zeros ▷ Output the initial  $n$  zeros
3:  $seen \leftarrow \{R\}$  ▷ Set of seen  $n$ -tuples, initially contains all zeros
4: while  $|seen| < 2^n$  do
5:    $R_{next1} \leftarrow$  last  $n - 1$  bits of  $R$  followed by 1
6:   if  $R_{next1} \notin seen$  then
7:      $R \leftarrow R_{next1}$  ▷ Shift in a 1
8:      $S \leftarrow S \cdot 1$  ▷ Append 1 to output
9:   else
10:     $R \leftarrow$  last  $n - 1$  bits of  $R$  followed by 0 ▷ Shift in a 0
11:     $S \leftarrow S \cdot 0$  ▷ Append 0 to output
12:   end if
13:    $seen \leftarrow seen \cup \{R\}$ 
14: end while
15: return  $S$ 
```

Remark 2. *The Prefer-1 algorithm can be viewed as a greedy approach that traverses a De Bruijn graph by always choosing the edge labeled 1 when possible. It represents a special case of the more general Prefer-d algorithm for arbitrary alphabet sizes.*

Theorem 8 (Prefer-Opposite Algorithm). The Prefer-Opposite algorithm generates a binary De Bruijn sequence $B(2, n)$ by starting with n zeros and repeatedly appending the opposite of the last bit (when possible) or the same as the last bit, choosing whichever creates a new n -tuple that hasn't appeared before in the sequence.

Algorithm 5 Prefer-Opposite Algorithm for $B(2, n)$

Require: Integer $n \geq 2$

Ensure: Binary De Bruijn sequence $B(2, n)$

```

1:  $R \leftarrow$  n-bit register initialized to all zeros
2:  $S \leftarrow$  sequence of  $n$  zeros ▷ Output the initial  $n$  zeros
3:  $last \leftarrow 0$  ▷ Last bit output
4:  $seen \leftarrow \{R\}$  ▷ Set of seen  $n$ -tuples, initially contains all zeros
5: while  $|seen| < 2^n$  do
6:    $opposite \leftarrow 1 - last$  ▷ Opposite of the last bit
7:    $R_{next} \leftarrow$  last  $n - 1$  bits of  $R$  followed by  $opposite$ 
8:   if  $R_{next} \notin seen$  then
9:      $last \leftarrow opposite$  ▷ Prefer the opposite
10:  else
11:     $R_{next} \leftarrow$  last  $n - 1$  bits of  $R$  followed by  $last$ 
12:  end if
13:   $R \leftarrow R_{next}$  ▷ Update register
14:   $S \leftarrow S \cdot last$  ▷ Append bit to output
15:   $seen \leftarrow seen \cup \{R\}$ 
16: end while
17: return  $S$ 

```

Theorem 9 (Correctness). The Prefer-Opposite algorithm correctly generates a binary De Bruijn sequence of order n with length 2^n .

Remark 3. *The Prefer-Opposite algorithm creates sequences with different balance properties than the Prefer-1 algorithm, often resulting in a more even distribution of 0s and 1s. This approach can be generalized to larger alphabets by defining a suitable notion of "opposite."*

Theorem 10 (Huang's Cycle-Joining Algorithm). Huang's algorithm constructs a De Bruijn sequence $B(k, n)$ by first generating all disjoint cycles of a feedback shift register, then identifying a spanning tree (refer to def 33) of the cycle graph, and systematically joining cycles according to the spanning tree to form a single cycle containing all states.

Theorem 11 (Correctness). Huang's Cycle-Joining algorithm correctly generates a De Bruijn sequence of order n over an alphabet of size k with length k^n .

Remark 4. *Huang's algorithm provides a general approach that works for any alphabet size and offers flexibility in how cycles are joined, enabling the construction of De Bruijn sequences with specific properties by choosing different spanning trees of the cycle graph.*

Remark 5 (On Spanning Trees). *The foundational concepts of spanning trees are addressed in Section 5.2 as part of our characterization of trees. Readers may find this material particularly relevant for understanding the tree-based algorithms presented in subsequent sections, including breadth-first search and depth-first search (Section 5.3), as well as Prim's and Kruskal's algorithms (Sections 5.4 and 5.5). Those unfamiliar with tree structures may wish to review the fundamental properties outlined in Section 5.1 before proceeding.*

Algorithm 6 Huang's Cycle-Joining Algorithm for $B(k, n)$

Require: Integer $n \geq 2$, alphabet size $k \geq 2$

Ensure: De Bruijn sequence $B(k, n)$

```
1: function GENERATEFSRCYCLES( $n, k$ )
2:   Generate all disjoint cycles formed by the feedback shift register
3:   return set of all cycles  $C = \{C_1, C_2, \dots, C_m\}$ 
4: end function
5: function BUILDCYCLEGRAPH( $C$ )
6:    $G \leftarrow$  empty graph with vertices representing cycles
7:   for each cycle  $C_i$  in  $C$  do
8:     Add a vertex  $v_i$  to  $G$  representing cycle  $C_i$ 
9:   end for
10:  for each pair of cycles  $C_i, C_j$  with a potential joining state do
11:    Add an edge  $(v_i, v_j)$  to  $G$  labeled with the joining state
12:  end for
13:  return  $G$ 
14: end function
15: function JOINCYCLES( $C_i, C_j, s$ )
16:    $s$  is a state appearing in both cycles at different predecessors
17:   Redirect transitions to merge  $C_i$  and  $C_j$  into a single cycle
18:   return the merged cycle
19: end function
20:  $cycles \leftarrow$  GenerateFSRCycles( $n, k$ )
21:  $G \leftarrow$  BuildCycleGraph( $cycles$ )
22:  $connectedCycles \leftarrow \emptyset$ 
23:  $joinEdges \leftarrow \emptyset$ 
24:  $result \leftarrow$  first cycle in  $cycles$ 
25:  $connectedCycles \leftarrow connectedCycles \cup \{result\}$ 
26: while  $|connectedCycles| < |cycles|$  do
27:    $bestEdge \leftarrow$  null
28:   for each edge  $(v_i, v_j)$  in  $G$  where  $v_i$  represents a cycle in  $connectedCycles$  and  $v_j$  doesn't do
29:     if  $bestEdge$  is null or edge  $(v_i, v_j)$  is preferred for joining then
30:        $bestEdge \leftarrow (v_i, v_j)$ 
31:     end if
32:   end for
33:   Add cycle represented by  $v_j$  to  $connectedCycles$ 
34:    $joinEdges \leftarrow joinEdges \cup \{bestEdge\}$ 
35:    $s \leftarrow$  joining state associated with  $bestEdge$ 
36:    $result \leftarrow$  JoinCycles( $result$ , cycle for  $v_j, s$ )
37: end while
38: return sequence representation of  $result$ 
```

▷ Set of cycles already joined
▷ Set of edges used to join cycles

Remark 6 (Non-uniqueness). *De Bruijn sequences are not unique for $n > 2$. The number of distinct De Bruijn sequences $B(2, n)$ is given by $2^{2^{n-1}-n}$, which grows rapidly with n . For example, there are 2 distinct $B(2, 3)$ sequences and 16 distinct $B(2, 4)$ sequences.*

Table 1: De Bruijn Sequence Algorithm Comparison

	Huang	Prefer-Opp.	Prefer-1	Lyndon
Purpose	General	Binary	Binary	Mathematical
Time	$O(k^n)$	$O(2^n)$	$O(2^n)$	$O(2^n)$
Space	$O(k^n)$	$O(n)$	$O(n)$	$O(n)$
Key Feature	Any k	In-place	Simplest	Elegant

2.3 De Bruijn Graphs

Definition 19 (De Bruijn Graph). A De Bruijn graph $G(k, n)$ is a directed graph representing all possible sequences of a given length over a given alphabet:

1. The graph has k^{n-1} vertices, where k is the alphabet size.
2. Each vertex represents a sequence of length $n - 1$ over the alphabet.
3. There is a directed edge from vertex u to vertex v if the last $n - 2$ symbols of u match the first $n - 2$ symbols of v .
4. Each edge is labeled with the symbol that extends the sequence represented by u to obtain the sequence for the destination vertex v .
5. The graph has exactly k^n edges.
6. For every vertex, the in-degree equals the out-degree (both equal to k).
7. A De Bruijn sequence $B(k, n)$ corresponds to an Eulerian cycle in this graph.

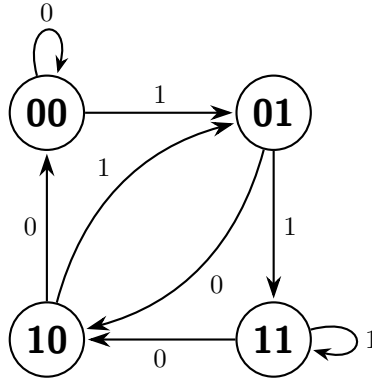


Figure 41: De Bruijn Graph $B(2, 3)$. This directed graph has 4 vertices representing all 2-bit sequences, and 8 edges labeled by the appended bit. An Eulerian circuit on this graph yields a de Bruijn sequence of order 3 over a binary alphabet.

Lemma 2 (Connection to Eulerian Circuits). A De Bruijn sequence $B(k, n)$ corresponds exactly to an Eulerian circuit in the De Bruijn graph $B(k, n - 1)$.

Proof sketch. Each edge in the De Bruijn graph $B(k, n - 1)$ represents the transition from one $(n - 1)$ -length sequence to another by appending a symbol and removing the first symbol. Following an Eulerian circuit through all edges generates each possible n -length sequence exactly once, which is precisely the definition of a De Bruijn sequence. $\square$

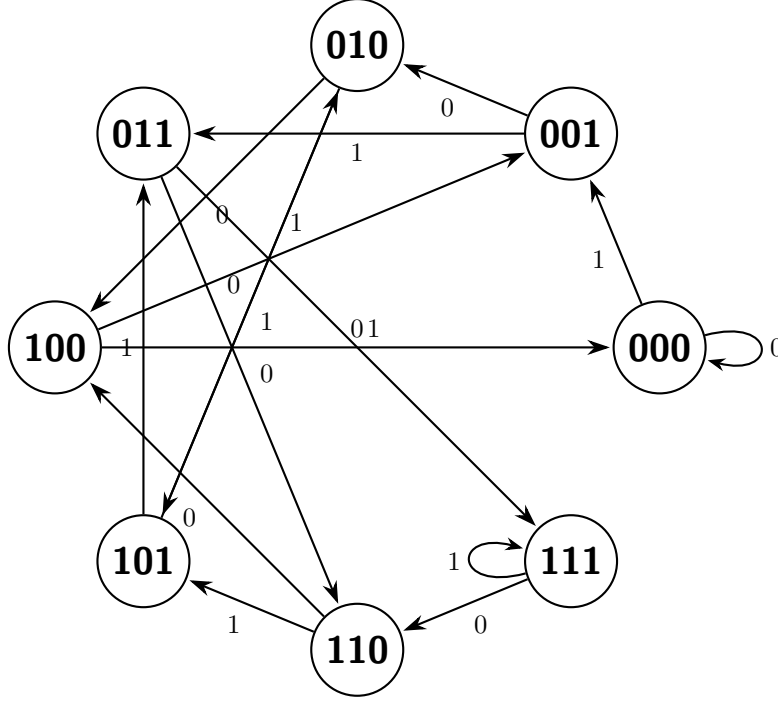


Figure 42: De Bruijn Graph $B(2, 4)$. This directed graph has 8 vertices representing all possible 3-bit binary sequences (000 through 111). Each edge is labeled with the bit (0 or 1) that is appended when transitioning between states. The graph has 16 edges in total, and a De Bruijn sequence $B(2, 4)$ corresponds to an Eulerian circuit traversing all edges exactly once.

Lemma 3 (Fundamental Property of De Bruijn Graphs). For any alphabet size k and order n , the De Bruijn graph $B(k, n)$ has exactly k^n vertices and k^{n+1} edges.

Proof. Each vertex in $B(k, n)$ represents a sequence of n symbols from an alphabet of size k , giving exactly k^n possible vertices. From each vertex, there are k possible transitions (one for each symbol in the alphabet), resulting in exactly $k \cdot k^n = k^{n+1}$ edges. $\square$

Lemma 4 (Connection to Eulerian Circuits). A De Bruijn sequence $B(k, n)$ corresponds exactly to an Eulerian circuit in the De Bruijn graph $B(k, n - 1)$.

Proof sketch. Each edge in the De Bruijn graph $B(k, n - 1)$ represents the transition from one $(n - 1)$ -length sequence to another by appending a symbol and removing the first symbol. Following an Eulerian circuit through all edges generates each possible n -length sequence exactly once, which is precisely the definition of a De Bruijn sequence. $\square$

Remark 7 (Relationship to Universal Cycles). *De Bruijn sequences are a special case of universal cycles, which are cyclic sequences where each object from a set appears exactly once as a consecutive substring. When the objects are all possible n -tuples from a k -symbol alphabet, the universal cycle is precisely a De Bruijn sequence of order n .*

Theorem 12 (Line Graph Relationship). The De Bruijn graph $B(k, n + 1)$ is isomorphic to the line graph of $B(k, n)$.

Proof. In the line graph of $B(k, n)$, each vertex corresponds to an edge in $B(k, n)$. An edge in $B(k, n)$ represents a transition from an n -length sequence to another by appending a symbol and removing the first symbol, which results in an $(n + 1)$ -length sequence. This exactly corresponds to a vertex in $B(k, n + 1)$. Two vertices in the line graph are adjacent if their corresponding edges in $B(k, n)$ form a directed path of length 2, which precisely matches the adjacency condition in $B(k, n + 1)$. $\square$

Theorem 13 (Shift Register Implementation). Every binary De Bruijn sequence of order n can be generated by a linear feedback shift register of length n with an appropriate feedback function.

Proof outline. Consider an n -bit shift register with bits $(b_1, b_2, \dots, b_n)$. For each state of the register, we compute the next bit b_{n+1} using a feedback function $f(b_1, b_2, \dots, b_n)$. This bit is then shifted into the register, and b_1 is discarded.

For a De Bruijn sequence, we need a feedback function f such that the shift register cycles through all 2^n possible states before repeating. This can be achieved by defining f such that:

1. For the state $(0, 0, \dots, 0)$, f outputs 1 (to avoid the cycle containing only zeros)
2. For all other states, f is chosen strategically to ensure a maximal cycle

One such function for binary De Bruijn sequences is:

$$f(b_1, b_2, \dots, b_n) = b_1 \oplus b_2 \oplus \dots \oplus b_{n-1} \oplus (b_n \vee 1)$$

where $\oplus$ is XOR and $\vee$ is OR. This produces a modified maximum-length sequence that includes all 2^n possible states. $\square$

2.4 Chapter 2 Exercises

2.4.1 Easy Problems: Eulerian Graphs

1. Determine whether the following graph is Eulerian, semi-Eulerian, or neither: A graph with vertices $\{A, B, C, D, E\}$ and edges $\{(A, B), (B, C), (C, D), (D, E), (E, A), (A, C), (C, E)\}$
2. Prove that a connected graph has an Eulerian path if and only if it has exactly 0 or 2 vertices with odd degree.
3. Construct a simple graph with 6 vertices that has an Eulerian circuit. How many edges must such a graph have at minimum?
4. Given a connected graph where all vertices have even degree, prove that it has an Eulerian circuit.

2.4.2 Medium Problems: Eulerian Graphs

5. A village has 7 bridges connecting different areas. Is it possible to walk through the village crossing each bridge exactly once? If so, find such a path. If not, explain why.
6. Show that if G is a connected graph where every vertex has even degree, then removing any edge results in a graph that has an Eulerian path but not an Eulerian circuit.
7. Given a connected graph G with $2k$ vertices of odd degree ($k \geq 1$), what is the minimum number of paths needed to cover all edges of G exactly once?
8. Prove that if G is a connected graph with exactly two vertices of odd degree, say u and v , then there exists an Eulerian path from u to v .

2.4.3 Hard Problems: Eulerian Graphs

9. Prove that a connected graph G has an Eulerian circuit if and only if its edge set can be partitioned into simple cycles.
10. Consider a complete graph K_n . For which values of n does K_n have an Eulerian circuit? Provide a proof for your answer.
11. Let G be a connected graph with an even number of edges. Prove that the edges of G can be partitioned into paths of length 2 if and only if G has no vertices of odd degree.
12. For a connected graph G , let $o(G)$ denote the number of vertices with odd degree. If $G_1, G_2, \dots, G_k$ are the connected components of G , prove that $\sum_{i=1}^k o(G_i)$ is even.

2.4.4 Easy Problems: De Bruijn Sequences

13. Construct a binary De Bruijn sequence of order 3 (i.e., a circular sequence where every possible 3-length binary string appears exactly once as a substring).
14. How many distinct binary De Bruijn sequences of order 4 exist? Explain your reasoning.
15. For a ternary alphabet $\{0, 1, 2\}$, what is the length of a De Bruijn sequence of order 2? List all possible substrings that must appear exactly once.

2.4.5 Medium Problems: De Bruijn Sequences

16. Prove that for any alphabet of size k , a De Bruijn sequence of order n has length k^n .
17. Construct a De Bruijn sequence of order 2 for an alphabet of size 3. Then describe an algorithm to generate such sequences for any alphabet size.
18. Show how to construct a De Bruijn sequence of order $n + 1$ from a De Bruijn sequence of order n .
19. Given a binary De Bruijn sequence of order 4, describe how many different such sequences exist and explain an algorithm to generate them.

2.4.6 Hard Problems: De Bruijn Sequences

20. Prove that any De Bruijn sequence of order n for an alphabet of size k can be represented as an Eulerian circuit in a directed graph with k^{n-1} vertices.
21. Given a De Bruijn sequence of order n over alphabet $\{0, 1\}$, describe an algorithm to find the position of any n -length binary string within the sequence without examining the entire sequence.
22. Prove that for any binary De Bruijn sequence of order n , there exists a corresponding binary necklace of length $2^n - 1$.

2.4.7 Easy Problems: De Bruijn Graphs

23. Draw the De Bruijn Graph of order 1 for the binary alphabet $\{0, 1\}$. How many vertices and edges does it have?
24. For a ternary alphabet $\{0, 1, 2\}$, construct the De Bruijn Graph of order 1. How many vertices and edges does it have?
25. Draw the De Bruijn Graph of order 2 for the binary alphabet $\{0, 1\}$. Then trace a path through this diagram that generates a De Bruijn sequence of order 3.
26. Draw a binary De Bruijn Graph of order 1 (with vertices 0 and 1). Then draw the corresponding De Bruijn Graph of order 2 using the line graph construction.
27. Draw a De Bruijn Graph for a ternary alphabet $\{0, 1, 2\}$ of order 1. Find an Eulerian circuit in this diagram and write down the corresponding De Bruijn sequence.

2.4.8 Medium Problems: De Bruijn Graphs

28. Prove that the De Bruijn Graph $B(k, n)$ has k^n vertices and k^{n+1} edges.
29. Consider a modified De Bruijn graph where certain transitions are forbidden. Specifically, in a binary De Bruijn graph of order 3, transitions to states containing "111" are prohibited. Is it still possible to find an Eulerian path in this modified diagram? If so, construct one. If not, explain why.

2.4.9 Hard Problems: De Bruijn Graphs

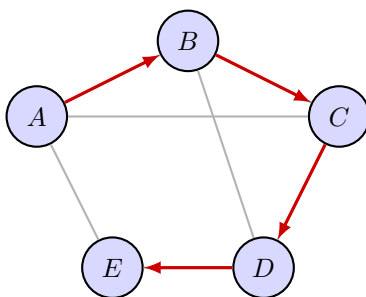
30. Let $B(k, n)$ be the De Bruijn graph of order n over an alphabet of size k . Prove that the line graph of $B(k, n)$ is isomorphic to $B(k, n + 1)$.
31. For a De Bruijn Graph $B(2, n)$, determine the chromatic number of the underlying undirected graph and prove your answer.

3 Hamiltonian Graphs

3.1 Hamiltonian Walks, Paths, Cycles

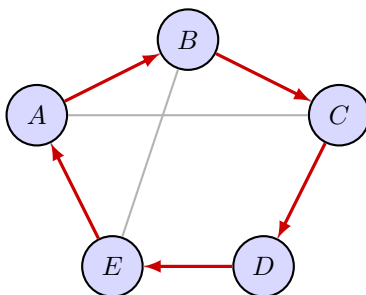
Definition 20 (Hamiltonian Cycle). A Hamiltonian cycle in a graph G is a cycle that visits each vertex of G exactly once (except for the starting vertex, which is also the ending vertex). Equivalently, a Hamiltonian cycle is a spanning cycle of the graph, meaning $V(C) = V(G)$. A graph is called Hamiltonian if it contains a Hamiltonian cycle.

Definition 21 (Hamiltonian Path). A Hamiltonian path in a graph G is a path that visits each vertex of G exactly once. Equivalently, a Hamiltonian path is a spanning path in the graph, meaning $V(P) = V(G)$. A graph is called traceable if it contains a Hamiltonian path.



Hamiltonian Path: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

Figure 43: Example of a Hamiltonian path. The red path visits every vertex of the graph exactly once, making it a Hamiltonian path. Unlike Eulerian paths which must traverse every edge exactly once, a Hamiltonian path must visit every vertex exactly once.



Hamiltonian Cycle: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$

Figure 44: Example of a Hamiltonian cycle. The red cycle visits every vertex of the graph exactly once before returning to the starting vertex, forming a closed loop.

Lemma 5 (Difference Between Hamiltonian and Eulerian). There exists no known simple necessary and sufficient condition for a graph to be Hamiltonian, unlike Eulerian graphs which can be characterized by the parity of vertex degrees.

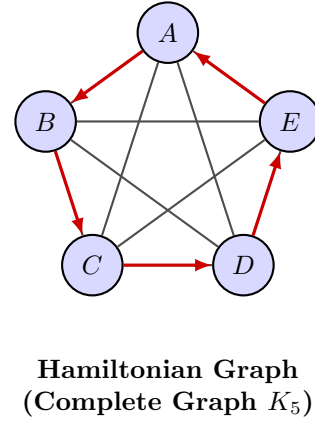
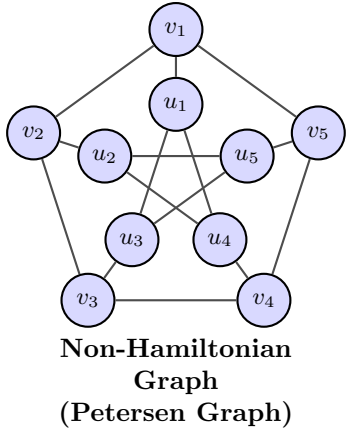


Figure 45: Comparison of a non-Hamiltonian graph (Petersen graph) and a Hamiltonian graph (complete graph K_5). The complete graph contains a Hamiltonian cycle (shown with red arrows) that visits every vertex exactly once before returning to the starting vertex.

Theorem 14. The Petersen graph is not Hamiltonian.

Proof. We present a proof by contradiction. Suppose that the Petersen graph contains a Hamiltonian cycle C .

Let us recall that the Petersen graph has 10 vertices and 15 edges, where each vertex has exactly 3 neighbors. The graph can be constructed as follows:

- Create an outer 5-cycle with vertices labeled $\{v_1, v_2, v_3, v_4, v_5\}$
- Create an inner 5-pointed star with vertices labeled $\{u_1, u_2, u_3, u_4, u_5\}$
- Connect each outer vertex v_i to the corresponding inner vertex u_i

Now, if a Hamiltonian cycle C exists, it must use exactly 10 edges of the graph to visit all 10 vertices exactly once. Since the Petersen graph has 15 edges in total, the cycle C omits exactly 5 edges.

Let us classify the edges of the Petersen graph into three types:

1. Type A: The 5 edges of the outer 5-cycle $\{v_1, v_2, v_3, v_4, v_5\}$
2. Type B: The 5 "spoke" edges connecting v_i to u_i for $i \in \{1, 2, 3, 4, 5\}$
3. Type C: The 5 edges of the inner star connecting u_i to u_{i+2} (with indices taken modulo 5)

For a Hamiltonian cycle to exist, it must use exactly 2 edges incident to each vertex (entering and exiting). Since each vertex has degree 3, exactly one edge at each vertex must be omitted from the cycle. This means that the 5 omitted edges form a perfect matching (or 1-factor) of the Petersen graph.

However, we can verify that no perfect matching can consist entirely of edges of a single type:

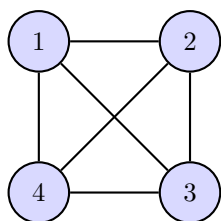
- Type A edges form a 5-cycle, not a matching
- Type B edges form a perfect matching, but if all Type B edges were omitted, the outer and inner vertices would be disconnected, making a Hamiltonian cycle impossible
- Type C edges form a 5-cycle among inner vertices, not a matching

Therefore, the 5 omitted edges must include edges of different types. But this creates a problem: if we omit some edges of each type, we can verify (by exhaustive case analysis or by using the symmetry of the graph) that it's impossible to form a Hamiltonian cycle with the remaining edges.

For example, if we omit 2 Type A edges, 2 Type B edges, and 1 Type C edge, then some vertex will be either isolated or have both its cycle edges from the same component (inner or outer), making it impossible to form a cycle that visits all vertices.

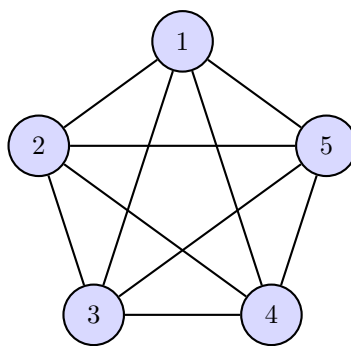
This contradiction proves that the Petersen graph cannot contain a Hamiltonian cycle. $\square$

Remark 8 (Complete Graphs). Recall that a complete graph K_n , introduced in Chapter 1, is a simple graph where every pair of vertices is connected by an edge. This property makes complete graphs particularly important for studying Hamiltonian cycles.



K_4 : Complete graph
on 4 vertices

Each vertex has degree 3 ($n - 1$)
Total of 6 edges $\binom{4}{2}$



K_5 : Complete graph
on 5 vertices

Each vertex has degree 4 ($n - 1$)
Total of 10 edges $\binom{5}{2}$

Figure 46: Examples of complete graphs K_4 and K_5 . In a complete graph, every pair of distinct vertices is connected by exactly one edge. Each vertex has degree $n - 1$, and the total number of edges is $\binom{n}{2} = \frac{n(n-1)}{2}$.

Corollary 2. Every complete graph K_n with $n \geq 3$ is Hamiltonian.

Remark 9 (Applications of Hamiltonian Cycles in Complete Graphs). Complete graphs have numerous practical applications involving Hamiltonian cycles. In particular, the Traveling Salesperson Problem (TSP) can be modeled using complete graphs, where vertices represent cities and edges represent paths between them, with weights indicating distances. Finding a minimum-weight Hamiltonian cycle in this complete graph corresponds to finding the optimal route for the salesperson.

Theorem 15 (NP-Completeness of Hamiltonian Cycle Problem). The problem of determining whether a given graph contains a Hamiltonian cycle is NP-complete. However, for complete graphs, the existence of a Hamiltonian cycle is guaranteed for $n \geq 3$, though finding the minimum-weight Hamiltonian cycle remains NP-hard.

Example 3 (Number of Hamiltonian Cycles). The number of distinct Hamiltonian cycles in a complete graph K_n is $\frac{(n-1)!}{2}$. For example:

- K_3 has exactly 1 Hamiltonian cycle (up to equivalence)
- K_4 has exactly 3 Hamiltonian cycles
- K_5 has exactly 12 Hamiltonian cycles
- K_{10} has approximately 181,440 distinct Hamiltonian cycles

This rapid growth illustrates why brute-force approaches to the TSP quickly become impractical as the number of vertices increases.

3.2 Dirac's Theorem

Theorem 16 (Dirac, 1952). Let G be a simple graph with $n \geq 3$ vertices. If every vertex has degree at least $\frac{n}{2}$ (equivalently, if the minimum degree $\delta(G) \geq \frac{n}{2}$), then G contains a Hamiltonian cycle (i.e., G is Hamiltonian).

Proof. Suppose the theorem is false. So there exists an n -vertex graph G with minimum degree at least $\frac{n}{2}$ and no Hamiltonian cycle. Let G be a maximal counterexample to the theorem, i.e., with as many edges as possible. Note: K_n is Hamiltonian, so $G \neq K_n$. Thus, if $\{u, v\} \notin E(G)$, then $G + \{u, v\}$ is Hamiltonian. So $G + \{u, v\}$ has a Hamiltonian path from u to v . For a vertex $w \in V \setminus \{u, v\}$, let w^+ be the vertex after w on the path from u to v . Let $S = \{w^+ : w \in N_G(u)\}$. We want $N_G(u) \cap N_G(v) \neq \emptyset$. Note that $N_G(u) \cup N_G(v) \subseteq V \setminus \{u, v\}$, so $|N_G(u) \cup N_G(v)| \leq n - 2$. Since $|N_G(u)| \geq \frac{n}{2}$ and $|N_G(v)| \geq \frac{n}{2}$, we have:

$$\begin{aligned} |N_G(u) \cap N_G(v)| &= |N_G(u)| + |N_G(v)| - |N_G(u) \cup N_G(v)| \\ &\geq \frac{n}{2} + \frac{n}{2} - (n - 2) \\ &= 2 \end{aligned}$$

Therefore, there exists $w \in N_G(u) \cap N_G(v)$. Now let C be the cycle consisting of the edges $\{u, w\}$, $\{w, v\}$, and the path from v to u in the Hamiltonian path of $G + \{u, v\}$. This creates a Hamiltonian cycle in G , a contradiction. $\square$

Remark 10 (Algorithmic Verification). *Dirac's condition is particularly valuable because it can be verified in $O(n)$ time by simply checking the minimum degree of the graph, whereas determining if a general graph has a Hamiltonian cycle is NP-complete.*

Algorithm 7 Verify Dirac's Condition

Require: Graph $G = (V, E)$ with $n \geq 3$ vertices

```

1: min_degree  $\leftarrow n$ 
2: for each  $v \in V$  do
3:   min_degree  $\leftarrow \min(\text{min\_degree}, |N(v)|)$ 
4: end for
5: dirac_satisfied  $\leftarrow \text{min\_degree} \geq \lceil n/2 \rceil$ 
6: return dirac_satisfied
```

Remark 11 (Algorithmic Efficiency). *The algorithm's time complexity is $O(n)$ assuming the graph is represented with adjacency lists and degree information is readily available. This is significantly more efficient than directly testing for Hamiltonicity, which is an NP-complete problem.*

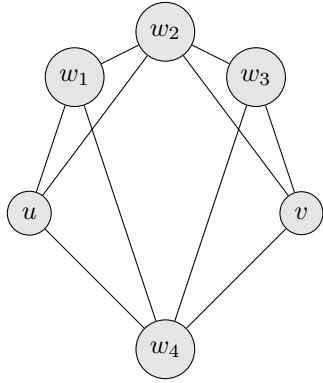
Remark 12 (Sufficiency vs. Necessity). *Dirac's condition is sufficient but not necessary for a graph to be Hamiltonian. Many graphs with minimum degree less than $n/2$ still contain Hamiltonian cycles.*

Theorem 17 (Disjoint Union Property). If G_1 and G_2 are two graphs that satisfy Dirac's condition individually, their disjoint union $G_1 \cup G_2$ does not necessarily satisfy Dirac's condition, highlighting that Hamiltonicity is not preserved under disjoint union.

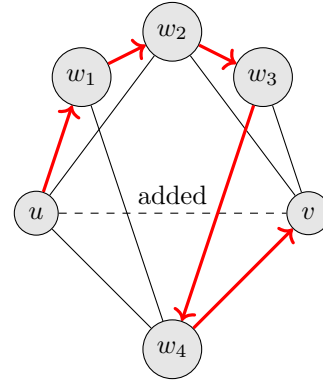
Remark 13 (Implementation Note). *When implementing this algorithm, special care should be taken with disconnected graphs, as no disconnected graph can have a Hamiltonian cycle regardless of minimum degree.*

Lemma 6 (Approximating Hamiltonicity). When $\delta(G) \geq \frac{n}{2} - \epsilon$ for small ϵ , even though Dirac's condition isn't satisfied, the graph may still contain a path or cycle covering most vertices.

3.3 Visual Illustration of Dirac's Theorem Proof

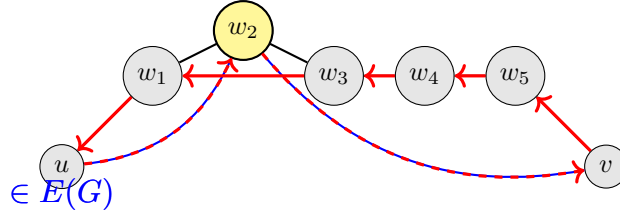


G (with $\{u, v\} \notin E(G)$)



$G + \{u, v\}$ with Hamiltonian path

Figure 47: Illustration of the graph closure concept for Hamiltonicity. Left: Original graph G where vertices u and v are not adjacent but have combined degree $\deg(u) + \deg(v) \geq n$. Right: After adding edge $\{u, v\}$ (shown as dashed), a Hamiltonian path (shown in red) exists from u to v . The Bondy-Chvátal theorem proves that if such vertices have sufficient combined degree, adding the edge between them preserves the Hamiltonian property.



Finding a Hamiltonian cycle in G using a common neighbor w_2

Figure 48: Illustration of the "common neighbor" technique in the proof of Dirac's Theorem. When vertices u and v both have degree at least $n/2$, they must share at least one common neighbor (here w_2 , highlighted in yellow). This allows us to construct a Hamiltonian cycle (red arrows) by rerouting the original path. The dashed red edges show how we can bypass the original path segment through w_2 to create a cycle that visits all vertices exactly once.

Theorem 18 (Closure Properties). Let G be a graph with n vertices and let $c(G)$ be its closure. Then:

1. G is Hamiltonian if and only if $c(G)$ is Hamiltonian.
2. If G satisfies Dirac's condition (minimum degree $\delta(G) \geq n/2$), then $c(G)$ is a complete graph.
3. The closure operation is uniquely defined, independent of the order in which edges are added.

Lemma 7 (Las Vergnas, 1972). Let G be a simple graph and let u, v be non-adjacent vertices in G . If there exists a Hamiltonian path from u to v in G , then $G + \{u, v\}$ has a Hamiltonian cycle.

Corollary 3. If G has a Hamiltonian path and its endpoints have degree sum at least n , then G has a Hamiltonian cycle.

Lemma 8 (Degree Sum of Common Neighbors). Let $P = (v_1, v_2, \dots, v_n)$ be a Hamiltonian path in a graph G . If $N(v_1) \cap N(v_n) \neq \emptyset$, then G has a Hamiltonian cycle.

3.4 Alternative Form of Dirac's Theorem

Theorem 19 (Dirac, 1952). Let G be a simple graph with $n \geq 3$ vertices. If every vertex has degree at least $\frac{n}{2}$ (equivalently, if the minimum degree $\delta(G) \geq \frac{n}{2}$), then G contains a Hamiltonian cycle (i.e., G is Hamiltonian).

Lemma 9. The degree condition in Dirac's theorem is tight, meaning that there exist graphs with minimum degree $\delta(G) = \frac{n}{2} - 1$ that are not Hamiltonian.

Proof Sketch. If G is not Hamiltonian, consider a longest path $P = (v_1, \dots, v_k)$. Since $\deg(v_1) \geq \frac{n}{2}$ and $\deg(v_k) \geq \frac{n}{2}$, by the pigeonhole principle, there must exist vertices v_i, v_{i+1} on P such that $v_1 v_{i+1}, v_k v_i \in E(G)$. This creates cycle $(v_1, \dots, v_i, v_k, \dots, v_{i+1}, v_1)$, contradicting P 's maximality. $\square$

Example 4 (Tightness of Dirac's Condition). Consider the complete bipartite graph $K_{m,m+1}$ with $m \geq 2$. This graph has $n = 2m + 1$ vertices, and each vertex has degree either m or $m + 1$. The minimum degree is $\delta(G) = m = \frac{n-1}{2} = \frac{n}{2} - \frac{1}{2}$, which is just below Dirac's threshold. This graph cannot contain a Hamiltonian cycle because any cycle must alternate between the two partite sets, which is impossible when the sets have different sizes.

Remark 14 (Applications). *Dirac's Theorem provides a simple sufficient condition for Hamiltonicity that can be verified in linear time, making it valuable in network design applications where robustness requires the existence of a cycle visiting all nodes.*

Remark 15 (Related Results). *Dirac's Theorem was generalized by Ore (1960), who showed that a graph is Hamiltonian if, for every pair of non-adjacent vertices u and v , we have $\deg(u) + \deg(v) \geq n$. This is a strictly stronger result, as there exist graphs that satisfy Ore's condition but not Dirac's.*

3.5 Examples of Dirac's Theorem

Example 5 (Minimum degree exactly $\frac{n}{2}$). Consider a graph on $n = 6$ vertices where each vertex has degree exactly $\frac{n}{2} = 3$.

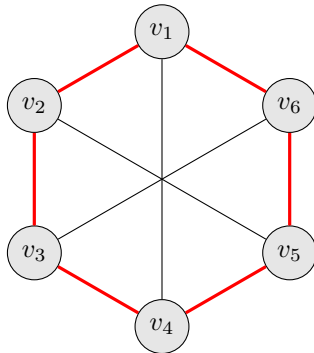


Figure 49: A 3-regular graph on 6 vertices that satisfies Dirac's condition. Each vertex has degree 3, which equals $n/2$. The red cycle highlights one of the Hamiltonian cycles in this graph. This example demonstrates that cubic (3-regular) graphs can be Hamiltonian when they meet the minimum degree threshold.

Lemma 10 (Cubic Graph Hamiltonicity). Not all cubic graphs are Hamiltonian. The Petersen graph is a well-known 3-regular graph with 10 vertices that has no Hamiltonian cycle, demonstrating that regularity alone is insufficient for Hamiltonicity.

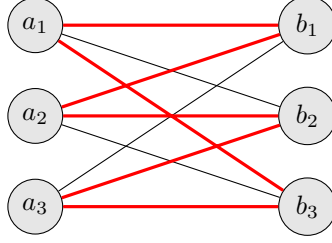


Figure 50: The complete bipartite graph $K_{3,3}$ with $n = 6$ vertices. Each vertex has degree exactly 3, which equals $\frac{n}{2}$, satisfying Dirac's condition. A Hamiltonian cycle is highlighted in red: $a_1 - b_1 - a_2 - b_2 - a_3 - b_3 - a_1$. This example illustrates that bipartite graphs can be Hamiltonian when they have equal-sized partitions.

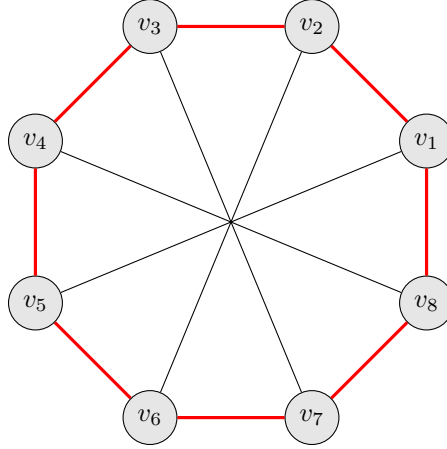


Figure 51: A graph on $n = 8$ vertices where each vertex has degree exactly 4 ($= \frac{n}{2}$), precisely meeting Dirac's minimum degree condition. The graph contains a Hamiltonian cycle (highlighted in red) along the outer cycle. This example illustrates a family of graphs that can be constructed for any even n where each vertex has degree exactly $\frac{n}{2}$ by connecting vertices to their opposites in a regular n -gon.

Definition 22 (Circulant Graph). A circulant graph $\text{Circ}(n; S)$ is a graph with vertex set $\{0, 1, 2, \dots, n-1\}$ and an edge between vertices i and j if and only if $|i - j| \equiv s \pmod{n}$ for some s in the connection set $S \subseteq \{1, 2, \dots, \lfloor n/2 \rfloor\}$. In other words, it's a graph where vertices are arranged in a circle, and each vertex is connected to specific vertices at fixed distances around the circle.

Definition 23 (Graph Toughness). The toughness $t(G)$ of a graph G is defined as:

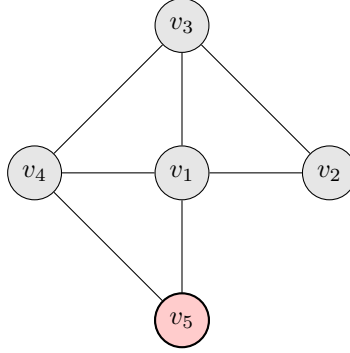
$$t(G) = \min \left\{ \frac{|S|}{c(G-S)} : S \subset V(G), c(G-S) > 1 \right\}$$

where $c(G-S)$ is the number of connected components in the graph after removing vertex set S . Intuitively, toughness measures how difficult it is to disconnect a graph by removing vertices.

Theorem 20 (Circulant Graphs and Hamiltonicity). The graph shown in Figure 51 is an example of a circulant graph $\text{Circ}(8; \{1, 4\})$. A circulant graph with n vertices is Hamiltonian if and only if it is connected.

Problem 1. Show that in the graph of Figure 51, there are exactly 8 distinct Hamiltonian cycles. Hint: Count the cycles that use only the outer edges versus those that must use the "diagonal" connections.

Lemma 11 (Toughness of Regular Graphs). The graph in Figure 51 has toughness exactly 1. In general, any k -regular graph with $k \geq 2$ has toughness at least $\frac{k}{2}$.



$$\deg(v_5) = 1 < \frac{n}{2} = 2.5$$

Figure 52: A graph that fails to satisfy Dirac's condition. This graph has $n = 5$ vertices, with vertex v_5 (highlighted in red) having degree 1, which is less than $\frac{n}{2} = 2.5$. The graph cannot contain a Hamiltonian cycle because any cycle including v_5 must use both its only edge (to v_1) as entry and exit, making it impossible to continue the cycle. This demonstrates why the minimum degree requirement in Dirac's theorem is necessary.

Lemma 12 (Connectivity Requirement). If a graph G has a Hamiltonian cycle, then G is 2-connected.

Proof. Suppose G has a Hamiltonian cycle C . If we remove any vertex v from G , the cycle C becomes a path that connects all remaining vertices. Therefore, $G - v$ is connected, meaning G is 2-connected. $\square$

Theorem 21 (Pendant Vertices). Any graph G containing a pendant vertex cannot contain a Hamiltonian cycle.

Proof. Let v be a pendant vertex in G with its only neighbor u . Any cycle containing v must use the edge $\{u, v\}$ twice: once to enter v and once to exit. Since a cycle can use each edge at most once, this creates a contradiction. $\square$

Lemma 13 (Cut Vertices). The vertex v_1 in Figure 52 is a cut vertex; hence no Hamiltonian cycle can exist in this graph.

Proof. Removing v_1 disconnects the graph into separate components containing $\{v_3, v_4\}$ and $\{v_2, v_5\}$. By Lemma 1, no Hamiltonian cycle can exist in a graph with a cut vertex. $\square$

Theorem 22 (Relaxed Degree Condition). If $\delta(G) \geq \frac{n-1}{2}$, then G is either Hamiltonian or isomorphic to $K_m + \bar{K}_m$ with $n = 2m - 1$.

Remark 16 (Wheel-Graph Relation). The graph in Figure 52 can be viewed as a wheel graph W_5 with one edge removed. This removal causes the graph to fall just below Dirac's threshold, eliminating its Hamiltonian property.

Remark 17 (Dirac-Critical). The graph is Dirac-critical: adding edge $\{v_2, v_5\}$ increases the minimum degree to $\lceil n/2 \rceil$ and makes the graph Hamiltonian.

Problem 2. Determine which pairs of edges can be added to Figure 52 to meet Dirac's condition, and list all such possible pairs.

Lemma 14 (Dirac's Condition and Pendant Vertices). Let G be the graph in Figure 52 with a pendant vertex v_5 of degree 1. Then:

1. G fails Dirac's condition since $\deg(v_5) = 1 < \frac{n}{2} = 2.5$
2. G cannot contain a Hamiltonian cycle

3. Adding the edge $\{v_2, v_5\}$ would increase $\deg(v_5)$ to 2, but this still falls short of Dirac's threshold
4. G requires at least two additional edges to satisfy Dirac's condition

Proof. A Hamiltonian cycle requires each vertex to have degree at least 2, which v_5 fails to satisfy. Furthermore, v_1 is a cut vertex, dividing the graph into two components upon its removal, which prevents any Hamiltonian cycle from existing. $\square$

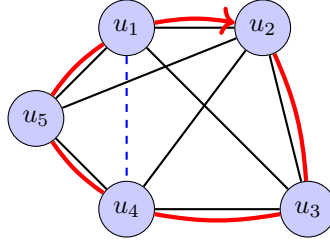
3.6 Related Results and Extensions

3.6.1 Ore's Theorem

Theorem 23 (Ore's Theorem, 1960). Let G be a simple graph with $n \geq 3$ vertices. If for every pair of non-adjacent vertices u and v in G ,

$$\deg(u) + \deg(v) \geq n,$$

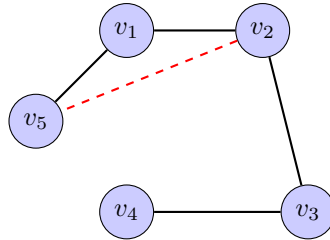
then G is Hamiltonian.



$$\deg(u_1) = 3, \deg(u_2) = 4, \deg(u_3) = 3, \deg(u_4) = 3, \deg(u_5) = 3$$

For non-adjacent vertices u_1 and u_4 : $\deg(u_1) + \deg(u_4) = 3 + 3 = 6 > 5 = n$

Figure 52: A graph with $n = 5$ vertices satisfying Ore's condition. The red path shows a Hamiltonian cycle, and the blue dashed line indicates a non-adjacent vertex pair whose degree sum exceeds n .



$$\deg(v_1) = 2, \deg(v_2) = 2, \deg(v_3) = 2, \deg(v_4) = 1, \deg(v_5) = 1$$

For non-adjacent vertices v_2 and v_5 : $\deg(v_2) + \deg(v_5) = 2 + 1 = 3 < 5 = n$

No Hamiltonian cycle exists in this graph

Figure 53: A graph with $n = 5$ vertices that fails both Dirac's and Ore's conditions. The red dashed line shows a non-adjacent vertex pair whose degree sum is less than n .

Lemma 15 (Ore's Condition and Path Graphs). Let P_n be a path graph with $n \geq 4$ vertices. Then:

1. For any non-adjacent vertices u and v in P_n , $\deg(u) + \deg(v) \leq 4 < n$
2. P_n fails to satisfy Ore's condition
3. The graph in Figure 53 contains a path P_4 as a subgraph (v_2, v_3, v_4, v_5)
4. Adding any single edge to Figure 53 is insufficient to satisfy Ore's condition

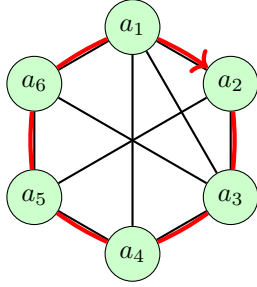
Proof. In any path graph P_n with $n \geq 4$, the maximum degree of any vertex is 2. Consider any pair of non-adjacent vertices u and v . Their degree sum is at most $2 + 2 = 4$, which is less than n . Thus, P_n always fails Ore's condition and cannot be Hamiltonian. The graph in Figure 53 contains the path (v_2, v_3, v_4, v_5) and exhibits similar properties, with vertices v_4 and v_5 having degree only 1, making it even further from satisfying Ore's condition. $\square$

3.6.2 Chvátal's Theorem

Theorem 24 (Chvátal, 1972). Let G be a simple graph with n vertices and degree sequence $d_1 \leq d_2 \leq \dots \leq d_n$. If for all $i < \frac{n}{2}$,

$$d_i \leq i \implies d_{n-i} \geq n - i,$$

then G is Hamiltonian.

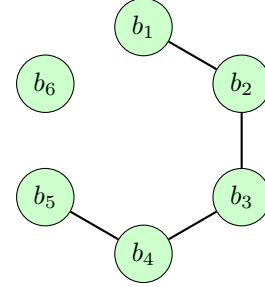


Example: Satisfies Chvátal's condition

$\deg(a_1) = 4, \deg(a_2) = 3, \deg(a_3) = 4,$
 $\deg(a_4) = 3, \deg(a_5) = 3, \deg(a_6) = 3$

Degree sequence: 3, 3, 3, 3, 4, 4

i	$d_i \leq i?$	If yes, $d_{n-i} \geq n - i?$
1	$3 > 1$	Not needed
2	$3 > 2$	Not needed



Counter-example: Fails Chvátal's condition

$\deg(b_1) = 1, \deg(b_2) = 2, \deg(b_3) = 2,$
 $\deg(b_4) = 2, \deg(b_5) = 1, \deg(b_6) = 0$

Degree sequence: 0, 1, 1, 2, 2, 2

i	$d_i \leq i?$	If yes, $d_{n-i} \geq n - i?$
1	$0 < 1$	$d_5 = 2 < 5$ (Fails)
2	$1 < 2$	$d_4 = 2 < 4$ (Fails)

Figure 54: Two examples illustrating Chvátal's condition: the left graph satisfies the condition and contains a Hamiltonian cycle (shown in red), while the right graph fails the condition and has no Hamiltonian cycle.

Lemma 16 (Bondy-Chvátal Closure). Let G be a graph with n vertices. The *Chvátal closure* of G , denoted $cl(G)$, is obtained by repeatedly adding edges between non-adjacent vertices u and v whenever $\deg(u) + \deg(v) \geq n$. This process is continued until no more edges can be added.

Corollary 4 (Bondy-Chvátal Theorem). A graph G with n vertices is Hamiltonian if and only if its closure $cl(G)$ is Hamiltonian.

Proof. The key insight is that adding an edge between vertices u and v where $\deg(u) + \deg(v) \geq n$ preserves the Hamiltonian property. That is, G has a Hamiltonian cycle if and only if $G + uv$ has a Hamiltonian cycle. By repeatedly applying this operation until no more edges can be added, we obtain $cl(G)$, which is Hamiltonian if and only if G is Hamiltonian. $\square$

Corollary 5 (Dirac's Theorem). Let G be a simple graph with $n \geq 3$ vertices. If $\delta(G) \geq \frac{n}{2}$, where $\delta(G)$ denotes the minimum degree of G , then G is Hamiltonian.

Proof. Suppose $\delta(G) \geq \frac{n}{2}$. Then for every vertex $v \in G$, we have $\deg(v) \geq \frac{n}{2}$. Consider the degree sequence $d_1 \leq d_2 \leq \dots \leq d_n$. Since the minimum degree $d_1 \geq \frac{n}{2} > i$ for all $i < \frac{n}{2}$, the condition in Chvátal's theorem is satisfied trivially (the premise is false). Therefore, G is Hamiltonian. $\square$

Lemma 17 (Cycle Extension). Let C be a cycle in a graph G . If there exists a vertex $v \notin C$ that is adjacent to at least two vertices of C , then there exists a cycle C' in G such that $V(C) \cup \{v\} \subseteq V(C')$.

Proof. Let $C = v_1, v_2, \dots, v_k, v_1$ be a cycle in G , and let $v \notin C$ be a vertex adjacent to at least two vertices of C , say v_i and v_j with $i < j$. Then we can form a new cycle $C' = v_1, v_2, \dots, v_i, v, v_j, v_{j+1}, \dots, v_k, v_1$, which includes all vertices of C plus v . $\square$

Corollary 6 (Fan's Theorem). Let G be a 2-connected graph with n vertices. If $\max\{\deg(u), \deg(v)\} \geq \frac{n}{2}$ for every pair of vertices u and v with distance $d(u, v) = 2$, then G is Hamiltonian.

Lemma 18 (Pancyclicity). If G is a graph with n vertices and contains a Hamiltonian cycle, and if $\deg(u) + \deg(v) \geq n + 1$ for every pair of non-adjacent vertices u and v , then G is pancyclic (i.e., contains cycles of all lengths from 3 to n).

Corollary 7 (Hamiltonian-Connected). If G is a graph with n vertices and $\delta(G) \geq \frac{n+1}{2}$, then G is Hamiltonian-connected (i.e., for any two vertices u and v , there exists a Hamiltonian path from u to v).

Remark 18. *These sufficient conditions create a hierarchy:*

$$\text{Dirac} \Rightarrow \text{Ore} \Rightarrow \text{Chvátal}$$

That is, Dirac's condition implies Ore's condition, which in turn implies Chvátal's condition.

Remark 19 (Sharpness). *All of these conditions are sharp, in the sense that they cannot be improved. For each theorem, there exist non-Hamiltonian graphs that almost satisfy the given condition but fail by the smallest possible margin.*

Remark 20 (Computational Complexity). *Despite these sufficient conditions, determining whether an arbitrary graph is Hamiltonian remains NP-complete. These theorems provide easily verifiable sufficient conditions but not necessary conditions for Hamiltonicity.*

Remark 21 (Closure Algorithm). *The Bondy-Chvátal closure can be computed in $O(n^3)$ time, where n is the number of vertices, by repeatedly checking all pairs of non-adjacent vertices and adding edges when the degree sum condition is met.*

Theorem 25 (Necessary Conditions). Let G be a graph. If G contains a Hamiltonian cycle, then the following must be true:

1. G is connected.
2. G is 2-connected (has no cut vertices).
3. G is 1-tough. That is, for any set S of vertices, removing S from G results in at most $|S|$ connected components.
4. If G is bipartite with partite sets of sizes a and b , then $a = b$.
5. No vertex in G has degree less than 2.

Remark 22. *While these conditions are necessary, they are not sufficient. There exist graphs satisfying all these conditions that still do not contain a Hamiltonian cycle.*

Corollary 8. A vertex of degree 1 cannot be part of any Hamiltonian cycle.

Definition 24. The **connectivity** of a graph G , denoted $\kappa(G)$, is the minimum number of vertices whose removal disconnects the graph. If G is a complete graph K_n , then $\kappa(G) = n - 1$.

Definition 25. The **independence number** of a graph G , denoted $\alpha(G)$, is the size of the largest independent set in G , i.e., the maximum number of vertices such that no two are adjacent.

Theorem 26 (Chvátal-Erdős Theorem). Let G be a graph with $n \geq 3$ vertices. If the connectivity $\kappa(G)$ and independence number $\alpha(G)$ satisfy $\kappa(G) \geq \alpha(G)$, then G contains a Hamiltonian cycle.

Sketch. The proof is based on showing that when $\kappa(G) \geq \alpha(G)$, the graph must satisfy Chvátal's condition on its degree sequence. The key insight is that high connectivity relative to the independence number forces a distribution of degrees that ensures Hamiltonicity. $\square$

Corollary 9. If $\kappa(G) > \alpha(G)$, then G is pancyclic (contains cycles of all lengths from 3 to n).

Remark 23. Intuitively, the Chvátal-Erdős theorem states that if a graph is "highly connected" relative to how many vertices can be independent, then it must contain a Hamiltonian cycle. High connectivity prevents the graph from being disconnected too easily, while a low independence number ensures sufficient edge density.

Remark 24. This theorem is particularly useful because it can identify Hamiltonian graphs that might not satisfy other conditions like those of Dirac or Ore. For example, the complete bipartite graph $K_{n,n}$ has minimum degree n but might not satisfy Dirac's condition when $n < \frac{n}{2}$. However, it has $\kappa(K_{n,n}) = n$ and $\alpha(K_{n,n}) = n$, so it satisfies the Chvátal-Erdős condition.

Remark 25. The lack of simple necessary and sufficient conditions for Hamiltonicity is connected to the computational complexity of the problem. The Hamiltonian cycle problem is NP-complete, which suggests that no easily verifiable characterization exists unless $P = NP$.

Theorem 27 (Hierarchy of Sufficient Conditions). The following implications hold for sufficient conditions of Hamiltonicity:

$$\text{Dirac} \Rightarrow \text{Ore} \Rightarrow \text{Chvátal} \Leftarrow \text{Chvátal-Erdős}$$

Remark 26. While there are many sufficient conditions for Hamiltonicity, finding a complete characterization remains an open problem in graph theory.

3.7 Traveling Salesman Problem

Definition 26 (NP-hard). A problem X is NP-hard if every problem $Y \in \mathbf{NP}$ can be reduced to X in polynomial time. Formally, for all $Y \in \mathbf{NP}$, $Y \leq_p X$, where $\leq_p$ denotes polynomial-time reduction.

In graph-theoretic context, a graph problem X is NP-hard if there exists a polynomial-time reduction from every problem in $\mathbf{NP}$ to X . This means that if we could solve X efficiently, we could solve all problems in $\mathbf{NP}$ efficiently.

Theorem 28. Many fundamental graph problems are NP-hard, including:

1. Determining if a graph contains a Hamiltonian cycle
2. Finding the minimum number of colors needed for proper vertex coloring
3. Finding the maximum clique in a graph
4. Computing the minimum vertex cover

Definition 27 (NP-complete). A problem X is NP-complete if:

1. $X \in \mathbf{NP}$, i.e., solutions to X can be verified in polynomial time, and
2. X is NP-hard, i.e., every problem in $\mathbf{NP}$ can be reduced to X in polynomial time.

A graph problem is NP-complete if it is both in $\mathbf{NP}$ and NP-hard. This means that a solution can be verified efficiently, but finding such a solution is believed to require exponential time in the worst case.

Remark 27. The distinction between NP-hard and NP-complete is that NP-complete problems must be in $\mathbf{NP}$, while NP-hard problems may or may not be in $\mathbf{NP}$. For example, the graph isomorphism problem is in $\mathbf{NP}$, while determining if a graph has a chromatic number equal to its clique number is NP-hard but not known to be in $\mathbf{NP}$.

Corollary 10 (Hamiltonian Path vs. Cycle). The Hamiltonian path problem and Hamiltonian cycle problem are polynomially equivalent; either both are in $\mathbf{P}$ or both are NP-complete. In fact, both are NP-complete.

Theorem 29 (Traveling Salesman Problem (TSP)). The **Traveling Salesman Problem (TSP)** asks for a minimum cost Hamiltonian cycle in a weighted graph. This problem is computationally hard; the fastest algorithm on an n -vertex graph runs in exponential time.

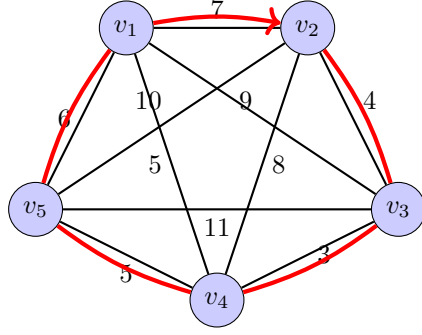


Figure 55: Complete graph with edge weights. The red arrows indicate an optimal TSP tour with cost $7 + 4 + 3 + 5 + 6 = 25$.

$$\text{Minimize } \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (23)$$

$$\text{Subject to } \sum_{j=1}^n x_{ij} = 1 \quad \forall i \in \{1, 2, \dots, n\} \quad (24)$$

$$\sum_{i=1}^n x_{ij} = 1 \quad \forall j \in \{1, 2, \dots, n\} \quad (25)$$

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1 \quad \forall S \subset \{1, 2, \dots, n\}, 2 \leq |S| < n \quad (26)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in \{1, 2, \dots, n\} \quad (27)$$

where $x_{ij} = 1$ if the path includes the edge from vertex i to vertex j , and 0 otherwise.

Corollary 11. The Traveling Salesman Problem is a direct extension of the Hamiltonian cycle problem, which is NP-complete.

Proof. The Hamiltonian cycle problem can be reduced to TSP by setting all edge weights to 1 for existing edges and ∞ for non-existing edges. Finding the minimum weight Hamiltonian cycle would then be equivalent to determining if a Hamiltonian cycle exists. $\square$

3.7.1 Approximation Algorithms

Theorem 30 (2-Approximation using MST). For metric TSP (where edge weights satisfy the triangle inequality), a 2-approximation can be obtained by:

1. Finding a minimum spanning tree T of the graph
2. Performing a preorder traversal of T to get a tour

This tour has cost at most twice the optimal tour.

Corollary 12. The Traveling Salesman Problem is a direct extension of the Hamiltonian cycle problem, which is NP-complete. Thus, TSP is also NP-hard.

Lemma 19 (MST Lower Bound). For any graph G with metric distances, the cost of a minimum spanning tree $MST(G)$ is a lower bound on the cost of an optimal TSP tour $OPT(G)$.

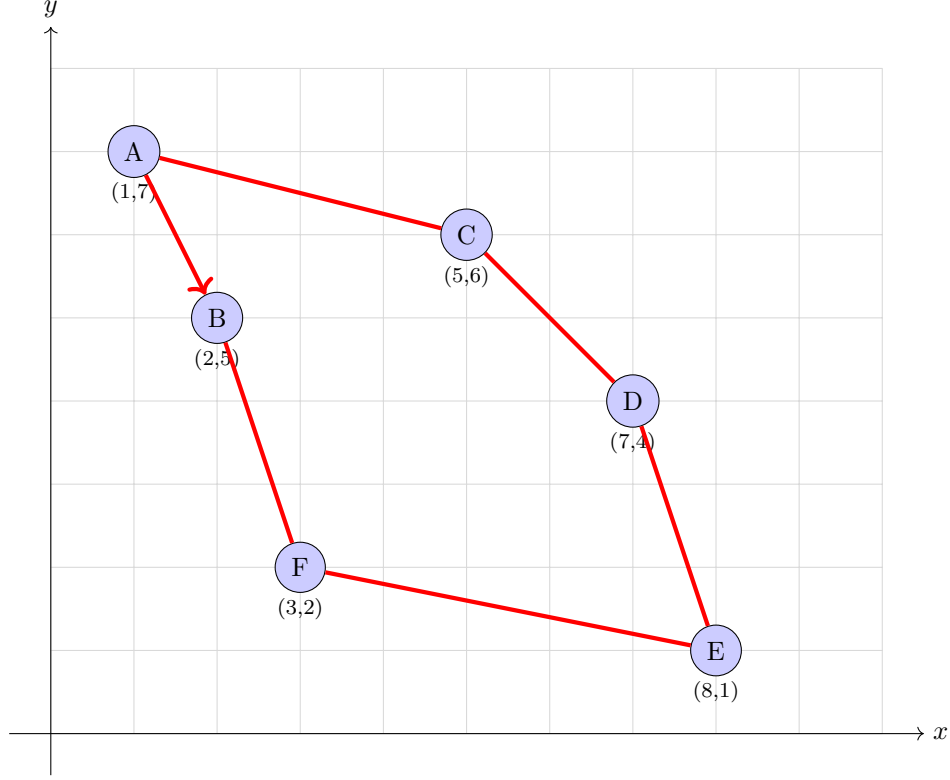


Figure 56: Euclidean TSP instance where cities are points in the plane and distances are Euclidean. The red path shows an optimal tour.

Proof. Let T^* be an optimal TSP tour of G . Removing any single edge from T^* produces a spanning path, which is a spanning tree. Since $MST(G)$ is the minimum-cost spanning tree, we have $c(MST(G)) \leq c(T^* - e) < c(T^*) = c(OPT(G))$. $\square$

Theorem 31 (Christofides' Algorithm). For metric TSP, Christofides' algorithm achieves a $\frac{3}{2}$ -approximation by:

1. Computing an MST of the graph
2. Finding a minimum-weight perfect matching of odd-degree vertices in the MST
3. Combining the MST and matching to form an Eulerian graph
4. Converting the Eulerian tour to a Hamiltonian cycle using shortcuts

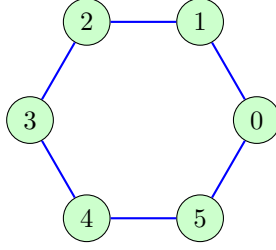
Corollary 13 (Approximation Hierarchy). For metric TSP, the following approximation bounds hold:

$$1 \leq \frac{c(OPT)}{c(MST)} \leq \frac{c(Christofides)}{c(MST)} \leq \frac{3}{2} < 2 \leq \frac{c(MST-preorder)}{c(MST)} \quad (28)$$

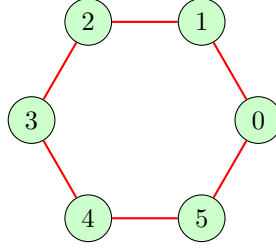
Remark 28. The best-known approximation ratio for metric TSP has remained at $\frac{3}{2}$ (Christofides' algorithm, 1976) for over 45 years, despite significant research efforts.

Remark 29. For the Euclidean TSP (where cities are points in the plane and distances are Euclidean), a polynomial-time approximation scheme (PTAS) exists, allowing approximation to within $(1 + \epsilon)$ of optimal for any $\epsilon > 0$.

Lemma 20 (Triangle Inequality). The triangle inequality is essential for the 2-approximation guarantee. Without it, TSP cannot be approximated within any constant factor unless $P = NP$.



Minimum
Spanning Tree



Approximated
TSP Tour

Relationship

$$\begin{aligned} c(MST) &\leq c(OPT) \\ c(APPROX) &\leq 2 \cdot c(MST) \\ \Rightarrow c(APPROX) &\leq 2 \cdot c(OPT) \end{aligned}$$

Figure 57: 2-approximation algorithm for TSP using Minimum Spanning Tree. **Left:** The MST shown is actually a cycle in this example, connecting all vertices with minimum total edge weight. In a general graph, the MST would be a tree (no cycles) with $n - 1$ edges. **Right:** The approximated TSP tour follows a preorder traversal of the MST, which in this example yields the same path since the MST already forms a cycle. The approximation guarantee works because (1) the MST cost is a lower bound on the optimal TSP tour cost since removing any edge from a tour produces a spanning tree, and (2) doubling the MST and shortcutting (via triangle inequality) produces a tour of cost at most twice optimal.

Remark 30 (Special Case). *The diagram shows a special case where the MST is itself a cycle, making the preorder traversal identical to the MST. In general graphs, the MST will have fewer edges than the tour, requiring the preorder traversal to create a valid Hamiltonian cycle.*

Theorem 32 (Tight Example). The factor-2 approximation of the MST-based algorithm is tight: there exist graphs where the approximation produces a tour whose cost is arbitrarily close to twice the optimal.

Remark 31 (Implementation Insight). *In practice, the MST-based approximation often performs much better than its worst-case factor-2 guarantee, typically producing tours within 15-20% of optimal on random Euclidean instances.*

While the TSP is NP-hard, several approximation algorithms provide practical solutions with guaranteed bounds on the quality of the result. We begin with an exact algorithm before exploring approximation techniques.

Definition 28 (Held-Karp Algorithm). The Held-Karp algorithm is an exact dynamic programming approach to solving the Traveling Salesman Problem. For a graph $G = (V, E)$ with edge weights w , it computes the minimum-cost tour by:

1. Defining $C(S, i)$ as the cost of the shortest path starting at vertex 1, visiting all vertices in S exactly once, and ending at vertex $i \in S$
2. Computing values using the recurrence relation:

$$C(S, i) = \begin{cases} w(1, i) & \text{if } S = \{i\} \\ \min_{j \in S, j \neq i} \{C(S \setminus \{i\}, j) + w(j, i)\} & \text{otherwise} \end{cases} \quad (29)$$

3. Finding the optimal tour cost as $\min_{i \in V \setminus \{1\}} \{C(V \setminus \{1\}, i) + w(i, 1)\}$

The algorithm has time complexity $O(n^2 \cdot 2^n)$ and space complexity $O(n \cdot 2^n)$, making it tractable only for instances with approximately 20-25 cities.

Algorithm 8 Held-Karp Algorithm

Require: Complete graph $G = (V, E)$ with edge weights w

Ensure: Optimal TSP tour and its cost

```
1: function HELDKARP( $G = (V, E), w$ )
2:   Initialize  $C[\{i\}][i] \leftarrow w(1, i)$  for all  $i \in \{2, \dots, n\}$ 
3:   for  $s \leftarrow 2$  to  $n - 1$  do
4:     for all subsets  $S \subset \{2, \dots, n\}$  with  $|S| = s$  do
5:       for each  $i \in S$  do
6:          $C[S][i] \leftarrow \min_{j \in S, j \neq i} \{C[S \setminus \{i\}][j] + w(j, i)\}$ 
7:       end for
8:     end for
9:   end for
10:   $opt\_cost \leftarrow \min_{i=2}^n \{C[\{2, \dots, n\}][i] + w(i, 1)\}$ 
11:   $tour \leftarrow$  Reconstruct tour using parent pointers
12:  return ( $tour, opt\_cost$ )
13: end function
```

Definition 29 (Approximation Algorithms for TSP). For instances where exact solutions are impractical, the following approximation algorithms offer efficient alternatives:

1. **Christofides Algorithm:** Provides a $\frac{3}{2}$ -approximation for metric TSP by augmenting a minimum spanning tree with a minimum-weight perfect matching on odd-degree vertices.
2. **Double-Tree Algorithm:** Achieves a 2-approximation for metric TSP by doubling each edge in a minimum spanning tree to create an Eulerian circuit, then shortcutting to form a tour.
3. **Nearest Neighbor:** A greedy heuristic without general approximation guarantees that iteratively selects the closest unvisited vertex.
4. **k-opt:** A local search method that improves tours by replacing k edges with k different edges, with no general approximation ratio but good practical performance.

3.8 Chapter 3 Exercises

3.8.1 Easy Problems

1. Explain the difference between a Hamiltonian path and a Hamiltonian cycle using a simple example.
2. Prove that every complete graph K_n with $n \geq 3$ is Hamiltonian.
3. Explain why a bipartite graph with unequal partition sizes cannot contain a Hamiltonian cycle.
4. Consider a graph with 8 vertices where each vertex has degree 3. Does Dirac's theorem guarantee that this graph is Hamiltonian? Explain your reasoning.
5. Prove that a graph with a pendant vertex (degree 1) cannot have a Hamiltonian cycle.
6. Design an algorithm to verify if a graph satisfies Dirac's condition in $O(n)$ time.
7. If a graph has minimum degree $k = 4$, what is the minimum length of a cycle guaranteed to exist in the graph?
8. How many distinct Hamiltonian cycles does K_5 contain?

3.8.2 Medium Problems

9. Consider a graph with 9 vertices. For every pair of non-adjacent vertices, the sum of their degrees is at least 8. Apply Ore's theorem to determine if this graph is Hamiltonian.
10. Given a graph with degree sequence $(2, 2, 3, 3, 3, 5)$, determine whether it satisfies Chvátal's condition for Hamiltonicity.
11. Prove that the Petersen graph is not Hamiltonian using the concept of matchings.
12. Given a graph G , explain the process of computing its closure $\text{cl}(G)$ and how this relates to Hamiltonicity.
13. For a graph with 10 vertices, which is stronger: Dirac's condition or Ore's condition? Give an example of a graph that satisfies one but not the other.
14. Prove that if a graph has a Hamiltonian path with endpoints u and v , and $\{u, v\} \in E(G)$, then G has a Hamiltonian cycle.
15. Explain the relationship between a graph's toughness and its Hamiltonian properties.
16. Let G be a graph with a cycle C of length $k < n$. Give sufficient conditions for when G contains a cycle of length $k + 1$ that includes all vertices of C .
17. Prove that the circulant graph $\text{Circ}(n; \{1, 2\})$ is Hamiltonian for all $n \geq 3$.
18. Describe how to construct a Hamiltonian cycle in the complete bipartite graph $K_{3,3}$.

3.8.3 Hard Problems

19. Construct a graph that has minimum degree $\lfloor \frac{n}{2} \rfloor - 1$ and is not Hamiltonian.
20. Prove that if a graph G has connectivity $\kappa(G) \geq \alpha(G)$ (independence number), then G is Hamiltonian.
21. Prove that the factor-2 approximation of the MST-based algorithm for metric TSP is tight by constructing an example.
22. If a graph G on n vertices is Hamiltonian and $\delta(G) \geq \frac{n+1}{2}$, prove that G contains cycles of all lengths from 3 to n .
23. Prove that a graph G is Hamiltonian if and only if its closure $\text{cl}(G)$ is Hamiltonian.

24. Give a complexity analysis for the Held-Karp algorithm for the TSP, explaining why the dynamic programming approach requires $O(n^2 \cdot 2^n)$ time.
25. Prove that an 8×8 chessboard has a closed knight's tour (a Hamiltonian cycle in the knight's graph).
26. Analyze why Christofides' algorithm achieves a $\frac{3}{2}$ -approximation for metric TSP rather than a 2-approximation.

3.8.4 Diagram Problems

27. Draw a non-Hamiltonian graph with 6 vertices where each vertex has degree at least 2.
28. Draw a graph that is Hamiltonian but not Eulerian, and another graph that is Eulerian but not Hamiltonian. Explain why each has one property but not the other.
29.
 - (a) Create a weighted graph with 5 vertices.
 - (b) Find the optimal TSP tour.
 - (c) Apply the MST-based 2-approximation algorithm.
 - (d) Compare the results of the optimal solution and the approximation.
30. Draw a diagram illustrating the key step in Dirac's theorem proof where vertices u and v are non-adjacent but share a common neighbor. Show how this leads to a Hamiltonian cycle.

4 Long Cycles

4.1 Long Cycles and Hamiltonian

The study of long cycles in graph theory extends our understanding beyond the well-established theory of Hamiltonian cycles, offering both theoretical insights and practical applications:

- **Extension of Hamiltonian theory:** While Dirac's Theorem and the Bondy-Chvátal Closure Theorem provide sufficient conditions for Hamiltonicity, many graphs fail to meet these conditions. Long cycle theory addresses the natural follow-up question: what is the best we can do when a Hamiltonian cycle doesn't exist?
- **Connection to the Traveling Salesman Problem:** The TSP seeks optimal Hamiltonian cycles in weighted graphs. However, in contexts where Hamiltonian cycles don't exist, finding the longest possible cycle becomes the natural alternative objective.
- **Structural guarantees:** The theorem establishing that graphs with minimum degree $k \geq 2$ must contain a cycle of length at least $k + 1$ provides a fundamental relationship between local properties (vertex degrees) and global structures (cycle lengths).
- **Optimality and tightness of bounds:** The complete graph K_{k+1} demonstrates that the bound of $k + 1$ is tight—we cannot guarantee longer cycles based solely on minimum degree. This reveals both the power and limitations of the minimum degree parameter.
- **Bridge to extremal graph theory:** The study of longest cycles connects to broader questions in extremal graph theory: what structural properties guarantee the existence of specific subgraphs or patterns?

This exploration creates a spectrum between graphs containing no cycles and those with Hamiltonian cycles, offering a more nuanced understanding of cycle structures in graphs.

Theorem 33 (Minimum Degree and Cycle Length). If G is a graph with minimum degree $k \geq 2$, it contains a cycle of length at least $k + 1$.

Proof. Let P be a longest path in the graph, with endpoints u and v . We first observe that:

$$N_G(u) \subseteq V(P) \quad \text{and} \quad N_G(v) \subseteq V(P) \quad (30)$$

This must be true, as any neighbor outside P would allow us to extend the path, contradicting its maximality. Since G has minimum degree at least k , we know that $|N_G(v)| \geq k$.

Let $N_G(v) = \{v_1, v_2, \dots, v_\ell\}$ where $\ell \geq k$. We can list these neighbors in the order they appear along the path P from u to v :

$$v_k, v_{k-1}, \dots, v_2, v_1 \quad (31)$$

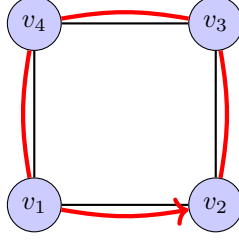
Now, consider the cycle C' formed by:

- The subpath of P from v_k to v (which passes through all vertices $v_{k-1}, \dots, v_1$)
- The edge $\{v_k, v\}$

This cycle C' contains at least the vertices $\{v, v_1, v_2, \dots, v_k\}$, so $|V(C')| \geq k + 1$, and therefore C' has length at least $k + 1$.

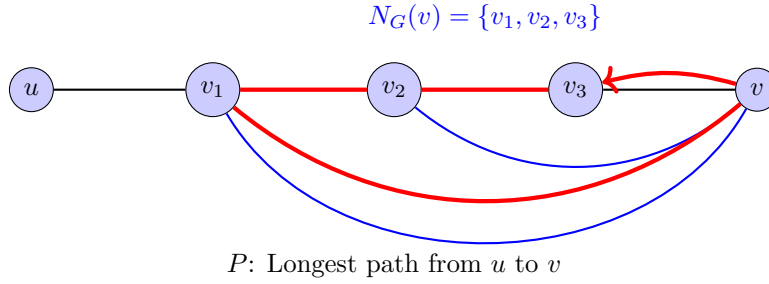
To show this bound is tight, consider the complete graph K_{k+1} . It has minimum degree k and its longest cycle has length exactly $k + 1$, demonstrating that our bound cannot be improved in general. $\square$

This C_4 example shows that while the theorem guarantees a cycle of length at least $k + 1$ (which would be 3 in this case), we can sometimes find longer cycles. Here, the minimum degree is $k = 2$, but the cycle has length 4. This illustrates that the theorem provides a lower bound that is tight in general, but specific graph structures may contain even longer cycles.



Edges: $(v_1, v_2) = e_1, (v_2, v_3) = e_2, (v_3, v_4) = e_3, (v_4, v_1) = e_4$
 Path: $v_1 \xrightarrow{e_1} v_2 \xrightarrow{e_2} v_3 \xrightarrow{e_3} v_4 \xrightarrow{e_4} v_1$

Figure 58: A cycle C_4 where each vertex has degree $k = 2$. The cycle length is $4 = k + 2$, demonstrating that cycles longer than $k + 1$ are possible in specific graphs.



Cycle C' : $v \rightarrow v_3 \rightarrow v_2 \rightarrow v_1 \rightarrow v$ of length $4 = k + 1$ for $k = 3$

Figure 59: Applying the long cycle theorem to a graph with minimum degree 3.

Example 6 (Applying the Proof to a Concrete Graph). In this example, we illustrate the proof technique on a concrete graph. The vertex v has three neighbors on the longest path P : v_1, v_2 , and v_3 . Following the construction in our proof, we form cycle C' by taking the path from v_1 to v along P , and then adding the edge $\{v_1, v\}$.

The resulting cycle C' contains vertices $\{v, v_3, v_2, v_1\}$ and thus has length 4, which is exactly $k + 1$ where $k = 3$ is the minimum degree of the graph. This demonstrates the theorem's guarantee in action, showing that the minimum degree directly determines the length of the longest guaranteed cycle.

Theorem 34 (Long Cycle Theorem). If G is a graph with minimum degree at least $k \geq 2$, then G contains a cycle of length at least $k + 1$.

Lemma 21 (Path Extension). Let P be a path in a graph G . If a vertex v has a neighbor $w \notin V(P)$, then there exists a path P' that is longer than P .

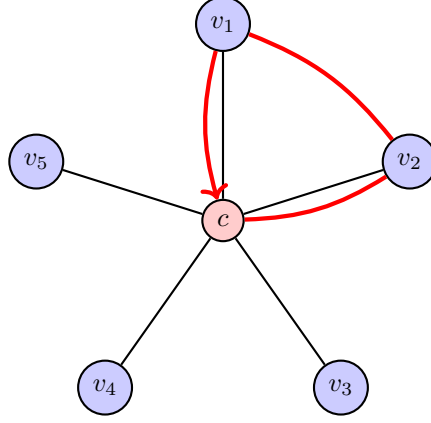
Proof. We can construct P' by adding the edge $\{v, w\}$ to P . Since $w \notin V(P)$, this creates a strictly longer path. $\square$

Lemma 22 (Endpoint Neighborhood Containment). If P is a longest path in a graph G with endpoints u and v , then $N_G(u) \subseteq V(P)$ and $N_G(v) \subseteq V(P)$.

Corollary 14 (Degree-Length Relationship). If G has minimum degree k and P is a longest path with endpoint v , then P contains at least k vertices adjacent to v .

Remark 32. The Long Cycle Theorem is tight in the sense that for any $k \geq 2$, there exists a graph with minimum degree k whose longest cycle has length exactly $k + 1$. The complete bipartite graph $K_{1,k+1}$ (star graph) demonstrates this, as shown in Figure 59.

Remark 33. While the theorem guarantees a cycle of length at least $k + 1$, specific graph structures may contain longer cycles. For instance, our example with minimum degree $k = 3$ contains a cycle of length 4, which equals $k + 1$. However, other graphs with minimum degree 3 might contain cycles of length 5 or more.



$K_{1,5}$ (Star Graph)

Degree Information: $d(c) = 5$, $d(v_i) = 1$ for all i , Minimum degree $k = 1$

Cycle: $v_1 \rightarrow c \rightarrow v_2 \rightarrow v_1$ has length $3 = k + 2$

Figure 60: A star graph $K_{1,5}$ demonstrating the tightness of the Long Cycle Theorem. The red cycle $C = (v_1, c, v_2, v_1)$ has length 3, which equals $k + 2$ where $k = 1$ is the minimum degree. This shows the theorem's bound is tight, as no cycle longer than $k + 2$ exists in this graph.

Example (Star Graph - Tight Bound Example).

This example demonstrates that the Long Cycle Theorem's bound cannot be improved in general. In the star graph $K_{1,5}$:

- The center vertex c has degree 5
- Each leaf vertex v_i has degree 1, making the minimum degree $k = 1$
- The longest possible cycle has length 3, consisting of the center and any two leaves
- This cycle length equals $k + 2 = 3$, showing we cannot improve the theorem to guarantee cycles longer than $k + 1$

While some graphs with minimum degree k may contain cycles of length greater than $k + 1$, this example proves that $k + 1$ is the best possible universal guarantee for all graphs with minimum degree k .

Remark 34 (Star Graphs and the Cycle Bound). *The star graph $K_{1,n}$ provides a valuable extremal case for understanding cycle structure. For any star graph, the minimum degree is always $k = 1$ (at the leaves), and the maximum possible cycle length is always 3, regardless of how many leaves are added. This demonstrates that increasing the number of vertices while maintaining the same minimum degree does not necessarily lead to longer cycles.*

Corollary 15 (Improved Bound for 2-Connected Graphs). If G is a 2-connected graph with minimum degree $k \geq 2$, then G contains a cycle of length at least $\min\{2k, |V(G)|\}$.

Sketch. In a 2-connected graph, for any vertex v , there exists a cycle containing v . Consider a longest cycle C and assume for contradiction that $|C| < \min\{2k, |V(G)|\}$. Then either $|C| < 2k$ or $|C| < |V(G)|$. In the latter case, there exists a vertex $w \notin V(C)$. By 2-connectivity, there exist two vertex-disjoint paths from w to C , which would allow us to create a longer cycle, contradicting the maximality of C . For the former case, a more detailed analysis of neighbor distribution along the cycle leads to a similar contradiction. $\square$

Theorem 35 (Extremal Graphs for the Long Cycle Theorem). For any $k \geq 1$, the star graph $K_{1,k}$ is an extremal graph for the Long Cycle Theorem in the following sense: it has minimum degree k and maximum cycle length $k + 1$, and removing any edge would disconnect the graph and eliminate all cycles.

4.2 Algorithm for Finding Long Cycles

A simple algorithm to find a cycle of length at least $k + 1$ in a graph with minimum degree $k \geq 2$:

Algorithm 9 FindLongCycle: Finding a cycle of length at least $k + 1$

Require: Graph $G = (V, E)$ with minimum degree $k \geq 2$

Ensure: A cycle C of length at least $k + 1$

```

1: function FINDLONGCYCLE( $G = (V, E)$ )
2:   /* Phase 1: Find longest path */
3:    $P \leftarrow$  longest path  $(v_1, \dots, v_\ell)$  found via DFS from arbitrary start
4:   /* Phase 2: Find neighbors of endpoint */
5:    $N \leftarrow \{v_j \in P \setminus \{v_\ell\} : \{v_j, v_\ell\} \in E\}$  ▷  $|N| \geq k$ 
6:   /* Phase 3: Sort neighbors by position */
7:   Build position map  $pos$  where  $pos[v_i] = i$  for all  $v_i \in P$ 
8:   Sort  $N$  by increasing position value
9:    $v_{earliest} \leftarrow$  first vertex in sorted  $N$ 
10:   $j \leftarrow pos[v_{earliest}]$ 
11:  /* Phase 4: Build and return cycle */
12:   $C \leftarrow (v_j, v_{j+1}, \dots, v_\ell, v_j)$ 
13:  return  $C$  ▷  $|C| = \ell - j + 2 \geq k + 1$ 
14: end function
15: function FINDLONGESTPATH( $G, start$ )
16:   Initialize  $visited \leftarrow \{start\}$ ,  $curr \leftarrow (start)$ ,  $best \leftarrow (start)$ 
17:   Call DFSPTHHELPER( $G, start, visited, curr, best$ )
18:   return  $best$ 
19: end function
20: function DFSPTHHELPER( $G, v, visited, curr, best$ )
21:   for each neighbor  $w$  of  $v$  in  $G$  do
22:     if  $w \notin visited$  then
23:       Add  $w$  to  $visited$  and append to  $curr$ 
24:       DFSPTHHELPER( $G, w, visited, curr, best$ )
25:       Remove  $w$  from  $visited$  and  $curr$  ▷ Backtrack
26:     end if
27:   end for
28:   if  $|curr| > |best|$  then
29:     Update  $best \leftarrow$  copy of  $curr$ 
30:   end if
31: end function

```

Remark 35. DFS and BFS will be covered later in this book. Come back to this section after we cover DFS and BFS to gain a deeper understanding of the path-finding technique employed.

Remark 36. While the algorithm is presented using DFS for finding a longest path, it's important to note that finding a truly longest path in a general graph is NP-hard. However, any maximal path (one that cannot be extended further) provides sufficient properties for the algorithm to work correctly.

Remark 37. The correctness of this algorithm relies on the theorem that in a graph with minimum degree $k \geq 2$, there exists a cycle of length at least $k + 1$. The proof of this theorem follows from the properties of longest paths and the pigeonhole principle.

Remark 38. For practical implementations, approximation algorithms for longest paths may be substituted. Even a greedy path-building approach will satisfy our needs, as long as the resulting path is maximal (cannot be extended).

4.3 Chapter 4 Exercises

4.3.1 Easy Problems

1. State the theorem on minimum cycle length in a graph with minimum degree $k \geq 2$. What is the guaranteed minimum cycle length?
2. In a graph with minimum degree 5, what is the minimum cycle length guaranteed by the long cycle theorem?
3. Prove that a graph with maximum degree 2 cannot have a cycle of length greater than the number of vertices.
4. What is the minimum cycle length in a 3-regular graph with 8 vertices? Is this bound tight? Explain.
5. Explain why the minimum cycle length in a graph must be at least 3.
6. Find the length of the longest cycle in the graph $K_{2,4}$ (complete bipartite graph with partite sets of sizes 2 and 4).
7. Prove that any graph with minimum degree $\delta(G) \geq 2$ must contain at least one cycle.

4.3.2 Medium Problems

8. Let P be a longest path in a graph G with minimum degree $k \geq 2$. Prove that the endpoints of P have all their neighbors on P .
9. Explain why the bound in the long cycle theorem is tight by constructing an example of a graph with minimum degree k where the longest cycle has length exactly $k + 1$.
10. Prove that in a 2-connected graph with minimum degree $k \geq 2$, there exists a cycle of length at least $\min\{2k, n\}$.
11. A graph G has minimum degree $k = 3$. Describe an algorithm to find a cycle of length at least $k + 1 = 4$ in G .
12. Let G be a graph with minimum degree $\delta(G) = 3$. If G has exactly 6 vertices, show that G must contain a cycle of length at least 4.
13. Find the relationship between the length of the longest cycle in a graph G and the length of the longest cycle in its complement $\overline{G}$.
14. Prove that in a Hamiltonian graph, the length of the longest cycle is exactly n . Then show that the converse is not true by providing a counterexample.

4.3.3 Hard Problems

15. Let G be a graph with minimum degree $\delta(G) \geq \frac{n}{r+1} + 1$ where $r \geq 2$. Prove that G contains a cycle of length at least $r + 2$.
16. Prove or disprove: if G is a k -connected graph with minimum degree $\delta(G) \geq k + 1$, then G contains a cycle of length at least $2k + 2$.
17. Let G be a graph with minimum degree $k \geq 2$. Prove that if G has girth g (the length of its shortest cycle), then G has a cycle of length at least $\min\{2k, g + k - 1\}$.
18. Design an algorithm to find the longest cycle in a graph G . Analyze its time complexity and explain why finding the longest cycle is generally an NP-hard problem.
19. Let G be a graph with n vertices and let $\lambda(G)$ denote the length of the longest cycle in G . Prove that if $\delta(G) > \frac{n}{2}$, then $\lambda(G) \geq 2\delta(G)$ unless G is Hamiltonian.
20. Prove that if G is a 3-connected cubic graph (3-regular), then G contains a cycle of length at least $4 \log n$.

4.3.4 Diagram Problems

21. Draw a graph with 8 vertices and minimum degree 3 that contains a cycle of length exactly 4 but no longer cycles. Mark the cycle in your diagram.
22. Illustrate the proof technique of the long cycle theorem by drawing:
 - (a) A longest path P in a graph with endpoints u and v
 - (b) The neighbors of v on the path P
 - (c) The earliest neighbor of v on the path
 - (d) The resulting cycle formed by the theorem

Label all relevant vertices and explain each step.

23. Draw a star graph $K_{1,4}$ and explain why the longest cycle in this graph has length 3. Then add one edge to create a cycle of length 4, and explain why this is now the longest possible cycle.
24. Create a sequence of graphs G_3, G_4, G_5 where each G_k has minimum degree k and contains a cycle of length exactly $k + 1$. Draw each graph and highlight the longest cycle in each.
25. Draw the Petersen graph and identify the longest cycle in it. Explain why this graph does not have a Hamiltonian cycle despite having minimum degree 3.
26. Draw a graph where the bound from the long cycle theorem is not tight, i.e., a graph with minimum degree k where the minimum cycle length is strictly greater than $k + 1$. Explain why this happens in your example.
27. Illustrate the relationship between the minimum degree and cycle length by drawing three different graphs with 6 vertices that have minimum degrees 2, 3, and 4 respectively. For each graph, highlight the longest cycle and discuss how the minimum degree affects the cycle length.

5 Trees and Forests

5.1 Trees, Forests, and Bridges

Definition 30 (Tree). A tree is a connected graph with no cycles.

Definition 31 (Acyclic Graph). A graph G is called *acyclic* if it contains no cycles.

Definition 32 (Forest). A forest is an acyclic graph, i.e., a graph with no cycles. Equivalently, a forest is a disjoint union of trees.

Definition 33 (Root). In a rooted tree, the *root* is a distinguished vertex that serves as the reference point for the hierarchical structure of the tree. From the root, there is a unique path to every other vertex in the tree. The root is the only vertex in a rooted tree that has no parent.

Lemma 23 (Tree Edge Count). Let T be a tree with n vertices. Then T has exactly $n - 1$ edges.

Proof. We proceed by induction on n . For $n = 1$, a tree with one vertex has 0 edges, and $1 - 1 = 0$.

Assume that for some $k \geq 1$, any tree with k vertices has $k - 1$ edges. Consider a tree T with $k + 1$ vertices. Choose any edge $e = \{u, v\}$ in T . Removing e disconnects T into two components, each of which is a tree. Let T_1 have n_1 vertices and T_2 have n_2 vertices, where $n_1 + n_2 = k + 1$. By the induction hypothesis, T_1 has $n_1 - 1$ edges and T_2 has $n_2 - 1$ edges. Therefore, T has $(n_1 - 1) + (n_2 - 1) + 1 = n_1 + n_2 - 1 = (k + 1) - 1 = k$ edges. $\square$

Lemma 24 (Unique Path in Trees). In a tree, there exists exactly one path between any two vertices.

Proof. Let T be a tree and let u and v be two distinct vertices in T . Since T is connected, there exists at least one path between u and v . Suppose for contradiction that there exist two distinct paths P_1 and P_2 from u to v . Then $P_1 \cup P_2$ contains a cycle, which contradicts the definition of a tree. Therefore, there can be only one path between any two vertices in a tree. $\square$

Lemma 25 (Leaf Existence). Every tree with at least two vertices has at least two leaves (vertices of degree 1).

Proof. Let T be a tree with $n \geq 2$ vertices. The sum of all vertex degrees in T is $2(n - 1)$ since T has $n - 1$ edges and each edge contributes 2 to the total degree sum. If every vertex had degree at least 2, then the sum of degrees would be at least $2n$, which is a contradiction. Therefore, T must have at least one vertex of degree 1.

Now consider a longest path P in T . Both endpoints of P must have degree 1 in T . If not, then an endpoint would have a neighbor outside P , and we could extend P to a longer path, contradicting our choice of P . Therefore, T has at least two leaves. $\square$

Lemma 26 (Removal of Leaf). If T is a tree and v is a leaf in T , then $T - v$ is also a tree.

Proof. Let T be a tree and v a leaf in T . Since v has degree 1, it is connected to exactly one other vertex. Removing v and its incident edge cannot create a cycle. Moreover, $T - v$ remains connected since v is not on any path between other vertices in T . Therefore, $T - v$ is a tree. $\square$

Lemma 27 (Equivalent Tree Characterizations). For a graph G with n vertices, the following statements are equivalent:

1. G is a tree.
2. G is connected and has $n - 1$ edges.
3. G is acyclic and has $n - 1$ edges.
4. G is acyclic, and adding any edge creates exactly one cycle.
5. G is connected, and removing any edge disconnects the graph.

Lemma 28 (Forest Component Count). If F is a forest with n vertices, m edges, and c components, then $n - m = c$.

Proof. Let F be a forest with c components. Each component is a tree. If the i -th component has n_i vertices, then it has $n_i - 1$ edges. The total number of vertices is $n = \sum_{i=1}^c n_i$, and the total number of edges is $m = \sum_{i=1}^c (n_i - 1) = \sum_{i=1}^c n_i - c = n - c$. Rearranging, we get $n - m = c$. $\square$

Lemma 29 (Bipartiteness of Trees). Every tree is bipartite.

Proof. Let T be a tree. Choose any vertex r in T as a "root." Partition the vertices of T into two sets: A consists of vertices whose distance from r is even, and B consists of vertices whose distance from r is odd. Since T has a unique path from r to any other vertex, this partition is well-defined. Now, consider any edge $\{u, v\}$ in T . The distances of u and v from r differ by exactly 1, so one is even and one is odd. Hence, one vertex is in A and the other is in B . Therefore, T is bipartite with partite sets A and B . $\square$

Lemma 30 (Spanning Trees). Every connected graph has at least one spanning tree.

Proof. Let G be a connected graph. If G is acyclic, then it is already a tree and thus its own spanning tree. If G contains cycles, we can repeatedly remove edges from cycles until no cycles remain. This process preserves connectivity because removing an edge from a cycle does not disconnect the graph. The resulting subgraph is a tree that spans all vertices of G , i.e., a spanning tree of G . $\square$

Lemma 31 (Forest Edge Bound). A graph with n vertices and fewer than $n - 1$ edges is not connected. A graph with n vertices and more than $n - 1$ edges contains at least one cycle.

Proof. Let G be a graph with n vertices and m edges. If $m < n - 1$, then G cannot be a tree (which would require exactly $n - 1$ edges), and therefore cannot be connected.

If $m > n - 1$, suppose for contradiction that G is acyclic. Then G is a forest with c components, where $c \geq 1$. By the formula $n - m = c$, we have $c = n - m < n - (n - 1) = 1$, which is a contradiction. Therefore, G must contain at least one cycle. $\square$

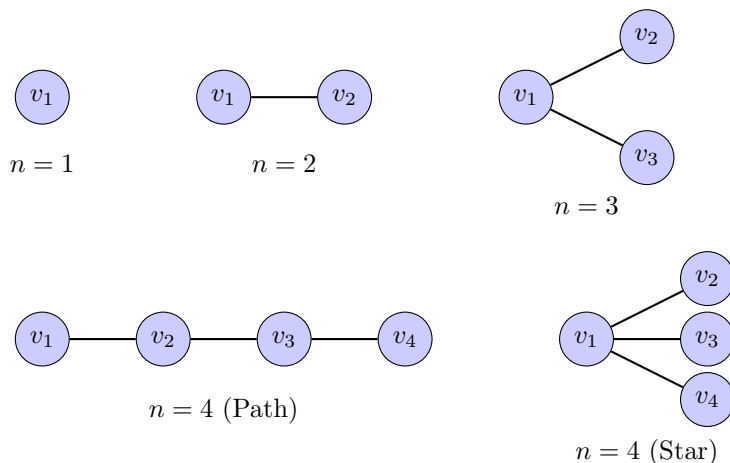


Figure 61: Examples of all possible tree structures with 1 to 4 vertices. Notice that each tree with n vertices has exactly $n - 1$ edges. As the number of vertices increases, the variety of possible tree structures grows rapidly.

Example 7 (Trees with at most five vertices). This figure shows the fundamental tree structures with small numbers of vertices. For $n = 1$, there is only one possible tree: a single isolated vertex. For $n = 2$, there is again only one possible structure: two vertices connected by a single edge. Starting from $n = 3$, we see multiple possible configurations emerging: a path with three vertices or a star with one central vertex connected to two leaves. For $n = 4$, we show two representative structures: a path of length 3 and a star with one central vertex connected to three leaves. Each of these structures satisfies the definition of a tree: they are connected and acyclic. No matter how complex a tree becomes, it will always have exactly $n - 1$ edges, where n is the number of vertices.

Definition 34 (Bridge). A bridge in a graph G is an edge $e \in E(G)$ such that $G - e$ has more components than G . If G is connected, then a bridge is an edge whose removal disconnects the graph.

Corollary 16. If G is a tree, then every edge is a bridge.

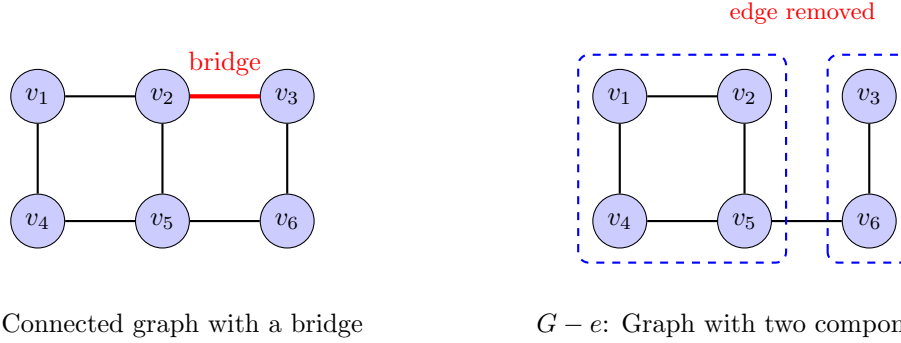


Figure 62: Illustration of a bridge in a graph. The left shows a connected graph G with a bridge highlighted in red between vertices v_2 and v_3 . The right shows the result after removing this bridge: the graph $G - e$ becomes disconnected with two separate components (outlined in blue). In a tree, every edge is a bridge because removing any edge would disconnect the tree.

Remark 39. A bridge is also sometimes called a cut-edge or isthmus. The presence of bridges in a graph indicates potential vulnerabilities in network connectivity. In applications such as communication networks, bridges represent critical connections whose failure would partition the network.

5.2 Characterization of Trees

Lemma 32 (Tree Characterization). For a graph G , the following conditions are equivalent:

1. G is a tree.
2. G is connected and acyclic.
3. G is connected and every edge is a bridge.
4. G is connected and has exactly $|V(G)| - 1$ edges.
5. G is acyclic and has exactly $|V(G)| - 1$ edges.
6. G is acyclic, and adding any edge creates exactly one cycle.

Proof. We'll prove the equivalence by showing $(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (4) \Rightarrow (5) \Rightarrow (1)$ and $(5) \Rightarrow (6) \Rightarrow (2)$.

1. $(1) \Rightarrow (2)$: This follows directly from the definition of a tree.
2. $(2) \Rightarrow (3)$: If G is connected and acyclic, then for any edge $e = \{u, v\}$, removing e would disconnect u and v . If there were still a path from u to v after removing e , then this path together with e would form a cycle, contradicting that G is acyclic.
3. $(3) \Rightarrow (4)$: We proceed by induction on $|V(G)|$.
 - For $|V(G)| = 1$, the statement is trivial.
 - Assume the statement holds for all connected graphs with n vertices where every edge is a bridge.
 - Consider a connected graph G with $n + 1$ vertices where every edge is a bridge.
 - Choose any leaf v (which must exist by the Leaf Existence Lemma) and let e be its incident edge.
 - The graph $G - v$ remains connected, has n vertices, and every edge is still a bridge.

- By the induction hypothesis, $G - v$ has $n - 1$ edges.
 - Therefore, G has $(n - 1) + 1 = n = |V(G)| - 1$ edges.
4. (4) $\Rightarrow$ (5): We need to show that if G is connected with $|V(G)| - 1$ edges, then G is acyclic.
- Suppose G contains a cycle.
 - Removing any edge e from this cycle would preserve connectivity.
 - Thus, $G - e$ would be connected with $|V(G)| - 2$ edges.
 - This contradicts the fact that a connected graph on $|V(G)|$ vertices needs at least $|V(G)| - 1$ edges.
5. (5) $\Rightarrow$ (1): If G is acyclic with $|V(G)| - 1$ edges, then G has exactly the right number of edges to be a tree.
- We need to show that G is connected.
 - If G were not connected, with $c > 1$ components, then each component would be a tree.
 - If component i has n_i vertices, it would have $n_i - 1$ edges.
 - The total number of edges would be $\sum_{i=1}^c (n_i - 1) = |V(G)| - c < |V(G)| - 1$, a contradiction.
6. (5) $\Rightarrow$ (6): If G is acyclic with $|V(G)| - 1$ edges, adding any edge $e = \{u, v\}$ would create at most one cycle.
- This cycle would be formed by e and the unique path from u to v in G .
 - We need to show that G is connected so that this path exists.
 - The proof is the same as for (5) $\Rightarrow$ (1).
7. (6) $\Rightarrow$ (2): If G is acyclic and adding any edge creates exactly one cycle, then for any two vertices u and v not adjacent in G :
- Adding the edge $\{u, v\}$ creates a cycle.
 - This means there must already be a path from u to v in G .
 - Since this holds for any two vertices, G is connected.

□

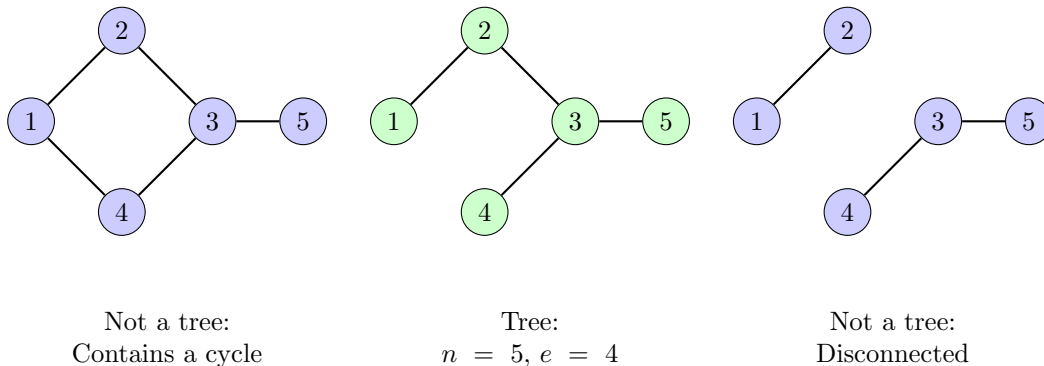


Figure 63: Illustration of tree characterization. Left: A connected graph with a cycle is not a tree. Center: A connected acyclic graph is a tree with $n - 1$ edges. Right: A disconnected acyclic graph is a forest, not a tree. The center graph satisfies all conditions from the lemma: it is connected, acyclic, has $n - 1$ edges, and every edge is a bridge.

Definition 35. (Spanning Tree) A spanning tree of a graph G is a spanning subgraph of G that is a tree.

Lemma 33 (Existence of Spanning Trees). Every connected graph has a spanning tree.

Proof. Let G be a connected graph. If G has no cycles, then it is already a tree and thus its own spanning tree. If G contains cycles, we can repeatedly remove edges from cycles until no cycles remain. This process preserves connectivity because removing an edge from a cycle does not disconnect the graph. The resulting subgraph is a tree that spans all vertices of G , i.e., a spanning tree of G . $\square$

Algorithm 10 Spanning Tree Construction

Require: A connected graph $G = (V, E)$

Ensure: A spanning tree $T = (V, E')$ of G

- 1: Initialize $T \leftarrow G$
 - 2: **while** T contains a cycle C **do**
 - 3: Choose any edge $e \in C$
 - 4: Remove e from T
 - 5: **end while**
 - 6: **return** T
-

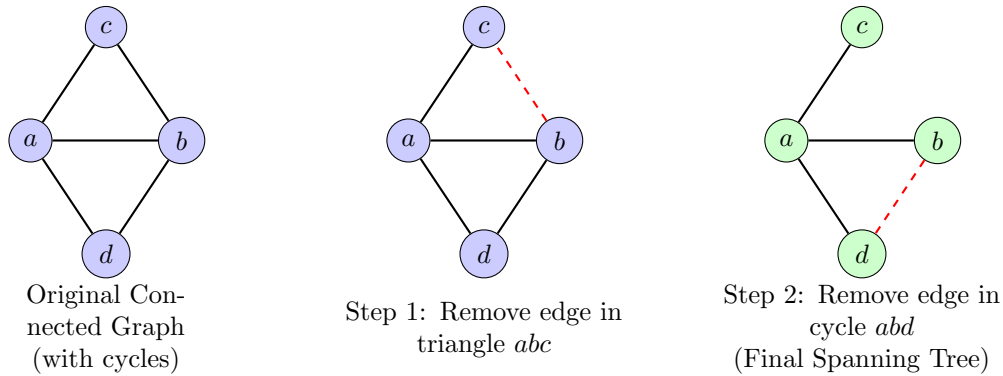


Figure 64: Construction of a spanning tree from a connected graph. Left: Original connected graph with two cycles (abc and abd). Center: After removing edge bc from the first cycle. Right: Final spanning tree after removing edge bd from the second cycle. The result is a connected acyclic subgraph that includes all vertices of the original graph.

Example 8 (Finding a Spanning Tree). To find a spanning tree in the graph shown in Figure 62:

1. We start with the connected graph on the left, which contains cycles.
2. First, we identify the cycle formed by vertices a , b , and c , and remove the edge $\{b, c\}$.
3. Next, we identify the remaining cycle formed by vertices a , b , and d , and remove the edge $\{b, d\}$.
4. After removing these edges, no cycles remain in the graph.
5. The resulting structure is a spanning tree of the original graph, with $n - 1 = 3$ edges for $n = 4$ vertices.

Remark 40. The spanning tree of a graph is not necessarily unique. Different choices of which edges to remove from cycles can lead to different spanning trees of the same graph.

Theorem 36 (Spanning Tree Edge Properties). If T is a spanning tree of graph G , then:

1. For any edge $e \in E(G) \setminus E(T)$, the graph $T + e$ contains exactly one cycle.
2. For any edge $e \in E(T)$, the graph $T - e$ has exactly two components.

5.3 Breadth-First Search and Depth-First Search Algorithms

Definition 36. If $u, v \in V(G)$, then the distance between u and v , denoted $d_G(u, v)$, is the length of a shortest u, v -path, or infinity if u and v are in different components of G .

Definition 37 (Diameter). The diameter of a connected graph G is:

$$\text{diam}(G) = \max\{d_G(u, v) : u, v \in V(G)\}$$

Definition 38 (Radius). The radius of a connected graph G is:

$$\text{rad}(G) = \min_{v \in V(G)} \max_{u \in V(G)} d_G(u, v)$$

Lemma 34 (Relationship Between Diameter and Radius). For any connected graph G :

$$\text{rad}(G) \leq \text{diam}(G) \leq 2 \cdot \text{rad}(G)$$

Remark 41. Some common diameter and radius values for standard graphs:

- Complete Graph K_n : $\text{diam}(K_n) = 1$, $\text{rad}(K_n) = 1$
- Path P_n : $\text{diam}(P_n) = n - 1$, $\text{rad}(P_n) = \lceil \frac{n-1}{2} \rceil$
- Cycle C_n : $\text{diam}(C_n) = \lfloor \frac{n}{2} \rfloor$, $\text{rad}(C_n) = \lceil \frac{n}{2} \rceil$
- Star $K_{1,n-1}$: $\text{diam}(K_{1,n-1}) = 2$, $\text{rad}(K_{1,n-1}) = 1$

Remark 42 (Applications). The diameter and radius have important applications in network analysis:

- Network Efficiency: Lower diameter and radius indicate more efficient communication networks
- Facility Location: The center of a graph (vertices with eccentricity equal to the radius) represents optimal locations for facilities to minimize maximum distance
- Social Networks: Diameter measures the maximum number of intermediaries needed to connect any two people
- Transportation Networks: Radius helps identify central hubs that provide efficient access to all locations

Definition 39 (Distance). The distance between vertices $u, v \in V(G)$, denoted $d_G(u, v)$, is the length of a shortest u, v -path, or infinity if u and v are in different components of G .

Definition 40 (Eccentricity). The eccentricity of a vertex v in a connected graph G , denoted $\text{ecc}(v)$, is the maximum distance from v to any other vertex:

$$\text{ecc}(v) = \max_{u \in V(G)} d_G(u, v)$$

Definition 41. The diameter of a connected graph G is:

$$\text{diam}(G) = \max\{d_G(u, v) : u, v \in V(G)\} = \max_{v \in V(G)} \text{ecc}(v)$$

In other words, the diameter is the maximum eccentricity of any vertex in G .

Definition 42. The radius of a connected graph G is:

$$\text{rad}(G) = \min_{v \in V(G)} \text{ecc}(v)$$

That is, the radius is the minimum eccentricity of any vertex in G .

5.3.1 BFS Algorithm and BFS Tree

Given a graph G and a starting vertex v_1 , we can construct a breadth-first search (BFS) tree as follows:

1. Designate v_1 as the root of the tree.
2. Initialize a queue Q with v_1 and mark v_1 as visited.
3. While Q is not empty:
 - (a) Dequeue a vertex u from Q .
 - (b) For each unvisited neighbor v of u :
 - i. Mark v as visited.
 - ii. Add edge $\{u, v\}$ to the BFS tree T .
 - iii. Enqueue v to Q .

The resulting tree T is a BFS spanning tree of G .

Example. Finding a BFS tree in the graph of Figure 65, starting from vertex a :

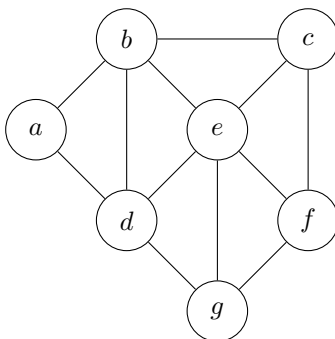


Figure 65: Example diagram for BFS Tree.

1. a is the root, $V(T) = \{a\}$, BFS ordering: (a)
2. Add b : $V(T) = \{a, b\}$, $E(T) = \{\{a, b\}\}$, BFS ordering: (a, b)
3. Add d : $V(T) = \{a, b, d\}$, $E(T) = \{\{a, b\}, \{a, d\}\}$, BFS ordering: (a, b, d)
4. Add c : $V(T) = \{a, b, d, c\}$, $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}\}$
BFS ordering: (a, b, d, c)
5. Add e : $V(T) = \{a, b, d, c, e\}$, $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}, \{b, e\}\}$
BFS ordering: (a, b, d, c, e)
6. Add g : $V(T) = \{a, b, d, c, e, g\}$
 $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}, \{b, e\}, \{d, g\}\}$
BFS ordering: (a, b, d, c, e, g)
7. Add f : $V(T) = \{a, b, d, c, e, g, f\}$
 $E(T) = \{\{a, b\}, \{a, d\}, \{b, c\}, \{b, e\}, \{d, g\}, \{c, f\}\}$
BFS ordering: (a, b, d, c, e, g, f)

Final BFS ordering: (a, b, d, c, e, g, f)

Note that some edges from the original graph, such as $\{b, d\}$, $\{c, e\}$, $\{e, f\}$, $\{e, g\}$, and $\{f, g\}$, are not part of the BFS tree because their endpoint vertices were already discovered through other paths when we encountered these edges.

Example. Finding a BFS tree in the graph of Figure 66, starting from vertex 2:

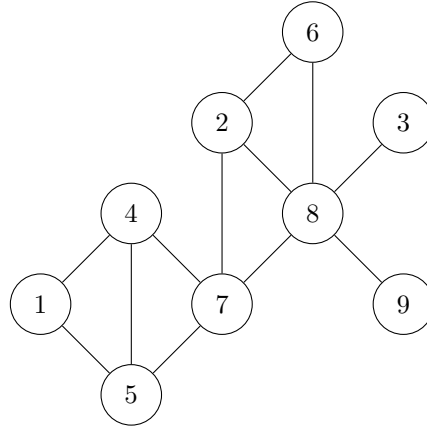


Figure 66: Example diagram for BFS Tree.

1. 2 is the root, $V(T) = \{2\}$, BFS ordering: (2)
2. Add 6: $V(T) = \{2, 6\}$, $E(T) = \{\{2, 6\}\}$
BFS ordering: (2, 6)
3. Add 7: $V(T) = \{2, 6, 7\}$, $E(T) = \{\{2, 6\}, \{2, 7\}\}$
BFS ordering: (2, 6, 7)
4. Add 8: $V(T) = \{2, 6, 7, 8\}$, $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}\}$
BFS ordering: (2, 6, 7, 8)
5. Add 4: $V(T) = \{2, 6, 7, 8, 4\}$, $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}\}$
BFS ordering: (2, 6, 7, 8, 4)
6. Add 3: $V(T) = \{2, 6, 7, 8, 4, 3\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{8, 3\}\}$
BFS ordering: (2, 6, 7, 8, 4, 3)
7. Add 9: $V(T) = \{2, 6, 7, 8, 4, 3, 9\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{8, 3\}, \{8, 9\}\}$
BFS ordering: (2, 6, 7, 8, 4, 3, 9)
8. Add 1: $V(T) = \{2, 6, 7, 8, 4, 3, 9, 1\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{8, 3\}, \{8, 9\}, \{4, 1\}\}$
BFS ordering: (2, 6, 7, 8, 4, 3, 9, 1)
9. Add 5: $V(T) = \{2, 6, 7, 8, 4, 3, 9, 1, 5\}$
 $E(T) = \{\{2, 6\}, \{2, 7\}, \{2, 8\}, \{7, 4\}, \{8, 3\}, \{8, 9\}, \{4, 1\}, \{1, 5\}\}$
BFS ordering: (2, 6, 7, 8, 4, 3, 9, 1, 5)

Final BFS ordering: (2, 6, 7, 8, 4, 3, 9, 1, 5)

Note that edges $\{6, 8\}$, $\{7, 8\}$, $\{4, 5\}$, $\{4, 7\}$, and $\{5, 7\}$ are not part of the BFS tree because their endpoints were already discovered when these edges were encountered. The BFS tree provides a minimum-distance path from the root (vertex 2) to every other vertex in the graph.

Lemma 35. If T is a BFS tree in a connected graph rooted at v , then:

$$d_T(v, w) = d_G(v, w)$$

i.e., BFS tree preserves distances from the root.

Definition 43 (BFS Layers). Let G be a graph and T be a BFS tree of G rooted at vertex v . The **layers** of T are defined as:

- Layer $L_0(v) = \{v\}$ (the root)
- Layer $L_i(v) = \{u \in V(G) : d_G(v, u) = i\}$ for $i \geq 1$

That is, the i^{th} layer consists of all vertices at distance exactly i from the root v .
The following properties hold:

1. All edges of G connect vertices either within the same layer or in adjacent layers.
2. The **height** of the BFS tree is the index of the largest non-empty layer.
3. The **eccentricity** of vertex v is equal to the height of the BFS tree rooted at v .
4. The **radius** of G is the minimum eccentricity among all vertices in G .

Graph with vertices $\{a, b, c, d, e, f, g\}$:

Let's calculate the eccentricity of each vertex by finding the height of the BFS tree rooted at that vertex:

Vertex	BFS Layers	Height	Eccentricity
a	$L_0 = \{a\}, L_1 = \{b, d\}, L_2 = \{c, e, g\}, L_3 = \{f\}$	3	3
b	$L_0 = \{b\}, L_1 = \{a, c, d, e\}, L_2 = \{g, f\}$	2	2
c	$L_0 = \{c\}, L_1 = \{b, e, f\}, L_2 = \{a, d, g\}$	2	2
d	$L_0 = \{d\}, L_1 = \{a, b, e, g\}, L_2 = \{c, f\}$	2	2
e	$L_0 = \{e\}, L_1 = \{b, c, d, f, g\}, L_2 = \{a\}$	2	2
f	$L_0 = \{f\}, L_1 = \{c, e, g\}, L_2 = \{b, d\}, L_3 = \{a\}$	3	3
g	$L_0 = \{g\}, L_1 = \{d, e, f\}, L_2 = \{a, b, c\}$	2	2

Properties of Figure 65:

- **Radius:** $\text{rad}(G) = \min\{\text{ecc}(v) : v \in V(G)\} = 2$
- **Diameter:** $\text{diam}(G) = \max\{\text{ecc}(v) : v \in V(G)\} = 3$
- **Central Vertices:** $\{b, c, d, e, g\}$ (vertices with eccentricity equal to radius)
- **Peripheral Vertices:** $\{a, f\}$ (vertices with eccentricity equal to diameter)

Graph with vertices $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$:

Let's examine key vertices to determine the graph's properties:

Vertex	BFS Layers	Height	Eccentricity
2	$L_0 = \{2\}, L_1 = \{6, 7, 8\}, L_2 = \{3, 4, 9\}, L_3 = \{1, 5\}$	3	3
7	$L_0 = \{7\}, L_1 = \{2, 4, 5, 8\}, L_2 = \{1, 3, 6, 9\}$	2	2
8	$L_0 = \{8\}, L_1 = \{2, 3, 6, 7, 9\}, L_2 = \{4\}, L_3 = \{1, 5\}$	3	3

A full analysis would include all vertices, but we've computed enough to determine the key properties.

Properties of Figure 66:

- **Radius:** $\text{rad}(G) = 2$ (achieved by vertex 7)
- **Diameter:** $\text{diam}(G) = 3$ (distance from vertex 1 to vertex 5)
- **A Central Vertex:** Vertex 7 has minimum eccentricity
- **BFS Tree Height from vertex 2:** 3

Remark 43. Both graphs share interesting structural properties:

- Both have radius 2 and diameter 3
- Both satisfy the theorem that $\text{rad}(G) \leq \text{diam}(G) \leq 2 \cdot \text{rad}(G)$ with equality on the right
- Figure 65 has multiple central vertices (5 out of 7 vertices), while Figure 66 has fewer central vertices relative to its size
- The BFS trees of both graphs efficiently reveal their distance properties

Algorithm 11 Breadth-First Search (BFS)

Require: Graph $G = (V, E)$, starting vertex s

Ensure: Vertices discovered in BFS order and a BFS tree T

```

1: procedure BFS( $G, s$ )
2:   Mark all vertices as unvisited
3:   Create an empty queue  $Q$ 
4:   Create an empty tree  $T$  with  $s$  as its root
5:   Mark  $s$  as visited
6:   Enqueue  $s$  into  $Q$ 
7:   while  $Q$  is not empty do
8:     Dequeue a vertex  $u$  from  $Q$ 
9:     for each neighbor  $v$  of  $u$  do
10:      if  $v$  is unvisited then
11:        Mark  $v$  as visited
12:        Add edge  $(u, v)$  to tree  $T$ 
13:        Enqueue  $v$  into  $Q$ 
14:      end if
15:    end for
16:  end while
17:  return discovered vertices and tree  $T$ 
18: end procedure

```

5.3.2 DFS Algorithm and DFS Tree

Given a graph G and a starting vertex v_1 , we can construct a depth-first search (DFS) tree as follows:

1. Designate v_1 as the root of the tree.
2. Call the recursive procedure DFS-VISIT(v_1):
 - (a) Mark vertex v_1 as visited.
 - (b) For each unvisited neighbor w of v_1 :
 - i. Add edge $\{v_1, w\}$ to the DFS tree T .
 - ii. Recursively call DFS-VISIT(w).

The resulting tree T is a DFS spanning tree of G .

DFS Tree Construction for Figure 65, starting from vertex a :

1. a is the root, $V(T) = \{a\}$, DFS ordering: (a)
2. Add b : $V(T) = \{a, b\}$, $E(T) = \{\{a, b\}\}$, DFS ordering: (a, b)
3. Add c : $V(T) = \{a, b, c\}$, $E(T) = \{\{a, b\}, \{b, c\}\}$
DFS ordering: (a, b, c)

4. Add e : $V(T) = \{a, b, c, e\}$, $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}\}$
DFS ordering: (a, b, c, e)
5. Add f : $V(T) = \{a, b, c, e, f\}$
 $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}, \{e, f\}\}$
DFS ordering: (a, b, c, e, f)
6. Add g : $V(T) = \{a, b, c, e, f, g\}$
 $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}, \{e, f\}, \{f, g\}\}$
DFS ordering: (a, b, c, e, f, g)
7. Add d : $V(T) = \{a, b, c, e, f, g, d\}$
 $E(T) = \{\{a, b\}, \{b, c\}, \{c, e\}, \{e, f\}, \{f, g\}, \{a, d\}\}$
DFS ordering: (a, b, c, e, f, g, d)

Final DFS ordering: (a, b, c, e, f, g, d)

Note that some edges from the original graph, such as $\{b, d\}$, $\{b, e\}$, $\{c, f\}$, $\{d, e\}$, $\{d, g\}$, and $\{e, g\}$, are not part of the DFS tree because their endpoint vertices were already discovered through other paths when we encountered these edges.

DFS Tree Construction for Figure 66, starting from vertex 2:

1. 2 is the root, $V(T) = \{2\}$, DFS ordering: (2)
2. Add 6: $V(T) = \{2, 6\}$, $E(T) = \{\{2, 6\}\}$, DFS ordering: $(2, 6)$
3. Add 8: $V(T) = \{2, 6, 8\}$, $E(T) = \{\{2, 6\}, \{6, 8\}\}$
DFS ordering: $(2, 6, 8)$
4. Add 3: $V(T) = \{2, 6, 8, 3\}$, $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}\}$
DFS ordering: $(2, 6, 8, 3)$
5. Add 9: $V(T) = \{2, 6, 8, 3, 9\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 9\}\}$
DFS ordering: $(2, 6, 8, 3, 9)$
6. Add 7: $V(T) = \{2, 6, 8, 3, 9, 7\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 9\}, \{2, 7\}\}$
DFS ordering: $(2, 6, 8, 3, 9, 7)$
7. Add 4: $V(T) = \{2, 6, 8, 3, 9, 7, 4\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 9\}, \{2, 7\}, \{7, 4\}\}$
DFS ordering: $(2, 6, 8, 3, 9, 7, 4)$
8. Add 1: $V(T) = \{2, 6, 8, 3, 9, 7, 4, 1\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 9\}, \{2, 7\}, \{7, 4\}, \{4, 1\}\}$
DFS ordering: $(2, 6, 8, 3, 9, 7, 4, 1)$
9. Add 5: $V(T) = \{2, 6, 8, 3, 9, 7, 4, 1, 5\}$
 $E(T) = \{\{2, 6\}, \{6, 8\}, \{8, 3\}, \{8, 9\}, \{2, 7\}, \{7, 4\}, \{4, 1\}, \{1, 5\}\}$
DFS ordering: $(2, 6, 8, 3, 9, 7, 4, 1, 5)$

Final DFS ordering: $(2, 6, 8, 3, 9, 7, 4, 1, 5)$

Note that some edges from the original graph, such as $\{2, 8\}$, $\{4, 7\}$, $\{5, 7\}$, $\{7, 8\}$, and $\{4, 5\}$, are not part of the DFS tree because their endpoint vertices were already discovered through other paths when we encountered these edges.

Remark 44 (DFS Properties Comparison). *Both graphs exhibit interesting structural properties when analyzed through DFS traversal:*

- DFS trees for both graphs have a maximum depth of 6 for Figure 65 and 8 for Figure 66

- The DFS trees reveal the connectivity patterns differently than BFS, highlighting branch-like structures
- In Figure 65, DFS from vertex a explores the a - b - c - e - f - g branch completely before backtracking to explore d
- In Figure 66, DFS from vertex 2 explores the 2-6-8-3-9 branch first before exploring the 2-7-4-1-5 branch
- Both DFS trees contain exactly $n - 1$ edges where n is the number of vertices
- The back edges (edges not in the DFS tree) reveal the cycle structure in both graphs
- While BFS trees minimize the distance from the root, DFS trees tend to create longer paths from the root to certain vertices

Algorithm 12 Depth-First Search (DFS) - Recursive

Require: Graph $G = (V, E)$, starting vertex s

Ensure: Vertices discovered in DFS order and a DFS tree T

```

1: procedure DFS( $G, s$ )
2:   Mark all vertices as unvisited
3:   Create an empty tree  $T$  with  $s$  as its root
4:   DFS-VISIT( $G, s, T$ )
5:   return discovered vertices and tree  $T$ 
6: end procedure
7: procedure DFS-VISIT( $G, u, T$ )
8:   Mark  $u$  as visited
9:   for each neighbor  $v$  of  $u$  do
10:    if  $v$  is unvisited then
11:      Add edge  $(u, v)$  to tree  $T$ 
12:      DFS-VISIT( $G, v, T$ )
13:    end if
14:  end for
15: end procedure

```

5.3.3 Comparison of BFS and DFS Traversals

Graph	BFS Traversal	DFS Traversal
Figure 65	(a, b, d, c, e, g, f)	(a, b, c, e, f, g, d)
Figure 66	$(2, 6, 7, 8, 4, 3, 9, 1, 5)$	$(2, 6, 8, 3, 9, 7, 4, 1, 5)$

Table 2: Comparison of BFS and DFS traversal orders starting from vertices a and 2

Remark 45 (Traversal Pattern Differences). *The BFS and DFS traversals reveal fundamentally different exploration strategies:*

- **Exploration Pattern:** BFS explores vertices in breadth (layer by layer), while DFS explores in depth (branch by branch).
- **Figure 65 Patterns:**
 - In BFS, vertices b and d are discovered first (immediate neighbors of a), then the next layer (c, e, g), and finally f .
 - In DFS, the algorithm explores the branch a - b - c - e - f - g completely before backtracking to explore d .
- **Figure 66 Patterns:**

- In BFS, immediate neighbors of 2 (vertices 6, 7, 8) are explored first, then their neighbors, creating a layered exploration.
- In DFS, the branch 2-6-8-3-9 is fully explored before backtracking to explore 7 and its branch.

Remark 46 (Structural Insights from Traversals). *The traversal trees reveal important structural differences:*

- **Tree Shape:** *BFS trees tend to be broader and less deep, while DFS trees are typically deeper with fewer branches.*
- **Path Lengths:**
 - *In Figure 65, the BFS tree guarantees shortest paths from a to all vertices, while the DFS tree creates a longer path to d (via a directly versus potentially through other vertices).*
 - *In Figure 66, the BFS path from 2 to 5 goes through vertices 7 and 4 (shorter), while the DFS path follows a longer route.*
- **Edge Selection:**
 - *BFS includes edges that minimize distance from the root.*
 - *DFS includes edges that extend the current exploration path as far as possible.*
 - *For example, in Figure 65, BFS includes edge {a, d} while DFS includes {f, g}.*
- **Cycle Detection:** *Both traversals identify different sets of back edges (edges not in the spanning tree) that form cycles.*

Remark 47 (Application Implications). *The different traversal orders suggest optimal algorithms for different applications:*

- **BFS Applications:** *Finding shortest paths, computing graph distances and eccentricity, detecting layers/levels in networks.*
- **DFS Applications:** *Detecting cycles, finding connected components, topological sorting, solving maze-like problems.*
- *For Figure 65, BFS would be more appropriate for determining central vertices, while DFS would be better for identifying the cycle formed by vertices a-b-d.*
- *For Figure 66, BFS would be better for finding the shortest path from vertex 2 to vertex 5, while DFS would be more efficient for detecting all cycles in the graph.*

5.4 Prim's Algorithm

Definition 44 (Subtree). Given a tree $T = (V, E)$, a subtree $T' = (V', E')$ is a connected subgraph of T that is itself a tree. Formally, T' is a subtree of T if:

1. $V' \subseteq V$ (the vertices of T' are a subset of the vertices of T)
2. $E' \subseteq E$ (the edges of T' are a subset of the edges of T)
3. T' is connected
4. T' has no cycles

In other words, a subtree is formed by taking a connected subset of vertices and edges from the original tree while maintaining the tree property.

Remark 48. Any connected subgraph of a tree is itself a tree. This is because removing edges or vertices from a tree cannot create cycles where none existed before.

Remark 49. For a tree with n vertices, if a subtree has k vertices, then it must have exactly $k - 1$ edges.

Example 9. Consider a tree T with vertices $V = \{a, b, c, d, e\}$ and edges $E = \{(a, b), (b, c), (b, d), (d, e)\}$. Examples of valid subtrees include:

- The subtree with vertices $\{a, b, d\}$ and edges $\{(a, b), (b, d)\}$
- The subtree with vertices $\{b, c, d, e\}$ and edges $\{(b, c), (b, d), (d, e)\}$
- The subtree with vertices $\{d, e\}$ and edge $\{(d, e)\}$
- Any single vertex, which forms a trivial subtree

Definition 45 (Prim's Algorithm). Prim's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected weighted undirected graph. It grows the minimum spanning tree one vertex at a time, starting from an arbitrary vertex and always adding the lowest-weight edge that connects a vertex in the tree to a vertex outside the tree. The algorithm proceeds as follows:

1. Select an arbitrary vertex s as the starting point and include it in the MST.
2. Initialize a set S to contain only the starting vertex s : $S = \{s\}$.
3. Initialize a set T of edges to be empty: $T = \emptyset$.
4. While $S \neq V$ (not all vertices are included in the MST):
 - (a) Consider all edges (u, v) where $u \in S$ and $v \in V \setminus S$.
 - (b) Choose the edge (u, v) with minimum weight among all crossing edges.
 - (c) Add vertex v to set S : $S = S \cup \{v\}$.
 - (d) Add edge (u, v) to set T : $T = T \cup \{(u, v)\}$.
5. The set T of edges forms the minimum spanning tree.

This algorithm can be efficiently implemented using a priority queue to select the minimum-weight edge at each step.

Remark 50. Unlike Kruskal's algorithm, Prim's algorithm maintains a single connected subtree at all times. It grows this tree by adding one vertex and one edge at each step, always ensuring connectivity.

Theorem 37 (Prim's Theorem). Let $G = (V, E)$ be a connected, undirected graph with positive edge weights $w : E \rightarrow \mathbb{R}^+$. The tree T constructed by Prim's Algorithm, which iteratively adds the minimum-weight edge connecting a vertex in the current tree to a vertex not yet in the tree until all vertices are included, is a minimum weight spanning tree of G .

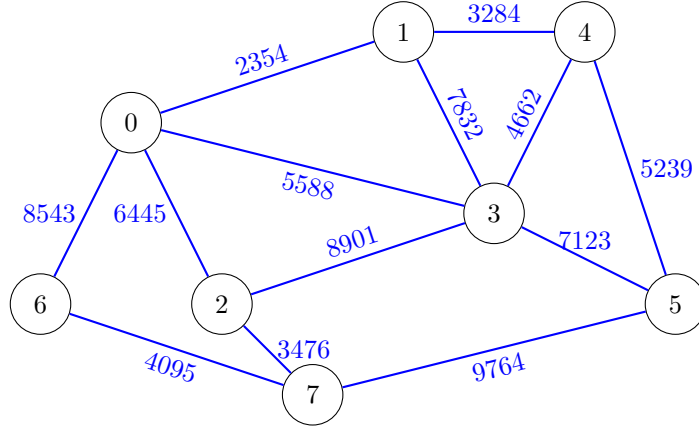


Figure 67: Example of a weighed graph

1. **Initialize:** Start with vertex 0.

- MST vertices: $\{0\}$
- MST edges: $\emptyset$

2. **Iteration 1:** Consider all edges from vertex 0 to unvisited vertices.

- Edge (0,1) with weight 2354
- Edge (0,2) with weight 6445
- Edge (0,3) with weight 5588
- Edge (0,6) with weight 8543
- **Add** the minimum weight edge: (0,1) with weight 2354
- MST vertices: $\{0,1\}$
- MST edges: $\{(0,1)\}$

3. **Iteration 2:** Consider all edges from vertices $\{0,1\}$ to unvisited vertices.

- Edge (0,2) with weight 6445
- Edge (0,3) with weight 5588
- Edge (0,6) with weight 8543
- Edge (1,3) with weight 7832
- Edge (1,4) with weight 3284
- **Add** the minimum weight edge: (1,4) with weight 3284
- MST vertices: $\{0,1,4\}$
- MST edges: $\{(0,1), (1,4)\}$

4. **Iteration 3:** Consider all edges from vertices $\{0,1,4\}$ to unvisited vertices.

- Edge (0,2) with weight 6445
- Edge (0,3) with weight 5588
- Edge (0,6) with weight 8543
- Edge (1,3) with weight 7832
- Edge (4,3) with weight 4662
- Edge (4,5) with weight 5239

- **Add** the minimum weight edge: $(4, 3)$ with weight 4662
- MST vertices: $\{0, 1, 4, 3\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3)\}$

5. **Iteration 4:** Consider all edges from vertices $\{0, 1, 4, 3\}$ to unvisited vertices.

- Edge $(0, 2)$ with weight 6445
- Edge $(0, 6)$ with weight 8543
- Edge $(3, 5)$ with weight 7123
- Edge $(4, 5)$ with weight 5239
- **Add** the minimum weight edge: $(4, 5)$ with weight 5239
- MST vertices: $\{0, 1, 4, 3, 5\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5)\}$

6. **Iteration 5:** Consider all edges from vertices $\{0, 1, 4, 3, 5\}$ to unvisited vertices.

- Edge $(0, 2)$ with weight 6445
- Edge $(0, 6)$ with weight 8543
- Edge $(5, 7)$ with weight 9764
- **Add** the minimum weight edge: $(0, 2)$ with weight 6445
- MST vertices: $\{0, 1, 4, 3, 5, 2\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5), (0, 2)\}$

7. **Iteration 6:** Consider all edges from vertices $\{0, 1, 4, 3, 5, 2\}$ to unvisited vertices.

- Edge $(0, 6)$ with weight 8543
- Edge $(2, 7)$ with weight 3476
- Edge $(5, 7)$ with weight 9764
- **Add** the minimum weight edge: $(2, 7)$ with weight 3476
- MST vertices: $\{0, 1, 4, 3, 5, 2, 7\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5), (0, 2), (2, 7)\}$

8. **Iteration 7:** Consider all edges from vertices $\{0, 1, 4, 3, 5, 2, 7\}$ to unvisited vertices.

- Edge $(0, 6)$ with weight 8543
- Edge $(6, 7)$ with weight 4095
- **Add** the minimum weight edge: $(6, 7)$ with weight 4095
- MST vertices: $\{0, 1, 4, 3, 5, 2, 7, 6\}$
- MST edges: $\{(0, 1), (1, 4), (4, 3), (4, 5), (0, 2), (2, 7), (6, 7)\}$

Result: The minimum spanning tree has the following edges:

- Edge $(0, 1)$ with weight 2354
- Edge $(1, 4)$ with weight 3284
- Edge $(4, 3)$ with weight 4662
- Edge $(4, 5)$ with weight 5239
- Edge $(0, 2)$ with weight 6445

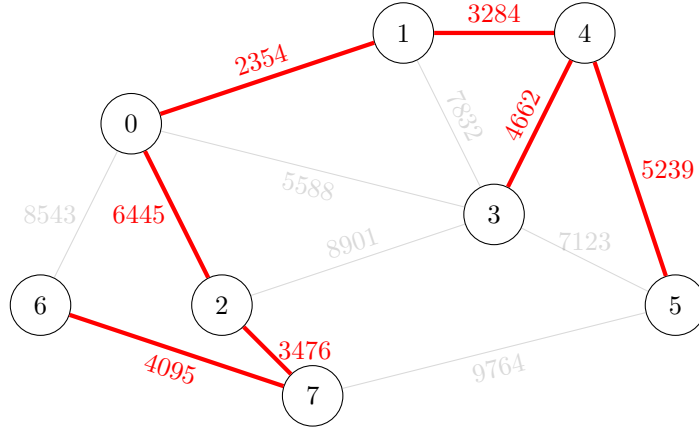


Figure 68: Finding the shortest spanning tree using Prim's Algorithm

Algorithm 13 Prim's Algorithm for Minimum Spanning Tree

Require: Connected, undirected graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$

Ensure: Minimum spanning tree $T = (V, E_T)$

```

1: procedure PRIM( $G, w$ )
2:   Select an arbitrary vertex  $s \in V$  as the starting vertex
3:    $V_T \leftarrow \{s\}$  ▷ Vertices in the MST
4:    $E_T \leftarrow \emptyset$  ▷ Edges in the MST
5:    $PQ \leftarrow \emptyset$  ▷ Priority queue of edges based on weight
6:   while  $V_T \neq V$  do
7:     for each edge  $(u, v) \in E$  where  $u \in V_T$  and  $v \notin V_T$  do
8:       Insert edge  $(u, v)$  into  $PQ$  with priority  $w(u, v)$ 
9:     end for
10:     $(u, v) \leftarrow \text{Extract-Min}(PQ)$  ▷ Extract edge with minimum weight
11:     $V_T \leftarrow V_T \cup \{v\}$  ▷ Add vertex  $v$  to the MST
12:     $E_T \leftarrow E_T \cup \{(u, v)\}$  ▷ Add edge  $(u, v)$  to the MST
13:  end while
14:  return  $T = (V, E_T)$ 
15: end procedure

```

- Edge (2,7) with weight 3476
- Edge (6,7) with weight 4095

The total weight of the MST is: $2354 + 3284 + 4662 + 5239 + 6445 + 3476 + 4095 = 29,555$.

Lemma 36 (Cut Property for Prim's Algorithm). At each step of Prim's algorithm, the edge selected is the minimum-weight edge crossing the cut between the vertices already in the MST and those not yet included. This edge must be in some minimum spanning tree of the graph.

Proof. Let S be the set of vertices already included in the MST, and let $e = (u, v)$ be the minimum-weight edge with $u \in S$ and $v \in V \setminus S$. Suppose for contradiction that e is not in any MST. Let T be an MST that doesn't contain e . Adding e to T creates a cycle, and this cycle must contain another edge e' that crosses the cut $(S, V \setminus S)$. Removing e' and adding e would create a spanning tree with weight less than T (since $w(e) < w(e')$), contradicting the minimality of T . $\square$

Lemma 37 (Uniqueness Condition). If all edge weights in a connected graph are distinct, then the minimum spanning tree is unique and will be found by Prim's algorithm regardless of the starting vertex.

5.5 Kruskal's Algorithm

Definition 46 (Kruskal's Algorithm). Kruskal's algorithm is a greedy algorithm that finds a minimum spanning tree for a connected weighted undirected graph. It builds the minimum spanning tree by adding edges in order of increasing weight, skipping edges that would create a cycle. The algorithm proceeds as follows:

1. Sort all edges of the graph $G = (V, E)$ in non-decreasing order of their weights.
2. Initialize a forest F where each vertex in the graph is a separate tree.
3. For each edge (u, v) in the sorted order:
 - (a) If vertices u and v belong to different trees in the forest F :
 - i. Add edge (u, v) to the forest F , merging the trees containing u and v .
 - (b) Otherwise, discard the edge (as it would create a cycle).
4. The forest F is the minimum spanning tree (or forest if the graph is not connected).

This algorithm uses the disjoint-set data structure to efficiently track which tree each vertex belongs to and to perform the merging operations.

Remark 51. *Kruskal's algorithm, unlike Prim's algorithm, does not necessarily build a connected subtree at each step. Instead, it may create a forest of trees that eventually merges into a single minimum spanning tree if the graph is connected.*

Demonstration of Kruskal's Algorithm for Finding a Minimum Spanning Tree

Starting with the weighted graph shown in Figure 67, we apply Kruskal's algorithm to find the minimum spanning tree (MST):

1. **Initialize:** Start with a forest where each vertex is a separate tree.
 - Forest trees: $\{\{0\}, \{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}\}$
 - MST edges: $\emptyset$
2. **Sort edges:** Arrange all edges by weight in ascending order.
 - Edge (0,1) with weight 2354
 - Edge (1,4) with weight 3284
 - Edge (2,7) with weight 3476
 - Edge (6,7) with weight 4095
 - Edge (3,4) with weight 4662
 - Edge (4,5) with weight 5239
 - Edge (0,3) with weight 5588
 - Edge (0,2) with weight 6445
 - Edge (3,5) with weight 7123
 - Edge (1,3) with weight 7832
 - Edge (2,3) with weight 8901
 - Edge (6,0) with weight 8543
 - Edge (7,5) with weight 9764
3. **Iteration 1:** Consider edge (0,1) with weight 2354.
 - Vertices 0 and 1 are in different trees.
 - **Add** this edge to the MST.

- Forest trees: $\{\{0, 1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}, \{7\}\}$
 - MST edges: $\{(0, 1)\}$
4. **Iteration 2:** Consider edge (1,4) with weight 3284.
- Vertices 1 and 4 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4\}, \{2\}, \{3\}, \{5\}, \{6\}, \{7\}\}$
 - MST edges: $\{(0, 1), (1, 4)\}$
5. **Iteration 3:** Consider edge (2,7) with weight 3476.
- Vertices 2 and 7 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4\}, \{2, 7\}, \{3\}, \{5\}, \{6\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7)\}$
6. **Iteration 4:** Consider edge (6,7) with weight 4095.
- Vertices 6 and 7 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4\}, \{2, 7, 6\}, \{3\}, \{5\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7)\}$
7. **Iteration 5:** Consider edge (3,4) with weight 4662.
- Vertices 3 and 4 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4, 3\}, \{2, 7, 6\}, \{5\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7), (3, 4)\}$
8. **Iteration 6:** Consider edge (4,5) with weight 5239.
- Vertices 4 and 5 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4, 3, 5\}, \{2, 7, 6\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7), (3, 4), (4, 5)\}$
9. **Iteration 7:** Consider edge (0,3) with weight 5588.
- Vertices 0 and 3 are already in the same tree.
 - **Skip** this edge (would create a cycle).
 - Forest trees remain: $\{\{0, 1, 4, 3, 5\}, \{2, 7, 6\}\}$
10. **Iteration 8:** Consider edge (0,2) with weight 6445.
- Vertices 0 and 2 are in different trees.
 - **Add** this edge to the MST.
 - Forest trees: $\{\{0, 1, 4, 3, 5, 2, 7, 6\}\}$
 - MST edges: $\{(0, 1), (1, 4), (2, 7), (6, 7), (3, 4), (4, 5), (0, 2)\}$
11. **Iteration 9:** All vertices are now in one tree (MST complete).

- Skip remaining edges: (3,5), (1,3), (2,3), (6,0), (7,5)

Result: The minimum spanning tree has the following edges:

- Edge (0,1) with weight 2354
- Edge (1,4) with weight 3284
- Edge (2,7) with weight 3476
- Edge (6,7) with weight 4095
- Edge (3,4) with weight 4662
- Edge (4,5) with weight 5239
- Edge (0,2) with weight 6445

The total weight of the MST is: $2354 + 3284 + 3476 + 4095 + 4662 + 5239 + 6445 = 29,555$.

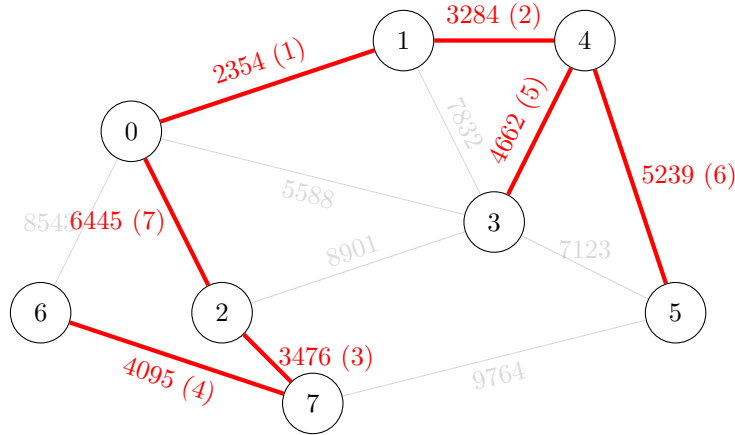


Figure 69: Finding the minimum spanning tree using Kruskal's Algorithm. Numbers in parentheses indicate the order in which edges were added to the MST.

Algorithm 14 Kruskal's Algorithm

```

1: procedure KRUSKAL( $G = (V, E)$ )
2:    $A \leftarrow \emptyset$  ▷ Initialize an empty set for the minimum spanning tree
3:   Create  $|V|$  trees, each containing a single vertex
4:   Sort all edges in  $E$  by weight in non-decreasing order
5:   for each edge  $e = (u, v) \in E$  in sorted order do
6:     if  $u$  and  $v$  are in different trees then
7:        $A \leftarrow A \cup \{e\}$  ▷ Add edge to the minimum spanning tree
8:       Merge the trees containing  $u$  and  $v$ 
9:     end if
10:  end for
11:  return  $A$  ▷ Return the minimum spanning tree
12: end procedure

```

Both Prim's and Kruskal's algorithms are greedy algorithms that find the minimum spanning tree (MST) of a connected, weighted graph. While they produce the same result when edge weights are unique, they differ in their approach and performance characteristics.

5.5.1 Key Similarities

- Both algorithms find a minimum spanning tree with the same total weight
- Both are greedy algorithms, making locally optimal choices at each step
- Both have the same time complexity of $O(E \log V)$ when implemented with appropriate data structures
- Both guarantee an optimal solution

Characteristic	Prim's Algorithm	Kruskal's Algorithm
Strategy	Grows a single tree	Maintains a forest of trees
Starting Point	Begins with a single vertex	Begins with all vertices as separate trees
Edge Selection	Selects the minimum-weight edge connecting the current tree to a new vertex	Selects the minimum-weight edge overall that doesn't create a cycle
Data Structure	Priority queue of edges or vertices	Disjoint-set data structure + sorted edges
Iteration Order	Based on graph structure	Based solely on edge weights
Optimal For	Dense graphs	Sparse graphs
Intermediate State	Always maintains a connected subtree	Maintains a forest that gradually merges

Table 3: Comparison of Prim's and Kruskal's Algorithms

As demonstrated in Figures 68 and 69:

1. Edge Addition Order:

- Prim's algorithm added edges: (0, 1), (1, 4), (4, 3), (4, 5), (0, 2), (2, 7), (6, 7)
- Kruskal's algorithm added edges: (0, 1), (1, 4), (2, 7), (6, 7), (3, 4), (4, 5), (0, 2)

2. Intermediate Structures:

- Prim's maintained a single connected component throughout
- Kruskal's initially had multiple components: {0, 1, 4}, {2, 7}, etc.

3. Edge Rejection Reasoning:

- Prim's rejected edges that didn't connect to new vertices
- Kruskal's rejected edges that would create cycles

4. Final Result:

- Both algorithms produced the same MST with total weight 29,555
- The resulting tree structure is identical
- Prim's algorithm requires efficient priority queue operations
- Kruskal's algorithm requires efficient disjoint-set operations with union-find
- For graphs with $|V|$ vertices and $|E|$ edges:

- Prim’s with binary heap: $O(E \log V)$
- Prim’s with Fibonacci heap: $O(E + V \log V)$
- Kruskal’s with efficient sorting and union-find: $O(E \log E) \approx O(E \log V)$
- Prim’s algorithm is often preferred for dense graphs
- Kruskal’s algorithm is often preferred for sparse graphs
- Both are used in network design, clustering algorithms, and approximation algorithms for other problems

Theorem 38 (MST Uniqueness). If all edge weights in a connected graph are distinct, then the minimum spanning tree is unique and will be found by both Prim’s and Kruskal’s algorithms.

Remark 52. *The choice between these algorithms typically depends on the specific graph characteristics and available data structures in the implementation environment.*

Corollary 17 (Exchange Property). If edge e is in an MST T and edge f is not in T , then adding f to T creates a unique cycle C . If the weight of f is less than the weight of any other edge in C , then $(T \setminus \{e\}) \cup \{f\}$ is also an MST.

Corollary 18 (Growth Property). Both algorithms build the MST by adding one edge at a time, never removing an edge once added.

Aspect	Prim’s Algorithm	Kruskal’s Algorithm
Best for	Dense graphs	Sparse graphs
Time Complexity	$O(E \log V)$ with binary heap	$O(E \log E)$ or $O(E \log V)$
Key Data Structure	Priority Queue	Disjoint-Set (Union-Find)
Growth Pattern	Single tree	Forest of trees

Table 4: Comparison of Prim’s and Kruskal’s Algorithms

Remark 53 (Kruskal’s Algorithm Optimizations). • *Using Union-Find with path compression and union by rank gives near-constant time operations*

- *Pre-sorting edges can be implemented efficiently using counting sort if weights are integers within a small range*

Remark 54 (Prim’s Algorithm Optimizations). • *Using a Fibonacci heap reduces time complexity to $O(E + V \log V)$*

- *For dense graphs, a simple array implementation of the priority queue can be more efficient than a binary heap*

Remark 55 (Algorithm Selection). • *Use Prim’s when $|E|$ is close to $|V|^2$*

- *Use Kruskal’s when $|E|$ is close to $|V|$*

5.6 Dijkstra's Algorithm

Definition 47 (Dijkstra's Algorithm). Let $G = (V, E)$ be a directed graph with a non-negative weight function $w : E \rightarrow \mathbb{R}^+$, and let $s \in V$ be a source vertex. Dijkstra's algorithm computes $\delta(s, v)$, the weight of the shortest path from s to each vertex $v \in V$, as follows:

1. Initialize $d[s] = 0$ and $d[v] = \infty$ for all other vertices $v \in V$.
2. Initialize a priority queue Q containing all vertices, with priority values equal to their d values.
3. Initialize the set of finalized vertices $S = \emptyset$.
4. While $Q \neq \emptyset$:
 - (a) Extract the vertex u with minimum distance value $d[u]$ from Q .
 - (b) Add u to S .
 - (c) For each edge $(u, v) \in E$:
 - i. If $d[v] > d[u] + w(u, v)$, then set $d[v] = d[u] + w(u, v)$ and update v 's position in Q .
5. Upon termination, $d[v] = \delta(s, v)$ for all $v \in V$.

The correctness of the algorithm follows from the invariant that at each iteration, $d[v] = \delta(s, v)$ for all $v \in S$.

Demonstration of Dijkstra's Algorithm for Finding Shortest Paths

Starting with the weighted graph shown in Figure 70, we apply Dijkstra's algorithm to find the shortest paths from source vertex 0 to all other vertices:

1. **Initialize:** Set distance to source vertex 0 as 0, and all other vertices as infinity.
 - Distance values: $d[0] = 0, d[1] = \infty, d[2] = \infty, d[3] = \infty, d[4] = \infty, d[5] = \infty, d[6] = \infty, d[7] = \infty$
 - Previous vertices: $prev[i] = null$ for all i
 - Priority queue $Q = \{0(0), 1(\infty), 2(\infty), 3(\infty), 4(\infty), 5(\infty), 6(\infty), 7(\infty)\}$
 - Visited set $S = \emptyset$
2. **Iteration 1:** Extract vertex 0 (smallest distance).
 - Remove vertex 0 from Q .
 - $S = \{0\}$
 - Update neighbors of vertex 0:
 - Edge (0, 1) with weight 2354: $d[1] = \min(\infty, 0 + 2354) = 2354, prev[1] = 0$
 - Edge (0, 2) with weight 6445: $d[2] = \min(\infty, 0 + 6445) = 6445, prev[2] = 0$
 - Edge (0, 3) with weight 5588: $d[3] = \min(\infty, 0 + 5588) = 5588, prev[3] = 0$
 - Edge (0, 6) with weight 8543: $d[6] = \min(\infty, 0 + 8543) = 8543, prev[6] = 0$
 - Updated priority queue: $Q = \{1(2354), 3(5588), 2(6445), 6(8543), 4(\infty), 5(\infty), 7(\infty)\}$
3. **Iteration 2:** Extract vertex 1 (smallest distance in Q).
 - Remove vertex 1 from Q .
 - $S = \{0, 1\}$
 - Update neighbors of vertex 1:
 - Edge (1, 3) with weight 7832: $d[3] = \min(5588, 2354 + 7832) = 5588$, no change
 - Edge (1, 4) with weight 3284: $d[4] = \min(\infty, 2354 + 3284) = 5638, prev[4] = 1$
 - Updated priority queue: $Q = \{3(5588), 4(5638), 2(6445), 6(8543), 5(\infty), 7(\infty)\}$
4. **Iteration 3:** Extract vertex 3 (smallest distance in Q).

- Remove vertex 3 from Q .
- $S = \{0, 1, 3\}$
- Update neighbors of vertex 3:
 - Edge (3, 2) with weight 8901: $d[2] = \min(6445, 5588 + 8901) = 6445$, no change
 - Edge (3, 4) with weight 4662: $d[4] = \min(5638, 5588 + 4662) = 5638$, no change
 - Edge (3, 5) with weight 7123: $d[5] = \min(\infty, 5588 + 7123) = 12711$, $prev[5] = 3$
- Updated priority queue: $Q = \{4(5638), 2(6445), 6(8543), 5(12711), 7(\infty)\}$

5. **Iteration 4:** Extract vertex 4 (smallest distance in Q).

- Remove vertex 4 from Q .
- $S = \{0, 1, 3, 4\}$
- Update neighbors of vertex 4:
 - Edge (4, 5) with weight 5239: $d[5] = \min(12711, 5638 + 5239) = 10877$, $prev[5] = 4$
- Updated priority queue: $Q = \{2(6445), 6(8543), 5(10877), 7(\infty)\}$

6. **Iteration 5:** Extract vertex 2 (smallest distance in Q).

- Remove vertex 2 from Q .
- $S = \{0, 1, 3, 4, 2\}$
- Update neighbors of vertex 2:
 - Edge (2, 7) with weight 3476: $d[7] = \min(\infty, 6445 + 3476) = 9921$, $prev[7] = 2$
- Updated priority queue: $Q = \{6(8543), 7(9921), 5(10877)\}$

7. **Iteration 6:** Extract vertex 6 (smallest distance in Q).

- Remove vertex 6 from Q .
- $S = \{0, 1, 3, 4, 2, 6\}$
- Update neighbors of vertex 6:
 - Edge (6, 7) with weight 4095: $d[7] = \min(9921, 8543 + 4095) = 9921$, no change
- Updated priority queue: $Q = \{7(9921), 5(10877)\}$

8. **Iteration 7:** Extract vertex 7 (smallest distance in Q).

- Remove vertex 7 from Q .
- $S = \{0, 1, 3, 4, 2, 6, 7\}$
- Update neighbors of vertex 7:
 - Edge (7, 5) with weight 9764: $d[5] = \min(10877, 9921 + 9764) = 10877$, no change
- Updated priority queue: $Q = \{5(10877)\}$

9. **Iteration 8:** Extract vertex 5 (smallest distance in Q).

- Remove vertex 5 from Q .
- $S = \{0, 1, 3, 4, 2, 6, 7, 5\}$
- No more updates needed (all vertices visited).

Result: The shortest paths from vertex 0 to all other vertices:

- Path to vertex 1: $0 \rightarrow 1$ with distance 2354
- Path to vertex 2: $0 \rightarrow 2$ with distance 6445

- Path to vertex 3: $0 \rightarrow 3$ with distance 5588
- Path to vertex 4: $0 \rightarrow 1 \rightarrow 4$ with distance 5638
- Path to vertex 5: $0 \rightarrow 1 \rightarrow 4 \rightarrow 5$ with distance 10877
- Path to vertex 6: $0 \rightarrow 6$ with distance 8543
- Path to vertex 7: $0 \rightarrow 2 \rightarrow 7$ with distance 9921

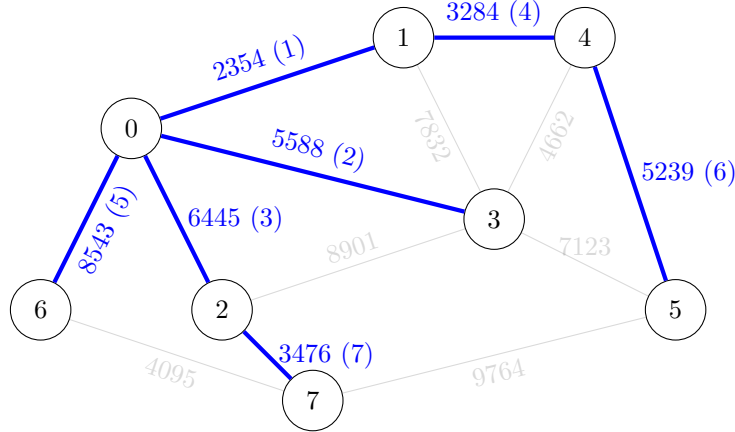


Figure 70: Finding the shortest paths using Dijkstra's Algorithm. Numbers in parentheses indicate the order in which vertices were processed.

Algorithm 15 Dijkstra's Algorithm

```

1: procedure DIJKSTRA( $G = (V, E), w, s$ )
2:   Input: Graph  $G$  with vertices  $V$  and edges  $E$ , weight function  $w : E \rightarrow \mathbb{R}^+$ , source vertex  $s \in V$ 
3:   Output: For each vertex  $v \in V$ , the shortest path distance  $d[v]$  from  $s$  to  $v$ 
4:    $d[v] \leftarrow \infty$  for all  $v \in V$  ▷ Initialize distances to infinity
5:    $d[s] \leftarrow 0$  ▷ Distance from source to itself is 0
6:    $Q \leftarrow V$  ▷ Initialize priority queue with all vertices
7:    $S \leftarrow \emptyset$  ▷ Set of vertices with finalized shortest path
8:   while  $Q \neq \emptyset$  do
9:      $u \leftarrow \text{Extract-Min}(Q)$  ▷ Extract vertex with minimum distance
10:     $S \leftarrow S \cup \{u\}$  ▷ Add  $u$  to set of processed vertices
11:    for each vertex  $v \in \text{Adj}[u]$  do
12:      if  $d[v] > d[u] + w(u, v)$  then ▷ Relaxation step
13:         $d[v] \leftarrow d[u] + w(u, v)$  ▷ Update distance
14:         $\pi[v] \leftarrow u$  ▷ Update predecessor
15:      end if
16:    end for
17:  end while
18:  return  $d, \pi$  ▷ Return distances and predecessors
19: end procedure

```

Demonstration of Dijkstra's Algorithm for Finding Shortest Paths

Important Note for Readers

Dijkstra's algorithm and Kruskal's algorithm solve fundamentally different graph problems:

- **Kruskal's algorithm** finds a Minimum Spanning Tree (MST) that connects all vertices with the minimum possible total edge weight (29,555 in our example).
- **Dijkstra's algorithm** finds the shortest paths from a single source vertex to all other vertices, creating a Shortest Path Tree (SPT).

These algorithms serve different purposes and will generally produce different tree structures with different total weights. The total weight of the MST (29,555) will not match the sum of the individual shortest path distances.

Theorem 39 (Correctness of Dijkstra's Algorithm). Let $G = (V, E)$ be a weighted graph with non-negative edge weights and $s \in V$ be a source vertex. Upon termination of Dijkstra's algorithm, for each vertex $v \in V$, the distance value $d[v]$ equals $\delta(s, v)$, the weight of the shortest path from s to v .

Theorem 40 (Optimality Principle). Any subpath of a shortest path is itself a shortest path between its endpoints.

Theorem 41 (Triangle Inequality). For any three vertices u , v , and w in a graph with non-negative edge weights, $\delta(u, w) \leq \delta(u, v) + \delta(v, w)$.

Theorem 42 (Negative Edge Weight Limitation). Dijkstra's algorithm may fail to find correct shortest paths if the graph contains any edges with negative weights.

Lemma 38 (Relaxation Property). If at any point during the execution of the algorithm, $d[u] = \delta(s, u)$ for some vertex u , then this equality will persist until termination of the algorithm.

Lemma 39 (Path Relaxation). If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from source $s = v_0$ to vertex v_k , and we relax the edges of p in order, then $d[v_k] = \delta(s, v_k)$ afterward, regardless of any other relaxation steps that occur.

Lemma 40 (Upper-Bound Property). For each vertex $v \in V$, we have $d[v] \geq \delta(s, v)$ at all times during the execution of the algorithm, and once $d[v] = \delta(s, v)$, it never changes.

Corollary 19 (Greedy Choice Property). The vertex u with the minimum distance value $d[u]$ among all unvisited vertices has $d[u] = \delta(s, u)$.

Corollary 20 (Subgraph Property). The collection of shortest paths from a source vertex s to all other vertices forms a tree rooted at s .

Corollary 21 (Convergence Time). For a graph with n vertices, Dijkstra's algorithm converges in exactly n iterations of the main loop.

Remark 56 (Time Complexity). *The standard implementation of Dijkstra's algorithm using a binary heap has time complexity $O(E \log V)$ for a graph with V vertices and E edges. Using a Fibonacci heap can reduce this to $O(V \log V + E)$.*

Remark 57 (Bidirectional Search). *For finding the shortest path between a single source and a single target, bidirectional Dijkstra's algorithm (which runs the algorithm simultaneously from both endpoints) can significantly reduce the search space.*

Remark 58 (A* Extension). *Dijkstra's algorithm can be extended to the A* algorithm by incorporating a heuristic function that estimates the distance from each vertex to the target, potentially improving performance for specific path-finding problems.*

5.7 Floyd–Warshall Algorithm

Definition 48 (Floyd-Warshall Algorithm). Let $G = (V, E)$ be a weighted directed graph with n vertices, where $V = \{1, 2, \dots, n\}$. The Floyd-Warshall algorithm computes the shortest path distances between all pairs of vertices as follows:

1. Initialize an $n \times n$ matrix $D^{(0)}$ where:
 - $D_{ij}^{(0)} = w(i, j)$ if there is an edge from i to j
 - $D_{ij}^{(0)} = \infty$ if there is no edge from i to j and $i \neq j$
 - $D_{ii}^{(0)} = 0$ for all $i \in V$
2. For $k = 1$ to n :
 - For $i = 1$ to n :
 - For $j = 1$ to n :
 - * $D_{ij}^{(k)} = \min(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)})$
3. The final matrix $D^{(n)}$ contains the shortest path distances, where $D_{ij}^{(n)}$ is the length of the shortest path from vertex i to vertex j .

The algorithm has time complexity $\Theta(n^3)$ and space complexity $\Theta(n^2)$, where n is the number of vertices in the graph.

Demonstration of Floyd-Warshall Algorithm for All-Pairs Shortest Paths

Starting with the weighted graph shown in Figure 71, we apply the Floyd-Warshall algorithm to find the shortest paths between all pairs of vertices:

1. **Initialize:** Create the distance matrix $D^{(0)}$ based on the direct edge weights.

- $D_{ii}^{(0)} = 0$ for all i (distance from a vertex to itself is 0)
- $D_{ij}^{(0)} = w(i, j)$ if there is an edge from i to j
- $D_{ij}^{(0)} = \infty$ if there is no edge from i to j and $i \neq j$

Initial distance matrix $D^{(0)}$:

$$\begin{pmatrix} 0 & 2354 & 6445 & 5588 & \infty & \infty & 8543 & \infty \\ \infty & 0 & \infty & 7832 & 3284 & \infty & \infty & \infty \\ \infty & \infty & 0 & 8901 & \infty & \infty & \infty & 3476 \\ \infty & \infty & \infty & 0 & 4662 & 7123 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 5239 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 & 4095 \\ \infty & \infty & \infty & \infty & \infty & 9764 & \infty & 0 \end{pmatrix}$$

2. **Iteration** $k = 0$ to $k = 1$: Consider vertex 1 (index 0) as an intermediate vertex.
 - For each pair of vertices (i, j) , check if path $i \rightarrow 0 \rightarrow j$ is shorter than current path $i \rightarrow j$
 - $D_{ij}^{(1)} = \min(D_{ij}^{(0)}, D_{i0}^{(0)} + D_{0j}^{(0)})$

Updated significant values in $D^{(1)}$:

- $D_{12}^{(1)} = \min(\infty, \infty + 6445) = \infty$
- $D_{13}^{(1)} = \min(7832, \infty + 5588) = 7832$
- $D_{16}^{(1)} = \min(\infty, \infty + 8543) = \infty$

(Most entries remain unchanged from $D^{(0)}$)

3. **Iteration** $k = 1$ to $k = 2$: Consider vertex 2 (index 1) as an intermediate vertex.

- For each pair of vertices (i, j) , check if path $i \rightarrow 1 \rightarrow j$ is shorter than current path $i \rightarrow j$
- $D_{ij}^{(2)} = \min(D_{ij}^{(1)}, D_{i1}^{(1)} + D_{1j}^{(1)})$

Updated significant values in $D^{(2)}$:

- $D_{04}^{(2)} = \min(\infty, 2354 + 3284) = 5638$
- $D_{13}^{(2)} = \min(7832, 0 + 7832) = 7832$

(Most entries remain unchanged from $D^{(1)}$)

4. **Iteration** $k = 2$ to $k = 3$: Consider vertex 3 (index 2) as an intermediate vertex.

- Calculate $D_{ij}^{(3)} = \min(D_{ij}^{(2)}, D_{i2}^{(2)} + D_{2j}^{(2)})$
- Notable update: $D_{07}^{(3)} = \min(\infty, 6445 + 3476) = 9921$

5. **Iteration** $k = 3$ to $k = 4$: Consider vertex 4 (index 3) as an intermediate vertex.

- Calculate $D_{ij}^{(4)} = \min(D_{ij}^{(3)}, D_{i3}^{(3)} + D_{3j}^{(3)})$
- Notable update: $D_{05}^{(4)} = \min(\infty, 5588 + 7123) = 12711$

6. **Iteration** $k = 4$ to $k = 5$: Consider vertex 5 (index 4) as an intermediate vertex.

- Calculate $D_{ij}^{(5)} = \min(D_{ij}^{(4)}, D_{i4}^{(4)} + D_{4j}^{(4)})$
- Notable update: $D_{05}^{(5)} = \min(12711, 5638 + 5239) = 10877$

7. **Iterations** $k = 6$ through $k = 8$: Consider remaining vertices as intermediates.

- Continue applying the recurrence relation for remaining vertices
- The algorithm completes after considering all vertices as potential intermediates

Final Distance Matrix $D^{(8)}$:

$$\begin{pmatrix} 0 & 2354 & 6445 & 5588 & 5638 & 10877 & 8543 & 9921 \\ \infty & 0 & \infty & 7832 & 3284 & 8523 & \infty & \infty \\ \infty & \infty & 0 & 8901 & \infty & \infty & \infty & 3476 \\ \infty & \infty & \infty & 0 & 4662 & 7123 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & 5239 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 0 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 13859 & 0 & 4095 \\ \infty & \infty & \infty & \infty & \infty & 9764 & \infty & 0 \end{pmatrix}$$

Result: The shortest path distances between all pairs of vertices are given in the final matrix $D^{(8)}$. For example:

- The shortest path from vertex 0 to vertex 5 has distance 10877.
- The shortest path from vertex 1 to vertex 5 has distance 8523.
- The shortest path from vertex 7 to vertex 5 has distance 9764.
- Many vertex pairs have no path between them (indicated by ∞).

Algorithm 16 Floyd-Warshall Algorithm

```
1: procedure FLOYDWARSHALL( $G = (V, E), w$ )
2:   Input: Graph  $G$  with vertices  $V = \{1, 2, \dots, n\}$  and edges  $E$ , weight function  $w$ 
3:   Output: Matrix  $D$  where  $D[i, j]$  is the shortest path distance from  $i$  to  $j$ 
4:   Initialize distance matrix  $D$ :
5:   for each  $i \in V$  do
6:     for each  $j \in V$  do
7:       if  $i = j$  then
8:          $D[i, j] \leftarrow 0$ 
9:       else if  $(i, j) \in E$  then
10:         $D[i, j] \leftarrow w(i, j)$ 
11:       else
12:         $D[i, j] \leftarrow \infty$ 
13:       end if
14:     end for
15:   end for
16:   Main algorithm:
17:   for  $k \leftarrow 1$  to  $n$  do
18:     for  $i \leftarrow 1$  to  $n$  do
19:       for  $j \leftarrow 1$  to  $n$  do
20:         if  $D[i, j] > D[i, k] + D[k, j]$  then
21:            $D[i, j] \leftarrow D[i, k] + D[k, j]$ 
22:            $P[i, j] \leftarrow P[i, k]$  ▷ Optional: store path information
23:         end if
24:       end for
25:     end for
26:   end for
27:   Negative cycle detection (optional):
28:   for  $i \leftarrow 1$  to  $n$  do
29:     if  $D[i, i] < 0$  then
30:       return "Graph contains a negative cycle"
31:     end if
32:   end for
33:   return  $D$  ▷ Return the matrix of shortest path distances
34: end procedure
```

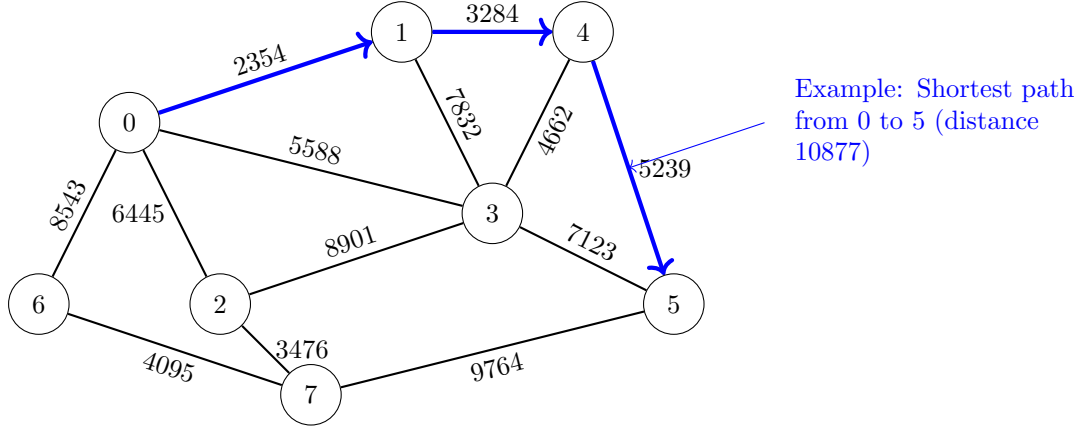


Figure 71: Graph for the Floyd-Warshall algorithm. The blue arrows highlight an example shortest path.

5.8 Bellman-Ford Algorithm

Definition 49 (Bellman-Ford Algorithm). Let $G = (V, E)$ be a weighted directed graph with n vertices and m edges, where edge weights may be negative. Let $s \in V$ be a source vertex. The Bellman-Ford algorithm computes the shortest path distances from s to all other vertices in V as follows:

1. Initialize distance values: $d[s] = 0$ and $d[v] = \infty$ for all other vertices $v \in V$.
2. Repeat the following relaxation step $n - 1$ times:
 - For each edge $(u, v) \in E$ with weight $w(u, v)$:
 - If $d[v] > d[u] + w(u, v)$, then $d[v] = d[u] + w(u, v)$ and $prev[v] = u$.
3. Check for negative weight cycles:
 - For each edge $(u, v) \in E$ with weight $w(u, v)$:
 - If $d[v] > d[u] + w(u, v)$, then report that the graph contains a negative weight cycle.

The algorithm has time complexity $O(nm)$ and space complexity $O(n)$, where n is the number of vertices and m is the number of edges in the graph.

Starting with the weighted graph shown in Figure 72, we apply the Bellman-Ford algorithm to find the shortest paths from source vertex 0 to all other vertices:

1. **Initialize:** Set distance to source vertex 0 as 0, and all other vertices as infinity.
 - Distance values: $d[0] = 0, d[1] = \infty, d[2] = \infty, d[3] = \infty, d[4] = \infty, d[5] = \infty, d[6] = \infty, d[7] = \infty$
 - Previous vertices: $prev[i] = null$ for all i
2. **Relaxation Iterations:** Apply relaxation step to all edges $n - 1 = 7$ times.
3. **Iteration 1:**
 - Edge (0, 1) with weight 2354: $d[1] = \min(\infty, 0 + 2354) = 2354, prev[1] = 0$
 - Edge (0, 2) with weight 6445: $d[2] = \min(\infty, 0 + 6445) = 6445, prev[2] = 0$
 - Edge (0, 3) with weight 5588: $d[3] = \min(\infty, 0 + 5588) = 5588, prev[3] = 0$
 - Edge (0, 6) with weight 8543: $d[6] = \min(\infty, 0 + 8543) = 8543, prev[6] = 0$
 - Edge (1, 3) with weight 7832: $d[3] = \min(5588, 2354 + 7832) = 5588$, no change
 - Edge (1, 4) with weight 3284: $d[4] = \min(\infty, 2354 + 3284) = 5638, prev[4] = 1$

- Edge (2, 3) with weight 8901: $d[3] = \min(5588, 6445 + 8901) = 5588$, no change
- Edge (2, 7) with weight 3476: $d[7] = \min(\infty, 6445 + 3476) = 9921$, $prev[7] = 2$
- Edge (3, 4) with weight 4662: $d[4] = \min(5638, 5588 + 4662) = 5638$, no change
- Edge (3, 5) with weight 7123: $d[5] = \min(\infty, 5588 + 7123) = 12711$, $prev[5] = 3$
- Edge (4, 5) with weight 5239: $d[5] = \min(12711, 5638 + 5239) = 10877$, $prev[5] = 4$
- Edge (6, 7) with weight 4095: $d[7] = \min(9921, 8543 + 4095) = 9921$, no change
- Edge (7, 5) with weight 9764: $d[5] = \min(10877, 9921 + 9764) = 10877$, no change

4. **Updated distances after Iteration 1:**

- $d[0] = 0, d[1] = 2354, d[2] = 6445, d[3] = 5588, d[4] = 5638, d[5] = 10877, d[6] = 8543, d[7] = 9921$

5. **Iteration 2:** Perform the same relaxation steps for all edges.

- No distance values change in this iteration.

6. **Iterations 3-7:** Repeat the relaxation process.

- No further changes in distance values occur in these iterations.

7. **Negative Cycle Check:** Check for negative weight cycles.

- For each edge $(u, v) \in E$, verify that $d[v] \leq d[u] + w(u, v)$
- All edges satisfy this condition, so there are no negative weight cycles.

Result: The shortest paths from vertex 0 to all other vertices:

- Path to vertex 1: $0 \rightarrow 1$ with distance 2354
- Path to vertex 2: $0 \rightarrow 2$ with distance 6445
- Path to vertex 3: $0 \rightarrow 3$ with distance 5588
- Path to vertex 4: $0 \rightarrow 1 \rightarrow 4$ with distance 5638
- Path to vertex 5: $0 \rightarrow 1 \rightarrow 4 \rightarrow 5$ with distance 10877
- Path to vertex 6: $0 \rightarrow 6$ with distance 8543
- Path to vertex 7: $0 \rightarrow 2 \rightarrow 7$ with distance 9921

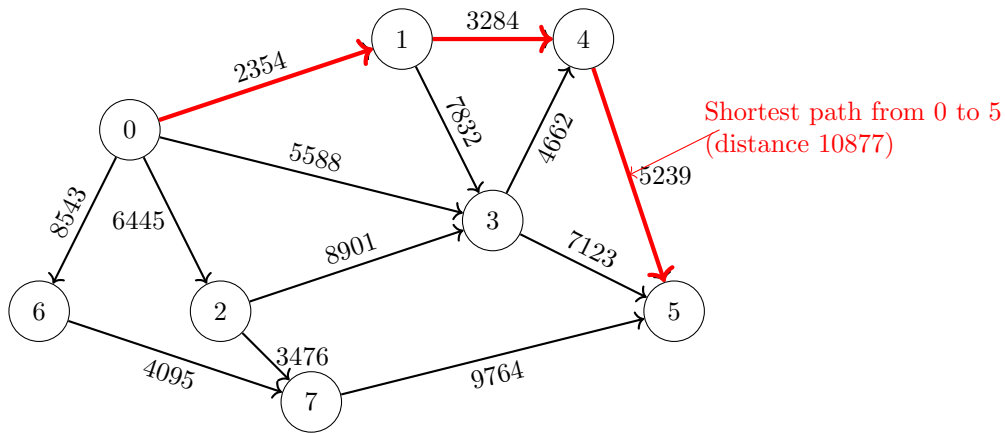


Figure 72: Graph for the Bellman-Ford algorithm. The red arrows highlight the shortest path from vertex 0 to vertex 5.

Key Theoretical Results for Bellman-Ford Algorithm

Algorithm 17 Bellman-Ford Algorithm

```
1: procedure BELLMANFORD( $G = (V, E), w, s$ )
2:   Input: Graph  $G$  with vertices  $V = \{1, 2, \dots, n\}$ , edges  $E$ , weight function  $w$ , source vertex  $s$ 
3:   Output: Array  $d$  where  $d[v]$  is the shortest path distance from  $s$  to  $v$ , or indication of a negative
   cycle
4:   Initialize:
5:   for each vertex  $v \in V$  do
6:      $d[v] \leftarrow \infty$  ▷ Set initial distance to infinity
7:      $predecessor[v] \leftarrow \text{NIL}$  ▷ Initialize predecessor array
8:   end for
9:    $d[s] \leftarrow 0$  ▷ Distance from source to itself is 0
10:  Relaxation phase:
11:  for  $i \leftarrow 1$  to  $|V| - 1$  do ▷ Repeat  $|V| - 1$  times
12:    for each edge  $(u, v) \in E$  do
13:      if  $d[v] > d[u] + w(u, v)$  then ▷ Relaxation step
14:         $d[v] \leftarrow d[u] + w(u, v)$ 
15:         $predecessor[v] \leftarrow u$ 
16:      end if
17:    end for
18:  end for
19:  Negative cycle detection:
20:  for each edge  $(u, v) \in E$  do
21:    if  $d[v] > d[u] + w(u, v)$  then
22:      return "Graph contains a negative weight cycle reachable from source"
23:    end if
24:  end for
25:  return  $d, predecessor$  ▷ Return shortest distances and predecessors
26: end procedure
```

Theorem 43 (Correctness of Bellman-Ford). If a graph contains no negative weight cycles reachable from the source vertex s , then after the Bellman-Ford algorithm terminates, $d[v] = \delta(s, v)$ for all vertices $v \in V$, where $\delta(s, v)$ is the weight of the shortest path from s to v .

Theorem 44 (Negative Cycle Detection). If the graph contains a negative weight cycle reachable from the source vertex s , then the Bellman-Ford algorithm will detect this after the relaxation phase.

Lemma 41 (Path Relaxation Lemma). Let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from $s = v_0$ to v_k . If we relax the edges of this path in order, then $d[v_k] = \delta(s, v_k)$ afterward, regardless of any other relaxation steps that occur.

Corollary 22 (Convergence Property). If the graph contains no negative weight cycles reachable from s , then after i iterations of the outer loop, the algorithm has correctly computed the shortest paths from s to all vertices reachable via paths with at most i edges.

Remark 59 (Comparison with Dijkstra's Algorithm). While Dijkstra's algorithm is more efficient for graphs with non-negative edge weights ($O(E \log V)$ vs. $O(VE)$ for Bellman-Ford), the Bellman-Ford algorithm can handle graphs with negative edge weights and detect negative cycles, which Dijkstra's algorithm cannot.

5.8.1 Algorithm Characteristics

Characteristic	Bellman-Ford Algorithm	Floyd-Warshall Algorithm
Primary purpose	Single-source shortest paths	All-pairs shortest paths
Time complexity	$O(V \cdot E)$	$O(V ^3)$
Space complexity	$O(V)$	$O(V ^2)$
Handles negative weights	Yes	Yes
Detects negative cycles	Yes	Yes (with additional checks)
Approach	Edge relaxation	Dynamic programming
Implementation complexity	Moderate	Simple

Table 5: Comparison of key characteristics between Bellman-Ford and Floyd-Warshall algorithms

- **Bellman-Ford Algorithm:** Uses the principle of edge relaxation, where each edge is "relaxed" to potentially improve the shortest path estimate. This process is repeated $|V| - 1$ times to ensure that shortest paths with up to $|V| - 1$ edges are found.
- **Floyd-Warshall Algorithm:** Uses dynamic programming to iteratively find the shortest path between all pairs of vertices. In each iteration k , it considers whether the path from vertex i to vertex j can be improved by going through vertex k .
- **Scalability:**
 - Bellman-Ford performs better on sparse graphs (where $|E| \approx |V|$)
 - Floyd-Warshall performs better when the graph is dense (where $|E| \approx |V|^2$)
- **Multiple Source Queries:**
 - If shortest paths are needed from a single source, Bellman-Ford is more efficient
 - If shortest paths are needed between many or all pairs of vertices, Floyd-Warshall is more efficient than running Bellman-Ford multiple times
- **Incremental Computation:**

- Bellman-Ford can be terminated early if only short paths are of interest
- Floyd-Warshall requires the full computation to complete before any results are available

Theorem 45 (Bellman-Ford Relaxation Property). If a graph $G = (V, E)$ contains no negative weight cycles reachable from the source vertex s , then after the Bellman-Ford algorithm performs $|V| - 1$ relaxation passes, $d[v] = \delta(s, v)$ for all vertices $v \in V$.

Theorem 46 (Floyd-Warshall Optimality Property). After the k -th iteration of the Floyd-Warshall algorithm, $d[i, j]$ equals the length of the shortest path from vertex i to vertex j using only vertices $\{1, 2, \dots, k\}$ as intermediate vertices.

Scenario	Recommendation	Rationale
Single source, sparse graph	Bellman-Ford	$O(V \cdot E)$ is better than $O(V ^3)$ when $ E $ is small
All pairs, dense graph	Floyd-Warshall	Cubic time is acceptable and implementation is simpler
Multiple sources, sparse graph	Multiple Bellman-Ford runs	May be faster than Floyd-Warshall if number of sources is small
Negative cycles detection	Bellman-Ford	More direct detection of negative cycles
Limited memory	Bellman-Ford	$O(V)$ space vs. $O(V ^2)$ space
Simple implementation	Floyd-Warshall	Conceptually simpler with fewer implementation pitfalls

Table 6: Practical recommendations for algorithm selection

5.9 Bipartiteness

Definition 50 (Bipartite Graph). A graph $G = (V, E)$ is bipartite if its vertex set V can be partitioned into two disjoint sets U and W such that every edge in E connects a vertex in U to a vertex in W . Equivalently, a graph is bipartite if and only if it does not contain any cycles of odd length.

Theorem 47 (Characterization of Bipartite Graphs). A graph is bipartite if and only if it is 2-colorable, i.e., its vertices can be colored using two colors such that no adjacent vertices share the same color.

Proof (Sketch). ($\Rightarrow$) Suppose G is bipartite with partite sets U and W . Any cycle must alternate between vertices in U and W . Since alternating between two sets always creates a path with an even number of edges (when returning to the starting set), any cycle must have even length.

($\Leftarrow$) Suppose G has no odd cycles. Select an arbitrary vertex $v \in V$ and assign it to set U . Perform a breadth-first search from v , assigning vertices to U or W based on their distance from v (odd distance to W , even distance to U). If G contains no odd cycles, this assignment creates a valid bipartite partition. $\square$

Theorem 48 (Bipartite Edge Bound). A bipartite graph with n vertices can have at most $\lfloor n^2/4 \rfloor$ edges. This bound is tight and achieved by the complete bipartite graph $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$.

Proof. Let $G = (U \cup W, E)$ be a bipartite graph with $|U| = a$, $|W| = b$, and $a + b = n$. The maximum number of edges is $a \cdot b$, which corresponds to a complete bipartite graph. To maximize $a \cdot b$ subject to $a + b = n$, we need $a \approx b \approx n/2$. More precisely, the maximum occurs at $a = \lfloor n/2 \rfloor$ and $b = \lceil n/2 \rceil$, giving $\lfloor n/2 \rfloor \cdot \lceil n/2 \rceil = \lfloor n^2/4 \rfloor$ edges. $\square$

The bipartiteness of a graph can be tested using either Breadth-First Search (BFS) or Depth-First Search (DFS). The following algorithm uses BFS to determine if a graph is bipartite:

1. **Initialize:**

- Create a color array for all vertices, initially set to "uncolored"
- Choose an arbitrary starting vertex s and color it (e.g., "red")
- Initialize a queue Q with vertex s

2. **BFS Traversal:**

- While queue Q is not empty:
 - Dequeue a vertex u from Q
 - For each neighbor v of u :
 - * If v is uncolored, assign it the opposite color of u and enqueue it
 - * If v is colored and has the same color as u , then the graph is not bipartite

3. **Repeat for Disconnected Components:**

- If there are uncolored vertices remaining, pick one and repeat the BFS process

4. **Result:**

- If no conflicts were found, the graph is bipartite
- Otherwise, the graph is not bipartite

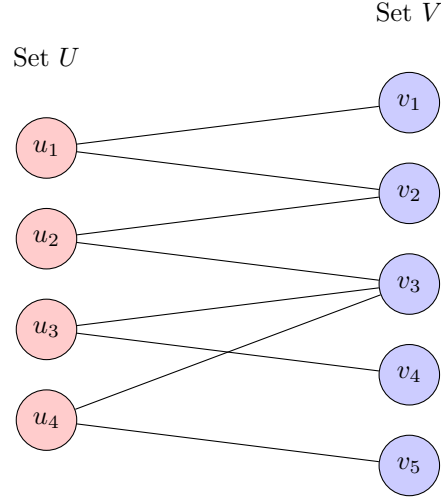


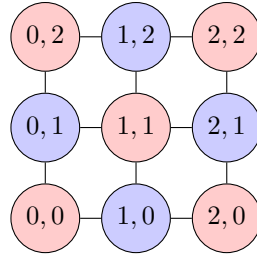
Figure 73: A bipartite graph with partite sets U and V

Example 10. A grid graph $P_m \times P_n$ is bipartite for any positive integers m and n .

Proof. A grid graph can be partitioned into two sets based on the parity of the sum of the row and column indices:

- Set A : Vertices (i, j) where $i + j$ is even
- Set B : Vertices (i, j) where $i + j$ is odd

Any edge in the grid connects a vertex (i, j) to either $(i + 1, j)$ or $(i, j + 1)$. In either case, the sum of the indices changes parity, meaning the edge connects a vertex in set A to a vertex in set B . Thus, there are no edges within set A or within set B , which proves the graph is bipartite. $\square$



3×3 Grid Graph

Figure 74: A 3×3 grid graph with bipartite coloring. Vertices are labeled with their coordinates (i, j) , and colored based on whether $i + j$ is even (red) or odd (blue).

Lemma 42. A cycle graph C_n is bipartite if and only if n is even.

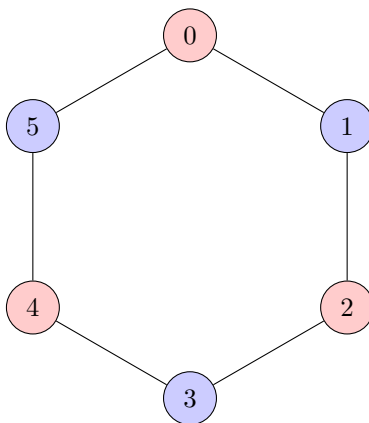
Proof. For a cycle with even length n , we can alternate colors around the cycle, placing adjacent vertices in different sets:

- Set A : Vertices at even positions $(0, 2, 4, \dots)$
- Set B : Vertices at odd positions $(1, 3, 5, \dots)$

Since n is even, when we return to the starting vertex after traversing the cycle, we will have used exactly $n/2$ vertices from set A and $n/2$ vertices from set B . The last vertex (at position $n - 1$) connects to the first

vertex (at position 0), and they belong to different sets because $n - 1$ has different parity than 0 when n is even.

For an odd cycle, it's impossible to create such a bipartition because the last vertex would need to connect to the first vertex, but they would have the same parity, forcing them into the same set—which violates the bipartite property. $\square$



Even Cycle C_6

Figure 75: An even cycle (C_6) with bipartite coloring. Even-indexed vertices are colored red, and odd-indexed vertices are colored blue.

Definition 51 (Distance Biregular Graph). A bipartite graph $G = (X \cup Y, E)$ is distance biregular if all vertices in X have degree k and all vertices in Y have degree l , where generally $k \neq l$.

X (degree $k = 2$) Y (degree $l = 3$)

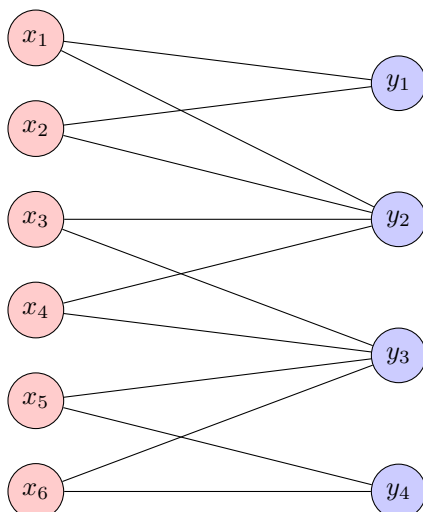


Figure 76: A $(2, 3)$ -biregular bipartite graph with $|X| = 6$ and $|Y| = 4$

5.10 Chapter 5 Exercises

5.10.1 Easy Questions

1. State the theorem on minimum cycle length in a graph with minimum degree $k \geq 2$. What is the guaranteed minimum cycle length?
2. In a graph with minimum degree 5, what is the minimum cycle length guaranteed by the long cycle theorem?
3. Prove that a graph with maximum degree 2 cannot have a cycle of length greater than the number of vertices.
4. What is the minimum cycle length in a 3-regular graph with 8 vertices? Is this bound tight? Explain.
5. Explain why the minimum cycle length in a graph must be at least 3.
6. Find the length of the longest cycle in the graph $K_{2,4}$ (complete bipartite graph with partite sets of sizes 2 and 4).
7. Prove that any graph with minimum degree $\delta(G) \geq 2$ must contain at least one cycle.

5.10.2 Medium Questions

8. Let P be a longest path in a graph G with minimum degree $k \geq 2$. Prove that the endpoints of P have all their neighbors on P .
9. Explain why the bound in the long cycle theorem is tight by constructing an example of a graph with minimum degree k where the longest cycle has length exactly $k + 1$.
10. Prove that in a 2-connected graph with minimum degree $k \geq 2$, there exists a cycle of length at least $\min\{2k, n\}$.
11. A graph G has minimum degree $k = 3$. Describe an algorithm to find a cycle of length at least $k + 1 = 4$ in G .
12. Let G be a graph with minimum degree $\delta(G) = 3$. If G has exactly 6 vertices, show that G must contain a cycle of length at least 4.
13. Find the relationship between the length of the longest cycle in a graph G and the length of the longest cycle in its complement $\overline{G}$.
14. Prove that in a Hamiltonian graph, the length of the longest cycle is exactly n . Then show that the converse is not true by providing a counterexample.

5.10.3 Hard Questions

15. Let G be a graph with minimum degree $\delta(G) \geq \frac{n}{r+1} + 1$ where $r \geq 2$. Prove that G contains a cycle of length at least $r + 2$.
16. Prove or disprove: if G is a k -connected graph with minimum degree $\delta(G) \geq k + 1$, then G contains a cycle of length at least $2k + 2$.
17. Let G be a graph with minimum degree $k \geq 2$. Prove that if G has girth g (the length of its shortest cycle), then G has a cycle of length at least $\min\{2k, g + k - 1\}$.
18. Design an algorithm to find the longest cycle in a graph G . Analyze its time complexity and explain why finding the longest cycle is generally an NP-hard problem.
19. Let G be a graph with n vertices and let $\lambda(G)$ denote the length of the longest cycle in G . Prove that if $\delta(G) > \frac{n}{2}$, then $\lambda(G) \geq 2\delta(G)$ unless G is Hamiltonian.
20. Prove that if G is a 3-connected cubic graph (3-regular), then G contains a cycle of length at least $4 \log n$.

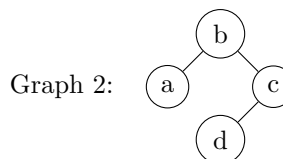
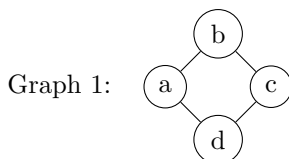
5.10.4 Diagram Questions

21. Draw a graph with 8 vertices and minimum degree 3 that contains a cycle of length exactly 4 but no longer cycles. Mark the cycle in your diagram.
22. Illustrate the proof technique of the long cycle theorem by drawing:
 - (a) A longest path P in a graph with endpoints u and v
 - (b) The neighbors of v on the path P
 - (c) The earliest neighbor of v on the path
 - (d) The resulting cycle formed by the theorem

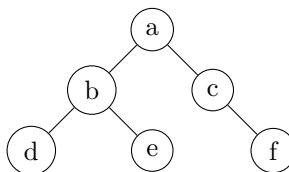
Label all relevant vertices and explain each step.
23. Draw a star graph $K_{1,4}$ and explain why the longest cycle in this graph has length 3. Then add one edge to create a cycle of length 4, and explain why this is now the longest possible cycle.
24. Create a sequence of graphs G_3, G_4, G_5 where each G_k has minimum degree k and contains a cycle of length exactly $k + 1$. Draw each graph and highlight the longest cycle in each.
25. Draw the Petersen graph and identify the longest cycle in it. Explain why this graph does not have a Hamiltonian cycle despite having minimum degree 3.
26. Draw a graph where the bound from the long cycle theorem is not tight, i.e., a graph with minimum degree k where the minimum cycle length is strictly greater than $k + 1$. Explain why this happens in your example.
27. Illustrate the relationship between the minimum degree and cycle length by drawing three different graphs with 6 vertices that have minimum degrees 2, 3, and 4 respectively. For each graph, highlight the longest cycle and discuss how the minimum degree affects the cycle length.

5.10.5 Visual Questions

1. For each of the following graphs, determine whether it is a tree and explain your reasoning:

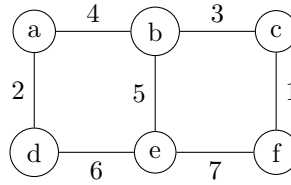


2. Consider the following tree:



Find all leaves, identify the center of the tree, and determine its diameter.

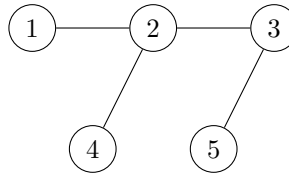
3. Draw all non-isomorphic trees with exactly 5 vertices.
4. Given the following weighted tree, find a minimum spanning tree and calculate its total weight:



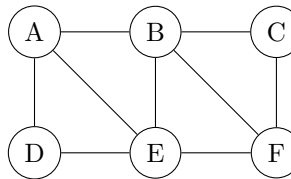
5. Determine the Prüfer code for the following tree:

(Hint 1: The Prüfer algorithm finds a unique sequence of $n-2$ integers for a labeled tree with n vertices. To construct the Prüfer code: repeatedly find the leaf with the smallest label, add its neighbor's label to the code, and remove the leaf from the tree. Continue until only 2 vertices remain.)

(Hint 2: The Prüfer algorithm's pseudo code is given by: 1. Initialize an empty sequence P 2. While the tree has more than 2 vertices: a. Find the leaf with the smallest label, call it v b. Let u be the only neighbor of v c. Add the label of u to sequence P d. Remove vertex v and its incident edge from the tree 3. Return the sequence P)

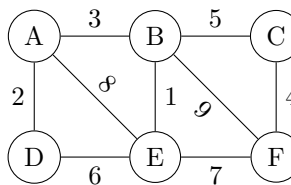


6. Draw a graph with 6 vertices that shows the difference between a spanning tree and a minimum spanning tree. Label the edge weights clearly.
7. Given the following graph:

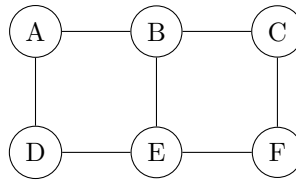


Illustrate step by step how Kruskal's algorithm would find a minimum spanning tree. Assume all edges have weight 1.

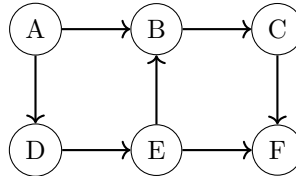
8. Draw a graph with exactly one bridge and explain how to identify it using the depth-first search algorithm. Show the discovery time and low values for each vertex.
9. Construct a graph with 7 vertices where the removal of any 2 vertices disconnects the graph, but the removal of any single vertex does not. Explain how this relates to vertex connectivity.
10. Illustrate Prim's algorithm step-by-step on the following weighted graph, starting from vertex A:



12. For the following graph, perform both BFS and DFS starting from vertex A. Show the resulting traversal order and the BFS/DFS trees:

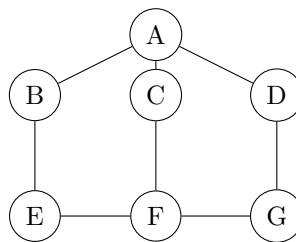


13. Consider the following directed graph:

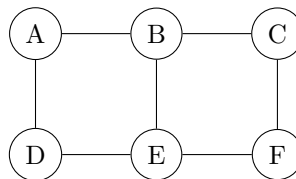


Perform a DFS traversal starting from vertex A and classify each edge as a tree edge, back edge, forward edge, or cross edge.

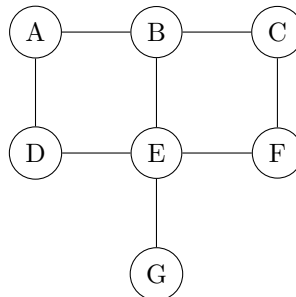
14. Explain the difference between BFS and DFS traversals by showing both traversal trees for the following graph, starting from vertex A:



15. Use BFS to determine if the following graph is bipartite. If it is, show the two partite sets with different colors:

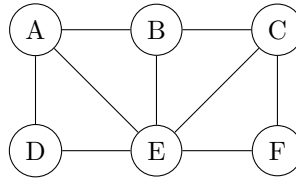


16. Find all articulation points in the following graph using the DFS algorithm. Show the discovery time and low value for each vertex:



6. Draw a directed graph with at least 8 vertices that has exactly 3 strongly connected components. Use DFS to identify these components.

7. Create a graph where the BFS tree and DFS tree (starting from the same vertex) are completely different. Highlight both trees in your diagram.
8. Illustrate the concept of DFS numbering and low values by drawing a graph with at least one bridge and one articulation point. Show the DFS tree with discovery times and low values for each vertex.
9. Draw an example showing how BFS can be used to find the shortest path between two vertices in an unweighted graph. Label the distances from the source vertex.
10. For the following graph, demonstrate how to detect cycles using DFS:



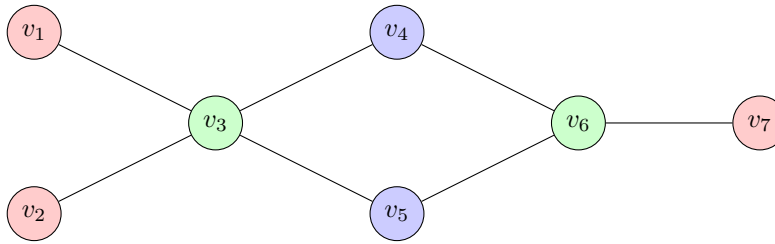
6 Structure of Connected Graphs

6.1 Blocks, Cut Vertices, and Connectedness

Definition 52 (Connected Graphs). A graph G is *connected* if for every pair of vertices $u, v \in V(G)$, there exists a path from u to v in G .

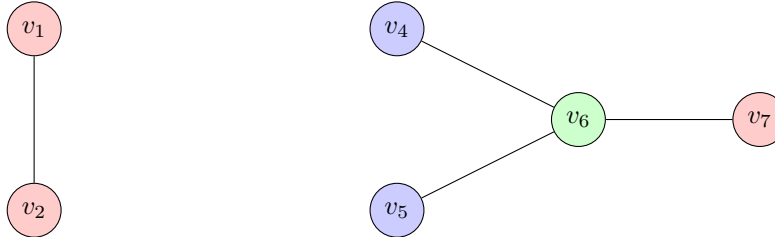
Definition 53 (Disconnected Graphs). A graph G is *disconnected* if there exist vertices $u, v \in V(G)$ such that there is no path from u to v in G .

Definition 54 (Cut Vertex). Let G be a connected graph. A vertex $v \in V(G)$ is called a *cut vertex* if the graph $G - \{v\}$ is disconnected.



Graph G with cut vertices v_3 and v_6

Figure 77: A graph G with cut vertices v_3 and v_6 (colored green). Removing either vertex disconnects the graph.



$G - \{v_3\}$ is disconnected

Figure 78: When vertex v_3 is removed, the graph becomes disconnected into separate components.

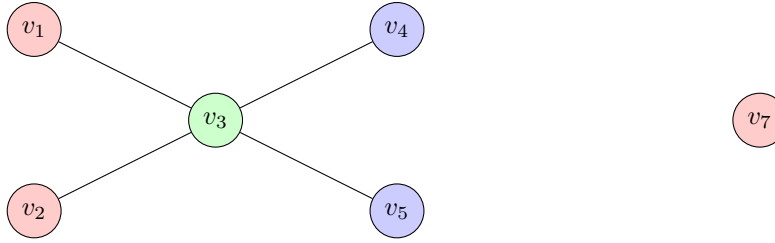
Definition 55 (Block). A block in a graph G is a maximal connected subgraph with no cut vertex.

Lemma 43. Every block of a graph is either a maximal 2-connected subgraph, or a bridge (an edge whose removal disconnects the graph), or an isolated vertex.

Remark 60. The blocks of a graph form a partition of its edges, meaning each edge belongs to exactly one block.

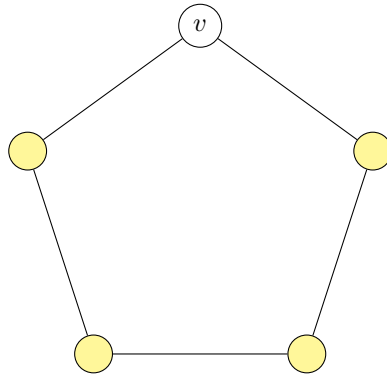
Remark 61. A vertex can belong to multiple blocks, and such a vertex must be a cut vertex of the graph.

Example 11. Consider the cycle graph C_5 shown below:



$G - \{v_6\}$ is disconnected

Figure 79: When vertex v_6 is removed, the graph becomes disconnected, isolating vertex v_7 .



C_5 has no cut vertices.

$C_5 - \{v\}$ is P_4

Figure 80: Cycle graph C_5 with vertex v highlighted. Removing any vertex from C_5 results in a path graph P_4 .

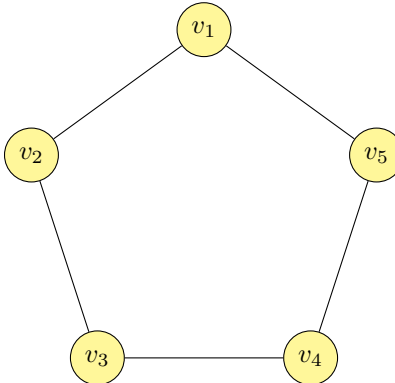


Figure 81: C_5 : The entire graph is a single block;
For the cycle graph C_5 :

- C_5 has no cut vertices.
- The entire graph is a single block (highlighted in yellow).
- For any vertex v in C_5 , the graph $C_5 - \{v\}$ is a path P_4 , which is connected.

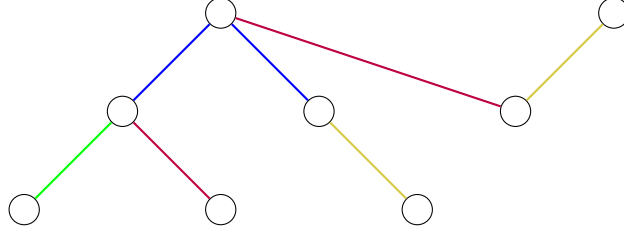
Lemma 44. In a cycle graph C_n with $n \geq 3$, the entire graph forms a single block.

Remark 62. The blocks of a connected graph are the "building blocks" that can be used to reconstruct the original graph. Understanding the block structure of a graph is fundamental to many graph algorithms and applications.

For a general connected graph G , we can identify its blocks as follows:

1. Identify all cut vertices of G .
2. Remove the cut vertices (one at a time) to find the connected components.
3. Each maximal subgraph with no cut vertices is a block.

Example 12. If T is a tree, then all blocks are K_2 (i.e., single edges).



Graph T

Figure 82: This graph T illustrates a fundamental property of trees in graph theory. It is a connected acyclic graph with multiple vertices joined by edges that are highlighted in different colors (blue, purple, green, and yellow). Each colored edge represents a distinct block in the graph.

Definition 56 (Block-Cutpoint Graph). Given a connected graph G , with blocks $B_1, B_2, \dots, B_k$ and cut vertices $v_1, v_2, \dots, v_c$, we form a new graph $\mathcal{B}(G)$ (also called the block-cutpoint graph of G) where:

$$V(\mathcal{B}(G)) = \{B_1, B_2, \dots, B_k, v_1, \dots, v_c\} \quad (32)$$

$$E(\mathcal{B}(G)) = \{\{B_i, v_j\} : v_j \in B_i\} \quad (33)$$

That is, $\mathcal{B}(G)$ has vertices corresponding to the blocks and cut vertices of G , and there is an edge between a block vertex B_i and a cut vertex v_j if and only if the cut vertex v_j belongs to the block B_i in the original graph G .

Theorem 49. For any connected graph G , its block-cutpoint graph $\mathcal{B}(G)$ is a tree.

Example 13. Consider a graph G that resembles a dumbbell (two triangles connected by a path). The block-cutpoint graph $\mathcal{B}(G)$ would have vertices representing the two triangular blocks, the edges of the connecting path as blocks, and the cut vertices that connect these blocks.

Example 14. Consider the following graph G with three blocks and two cut vertices:

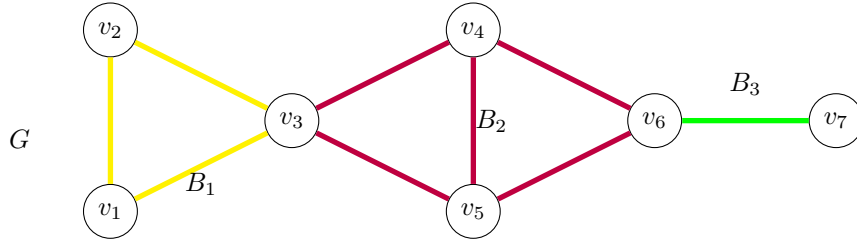


Figure 83: The cut vertices are $\{v_3, v_6\}$. The blocks are B_1, B_2, B_3 . The block-cutpoint graph $\mathcal{B}(G)$

Remark 63. The block-cutpoint graph $\mathcal{B}(G)$ is a bipartite graph, with one partition consisting of the blocks and the other partition consisting of the cut vertices.

Remark 64. The block-cutpoint graph is a useful tool for analyzing the structure of a graph. Many graph algorithms, such as finding bridges and articulation points, can be simplified by working with the block-cutpoint graph.

Theorem 50. A graph G is 2-connected if and only if it has at least 3 vertices and $\mathcal{B}(G)$ consists of a single block vertex (with no cut vertices).

Corollary 23. A graph G has no cut vertices if and only if either G is disconnected and each component is 2-connected, or G is connected and $\mathcal{B}(G)$ consists of a single vertex.

Block-Cutpoint Graph $\mathcal{B}(G)$

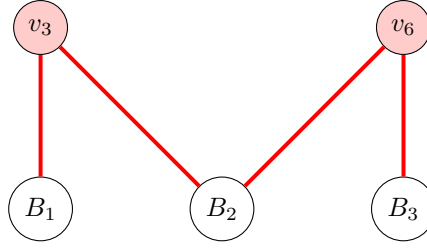


figure 84: The block-cutpoint graph represents the structure of G by showing how its blocks connect through cut vertices. Note that $\mathcal{B}(G)$ forms a tree with block vertices (white) and cut vertices (red).;

Theorem 51 (Tree Structure). The block-cutpoint graph $\mathcal{B}(G)$ of a connected graph G is always a tree.

Theorem 52 (Bipartite Property). The block-cutpoint graph $\mathcal{B}(G)$ is bipartite, with one partition consisting of block vertices and the other of cut vertices.

Lemma 45 (Leaf Property). Every leaf vertex in $\mathcal{B}(G)$ corresponds to a block in G , never to a cut vertex.

Lemma 46 (Path Uniqueness). For any two blocks B_i and B_j in G , there exists a unique path between the corresponding block vertices in $\mathcal{B}(G)$.

Remark 65 (Block Separation). *If v is a cut vertex in G , then removing v from G increases the number of connected components by exactly the number of blocks containing v minus 1.*

Remark 66 (Reconstruction). *The original graph G can be reconstructed from its block-cutpoint graph $\mathcal{B}(G)$ if we know the structure of each block.*

Corollary 24 (2-Connectivity). A graph G is 2-connected if and only if G has at least three vertices and $\mathcal{B}(G)$ consists of a single block vertex.

Corollary 25 (Distance Property). The distance between any two blocks B_i and B_j in $\mathcal{B}(G)$ is equal to twice the minimum number of cut vertices that must be traversed to go from a vertex in B_i to a vertex in B_j in G .

Theorem 53. For any connected graph G , the block graph $\mathcal{B}(G)$ is a tree.

Proof. We need the block graph $\mathcal{B}(G)$ to be connected. If $\mathcal{B}(G)$ has two components H_1 and H_2 , then for any block $B \in H_1$ and any block $B' \in H_2$, pick any vertex $v \in V(B)$ and $v' \in V(B')$, there is no v - v' path in G . So v and v' are in different components of G , contradicting that G is connected.

To show $\mathcal{B}(G)$ is a tree, we must show $\mathcal{B}(G)$ has no cycles.

Suppose C is a cycle in $\mathcal{B}(G)$. Then

$$\begin{aligned} V(C) &= \{v_1, B_1, v_2, B_2, \dots, v_r, B_r\} \\ E(C) &= \{\{v_1, B_1\}, \{B_1, v_2\}, \{v_2, B_2\}, \dots, \{v_r, B_r\}, \{B_r, v_1\}\} \end{aligned}$$

We claim $B = B_1 \cup B_2 \cup \dots \cup B_r \subseteq G$ has no cut vertex. This is a contradiction since each B_i is a block, but properly contained in B .

If we remove any vertex in this picture, we get a connected graph. So B has no cut vertex.

So $\mathcal{B}(G)$ has no cycles; it is a tree. □

intuition: This proof demonstrates why the block graph must be a tree. When we construct a block graph from our original graph, we're creating a structure that shows how the blocks and cut vertices are connected to each other.

First, the block graph must be connected because the original graph is connected. If the block graph weren't connected, that would mean some blocks in the original graph couldn't be reached from others,

contradicting our assumption that the original graph is connected. The key insight is understanding why the block graph cannot contain cycles. If it did contain a cycle, this cycle would alternate between blocks and cut vertices (since in the block graph, blocks connect only to cut vertices and vice versa). Following this cycle in the original graph would give us a sequence of blocks $B_1, B_2, \dots, B_r$, where consecutive pairs share a cut vertex. If we take the union of all these blocks, $B = B_1 \cup B_2 \cup \dots \cup B_r$, we'd get a larger subgraph. The contradiction arises because this union B would have no cut vertex! Any vertex that was a cut vertex for one of the original blocks is no longer a cut vertex in the union, because the cycle structure provides an alternate path around any potential cut vertex. This contradicts the definition of blocks as maximal 2-connected subgraphs. We've found a strictly larger 2-connected subgraph containing one of our blocks, which is impossible by the definition of maximality. Since we've reached a contradiction, our assumption must be wrong—the block graph cannot contain any cycles. And a connected graph with no cycles is precisely the definition of a tree. In essence, the tree structure of the block graph reflects how the original graph is “held together” by its cut vertices. Each block is like a solid component, and the cut vertices are the connectors that bind these components together in a tree-like arrangement.

Theorem 54. Let G be a connected graph containing no Θ -graph. Then G is a tree or a cycle.

A Θ -graph is a graph consisting of three uv -paths P , Q and R such that

$$V(P) \cap V(Q) = \{u, v\}$$

$$V(Q) \cap V(R) = \{u, v\}$$

$$V(R) \cap V(P) = \{u, v\}$$

A **tree of cycles** is a connected graph whose blocks are all cycles or K_2 .

Proof. It is enough to prove every block of G is a cycle or K_2 , by the last theorem.

Let B be a block of G . If $B \neq K_2$ and $B \neq$ a cycle, then B has a vertex of degree at least three.

Block B has a cycle C containing u , where $d(u) \geq 3$.

Since u is not a cut vertex of B , $B - \{u\}$ is connected. So there is a path P in $B - \{u\}$ from a neighbor of u not in C to a vertex v on the cycle.

Now P , Q , R form a Θ -graph, where $Q \cup R = C$ and Q and R are uv -paths. This is a contradiction.

So $B = K_2$ or $B =$ a cycle. $\square$

In other words: This proposition reveals something fascinating about graphs with a certain structural constraint. It tells us that if our graph doesn't contain a specific forbidden pattern (three distinct paths connecting the same two points), then the entire graph must be very simple - either just a tree or a single cycle.

Think about the forbidden pattern this way: imagine two cities connected by three completely separate roads. This creates a structure with multiple alternate routes between the same two points. What our theorem 54 says is that if a network doesn't have any such triple-path connections anywhere, the entire network must be either a tree (no loops at all) or just a single loop.

The proof uses a clever approach based on our previous result about block graphs. Remember that blocks are the maximal 2-connected pieces of a graph. The key insight is showing that each block in our graph must be either just a single edge or a cycle - nothing more complex is possible.

Here's why: If any block were more complex than just an edge or cycle, it would contain a vertex connecting to at least three other vertices. This high-degree vertex would be part of some cycle, but would also connect to something outside this cycle. Since the block is 2-connected, we could find a path that avoids this vertex, connecting its "outside neighbor" to the original cycle. But this would create exactly the triple-path pattern we're not allowed to have!

This contradiction tells us each block must be just an edge or a cycle. When we connect these simple blocks together at cut vertices, we get what's called a "tree of cycles" - essentially a tree-like structure where the "nodes" are either single edges or cycles.

Since the block graph forms a tree (from our previous theorem), the entire graph is either a single block (so just a cycle or a single edge) or multiple simple blocks arranged in a tree structure. Either way, the graph must be either a tree or a cycle, just as the proposition claims.

This elegant result completely characterizes graphs that avoid the triple-path pattern, showing they must have a remarkably simple structure.

6.2 Menger's Theorem

Theorem 55 (Menger's Theorem). Let G be a graph and u, v be two non-adjacent vertices in G . Then the minimum number of vertices whose removal disconnects u from v equals the maximum number of pairwise internally vertex-disjoint u - v paths in G .

Theorem 56 (Menger's Theorem - Vertex Form). For any graph G , and non-adjacent $u, v \in V(G)$, $\kappa(u, v)$ is equal to the maximum number of uv -paths $P_1, P_2, \dots, P_k$ such that

$$V(P_i) \cap V(P_j) = \{u, v\}$$

for all $i \neq j$.

Proof. Let $\kappa(u, v)$ be the minimum size of a uv -separator, and let $\pi(u, v)$ be the maximum number of internally vertex-disjoint uv -paths. We need to show that $\kappa(u, v) = \pi(u, v)$.

First, we prove that $\kappa(u, v) \geq \pi(u, v)$.

Consider any set of $\pi(u, v)$ internally vertex-disjoint uv -paths. Any uv -separator S must include at least one vertex from each path (excluding u and v), otherwise there would still be a uv -path after removing S . Since the paths are internally vertex-disjoint, S must contain at least $\pi(u, v)$ vertices. Therefore, $\kappa(u, v) \geq \pi(u, v)$.

Next, we prove that $\kappa(u, v) \leq \pi(u, v)$ by using induction on $|E(G)|$.

Base case: If G has no edges, then $\kappa(u, v) = 0 = \pi(u, v)$.

Inductive step: Assume the theorem holds for all graphs with fewer than $|E(G)|$ edges.

If $\kappa(u, v) = 0$, then u and v are in different components, so $\pi(u, v) = 0$ and we're done.

If $\kappa(u, v) > 0$, then there exists at least one uv -path. Let P be a shortest uv -path.

Case 1: If P consists of just the edge uv , remove this edge to get graph G' . By the induction hypothesis, $\kappa_{G'}(u, v) = \pi_{G'}(u, v)$. Note that $\kappa_G(u, v) = \kappa_{G'}(u, v) + 1$ and $\pi_G(u, v) = \pi_{G'}(u, v) + 1$. So $\kappa_G(u, v) = \pi_G(u, v)$.

Case 2: If $P = (u, w_1, w_2, \dots, w_k, v)$ where $k \geq 1$:

Let's modify G to create a new graph G' :

- Remove edge uw_1
- Add a new vertex w'
- Add edges uw' and $w'w_1$

This doesn't change $\kappa(u, v)$ or $\pi(u, v)$ because any uv -separator that contained w_1 can use w' instead, and any set of disjoint paths can be modified accordingly.

By repeating this process for each internal vertex of P , we get a graph where P has been replaced by a path where all internal vertices have degree 2 (they only connect to their neighbors on the path).

Now let's contract the path P into a single edge uv to get a graph G'' .

By the induction hypothesis, $\kappa_{G''}(u, v) = \pi_{G''}(u, v)$.

It can be shown that $\kappa_G(u, v) = \kappa_{G''}(u, v) + 1$ and $\pi_G(u, v) = \pi_{G''}(u, v) + 1$ (because P provides one additional path, and any minimum separator must include one vertex from P).

Therefore, $\kappa_G(u, v) = \pi_G(u, v)$.

This completes the induction, proving that $\kappa(u, v) = \pi(u, v)$ for all graphs. $\square$

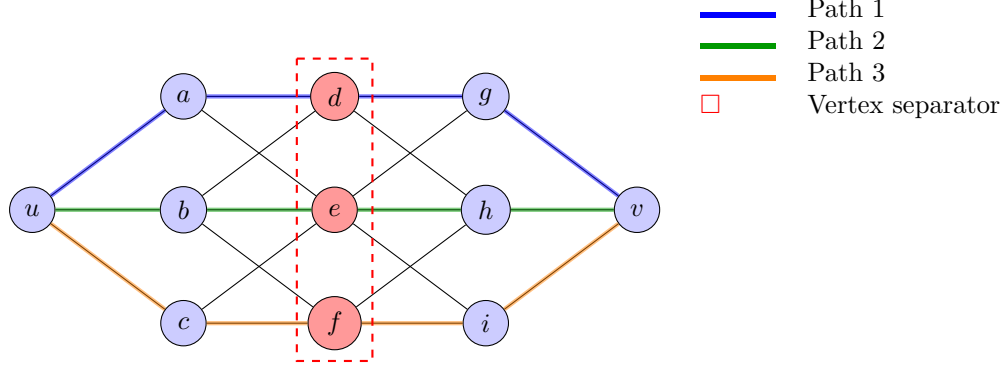
Definition 57. For $u, v \in V(G)$, a uv -edge-separator is a set $L \subseteq E(G)$ such that u and v are in different components of $G - L$.

Theorem 57 (Menger's Theorem - Edge Form). Let G be a graph and u, v be two vertices in G . Then the minimum number of edges whose removal disconnects u from v equals the maximum number of pairwise edge-disjoint u - v paths in G .

Definition 58. A graph G is k -connected if and only if $|V(G)| > k$ and for every set $S \subset V(G)$ with $|S| < k$, the graph $G - S$ is connected.

Lemma 47. A block is a 2-connected graph, except for K_2 .

Menger's Theorem - Example



This graph illustrates Menger's Theorem with $\kappa(u, v) = \pi(u, v) = 3$.

The three colored paths are internally vertex-disjoint.

The minimum vertex separator (vertices d, e, f) has size 3.

figure 85: An example illustrating Menger's Theorem with three internally vertex-disjoint paths and a minimum separator of size 3.

Definition 59. A uv -separator in a graph G with non-adjacent vertices $u, v \in V(G)$ is a set $X \subseteq V(G)$ such that u and v are in different components of $G - X$.

Definition 60. An **edge-cut** of a graph G is a collection of edges from $E(G)$ whose removal disconnects the graph.

Remark 67. Intuitively, an edge-cut is a collection of edges that, when removed from the graph, breaks it into separate pieces. Think of these edges as bridges between different regions of the graph—remove these bridges, and you can no longer travel between the regions. In real-world networks, edge-cuts represent vulnerable connection points; for example, in a communication network, an edge-cut might represent a set of communication links whose failure would divide the network into isolated parts.

Remark 68. To understand a uv -edge-separator, imagine two specific locations in a transportation network (vertices u and v). A uv -edge-separator is the smallest set of roads (edges) that, if closed, would make it impossible to travel between these two locations. The size of this separator tells us how robust the connection is between u and v —a larger separator means more alternative routes exist. Menger's Theorem elegantly shows that this robustness measure exactly equals the maximum number of separate paths we can find between u and v .

Lemma 48. If S is a minimal uv -separator in a graph G , then every vertex in S has at least one neighbor in each component of $G - S$ containing either u or v .

Remark 69. This lemma reveals an important property of minimal separators: every vertex in a minimal separator plays a critical role in the disconnection. If any vertex in the separator didn't have neighbors in both the u -component and v -component, we could remove it and still maintain the separation, contradicting minimality.

Corollary 26. For any non-adjacent vertices u and v in a graph G , the size of a minimum uv -separator equals the maximum number of internally disjoint uv -paths if and only if all internally disjoint uv -paths have minimum length.

Lemma 49 (Block Characterization). A graph is 2-connected if and only if it can be constructed from a cycle by successively adding an ear (a path that connects two existing vertices and is otherwise disjoint from the graph).

Corollary 27 (Cut Vertex Characterization). A vertex v in a connected graph G is a cut vertex if and only if there exist vertices u and w different from v such that v belongs to every uw -path in G .

Menger's Theorem - Before and After Removing Separator

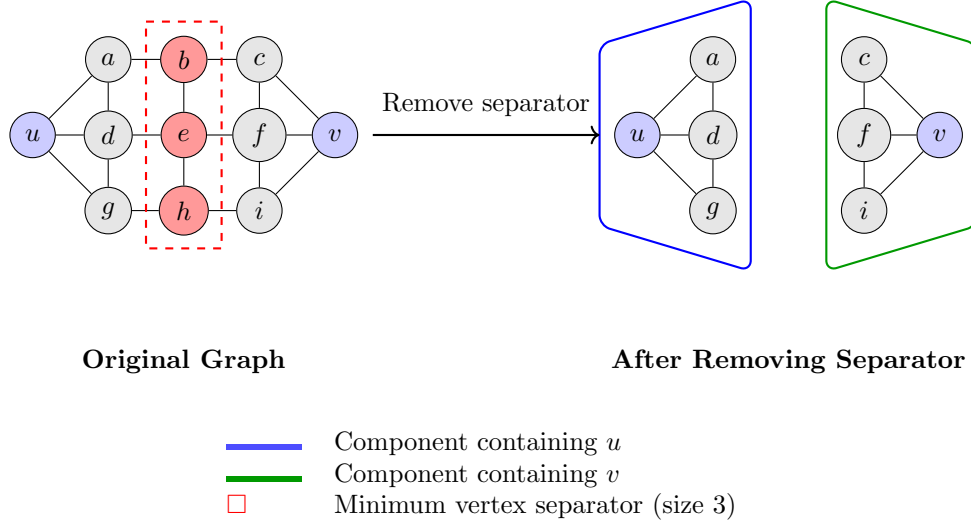


figure 86 : Illustration of a vertex separator in Menger's Theorem. When the minimum vertex separator (vertices b, e, h) is removed, vertices u and v become disconnected. The size of this separator equals the maximum number of internally vertex-disjoint paths between u and v .

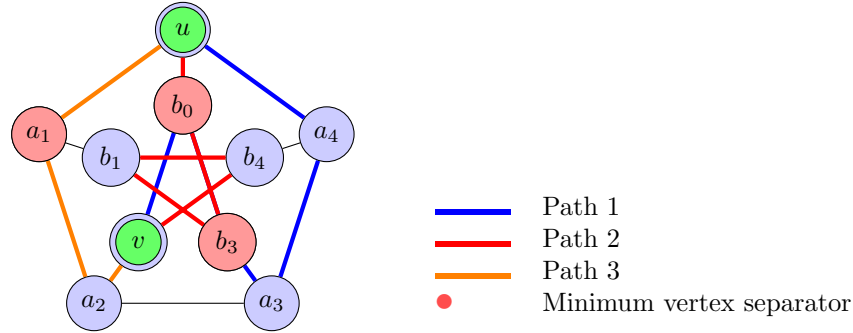


figure 87: Application of Menger's Theorem to the Petersen graph. Three internally vertex-disjoint paths are highlighted between vertices u and v . The minimum vertex separator (red vertices) has size 3, confirming that $\kappa(u, v) = \pi(u, v) = 3$.

Definition 61. For any vertices $u, v \in V(G)$, non-adjacent, let $\kappa(u, v)$ be the minimum size of a uv -separator.

Definition 62. A graph is k -connected if and only if $\kappa(u, v) \geq k$ for all u, v in the graph.

Definition 63. A graph is k -connected if and only if it is K_n , $n \geq k + 1$, or for any two vertices u, v in the graph, there exist k internally disjoint uv -paths.

Lemma 50 (Whitney's Theorem). A graph is k -connected if and only if it has at least $k+1$ vertices and the removal of any set of fewer than k vertices leaves the remaining subgraph connected.

Lemma 51 (Minimum Degree Bound). For any graph G , $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\kappa(G)$ is the vertex connectivity, $\lambda(G)$ is the edge connectivity, and $\delta(G)$ is the minimum degree of G .

Lemma 52 (Fan Lemma). If G is a k -connected graph and X is a set of at least k vertices, then from any vertex v not in X , there exist k internally disjoint paths from v to X (each path ends at a different vertex of X).

Lemma 53 (Expansion Lemma). In a k -connected graph G , if S is a set of k vertices, then every component of $G - S$ has at least k neighbors in S .

Example of a uv -separator

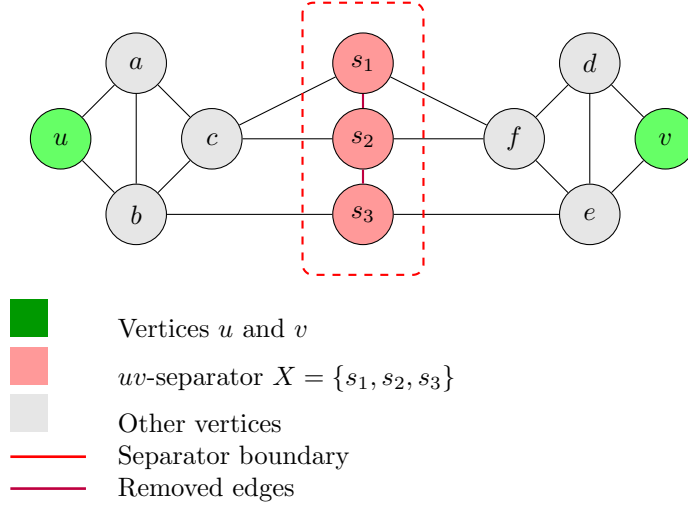


figure 88 : Example of a uv -separator $X = \{s_1, s_2, s_3\}$. When the vertices in X are removed, vertices u and v end up in different connected components of the resulting graph $G - X$.

Definition 64. The *edge-connectivity* between vertices u and v , denoted by $\lambda(u, v)$, is the minimum size of a uv -edge-separator.

Definition 65. Two paths are *edge-disjoint* if they do not share any edges (though they may share vertices).

Theorem 58 (Menger's Theorem (Edge Form)). For any vertices $u, v \in V(G)$, the minimum size $\lambda(u, v)$ of a uv -edge-separator equals the maximum number of edge-disjoint uv -paths in G .

This theorem has several important implications:

Corollary 28. A graph G is k -edge-connected if and only if for every pair of vertices $u, v \in V(G)$, there exist at least k edge-disjoint paths between u and v .

Example 15 (Cycle Graph). Consider a cycle graph C_n with $n \geq 3$ vertices. For any two vertices u and v on the cycle, there are exactly 2 edge-disjoint paths connecting them: one going clockwise and one going counterclockwise. Therefore, $\lambda(u, v) = 2$ for any u, v , and the cycle is 2-edge-connected.

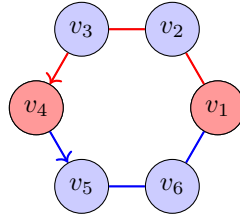


Figure 89 : C_6 with two edge-disjoint paths between v_1 and v_4

Example 16 (Complete Graph). In a complete graph K_n with $n \geq 2$ vertices, any two vertices u and v are connected by $n - 1$ edge-disjoint paths: one direct edge (u, v) and $n - 2$ paths of the form (u, w, v) for each vertex $w \neq u, v$. Therefore, $\lambda(u, v) = n - 1$ for any u, v , and K_n is $(n - 1)$ -edge-connected.

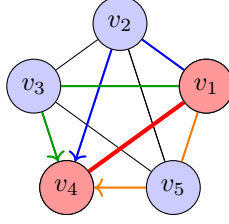


Figure 90 : K_5 with four edge-disjoint paths between v_1 and v_4

It's important to note the relationship between the edge and vertex forms of Menger's Theorem:

Theorem 59 (Relationship Between Edge and Vertex Connectivity). For any graph G , $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\kappa(G)$ is the vertex connectivity, $\lambda(G)$ is the edge connectivity, and $\delta(G)$ is the minimum degree of G .

Example 17 (Path Graph). Consider a path graph P_n with $n \geq 3$ vertices.

- Vertex connectivity: $\kappa(P_n) = 1$, since removing any internal vertex disconnects the graph.
- Edge connectivity: $\lambda(P_n) = 1$, since removing any edge disconnects the graph.

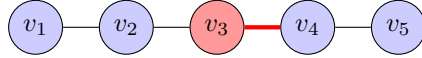


Figure 91: Removing vertex v_3 (in red) or edge (v_3, v_4) (red) disconnects the path graph, showing $\kappa(P_5) = \lambda(P_5) = 1$.

Example 18 (Cycle Graph). Consider a cycle graph C_n with $n \geq 3$ vertices.

- Vertex connectivity: $\kappa(C_n) = 2$, since we need to remove at least 2 vertices to disconnect the graph.
- Edge connectivity: $\lambda(C_n) = 2$, since we need to remove at least 2 edges to disconnect the graph.

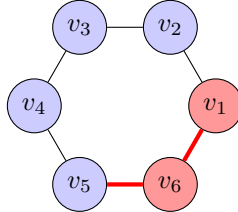


Figure 92: Removing vertices v_1 and v_6 (in red) or edges (v_5, v_6) and (v_6, v_1) (red) disconnects the cycle graph, showing $\kappa(C_6) = \lambda(C_6) = 2$.

Example 19 (Wheel Graph). Consider a wheel graph W_n with $n + 1$ vertices (a cycle with n vertices plus a central vertex connected to all cycle vertices).

- Vertex connectivity: $\kappa(W_n) = 3$ for $n \geq 4$, since removing the central vertex and any two adjacent cycle vertices disconnects the graph.
- Edge connectivity: $\lambda(W_n) = 3$ for $n \geq 4$, which equals the minimum degree of the graph.

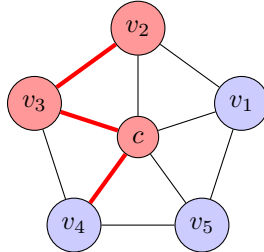


Figure 93: Removing center c and adjacent vertices v_2 and v_3 (in red) disconnects the wheel graph. Removing the three red edges also disconnects the graph, showing $\kappa(W_5) = \lambda(W_5) = 3$.

Example 20 (Petersen Graph). The Petersen graph is a 3-regular graph with 10 vertices.

- Vertex connectivity: $\kappa = 3$, since removing any 3 vertices can disconnect the graph, but no fewer will do so.
- Edge connectivity: $\lambda = 3$, matching the minimum degree.

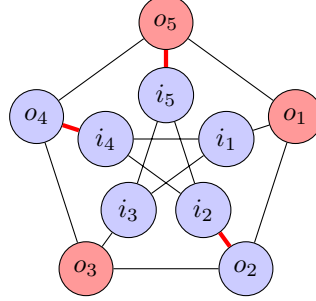


Figure 94: Removing vertices o_1 , o_3 , and o_5 (in red) disconnects the Petersen graph. Removing the three red edges also disconnects the graph, showing $\kappa = \lambda = 3$.

Example 21 (Grid Graph $P_2 \times P_n$). Consider a $2 \times n$ grid graph (ladder graph) with $n \geq 2$.

- Vertex connectivity: $\kappa = 2$, since removing any 2 vertices from different "rungs" can disconnect the graph.
- Edge connectivity: $\lambda = 2$, since removing any 2 edges from different "rungs" can disconnect the graph.

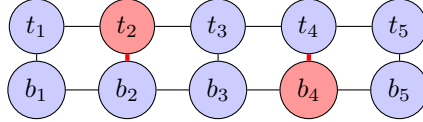


Figure 95: Removing vertices t_2 and b_4 (in red) disconnects the grid graph. Removing edges (t_2, b_2) and (t_4, b_4) (red) also disconnects the graph, showing $\kappa = \lambda = 2$.

Example 22 (Graph with $\kappa < \lambda$). Consider the graph formed by two copies of K_4 joined by two independent edges.

- Vertex connectivity: $\kappa = 2$, since removing the two endpoints of the connecting edges in one copy disconnects the graph.
- Edge connectivity: $\lambda = 3$, since we must remove both connecting edges plus at least one more edge to disconnect the graph.

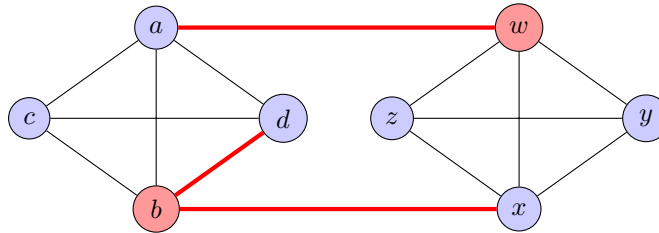


Figure 96 : Removing vertices b and w (in red) disconnects the graph, showing $\kappa = 2$. Removing the three red edges disconnects the graph, showing $\lambda = 3$.

Remark 70 (Connectivity Inequality). For any graph G , we have $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\delta(G)$ is the minimum degree. This inequality is tight, as demonstrated by complete graphs where $\kappa(K_n) = \lambda(K_n) = \delta(K_n) = n - 1$.

minimum edge-separator

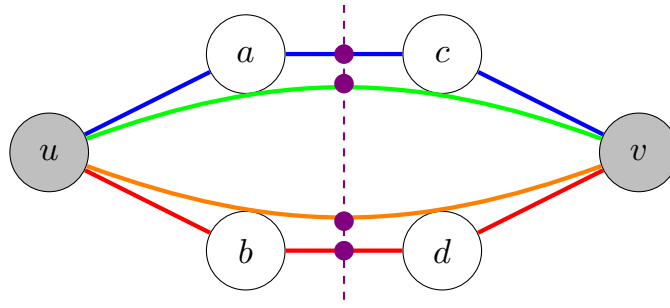


Figure 97 : Illustration of Menger's Theorem (Edge Form): The graph contains 4 edge-disjoint paths between u and v (shown in different colors). The minimum edge-separator (violet dashed line) cuts exactly 4 edges, demonstrating that $\lambda(u, v) = 4$.

Corollary 29 (Characterization of k -Connectivity). A graph G with at least $k + 1$ vertices is k -connected if and only if for every pair of distinct vertices u and v , there exist at least k internally vertex-disjoint paths between u and v .

Proof. By Menger's Theorem, the minimum size of a uv -separator equals the maximum number of internally vertex-disjoint uv -paths. A graph is k -connected if and only if the removal of fewer than k vertices does not disconnect the graph, which is equivalent to having at least k internally vertex-disjoint paths between any pair of vertices. $\square$

Theorem 60 (Menger's Theorem - Edge Form). For $u, v \in V(G)$, the minimum size $\lambda(u, v)$ of a uv -edge-separator equals the maximum number of edge-disjoint uv -paths in G .

Theorem 61. If G is a connected graph with a bridge, then $\lambda(u, v) = 1$ when $\{u, v\}$ is the bridge.

Corollary 30. If G is connected with no bridge, then $\lambda(u, v) \geq 2$ for every $\{u, v\} \subseteq V(G)$.

Definition 66. An **edge-cut** of a graph G is a set $L \subseteq E(G)$ such that $G - L$ is disconnected.

Definition 67. A graph G is k -edge-connected if and only if every edge cut in G has size at least k .

Theorem 62. The following statements are equivalent:

1. A graph is connected.
2. A graph is 1-edge-connected.

Theorem 63. The following statements are equivalent:

1. A graph is 2-edge-connected.
2. A graph is connected and has no bridge.
3. Every edge in the graph is part of at least one cycle.

Definition 68. If G is a graph, then the vertex connectivity $\kappa(G)$ is $n - 1$ if $G = K_n$, otherwise it is the minimum size of a vertex cut in G .

Definition 69. The edge-connectivity $\lambda(G)$ is the minimum size of an edge cut in G .

Example 23. For the cycle graph G_1 shown:

$$\lambda(G_1) = 2 \quad \kappa(G_1) = 2$$

For the complete bipartite graph $G_2 = K_{5,7}$ (simplified version of $K_{12,12}$ for clarity):

$$\text{For the actual } K_{12,12} \text{ graph described: } \lambda(G_2) = 7 \quad \kappa(G_2) = 5 \quad \delta(G_2) = 11$$

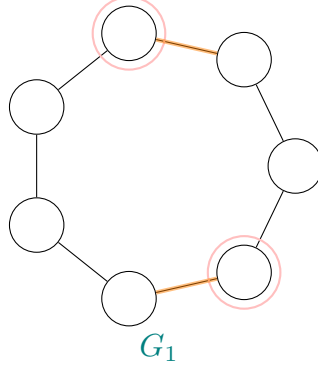


Figure 98 :A cycle graph with $\lambda(G_1) = 2$ and $\kappa(G_1) = 2$. The minimum vertex cut (two vertices highlighted in pink) and minimum edge cut (two edges highlighted in orange) both have size 2.

Corollary 31. For any graph G , $\kappa(G) \leq \lambda(G) \leq \delta(G)$.

Lemma 54. The vertex connectivity $\kappa(G)$ is $\min\{|X| : X \subseteq V(G) \text{ is a vertex cut}\}$.

Remark 71. Recall that a graph G is k -connected if and only if $\kappa(G) \geq k$.

Lemma 55. The edge-connectivity $\lambda(G)$ is $\min\{|L| : L \subseteq E(G) \text{ is an edge cut}\}$.

Remark 72. Recall that a graph G is k -edge-connected if and only if $\lambda(G) \geq k$.

Proof of Lemma 51 (Minimum Degree Bound). Recall that for any graph G , we need to prove that $\kappa(G) \leq \lambda(G) \leq \delta(G)$, where $\kappa(G)$ is the vertex connectivity, $\lambda(G)$ is the edge connectivity, and $\delta(G)$ is the minimum degree of G .

We first prove $\lambda(G) \leq \delta(G)$. Consider any vertex $v \in V(G)$ with $d(v) = \delta(G)$. Let $L = \{\{v, w\} : w \in N(v)\}$ be the set of all edges incident to v . This is an edge cut of size $\delta(G)$. Therefore, $\lambda(G) \leq \delta(G)$.

For $\kappa(G) \leq \lambda(G)$, we use Menger's theorem.

Recall Menger's Theorem in vertex form, which states that for non-adjacent vertices u, v in a graph G :

$$\kappa(u, v) = \max\{k : \text{there are } k \text{ internally disjoint } uv\text{-paths}\}.$$

Note that internally disjoint uv -paths are also edge-disjoint.

Using these facts, we can establish:

$$\begin{aligned} \kappa(G) &\leq \min_{\{u,v\} \notin E(G)} \max\{k : \text{there are } k \text{ edge-disjoint } uv\text{-paths}\} \\ &\leq \min_{u,v} \min\{|L| : L \subseteq E(G) \text{ is a } uv\text{-edge-separator}\} \\ &= \min\{|L| : L \subseteq E(G) \text{ is an edge cut}\} \\ &= \lambda(G) \end{aligned}$$

Therefore, $\kappa(G) \leq \lambda(G)$.

This completes the proof. □

Theorem 64. There exists a graph G such that given $k \leq \ell \leq d$, $\kappa(G) = k$, $\lambda(G) = \ell$, and $\delta(G) = d$.

Proof. We construct a graph G as follows:

1. Start with two complete graphs K_{k+1} , which we will call X and Y .
2. Select k vertices $\{x_1, x_2, \dots, x_k\}$ from X and k vertices $\{y_1, y_2, \dots, y_k\}$ from Y .

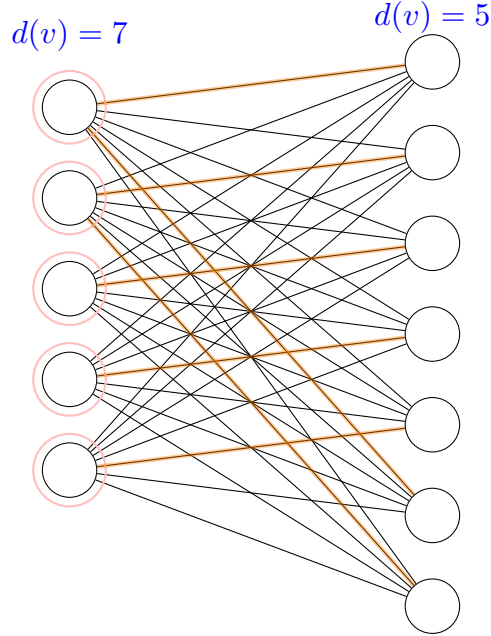


Figure 99 : A complete bipartite graph (simplified from $K_{12,12}$ to $K_{5,7}$ for clarity). The minimum vertex cut (vertices on left side, highlighted in pink) has size 5. The minimum edge cut (some edges highlighted in orange) has size 7. The minimum degree $\delta(G_2) = 5$.

3. Connect these selected vertices with edges such that:

- There are exactly $(\ell - k)$ edges between vertex x_i and vertex y_i for $i = 1, 2, \dots, k$
- Additionally, add edges between X and Y to ensure each vertex has degree at least d

4. Finally, add self-loops or additional edges within X and Y as needed to ensure every vertex has degree exactly d .

Now, we verify the three parameters:

For minimum degree $\delta(G)$:

By construction, every vertex has degree at least d , and we've ensured none has higher degree, so $\delta(G) = d$.

For edge connectivity $\lambda(G)$:

The total number of edges between X and Y is $k \cdot (\ell - k) + k = k \cdot (\ell - k + 1) = \ell$. Removing these ℓ edges disconnects the graph into two components (X and Y). We cannot disconnect the graph by removing fewer than ℓ edges because:

- Within X and Y , we have complete graphs K_{k+1} , which require more than ℓ edge removals to disconnect (since $k \leq \ell$)
- Between X and Y , we've constructed exactly ℓ edges

Therefore, $\lambda(G) = \ell$.

For vertex connectivity $\kappa(G)$:

If we remove the k vertices $\{x_1, x_2, \dots, x_k\}$ from X , the graph becomes disconnected because there are no remaining edges between X and Y . We cannot disconnect the graph by removing fewer than k vertices because:

- The k vertices in X that connect to Y form a complete graph with other vertices in X
- Removing fewer than k vertices will leave at least one vertex in X connected to Y

Therefore, $\kappa(G) = k$.

This completes the proof, showing that we can construct a graph G with $\kappa(G) = k$, $\lambda(G) = \ell$, and $\delta(G) = d$ for any values satisfying $k \leq \ell \leq d$. $\square$

6.3 Structure of Connected Graphs Exercises

6.3.1 Easy Questions

1. State the definition of a cut vertex. Give an example of a graph with exactly two cut vertices.
2. Define the terms “block” and “block-cutpoint graph” in the context of graph theory.
3. If a graph G has exactly one block, what can you say about G ? Justify your answer.
4. True or False: Every cut vertex in a connected graph must belong to at least two blocks. Prove your answer.
5. If T is a tree, characterize all of its blocks. How many blocks does a tree with n vertices have?
6. State Menger’s Theorem (vertex form). What is the relationship between the minimum number of vertices whose removal disconnects u from v and the maximum number of internally vertex-disjoint u - v paths?
7. Given a graph with $\kappa(G) = 2$, $\lambda(G) = 3$, and $\delta(G) = 4$, explain what each of these values represents.

6.3.2 Medium Questions

8. Let G be a connected graph with n vertices. If G has exactly $n - 1$ edges, prove that every edge of G is a block.
9. Prove that the block-cutpoint graph $\mathcal{B}(G)$ of a connected graph G is always a tree.
10. Let G be a graph with vertex connectivity $\kappa(G) = k$ and edge connectivity $\lambda(G) = \ell$. Prove that $k \leq \ell$.
11. If a graph G has minimum degree $\delta(G) = 3$ and vertex connectivity $\kappa(G) = 2$, construct a concrete example of such a graph with the minimum possible number of vertices.
12. Prove that in a 2-connected graph, any two vertices lie on a common cycle.
13. Let G be a connected graph. Prove that G is 2-connected if and only if for any three vertices u , v , and w , there exists a path from u to v that does not contain w .
14. A vertex v in a connected graph G is called an articulation point if removing v disconnects the graph. Prove that v is an articulation point if and only if there exist vertices a and b such that every path from a to b passes through v .

6.3.3 Hard Questions

15. Let G be a connected graph containing no Θ -graph. Prove that G is either a tree or a cycle.
16. Prove that if G is a k -connected graph, then for any set S of k vertices and any vertex $v \notin S$, there exist k internally vertex-disjoint paths from v to the vertices of S .
17. Given values $k \leq \ell \leq d$, construct a graph with vertex connectivity $\kappa(G) = k$, edge connectivity $\lambda(G) = \ell$, and minimum degree $\delta(G) = d$. Prove that your construction works.
18. Let G be a graph with $\lambda(G) = k$. Prove that if G has a vertex cut of size k , then G also has an edge cut of size k .
19. Let G be a graph with n vertices. Prove that if $\delta(G) > \frac{n-1}{2}$, then G is 2-connected.
20. Develop an algorithm to find all cut vertices in a connected graph G . Analyze its time complexity.
21. For a connected graph G , let $b(G)$ be the number of blocks and $c(G)$ be the number of cut vertices. Prove that if G is not a block, then $b(G) = c(G) + 1$.

6.3.4 Diagram Questions

22. Construct a graph G with vertex connectivity $\kappa(G) = 2$, edge connectivity $\lambda(G) = 3$, and minimum degree $\delta(G) = 4$. Clearly label the graph and explain why it satisfies the required properties.
23. Draw a graph with at least 3 blocks. Identify all cut vertices and blocks. Then draw its block-cutpoint graph $\mathcal{B}(G)$.
24. Draw a 3-connected graph with 8 vertices where every vertex has degree exactly 3. Highlight three internally vertex-disjoint paths between two non-adjacent vertices to illustrate Menger's Theorem.
25. Construct a graph that has exactly 3 cut vertices and 4 blocks. Draw the corresponding block-cutpoint graph.
26. Consider the Petersen graph. Choose two non-adjacent vertices u and v , and illustrate Menger's Theorem by highlighting: (a) a minimum vertex separator between u and v , and (b) the maximum number of internally vertex-disjoint paths between u and v .
27. Draw a graph where $\kappa(G) < \lambda(G) < \delta(G)$. Clearly indicate the values of each parameter and explain how your graph satisfies these inequalities.
28. Draw an example of a connected graph that contains no Θ -graph and is neither a tree nor a cycle. Explain why such a graph cannot exist, referring to the relevant theorem.

6.3.5 Additional Questions

29. Let C be a cut set of vertices in a connected graph G . Prove that if C is minimal (i.e., no proper subset of C is a cut set), then every vertex in C has at least one neighbor in each connected component of $G - C$.
30. Prove that a k -connected graph on n vertices must have at least $\lceil \frac{kn}{2} \rceil$ edges.
31. For any two distinct vertices u and v in a graph G , define $\kappa(u, v)$ as the minimum number of vertices whose removal disconnects u from v (or makes u and v belong to the same component if they are adjacent). Prove that $\kappa(G) = \min_{u, v \in V(G), u \neq v} \kappa(u, v)$.
32. If G is a k -connected graph and H is obtained from G by adding a new vertex adjacent to at least k vertices of G , prove that H is also k -connected.
33. Let G be a connected graph where every vertex has even degree. Prove that G has no bridges.
34. Let G be a graph where every vertex has degree at least k . Prove that G contains a path of length at least k .
35. Define a uv -separator in a graph G with non-adjacent vertices u and v as a set $X \subseteq V(G) \setminus \{u, v\}$ such that u and v are in different components of $G - X$. If S is a minimal uv -separator, prove that every vertex in S has at least one neighbor in the component of $G - S$ containing u and at least one neighbor in the component containing v .
36. Let G be a graph with vertex connectivity $\kappa(G) = k$. Prove that G has at least $k + 1$ vertices of degree at least k .
37. Prove that a graph G is 2-edge-connected if and only if it has an orientation such that the resulting directed graph is strongly connected.
38. Use Menger's Theorem to prove that if G is a k -connected graph and S is a subset of $V(G)$ with $|S| \geq k$, then for any vertex $v \notin S$, there exist k internally vertex-disjoint paths from v to k distinct vertices in S .
39. Let G be a 3-connected graph. Prove that any two cycles in G have a common vertex or there exists a path connecting them that is internally disjoint from both cycles.

40. A block-cut tree of a connected graph G is a tree where each vertex corresponds to either a block or a cut vertex of G , with edges connecting each cut vertex to the blocks containing it. Prove that removing a cut vertex v from G creates exactly $\deg_T(v) - 1$ components, where $\deg_T(v)$ is the degree of v in the block-cut tree.

7 Matchings and Factors

Definition 70 (Matching). A matching in a graph is a set of vertex disjoint edges in the graph.

Definition 71 (Maximum Matching). A maximum matching in a graph G is a matching M such that $|E(M)|$ is a maximum.

Definition 72 (Perfect Matching). A perfect matching or 1-factor is a matching M such that

$$V(M) = V(G).$$

Definition 73 (1-Factor). A 1-factor of a graph G is a spanning 1-regular subgraph of G . In other words, it is a subgraph that includes all vertices of G where each vertex has degree exactly 1.

Theorem 65 (Tutte's 1-Factor Theorem). A graph $G = (V, E)$ has a perfect matching (also called a 1-factor) if and only if for every subset $S \subseteq V$, the number of odd-sized components in $G - S$ is at most $|S|$.

In other words, a graph G has a perfect matching if and only if:

$$\text{odd}(G - S) \leq |S| \quad \text{for all } S \subseteq V$$

where $\text{odd}(G - S)$ denotes the number of connected components in $G - S$ that have an odd number of vertices.

Corollary 32 (Tutte-Berge Formula). For any graph $G = (V, E)$, the size of a maximum matching equals:

$$\nu(G) = \frac{1}{2} \min_{S \subseteq V} \{|V| + |S| - \text{odd}(G - S)\}$$

where $\nu(G)$ is the matching number of G (the size of a maximum matching).

Remark 73. A 1-factor is equivalent to a perfect matching, as each vertex must be incident to exactly one edge, and the collection of these edges forms a matching that covers all vertices of the graph.

Remark 74. The Tutte-Berge formula generalizes Tutte's 1-factor theorem by quantifying the maximum matching size when a perfect matching does not exist. It shows that the "deficiency" is determined by the subset S that maximizes $\text{odd}(G - S) - |S|$.

Remark 75. For bipartite graphs, Tutte's condition simplifies to Hall's Marriage Theorem, which states that a bipartite graph $G = (A \cup B, E)$ with $|A| = |B|$ has a perfect matching if and only if for all subsets $S \subseteq A$, $|N(S)| \geq |S|$.

Remark 76. For a graph to have a 1-factor:

- The graph must have an even number of vertices (since each edge covers exactly two vertices)
- The graph must satisfy Tutte's condition: for any subset S of vertices, the number of odd components in $G - S$ is at most $|S|$

Example 24. The complete graph K_{2n} always has a 1-factor. In fact, it can be decomposed into $2n - 1$ disjoint 1-factors.

Example 25. Consider the following cycle graph G with 7 vertices:

The highlighted edges form a matching because they don't share any vertices. This matching is not maximum.

This graph does not have a perfect matching because it has 7 vertices (an odd number), and in a perfect matching, each edge covers exactly 2 vertices. With an odd number of vertices, at least one vertex must be left uncovered.

The Petersen graph shown above has 10 vertices, which is an even number. However, it still cannot have a perfect matching due to its specific structure. A key property of graphs that can have perfect matchings is that for any subset S of vertices, the number of connected components in $G - S$ that have an odd number of vertices is at most $|S|$ (a consequence of Tutte's theorem).

Example 26. Here is an example of a graph with a perfect matching:

The complete bipartite graph $K_{3,3}$ has 6 vertices and the highlighted edges form a perfect matching that covers all vertices.

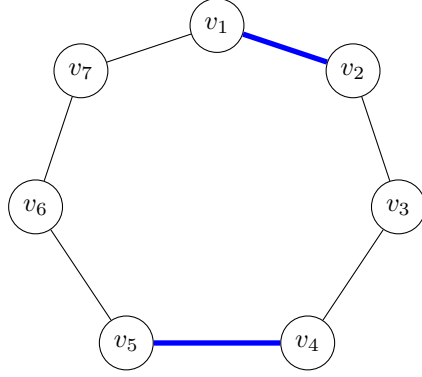


Figure 100 : A matching with edges $\{\{v_1, v_2\}, \{v_4, v_5\}\}$

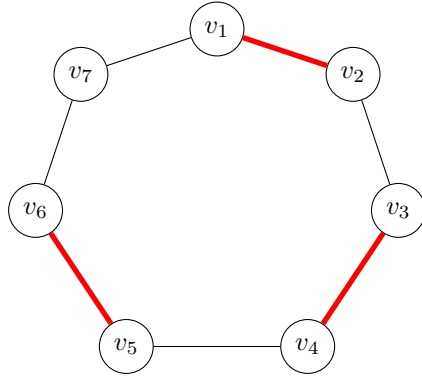


Figure 101 : A maximum matching in G with 3 edges

Problem 3. Suppose we have persons $p_1, \dots, p_n$ and jobs $j_1, j_2, \dots, j_m$.

Place an edge $\{p_i, j_k\}$ if person p_i can do job j_k .

What is the maximum number of jobs that can be done, if each person is assigned exactly one job, and each job is done by one person?

Remark 77. This is equivalent to finding a maximum matching in the bipartite graph $G = (P \cup J, E)$ where:

- $P = \{p_1, p_2, \dots, p_n\}$ is the set of people
- $J = \{j_1, j_2, \dots, j_m\}$ is the set of jobs
- $E = \{\{p_i, j_k\} : \text{person } p_i \text{ can do job } j_k\}$

Example 27. In the example shown in the figure, the maximum number of jobs that can be done is 3, as shown by the highlighted matching where:

- Person p_1 is assigned job j_1 (green edge)
- Person p_2 is assigned job j_3 (red edge)
- Person p_3 is assigned job j_2 (blue edge)

This is a perfect matching since all persons are assigned a job and all jobs are filled.

Remark 78. If $n \neq m$ (i.e., the number of people doesn't equal the number of jobs), then we can't have a perfect matching. The maximum number of assignments possible will be $\min(n, m)$.

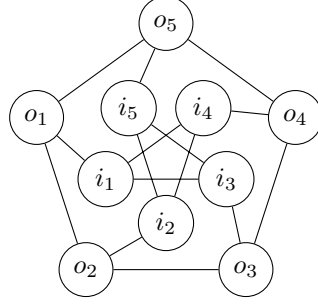


Figure 102 : Petersen graph - has an odd number of vertices (5+5)

Cannot have a perfect matching

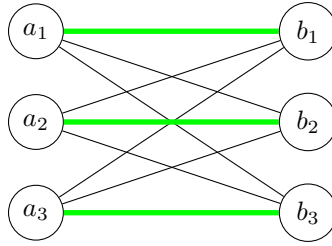


Figure 103 : $K_{3,3}$ with a perfect matching (highlighted in green)

7.1 Independent Sets and Covers

Definition 74 (Independent Set). An independent set in a graph is a set of vertices containing no edge of the graph.

Definition 75 (Independence Number). Let $\alpha(G)$ be the size of a maximum independent set in G .

Definition 76 (Matching Number). Let $\alpha'(G)$ be the number of edges in a maximum matching.

Example 28. For the cycle graph C_7 , we have $\alpha(C_7) = 3$ and $\alpha'(C_7) = 3$.

Definition 77 (Vertex Cover). A vertex cover of a graph G is a set $X \subseteq V(G)$ such that for every edge $e \in E(G)$, $e \cap X \neq \emptyset$.

Definition 78 (Vertex Cover Number). Let $\beta(G)$ be the smallest size of a vertex cover in G .

Definition 79 (Edge Cover). An edge cover is a set of edges of a graph G such that every vertex is contained in at least one of the edges.

Definition 80 (Edge Cover Number). Let $\beta'(G)$ be the smallest size of an edge cover in G .

Remark 79. We can verify these relationships in our C_7 example:

$$\alpha(C_7) + \beta(C_7) = 3 + 4 = 7 = |V(C_7)| \quad (34)$$

$$\alpha'(C_7) + \beta'(C_7) = 3 + 4 = 7 = |V(C_7)| \quad (35)$$

Remark 80. An independent set S is a maximum independent set if and only if its complement $V(G) \setminus S$ is a minimum vertex cover. This follows directly from Gallai's Theorem.

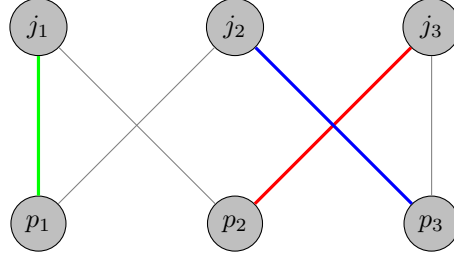


Figure 104 : Example of a job assignment problem with a maximum matching
(Each color represents a person-job assignment)

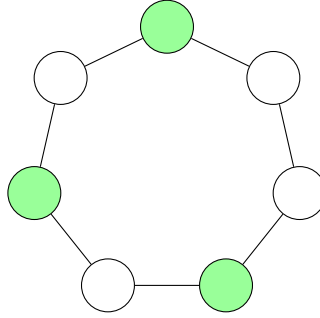


Figure 105 : C_7 with a maximum independent set of size $\alpha(C_7) = 3$

Lemma 56 (Gallai's Lemma). If G is a graph with no isolated vertices, then

$$\alpha'(G) + \beta'(G) = |V(G)|.$$

If G is any graph, then

$$\alpha(G) + \beta(G) = |V(G)|.$$

Proof. Let $I \subseteq V(G)$ be a maximum independent set in G ; hence $|I| = \alpha(G)$. Since I is independent, every edge of G has at least one endpoint in

$$V(G) \setminus I,$$

so $V(G) \setminus I$ is a vertex cover. By minimality of $\beta(G)$, we have

$$|V(G) \setminus I| \geq \beta(G).$$

But $|V(G) \setminus I| = |V(G)| - |I| = |V(G)| - \alpha(G)$, so

$$\alpha(G) + \beta(G) \leq |V(G)|.$$

Conversely, let $C \subseteq V(G)$ be a minimum vertex cover; hence $|C| = \beta(G)$. By definition of a vertex cover, no two vertices in

$$V(G) \setminus C$$

can be adjacent, so $V(G) \setminus C$ is an independent set. By maximality of $\alpha(G)$,

$$|V(G) \setminus C| \leq \alpha(G).$$

Again $|V(G) \setminus C| = |V(G)| - \beta(G)$, giving

$$\alpha(G) + \beta(G) \geq |V(G)|.$$

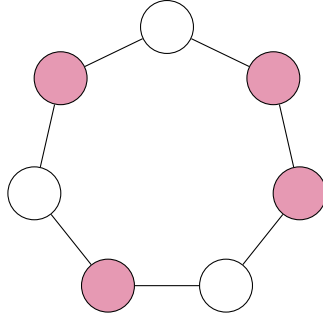


Figure 106 : C_7 with a minimum vertex cover of size $\beta(C_7) = 4$

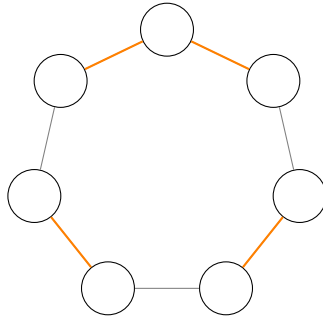


Figure 107 : C_7 with a minimum edge cover of size $\beta'(C_7) = 4$

Combining the two inequalities yields

$$\alpha(G) + \beta(G) = |V(G)|,$$

as claimed. □

Example 29. Consider the cycle graph C_5 shown below:

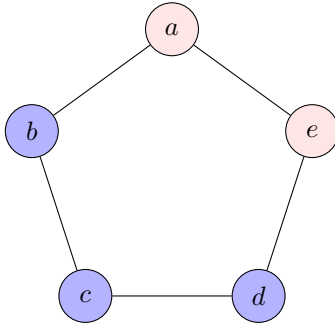


Figure 108 : Graph $G = C_5$ with $\alpha(G) = 2$ and $\beta(G) = 3$.
Note that $\alpha(G) + \beta(G) = 2 + 3 = 5 = |V(G)|$.

In the figure above:

- Pink vertices form a maximum independent set of size $\alpha(G) = 2$
- Blue vertices form a minimum vertex cover of size $\beta(G) = 3$

In the second diagram:

- The solid red edges form a maximum matching of size $\alpha'(G) = 2$

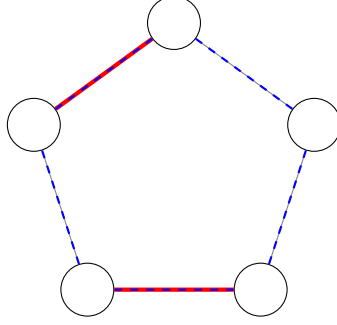


Figure 109 : Graph $G = C_5$ with a maximum matching (solid red) and a minimum edge cover (dashed blue).

$$\alpha'(G) = 2 \text{ and } \beta'(G) = 3.$$

$$\alpha'(G) + \beta'(G) = 2 + 3 = 5 = |V(G)|.$$

- The dashed blue edges form a minimum edge cover of size $\beta'(G) = 3$

Remark 81. The first part of Gallai's Lemma tells us that for any graph without isolated vertices, the sum of the size of a maximum matching and the size of a minimum edge cover equals the number of vertices in the graph.

The second part tells us that for any graph (even those with isolated vertices), the sum of the size of a maximum independent set and the size of a minimum vertex cover equals the number of vertices in the graph.

Remark 82. These relationships reflect the complementary nature of these graph parameters:

- A vertex is either in an independent set or in a vertex cover
- Each vertex must be covered by at least one edge in an edge cover, and can be covered by at most one edge in a matching

Definition 81 (Independence Number). The *independence number* of a graph G , denoted by $\alpha(G)$, is the size of a maximum independent set in G .

Definition 82 (Matching Number). The *matching number* of a graph G , denoted by $\alpha'(G)$, is the size of a maximum matching in G .

Definition 83 (Vertex Cover Number). The *vertex cover number* of a graph G , denoted by $\beta(G)$, is the size of a minimum vertex cover in G .

Definition 84 (Edge Cover Number). The *edge cover number* of a graph G , denoted by $\beta'(G)$, is the size of a minimum edge cover in G .

Recall Gallai's Lemma, which establishes an important relationship between these graph parameters:

For any graph G :

$$\alpha(G) + \beta(G) = |V(G)| \quad (36)$$

And if G has no isolated vertices:

$$\alpha'(G) + \beta'(G) = |V(G)| \quad (37)$$

Proof of the first part:

Let I be a maximum independent set in G , so $|I| = \alpha(G)$.

Since I is an independent set, the remaining vertices $V(G) \setminus I$ must form a vertex cover. This gives us:

$$|V(G) \setminus I| \geq \beta(G) \quad (38)$$

Therefore:

$$|I| + |V(G) \setminus I| = |V(G)| \geq \alpha(G) + \beta(G) \quad (39)$$

Conversely, let C be a minimum vertex cover, so $|C| = \beta(G)$. Then $V(G) \setminus C$ is an independent set, so $|V(G) \setminus C| \leq \alpha(G)$.

We have:

$$|V(G)| = |C| + |V(G) \setminus C| \leq \beta(G) + \alpha(G) \quad (40)$$

From (1) and (2), we get:

$$\alpha(G) + \beta(G) = |V(G)| \quad (41)$$

Proof of the second part:

Suppose now G has no isolated vertices. Let M be a maximum matching in G , so $|E(M)| = \alpha'(G)$.

Then $V(G) \setminus V(M)$ is an independent set of G . This is because if there were an edge between two vertices in $V(G) \setminus V(M)$, we could add this edge to M to create a larger matching, contradicting the maximality of M .

Now, to form an edge cover, we can: 1. Pick one edge containing each vertex of $V(G) \setminus V(M)$ 2. Add all edges from M to this set

This forms an edge cover, as every vertex is covered by at least one edge. The total number of edges in this cover is at most:

$$|V(G) \setminus V(M)| + |E(M)| = |V(G)| - |V(M)| + |E(M)| \quad (42)$$

Since each edge in M covers exactly two vertices, $|V(M)| = 2|E(M)|$. Therefore:

$$|V(G)| - |V(M)| + |E(M)| = |V(G)| - 2|E(M)| + |E(M)| = |V(G)| - |E(M)| \quad (43)$$

So the size of this edge cover is at most $|V(G)| - \alpha'(G)$, which means:

$$\beta'(G) \leq |V(G)| - \alpha'(G) \quad (44)$$

Conversely, starting with a minimum edge cover, we can construct a matching to show:

$$\alpha'(G) \geq |V(G)| - \beta'(G) \quad (45)$$

Combining these inequalities:

$$\alpha'(G) + \beta'(G) = |V(G)| \quad (46)$$

This completes the proof First part of Gallai's Lemma.

Proof of $\alpha'(G) + \beta'(G) = |V(G)|$ for graphs with no isolated vertices. Suppose G has no isolated vertices. Let M be a maximum matching in G , so $|E(M)| = \alpha'(G)$.

Then $V(G) \setminus V(M)$ is an independent set of G . This is because if there were an edge between two vertices in $V(G) \setminus V(M)$, we could add this edge to M to create a larger matching, contradicting the maximality of M .

Now, to form an edge cover, we can pick one edge containing each vertex of $V(G) \setminus V(M)$ and add all edges from M . Since G has no isolated vertices, such edges must exist. This gives us an edge cover of size at most:

$$\beta'(G) \leq |V(G) \setminus V(M)| + |E(M)| \quad (47)$$

$$= |V(G)| - |V(M)| + |E(M)| \quad (48)$$

$$= |V(G)| - 2|E(M)| + |E(M)| \quad (49)$$

$$= |V(G)| - |E(M)| \quad (50)$$

$$= |V(G)| - \alpha'(G) \quad (51)$$

So:

$$\alpha'(G) + \beta'(G) \leq |V(G)| \quad (1)$$

Conversely, let $F \subseteq E(G)$ be a minimum edge cover. Then every edge of F has an end which has degree 1 in F . This must be true, for otherwise we could remove an edge and still maintain an edge cover, contradicting the minimality of F .

This means that F is a disjoint union of stars: every component of F is a $K_{1,t}$ (a star graph with one central vertex connected to t leaf vertices).

If F has k components, then $\alpha'(G) \geq k$ (since we can pick one edge from each component to form a matching, and this matching can be extended to a maximum matching).

We have $|E(F)| = |V(G)| - k$ because in a disjoint union of k stars covering all vertices, the number of edges is exactly the number of vertices minus the number of stars. Therefore:

$$|E(F)| = \beta'(G) = |V(G)| - k \leq |V(G)| - \alpha'(G) \quad (52)$$

So:

$$\beta'(G) + \alpha'(G) \geq |V(G)| \quad (2)$$

From inequalities (1) and (2), we conclude:

$$\alpha'(G) + \beta'(G) = |V(G)| \quad (53)$$

This completes the proof of the second part of Gallai's Theorem. $\square$

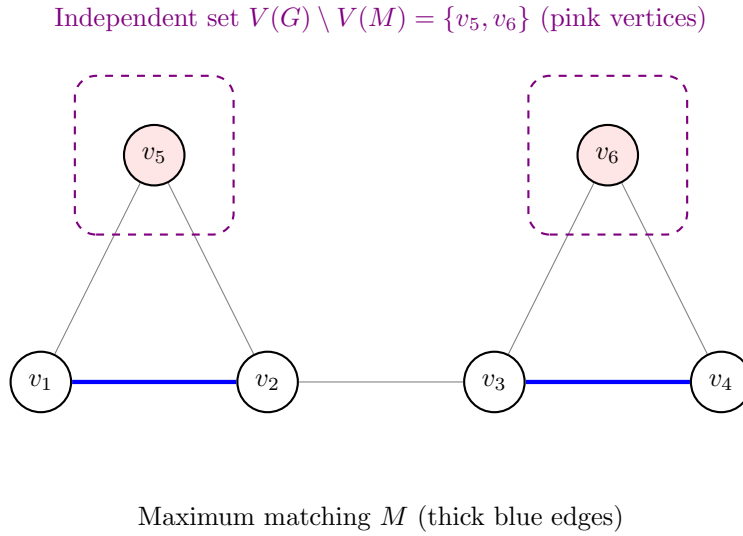


Figure 110 : Illustration for Gallai's Lemma: A graph G with a maximum matching M of size $\alpha'(G) = 2$ (blue edges). The vertices not covered by the matching (v_5 and v_6 , highlighted in pink) form an independent set. For an edge cover, we need the two edges in M plus one edge incident to each of v_5 and v_6 , giving a minimum edge cover of size $\beta'(G) = 4$. Note that $\alpha'(G) + \beta'(G) = 2 + 4 = 6 = |V(G)|$.

Theorem 66 (Hall's Theorem - Formal Statement). Let G be a bipartite graph with bipartition (A, B) where $|A| = |B|$. Then G has a perfect matching if and only if for every subset $X \subseteq A$:

$$|N(X)| \geq |X|$$

Theorem 67 (Hall's Marriage Theorem). Hall's Theorem gives a necessary and sufficient condition for a bipartite graph to have a perfect matching.

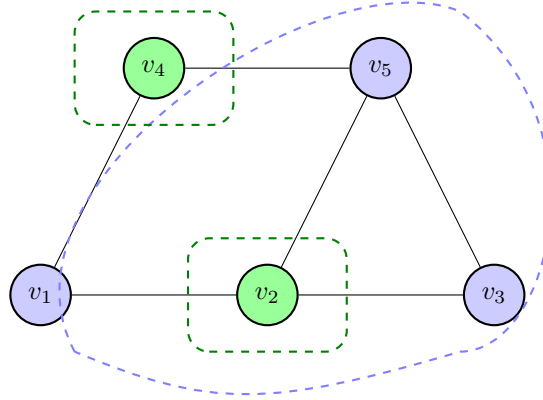
Remark 83. For bipartite graphs, Tutte's condition simplifies to Hall's Marriage Theorem, which states that a bipartite graph $G = (A \cup B, E)$ with $|A| = |B|$ has a perfect matching if and only if for all subsets $S \subseteq A$, $|N(S)| \geq |S|$.

Theorem 68 (König-Ore Formula). Let $G(A, B)$ be a bipartite graph. Then

$$\nu(G, A) = \max_{S \subseteq A} \{|S| - |N(S)|\}.$$

where:

Independent set $V(G) \setminus C = \{v_2, v_4\}$ (green vertices)



Vertex cover $C = \{v_1, v_3, v_5\}$ (blue vertices)

Figure 111 : Illustrating $\alpha(G) + \beta(G) = |V(G)|$: This graph has a minimum vertex cover C of size $\beta(G) = 3$ (blue vertices) and a maximum independent set of size $\alpha(G) = 2$ (green vertices). The vertex cover and independent set partition the vertex set, and $\alpha(G) + \beta(G) = 2 + 3 = 5 = |V(G)|$.

- $\nu(G, A)$ is the deficiency of G with respect to A (the maximum number of vertices in A that must remain unmatched in any matching)
- $N(S)$ is the neighborhood of set S (all vertices adjacent to at least one vertex in S)
- The maximum is taken over all subsets S of A

Theorem 69 (Relationship to König-Ore Formula). For bipartite graphs $G = (A \cup B, E)$ with $|A| = |B|$, Tutte's condition becomes equivalent to the König-Ore formula, as:

$$\text{odd}(G - S) \leq |S| \quad \text{for all } S \subseteq V$$

reduces to Hall's condition:

$$|N(T)| \geq |T| \quad \text{for all } T \subseteq A$$

when applied to bipartite graphs.

Corollary 33. A bipartite graph $G(A, B)$ has a matching that covers all vertices in A if and only if for every subset $S \subseteq A$, $|N(S)| \geq |S|$.

Remark 84. This is also known as Hall's Marriage Theorem when expressed in the contrapositive form, which states that a bipartite graph $G(A, B)$ has a matching covering all vertices in A if and only if for all $S \subseteq A$, $|N(S)| \geq |S|$.

Remark 85. In the context of the marriage problem, this means that a set of men can all be married if and only if any set of k men collectively know at least k women.

Remark 86. This condition is often called Hall's marriage condition and can be intuitively understood as: "A group of people can all get married if and only if any group of k people collectively know at least k potential spouses."

Example 30. Consider a bipartite graph representing possible marriages between n men and n women. Each edge represents that a particular man and woman would consider marrying each other. Hall's Theorem states that perfect matchings (where everyone gets married) are possible if and only if for any group of k men, there are at least k women who would consider marrying someone in that group.

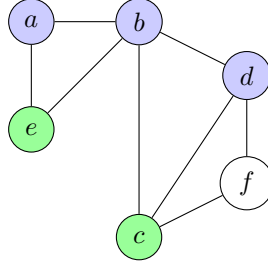


Figure 112 : Illustration of neighborhood $N(X)$ in a graph. The set $X = \{e, c\}$ (green vertices) has neighborhood $N(X) = \{a, b, d, f\}$ (blue vertices). Note that $|N(X)| = 4 > 2 = |X|$, satisfying Hall's condition for this subset.

Remark 87. The condition in Hall's Theorem is often easier to check by considering its contrapositive: a bipartite graph with bipartition (A, B) where $|A| = |B|$ has no perfect matching if and only if there exists a subset $X \subseteq A$ such that $|N(X)| < |X|$.

Corollary 34. If G is a k -regular bipartite graph, and $k \geq 1$, then G has a perfect matching.

Proof. Let G be a k -regular bipartite graph with bipartition (A, B) . Since G is k -regular, every vertex in A has exactly k neighbors in B , and every vertex in B has exactly k neighbors in A .

Let's verify Hall's condition. For any subset $X \subseteq A$, the vertices in X contribute exactly $k|X|$ edges to the graph. All these edges connect to vertices in $N(X) \subseteq B$. Since each vertex in $N(X)$ can be incident to at most k edges, we have:

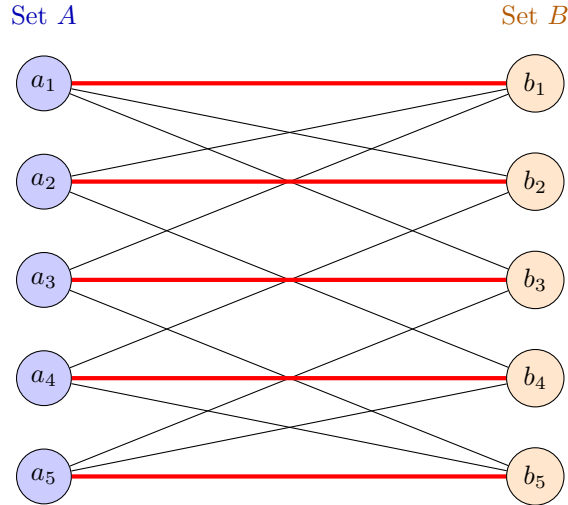
$$k|X| \leq k|N(X)|$$

This simplifies to:

$$|X| \leq |N(X)|$$

Thus, Hall's condition is satisfied for any subset $X \subseteq A$. Therefore, G has a perfect matching. $\square$

Example 31. Consider the following 3-regular bipartite graph G :



Perfect matching highlighted in red

Figure 113 : A 3-regular bipartite graph with a perfect matching. Each vertex has exactly 3 neighbors, and by the corollary, a perfect matching is guaranteed to exist.

In this 3-regular bipartite graph, each vertex is connected to exactly 3 vertices on the opposite side. According to the corollary, this graph must have a perfect matching. Indeed, a perfect matching is highlighted in red.

We can verify Hall's condition: for any subset X of the left partition, $|N(X)| \geq |X|$. For example:

- If $X = \{a_1\}$, then $N(X) = \{b_1, b_2, b_3\}$ and $|N(X)| = 3 > 1 = |X|$
- If $X = \{a_1, a_2\}$, then $N(X) = \{b_1, b_2, b_3, b_4\}$ and $|N(X)| = 4 > 2 = |X|$
- If $X = \{a_1, a_2, a_3, a_4, a_5\}$, then $N(X) = \{b_1, b_2, b_3, b_4, b_5\}$ and $|N(X)| = 5 = |X|$

Hall's condition is satisfied for all possible subsets of A , confirming the existence of a perfect matching.

Proof. We need to show if G has parts A and B , then for every $X \subseteq A$, $|N(X)| \geq |X|$.

Let $e(X, N(X))$ be the number of edges between X and $N(X)$.

Since G is k -regular, each vertex in X has exactly k neighbors in B . Therefore:

$$e(X, N(X)) = k|X|$$

On the other hand, each vertex in $N(X)$ is adjacent to at most k vertices in X (since the graph is k -regular):

$$e(N(X), X) \leq k|N(X)|$$

Since $e(X, N(X)) = e(N(X), X)$ (these represent the same set of edges), we have:

$$k|X| = e(X, N(X)) = e(N(X), X) \leq k|N(X)|$$

Dividing both sides by k (which is possible since $k \geq 1$):

$$|X| \leq |N(X)|$$

So $|N(X)| \geq |X|$ for all $X \subseteq A$.

By Hall's Theorem, there is a matching M covering A .

Since the graph is k -regular, the total number of edges in the graph is $k|A| = k|B|$, which implies $|A| = |B|$.

Therefore, M is a perfect matching. □

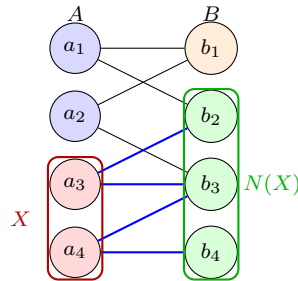


Figure 114 : 2-regular bipartite graph with $X = \{a_3, a_4\}$ (red) and $N(X) = \{b_2, b_3, b_4\}$ (green). Each vertex in X has $k = 2$ neighbors, giving $e(X, N(X)) = k|X| = 4$ edges (blue). Since $|N(X)| = 3 > 2 = |X|$, Hall's condition is satisfied.

7.2 Hall's Theorem and Various Matching Algorithms

Theorem 70 (Hall's Theorem). Let G be a bipartite graph with parts A and B . Then G has a matching covering A if and only if for every $X \subseteq A$:

Hall's condition: $|N(X)| \geq |X|$

Proof. ($\Rightarrow$) If G has a matching M covering A , then for $X \subseteq A$, $|N(X)| \geq |X|$.

($\Leftarrow$) If $|N(X)| \geq |X| \forall X \subseteq A$, we have to find a matching covering A .

Proceed by induction on $|A|$:

Base case: $|A| = 1$. Then by Hall's condition $|N(A)| \geq |A|$, so a has a neighbor $b \in B$. Now $\{a, b\}$ gives a matching covering A .

Inductive step: Suppose $|A| \geq 2$ and the theorem is true for bipartite graphs with parts A' and B where $|A'| < |A|$.

Case 1: $|N(X)| > |X|$ for all $X \subset A$, $X \neq \emptyset$

Pick any edge $ab \in E(G)$ where $a \in A$, $b \in B$. Let $G' = G - \{a\} - \{b\}$

Then G' has parts $A' = A - \{a\}$ and $B' = B - \{b\}$

For $X \subseteq A'$: $N_{G'}(X) \supseteq N_G(X) - \{b\}$ Therefore $|N_{G'}(X)| \geq |N_G(X)| - 1 \geq |X|$

Therefore Hall's condition holds for G' . By induction, G' has a matching M' covering A' . Now $M = M' \cup \{ab\}$ is a matching covering A .

Case 2: $|N(X)| = |X|$ for some $X \subset A$, $X \neq \emptyset$

If $Y \subseteq X$: $|N(Y)| \geq |Y|$ by Hall's condition in G

So Hall's condition holds in $G[X \cup N(X)]$

By induction, $G[X \cup N(X)]$ has a matching M_1 covering X .

Now let $Y \subseteq A - X$:

$$\begin{aligned} |N_{G_2}(Y)| &= |N_G(Y \cup X)| - |N_G(X)| \\ &\geq |Y \cup X| - |X| \\ &= |Y| \end{aligned}$$

So Hall's condition holds for G_2 . By induction there is a matching M_2 in G_2 covering $A - X$.

Now $M_1 \cup M_2$ is a matching in G covering A . □

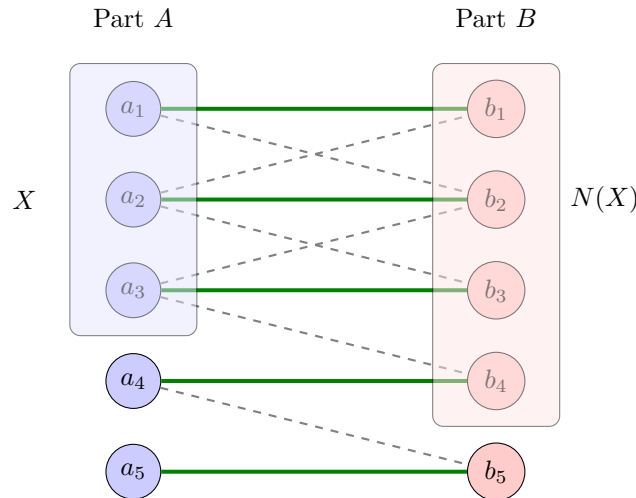


Figure 115: Illustration of Hall's Theorem. The green edges form a matching covering set A . The subset $X \subseteq A$ is highlighted in blue, and its neighborhood $N(X)$ is highlighted in red. Hall's condition requires that $|N(X)| \geq |X|$ for every $X \subseteq A$.

Case 1: $|N(X)| > |X|$ for all $X \subset A$

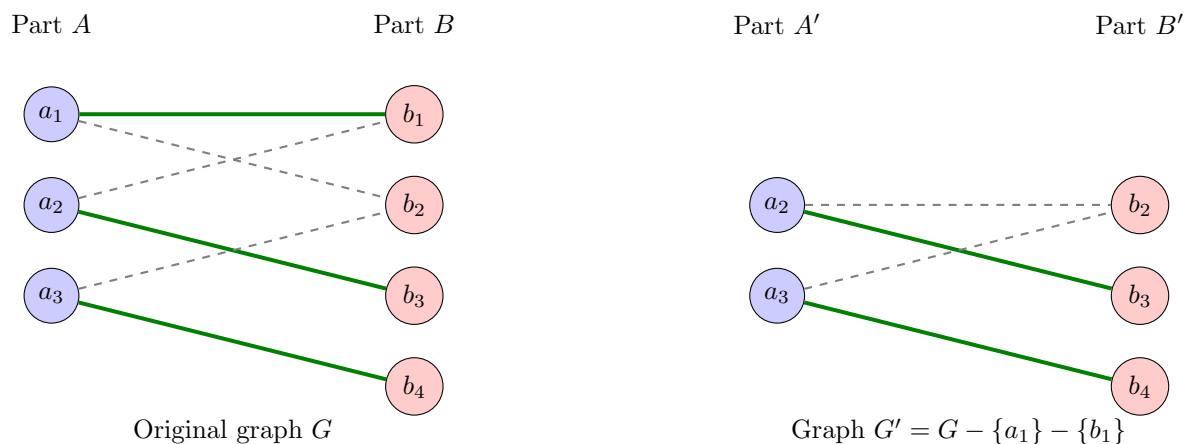


Figure 116 : Illustration of Case 1 in the proof of Hall's Theorem. We remove an edge a_1b_1 and the vertices a_1 and b_1 to create G' .

Case 2: $|N(X)| = |X|$ for some $X \subset A$

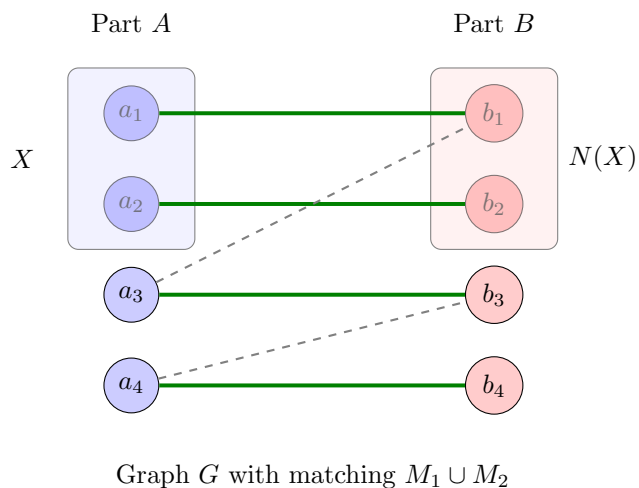


Figure 117 : Illustration of Case 2 in the proof of Hall's Theorem. We have a subset $X \subset A$ where $|N(X)| = |X|$. We find matchings M_1 in the subgraph induced by $X \cup N(X)$ and M_2 in the remaining graph.

Definition 85. Suppose we have people $p_1, p_2, \dots, p_n$ and committees $S_1, S_2, \dots, S_m$ which are sets of people.

A **System of Distinct Representatives (SDR)** is a selection of one person from each committee such that no person represents more than one committee.

For bipartite $G = (A \cup B, E)$ where:

- $A = \{S_1, S_2, \dots, S_m\}$
- $B = \{p_1, p_2, \dots, p_n\}$
- $(S_i, p_j) \in E$ if $p_j \in S_i$

An SDR in G is a matching covering A .

Theorem 71. Sets $S_1, S_2, \dots, S_m$ have an SDR if and only if for all $I \subseteq \{1, 2, \dots, m\}$:

$$\left| \bigcup_{i \in I} S_i \right| \geq |I| \quad (54)$$

Example 32. Suppose we have committees:

$$S_1 = \{p_1, p_2, p_3\}$$

$$S_2 = \{p_2, p_3, p_4\}$$

$$S_3 = \{p_1, p_4, p_5\}$$

Is there a system of distinct representatives?

Yes: $\{p_3, p_2, p_1\}$ where p_3 represents S_1 , p_2 represents S_2 , and p_1 represents S_3 .

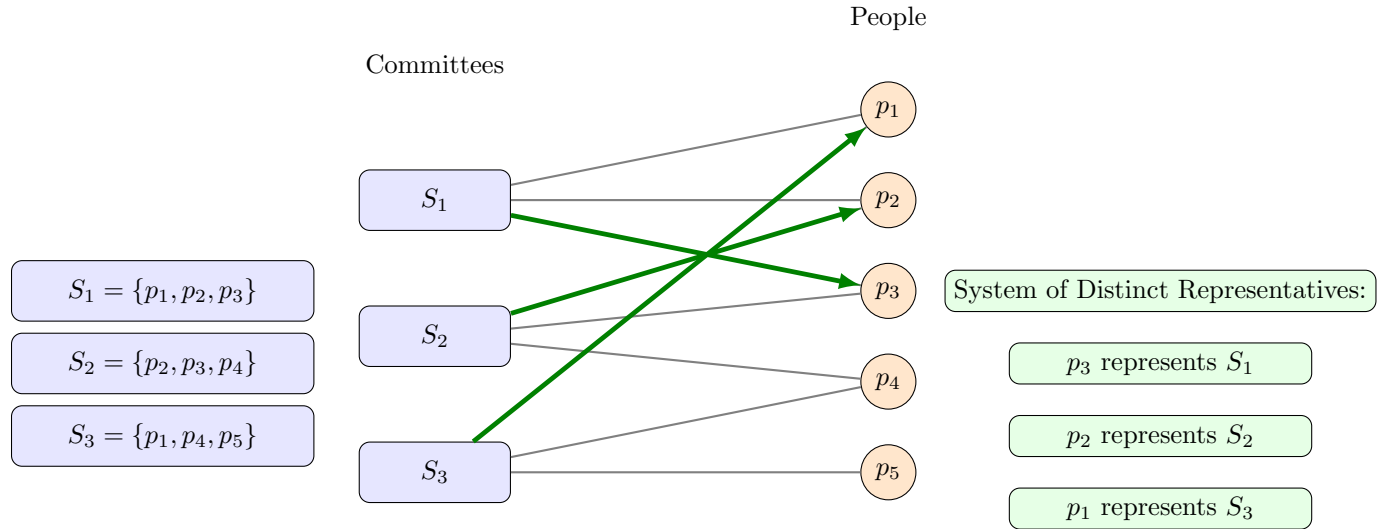


Figure 118 : Illustration of a System of Distinct Representatives (SDR) for the given committees. The green arrows show the selected representatives, where p_3 represents S_1 , p_2 represents S_2 , and p_1 represents S_3 .

Example 33. Suppose we have committees:

$$S_1 = \{1, 4\}$$

$$S_2 = \{3, 5\}$$

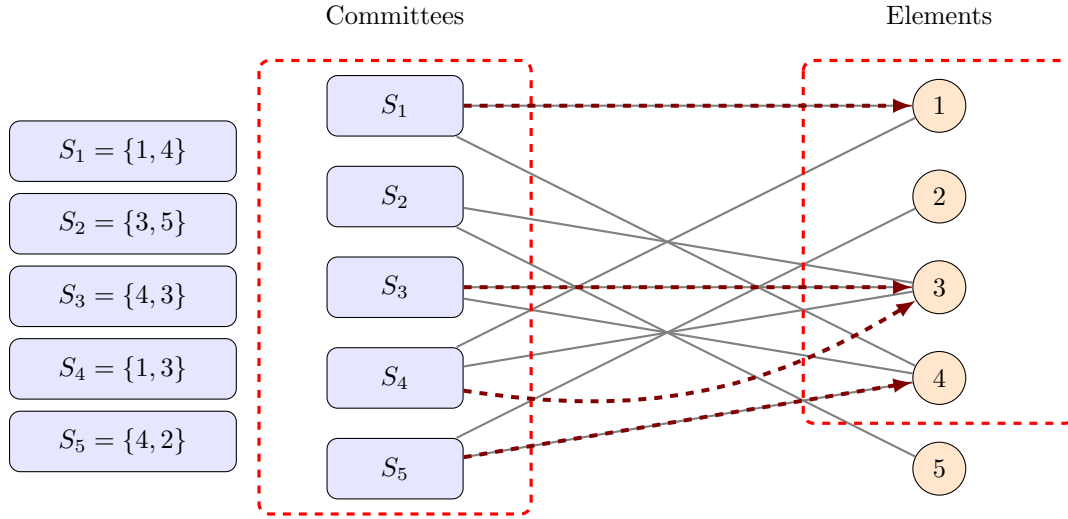
$$S_3 = \{4, 3\}$$

$$S_4 = \{1, 3\}$$

$$S_5 = \{4, 2\}$$

No SDR because: For $I = \{1, 3, 4, 5\}$ $|S_1 \cup S_3 \cup S_4 \cup S_5| = |\{1, 2, 3, 4\}| = 4 = |I|$

However, no valid SDR can be constructed due to the specific distribution of elements in the sets.



For the subset $I = \{S_1, S_3, S_4, S_5\}$, we have: $|S_1 \cup S_3 \cup S_4 \cup S_5| = |\{1, 2, 3, 4\}| = 4 = |I|$

Hall's condition is satisfied with equality, but due to the distribution:

- Element 3 needs to represent at least one of $\{S_3, S_4\}$
- Element 4 needs to represent at least one of $\{S_1, S_3, S_5\}$
- Element 1 needs to represent at least one of $\{S_1, S_4\}$
- Element 2 can only represent S_5

These constraints create a situation where no valid SDR exists.

Figure 119 : Illustration of a case where no System of Distinct Representatives exists. The red dashed boxes highlight the problematic subset of committees $I = \{S_1, S_3, S_4, S_5\}$ and their combined neighborhood $\{1, 2, 3, 4\}$.

Although $|I| = |\cup_{i \in I} S_i| = 4$ (Hall's condition is satisfied with equality), the specific distribution of elements prevents a valid SDR.

Example 34. Suppose:

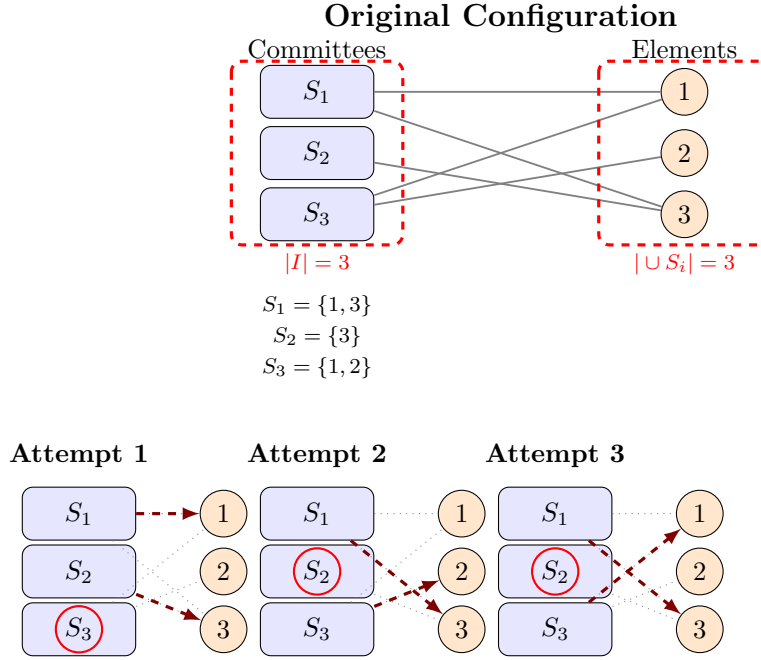
$$S_1 = \{1, 3\}$$

$$S_2 = \{3\}$$

$$S_3 = \{1, 2\}$$

No SDR exists. For $I = \{1, 2, 3\}$, we have: $|S_1 \cup S_2 \cup S_3| = |\{1, 2, 3\}| = 3 = |I|$

The condition is satisfied with equality, but the specific structure prevents a valid SDR.



Explanation: For $I = \{S_1, S_2, S_3\}$, we have $|S_1 \cup S_2 \cup S_3| = |\{1, 2, 3\}| = 3 = |I|$. Hall's condition is satisfied with equality, but no valid SDR exists:

- **Attempt 1:** If we assign element 1 to S_1 and element 3 to S_2 , then S_3 has no representative left.
- **Attempt 2:** If we assign element 3 to S_1 and element 2 to S_3 , then S_2 has no representative left.
- **Attempt 3:** If we assign element 3 to S_1 and element 1 to S_3 , then S_2 has no representative left.

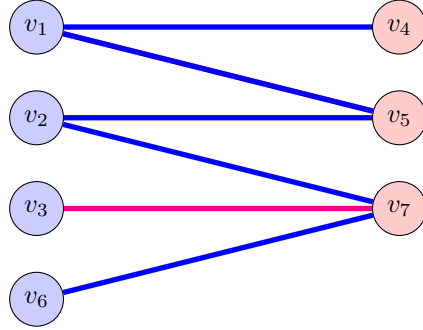
The fundamental issue is that S_2 contains only element 3, which means element 3 must represent S_2 in any valid SDR. But if element 3 represents S_2 , then S_1 must be represented by element 1, leaving no valid representative for S_3 .

Figure 120: Illustration of a case where no System of Distinct Representatives exists despite satisfying Hall's condition with equality ($|I| = |\cup_{i \in I} S_i| = 3$). The top diagram shows the committee-element relationships, while the bottom row shows all possible assignment attempts and why each fails.

Definition 86 (M-alternating path). Let G be a bipartite graph and M a matching in G . Then an **M-alternating path** is a path $P \subseteq G$ with

$$E(P) = \{\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{k-1}, v_k\}\}$$

such that edges are alternately in and not in M .

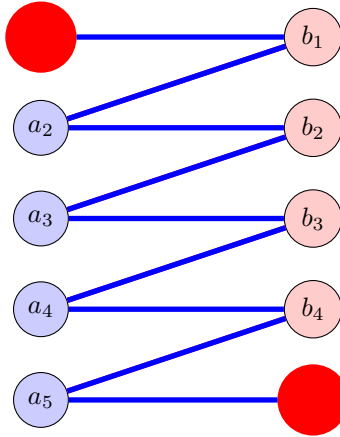


An M-alternating path: $P = (v_6, v_7, v_2, v_5, v_1, v_4)$

Edge sequence: not in M , in M , not in M , in M , not in M

Figure 121 : Example of an M-alternating path. Matching edges are shown in magenta. The path alternates between edges in and not in the matching.

Definition 87 (M-augmenting path). An **M-augmenting path** is an M-alternating path starting and ending with edges not in M , and the first and last vertices of the path are exposed by M (i.e., not in any edge of M).



An M-augmenting path from exposed vertex a_1 to exposed vertex b_5

The path alternates between edges not in M and edges in M

Figure 122 : Example of an M-augmenting path. The red circled vertices are exposed (not matched). The augmenting path starts and ends with exposed vertices.

Theorem 72 (Berge). If M is a matching in a graph G , then M is a maximum matching if and only if G contains no M-augmenting paths.

Hopcroft-Karp Algorithm Step by Step

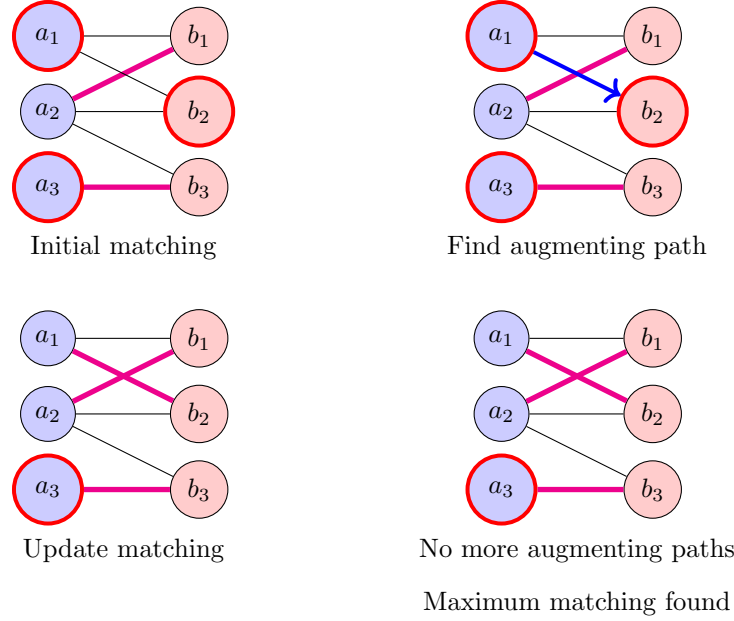


Figure 124 : Complete execution of the Hopcroft-Karp algorithm on a small example. We start with a matching, find augmenting paths, update the matching, and continue until no more augmenting paths exist. At that point, we have found a maximum matching.

Algorithm 18 Hopcroft-Karp Algorithm

```

1: procedure HOPCROFTKARP( $G = (U \cup V, E)$ )                                 $\triangleright G$  is a bipartite graph
2:    $M \leftarrow \emptyset$                                                    $\triangleright$  Initialize empty matching
3:   while true do
4:      $L \leftarrow \text{BuildLevelGraph}(G, M)$                                  $\triangleright$  BFS to build level graph
5:     if no augmenting path exists in  $L$  then
6:       return  $M$                                                           $\triangleright$  No more augmenting paths,  $M$  is maximum
7:     end if
8:      $P \leftarrow \emptyset$                                                  $\triangleright$  Set of vertex-disjoint augmenting paths
9:     for all exposed vertices  $u \in U$  do
10:      while DFS( $L, u, M, P$ ) do  $\triangleright$  Find augmenting path starting at  $u$   $\triangleright$  DFS returns true if it
      finds an augmenting path
11:      end while
12:    end for
13:    for all augmenting paths  $p \in P$  do
14:       $M \leftarrow M \oplus p$                                                $\triangleright$  Augment matching using symmetric difference
15:    end for
16:  end while
17: end procedure

```

Algorithm 19 Build Level Graph

```
1: procedure BUILDLEVELGRAPH( $G = (U \cup V, E), M$ )
2:    $L \leftarrow$  empty graph
3:   level  $\leftarrow$  array of distances, initialized to  $\infty$ 
4:   queue  $\leftarrow$  all exposed vertices from  $U$ 
5:   for all exposed vertices  $u \in U$  do
6:     level[ $u$ ]  $\leftarrow 0$ 
7:   end for
8:   foundAugmentingPath  $\leftarrow$  false
9:   while queue is not empty AND not foundAugmentingPath do
10:     $u \leftarrow$  queue.dequeue()
11:    if  $u \in U$  then ▷ Vertex in left partition
12:      for all edges  $(u, v) \in E$  where  $(u, v) \notin M$  do ▷ Only non-matching edges
13:        if level[ $v$ ] =  $\infty$  then
14:          level[ $v$ ]  $\leftarrow$  level[ $u$ ] + 1
15:          Add edge  $(u, v)$  to  $L$ 
16:          queue.enqueue( $v$ )
17:        end if
18:      end for
19:    else ▷  $u \in V$ , vertex in right partition
20:      if  $u$  is exposed then ▷ Found augmenting path
21:        foundAugmentingPath  $\leftarrow$  true
22:      else
23:        Let  $(v, u)$  be the unique edge in  $M$  containing  $u$ 
24:        if level[ $v$ ] =  $\infty$  then
25:          level[ $v$ ]  $\leftarrow$  level[ $u$ ] + 1
26:          Add edge  $(u, v)$  to  $L$  ▷ Add the matching edge to level graph
27:          queue.enqueue( $v$ )
28:        end if
29:      end if
30:    end if
31:  end while
32:  return  $L$ 
33: end procedure
```

Algorithm 20 DFS to Find Augmenting Path

```
1: procedure DFS( $L, u, M, P$ )
2:   if  $u$  is exposed in  $V$  then ▷ Found augmenting path ending at exposed vertex in  $V$ 
3:     Add current path to  $P$ 
4:     return true
5:   end if
6:   for all edges  $(u, v)$  in  $L$  do
7:     Mark  $(u, v)$  as visited
8:     if DFS( $L, v, M, P$ ) then
9:       return true
10:    end if
11:  end for
12:  return false
13: end procedure
```

Definition 88 (Hopcroft-Karp Algorithm). The Hopcroft-Karp algorithm is a method for finding a maximum matching in a bipartite graph $G = (U \cup V, E)$ where U and V are disjoint vertex sets and all edges in E connect a vertex in U to a vertex in V . The algorithm iteratively increases the size of a matching by finding multiple shortest augmenting paths in each phase until no more augmenting paths exist. An augmenting path is defined as a path that starts and ends at unmatched vertices and alternates between edges not in the matching and edges in the matching.

1. **Multiple Augmenting Paths:** Unlike the standard augmenting path algorithm, Hopcroft-Karp finds multiple shortest augmenting paths in each phase.
2. **Level Graph:**
 - The algorithm builds a level graph using BFS where layers alternate between vertices in U and V .
 - Each layer contains vertices at the same distance from the source.
 - Paths in the level graph are guaranteed to be shortest augmenting paths.
3. **DFS for Finding Disjoint Paths:** After building the level graph, the algorithm uses DFS to find a maximal set of vertex-disjoint augmenting paths.
4. **Symmetric Difference:** For each augmenting path p , the matching is updated using symmetric difference: $M \oplus p = (M \setminus p) \cup (p \setminus M)$.
5. **Efficiency:** The algorithm performs at most $O(\sqrt{|V|})$ phases, each taking $O(|E|)$ time, yielding the overall $O(|E|\sqrt{|V|})$ time complexity.

Theorem 73 (Alternating Path Decomposition). For any two matchings M and M' in a graph G , the symmetric difference $M \oplus M'$ can be decomposed into alternating paths and alternating cycles.

Corollary 35. For any matching M that is not maximum, there exists an augmenting path P such that $M \oplus P$ is a matching with $|M \oplus P| = |M| + 1$.

Lemma 57 (Hopcroft-Karp). If no augmenting paths of length less than or equal to $2k - 1$ exist, then the matching M is at most $\frac{|M^*|}{k}$ edges away from a maximum matching M^* .

Lemma 58 (Monotonicity of Shortest Augmenting Paths). In successive phases of the Hopcroft-Karp algorithm, the length of the shortest augmenting paths is strictly increasing.

Lemma 59 (Phase Count). The Hopcroft-Karp algorithm performs at most $O(\sqrt{|V|})$ phases, where each phase increases the length of the shortest augmenting path.

Theorem 74 (Disjoint Augmenting Paths). Given a set of vertex-disjoint augmenting paths, applying all of them simultaneously results in a valid matching.

Lemma 60 (Length of Augmenting Paths). If M is a non-maximum matching and M^* is a maximum matching in graph G , then the symmetric difference $M \oplus M^*$ contains at least one M -augmenting path.

Theorem 75 (König–Egerváry). In any bipartite graph $G = (U \cup V, E)$, the size of a maximum matching equals the size of a minimum vertex cover:

$$\nu(G) = \tau(G).$$

Theorem 76 (Alternating Path Decomposition). For any two matchings M and M' in a graph G , the symmetric difference

$$M \oplus M' = (M \setminus M') \cup (M' \setminus M)$$

can be decomposed into a set of vertex-disjoint alternating paths and alternating cycles.

Corollary 36. If M is a non-maximum matching in G , then there exists an augmenting path P such that

$$|M \oplus P| = |M| + 1.$$

Lemma 61 (Hopcroft–Karp Distance Bound). Let M be the current matching and M^* a maximum matching in a bipartite graph G . If there is no M -augmenting path of length $\leq 2k - 1$, then

$$|M| \geq |M^*| - k.$$

Equivalently, M is at most k edges short of optimal.

Lemma 62 (Phase Count). In the Hopcroft–Karp algorithm, the length of the shortest augmenting paths strictly increases between phases, and hence the total number of phases is

$$O(\sqrt{|V|}).$$

Theorem 77 (Disjoint Augmenting Paths). Given a set of vertex-disjoint M -augmenting paths in the level graph, applying the symmetric-difference update on all of them simultaneously produces a valid matching of size increased by the number of paths.

Lemma 63 (Existence in Symmetric Difference). If M is a non-maximum matching and M^* is a maximum matching, then $M \oplus M^*$ contains at least one M -augmenting path.

Lemma 64 (Blocking-Flow Phase Bound). In a unit-capacity network (equivalent to the bipartite level graph), each blocking-flow phase can be found in $O(|E|)$ time, and there are at most $O(\sqrt{|V|})$ such phases.

Definition 89 (Hungarian Algorithm). The Hungarian Algorithm (also known as the Kuhn-Munkres algorithm) is a combinatorial optimization method that solves the assignment problem in polynomial time. Given a weighted bipartite graph $G = (U \cup V, E)$ with weights $w : E \rightarrow \mathbb{R}$, the algorithm finds a perfect matching $M \subseteq E$ that minimizes (or maximizes) the sum of weights $\sum_{e \in M} w(e)$.

The algorithm works by iteratively improving a feasible vertex labeling and a matching until optimality is achieved. It maintains a set of feasible vertex labels $l : U \cup V \rightarrow \mathbb{R}$ that satisfy the condition $l(u) + l(v) \leq w(u, v)$ for all $(u, v) \in E$ (for minimization) or $l(u) + l(v) \geq w(u, v)$ (for maximization). The time complexity of the Hungarian Algorithm is $O(|V|^3)$.

Algorithm 21 Hungarian Algorithm (Minimization Version)

Require: Bipartite graph $G = (U \cup V, E)$ with weights $w : E \rightarrow \mathbb{R}$

Ensure: Perfect matching M with minimum total weight

```

1: Initialize feasible labeling  $l : U \cup V \rightarrow \mathbb{R}$  where:
2:    $l(u) = \min_{v \in V} w(u, v)$  for all  $u \in U$ 
3:    $l(v) = 0$  for all  $v \in V$ 
4: Initialize matching  $M = \emptyset$ 
5: while  $M$  is not a perfect matching do
6:   Construct equality subgraph  $G_l = (U \cup V, E_l)$  where  $E_l = \{(u, v) \in E : l(u) + l(v) = w(u, v)\}$ 
7:   if there exists an augmenting path  $P$  in  $G_l$  then
8:      $M \leftarrow M \oplus P$  ▷ Augment matching using path  $P$ 
9:   else
10:     $S \leftarrow \{ \text{unmatched vertices in } U \}$ 
11:     $T \leftarrow \{ \text{vertices in } V \text{ reachable from } S \text{ through alternating paths in } G_l \}$ 
12:     $\alpha \leftarrow \min_{u \in S, v \in V \setminus T} \{l(u) + l(v) - w(u, v)\}$ 
13:    Update labels:
14:     $l(u) \leftarrow l(u) - \alpha$  for all  $u \in S$ 
15:     $l(v) \leftarrow l(v) + \alpha$  for all  $v \in T$ 
16:   end if
17: end while
18: return  $M$ 
```

Lemma 65 (Feasible Labeling Optimality). If l is a feasible labeling and M is a perfect matching in the equality graph G_l , then M is a minimum-weight perfect matching in the original graph G .

Lemma 66 (Complementary Slackness). For a feasible labeling l and a matching M , if $(u, v) \in M$ implies $l(u) + l(v) = w(u, v)$, then M is an optimal matching.

Lemma 67 (Labeling Update Invariant). The update rule $l(u) \leftarrow l(u) - \alpha$ for $u \in S$ and $l(v) \leftarrow l(v) + \alpha$ for $v \in T$ (where $\alpha = \min_{u \in S, v \in V \setminus T} \{l(u) + l(v) - w(u, v)\}$) maintains the feasibility of the labeling.

Lemma 68 (Labeling Value and Matching Weight). For any feasible labeling l and any perfect matching M , the sum of all vertex labels $\sum_{v \in U \cup V} l(v)$ provides a lower bound on the total weight of M .

Lemma 69 (Duality Gap). For a feasible labeling l and matching M , the difference between the sum of labels $\sum_{v \in U \cup V} l(v)$ and the weight of M is zero if and only if M is optimal.

Lemma 70 (Augmenting Path Property). If P is an augmenting path in the equality graph G_l , then applying the symmetric difference $M \oplus P$ strictly increases the size of the matching while maintaining the property that matched edges are in the equality graph.

Definition 90 (Kuhn-Munkres Algorithm). The Kuhn-Munkres algorithm (also known as the Hungarian algorithm) is a combinatorial optimization algorithm that solves the assignment problem in polynomial time. Given a weighted bipartite graph $G = (U \cup V, E)$ with $|U| = |V| = n$ and weight function $w : E \rightarrow \mathbb{R}$, it finds a perfect matching $M \subseteq E$ that minimizes the total weight $\sum_{e \in M} w(e)$.

The algorithm maintains a feasible vertex labeling $l : U \cup V \rightarrow \mathbb{R}$ and iteratively improves both the labeling and a partial matching until a perfect matching is found in the equality subgraph $G_l = (U \cup V, E_l)$ where $E_l = \{(u, v) \in E : l(u) + l(v) = w(u, v)\}$. By the theory of complementary slackness, when a perfect matching exists in G_l , it is guaranteed to be an optimal solution to the original assignment problem.

The time complexity of the Kuhn-Munkres algorithm is $O(n^3)$, where n is the number of vertices in each partition of the bipartite graph.

Theorem 78 (Kuhn-Munkres Optimality). If l is a feasible labeling and M is a perfect matching in the equality graph G_l , then M is an optimal assignment (minimum-weight perfect matching) for the original weighted bipartite graph G .

Lemma 71 (Feasible Labeling Lower Bound). For any feasible labeling l and any perfect matching M in G , the following inequality holds:

$$\sum_{u \in U} l(u) + \sum_{v \in V} l(v) \leq \sum_{(u, v) \in M} w(u, v) \quad (55)$$

Theorem 79 (Strong Duality). The minimum weight of a perfect matching equals the maximum value of $\sum_{u \in U} l(u) + \sum_{v \in V} l(v)$ over all feasible labelings l .

Lemma 72 (Invariant During Algorithm). Throughout the execution of the Kuhn-Munkres algorithm, for any edge $(u, v) \in M$, the equality $l(u) + l(v) = w(u, v)$ is maintained.

Lemma 73 (Labeling Update Property). The labeling update rule:

$$\alpha = \min_{u \in S, v \in V \setminus T} \{l(u) + l(v) - w(u, v)\} \quad (56)$$

$$l'(x) = \begin{cases} l(x) - \alpha & \text{if } x \in S \\ l(x) + \alpha & \text{if } x \in T \\ l(x) & \text{otherwise} \end{cases} \quad (57)$$

preserves the feasibility of the labeling and adds at least one new edge to the equality graph G_l .

Lemma 74 (Alternating Path Property). If P is an alternating path in the equality graph G_l from an unmatched vertex in U to an unmatched vertex in V , then $M \oplus P$ is a matching of size $|M| + 1$ that maintains the property that all matched edges belong to the equality graph.

Algorithm 22 Kuhn-Munkres Algorithm

Require: Complete bipartite graph $G = (U \cup V, E)$ with $|U| = |V| = n$ and weights $w : E \rightarrow \mathbb{R}$

Ensure: Perfect matching M with minimum total weight

```
1: Initialize feasible labeling  $l$ :
2:  $l(u) = \min_{v \in V} w(u, v)$  for all  $u \in U$ 
3:  $l(v) = 0$  for all  $v \in V$ 
4: Initialize  $M = \emptyset$  ▷ Current matching
5: Initialize  $S = \emptyset, T = \emptyset$  ▷ Vertex sets for alternating tree
6: while  $|M| < n$  do ▷ While matching is not perfect
7:   if  $S = \emptyset$  then
8:     Choose an unmatched vertex  $u \in U$ 
9:      $S \leftarrow \{u\}$ 
10:     $T \leftarrow \emptyset$ 
11:   end if
12:   if  $N_l(S) = T$  then ▷  $N_l(S)$  is the neighbors of  $S$  in the equality subgraph
13:      $\alpha \leftarrow \min_{u \in S, v \in V \setminus T} \{l(u) + l(v) - w(u, v)\}$ 
14:     Update labels:
15:      $l(u) \leftarrow l(u) - \alpha$  for all  $u \in S$ 
16:      $l(v) \leftarrow l(v) + \alpha$  for all  $v \in T$ 
17:   else
18:     Choose  $v \in N_l(S) \setminus T$ 
19:     if  $v$  is not matched in  $M$  then
20:       Augment  $M$  along the alternating path from unmatched  $u \in S$  to  $v$ 
21:        $S \leftarrow \emptyset, T \leftarrow \emptyset$  ▷ Reset alternating tree
22:     else
23:       Let  $(z, v) \in M$  be the edge matching  $v$ 
24:        $S \leftarrow S \cup \{z\}$ 
25:        $T \leftarrow T \cup \{v\}$ 
26:     end if
27:   end if
28: end while
29: return  $M$ 
```

Remark 88 (Relation to Linear Programming). *The assignment problem can be formulated as a linear program:*

$$\text{minimize} \quad \sum_{(u,v) \in E} w(u,v) \cdot x_{uv} \quad (58)$$

$$\text{subject to} \quad \sum_{v \in V} x_{uv} = 1 \quad \forall u \in U \quad (59)$$

$$\sum_{u \in U} x_{uv} = 1 \quad \forall v \in V \quad (60)$$

$$x_{uv} \geq 0 \quad \forall (u,v) \in E \quad (61)$$

The dual of this linear program corresponds to finding an optimal feasible vertex labeling.

Remark 89 (Integrality). *The constraint matrix of the assignment problem is totally unimodular, which guarantees that the linear program has an integer optimal solution, justifying the combinatorial approach of the Kuhn-Munkres algorithm.*

Remark 90 (Maximization Version). *The Kuhn-Munkres algorithm can be adapted to solve the maximum-weight assignment problem by negating all weights or by modifying the feasible labeling condition to $l(u) + l(v) \geq w(u,v)$.*

Remark 91 (Initialization Optimization). *A better initial feasible labeling can improve practical performance:*

$$l(u) = \max_{v \in V} w(u,v) \text{ for all } u \in U \text{ and } l(v) = 0 \text{ for all } v \in V \quad (62)$$

for the maximization version.

Definition 91 (Edmonds' Matching Algorithm). Let $G = (V, E)$ be an undirected graph. Edmonds' matching algorithm, also known as the Blossom algorithm, is a polynomial-time algorithm that finds a maximum matching $M \subseteq E$ such that:

1. No two edges in M share a common vertex (the definition of a matching).
2. M has the maximum possible cardinality $|M|$ among all matchings in G .
3. The algorithm works for any undirected graph, not only bipartite graphs.

The algorithm operates by iteratively finding augmenting paths in G relative to the current matching M . An augmenting path is an alternating path (a path where edges alternate between being in M and not in M) that starts and ends with unmatched vertices. When an augmenting path P is found, the matching is updated by taking the symmetric difference $M \oplus P$, which increases the size of the matching by one edge.

The key innovation of Edmonds' algorithm is the handling of *blossoms* – odd-length cycles in the graph that require special treatment. When a blossom is detected, it is temporarily contracted into a single vertex, the matching problem is solved in the resulting contracted graph, and then the blossom is expanded to obtain a solution for the original graph.

The time complexity of the original algorithm is $O(|V|^3)$, where $|V|$ is the number of vertices in the graph. More efficient implementations can achieve $O(|V| \cdot |E|)$ complexity.

Lemma 75 (Blossom Lemma). Let $G = (V, E)$ be a graph with matching M . If $B \subset V$ forms a blossom with respect to M , and G' is the graph obtained by contracting B to a single vertex b , with matching M' derived from M , then G contains an augmenting path with respect to M if and only if G' contains an augmenting path with respect to M' .

Theorem 80 (Tutte-Berge Formula). For any graph $G = (V, E)$, the size of a maximum matching in G is given by:

$$\nu(G) = \min_{S \subseteq V} \frac{1}{2} (|V| - \text{odd}(G - S) + |S|)$$

where $\text{odd}(G - S)$ denotes the number of connected components in $G - S$ having an odd number of vertices, and the minimum is taken over all subsets S of V .

Algorithm 23 Find Augmenting Path

```
1: procedure FINDAUGMENTINGPATH( $G = (V, E), M$ )
2:   Let  $F$  be the forest of alternating trees rooted at free vertices
3:   while  $F$  can be expanded do
4:     if there exists an edge  $(u, v)$  where  $u$  is in  $F$ ,  $v$  is not in  $F$ , and  $v$  is free then
5:       Add  $v$  and edge  $(u, v)$  to  $F$ 
6:       return the augmenting path from the root of  $u$ 's tree to  $v$ 
7:     else if there exists an edge  $(u, v)$  where  $u$  is in  $F$ ,  $v$  is not in  $F$ , and  $v$  is matched then
8:       Let  $w$  be the vertex matched to  $v$  in  $M$ 
9:       Add  $v, w$ , and edges  $(u, v), (v, w)$  to  $F$ 
10:    else if there exists an edge  $(u, v)$  where  $u, v$  are in  $F$  but in different trees then
11:      return the augmenting path through  $u$  and  $v$ 
12:    else if there exists an edge  $(u, v)$  where  $u, v$  are in  $F$  and in the same tree then
13:      Contract the blossom containing  $u$  and  $v$  into a single vertex
14:      Update  $F$  and  $G$  accordingly
15:    else
16:      return No augmenting path exists
17:    end if
18:  end while
19:  return No augmenting path exists
20: end procedure
```

Lemma 76 (Gallai-Edmonds Decomposition). For any graph $G = (V, E)$, the vertex set V can be partitioned into three sets D , A , and C as follows:

- $D = \{v \in V : v \text{ is not covered by at least one maximum matching}\}$
- $A = \{v \in V \setminus D : v \text{ is adjacent to at least one vertex in } D\}$
- $C = V \setminus (D \cup A)$

Moreover, this decomposition has the following properties:

1. Each connected component of the subgraph induced by D has an odd number of vertices.
2. Every maximum matching in G contains a perfect matching on the subgraph induced by C .
3. Every maximum matching in G matches each vertex in A with a vertex in D , with each component of $G[D]$ matched to at most one vertex in A .
4. The size of a maximum matching in G is $\frac{1}{2}(|V| - \text{odd}(G[D]) + |A|)$, where $\text{odd}(G[D])$ is the number of odd components in $G[D]$.

Theorem 81 (Dulmage-Mendelsohn Decomposition). Let $G = (X \cup Y, E)$ be a bipartite graph with bipartition (X, Y) . Let M be a maximum matching in G . Then there exists a partition of X into sets X_1, X_2, X_3 and a partition of Y into sets Y_1, Y_2, Y_3 such that:

1. M matches X_1 to Y_1 , X_2 to Y_2 , and leaves all vertices in $X_3 \cup Y_3$ unmatched.
2. There are no edges between X_1 and Y_3 , or between X_3 and Y_1 .
3. All maximum matchings in G match X_1 exactly to Y_1 .
4. Every matching of maximum size in the subgraph induced by $X_2 \cup Y_2$ can be extended to a maximum matching of G by including a perfect matching between X_1 and Y_1 .

Theorem 82 (Petersen's Theorem). Every 3-regular graph with no bridges has a perfect matching.

Remark 92. *Tutte's theorem is a generalization of Petersen's theorem, which states that every 3-regular graph with no bridges has a perfect matching.*

Lemma 77 (Blossoms Intersection). Let B_1 and B_2 be two distinct blossoms in graph G with respect to matching M . If $B_1 \cap B_2 \neq \emptyset$, then either:

1. $|B_1 \cap B_2| = 1$, i.e., the blossoms share exactly one vertex, or
2. $B_1 \subset B_2$, i.e., one blossom is entirely contained within the other, or
3. $B_2 \subset B_1$.

Algorithm 24 Contract Blossom

```

1: procedure CONTRACTBLOSSOM( $G = (V, E)$ ,  $F$ ,  $u$ ,  $v$ )
2:   Find the lowest common ancestor  $w$  of  $u$  and  $v$  in  $F$ 
3:   Let  $B$  be the vertices in the cycle formed by the paths from  $w$  to  $u$  and  $w$  to  $v$ , plus the edge  $(u, v)$ 
4:   Create a new vertex  $b$  representing the blossom  $B$ 
5:   Replace all edges incident to vertices in  $B$  with edges to  $b$ 
6:   Update the matching  $M$  to reflect the contraction
7:   Update the forest  $F$  to replace the blossom with  $b$ 
8:   return the contracted graph and updated forest
9: end procedure

```

Algorithm 25 Expand Blossom

```

1: procedure EXPANDBLOSSOM( $G$ ,  $M$ ,  $B$ )
2:   Let  $b$  be the vertex representing blossom  $B$ 
3:   if  $b$  is not matched in  $M$  then
4:     Expand  $b$  back to the original vertices in  $B$ 
5:     Keep the matching edges within  $B$  as they were before contraction
6:   else
7:     Let  $(b, v)$  be the edge in  $M$  incident to  $b$ 
8:     Find the vertex  $u \in B$  such that  $(u, v)$  was an edge in the original graph
9:     Expand  $b$  back to the original vertices in  $B$ 
10:    Include  $(u, v)$  in  $M$ 
11:    Match the remaining vertices in  $B$  optimally along the blossom
12:   end if
13:   return the expanded graph and updated matching
14: end procedure

```

Remark 93. *The time complexity of Edmonds' algorithm is $O(|V|^3)$ where $|V|$ is the number of vertices in the graph. This can be improved to $O(|V|^2|E|)$ or even $O(|V||E|)$ with more sophisticated implementations.*

Definition 92 (Stable Matching). Let A and B be two disjoint sets of equal size, where each element in set A has a strict preference ordering over the elements in set B , and each element in set B has a strict preference ordering over the elements in set A . A matching M is a one-to-one correspondence between the elements of A and B .

A matching M is said to be *stable* if and only if there does not exist any pair $(a, b) \in A \times B$ such that:

1. $(a, b) \notin M$, i.e., a and b are not matched to each other in M , and
2. a prefers b to its current partner in M , and
3. b prefers a to its current partner in M .

Such a pair (a, b) is called a *blocking pair* or an *unstable pair*. A stable matching is thus a matching with no blocking pairs.

Theorem 83 (Gale-Shapley). For any instance of the stable matching problem with equal-sized sets, a stable matching always exists and can be found in $O(n^2)$ time, where n is the size of each set.

Definition 93 (Hospital-Resident Problem). The Hospital-Resident problem (also known as the College Admissions problem) is a generalization of the stable matching problem where:

- Set B (hospitals) may be matched to multiple elements from set A (residents).
- Each hospital $h \in B$ has a capacity c_h indicating the maximum number of residents it can accommodate.
- A matching M is stable if there is no resident-hospital pair (r, h) such that:
 1. r is unmatched or prefers h to their current assignment, and
 2. h is under capacity or prefers r to at least one of its currently matched residents.

Definition 94 (Stable Roommates Problem). The Stable Roommates problem is a variation of the stable matching problem where:

- There is only one set of individuals (rather than two disjoint sets).
- Each person has a preference ranking over all other individuals.
- A matching pairs individuals as roommates.
- A matching is stable if there are no two individuals who prefer each other to their assigned roommates.

Unlike the original stable matching problem, a stable solution is not guaranteed to exist in the Stable Roommates problem.

Definition 95 (Gale-Shapley Algorithm). The Gale-Shapley Algorithm, also known as the Deferred Acceptance Algorithm, is a procedure to find a stable matching between two equally sized sets, where each element has a strict preference ordering over elements in the other set.

Let $A = \{a_1, a_2, \dots, a_n\}$ and $B = \{b_1, b_2, \dots, b_n\}$ be the two sets, with each element having a preference list ranking all elements of the other set.

Algorithm 26 Gale-Shapley Algorithm

```

1: procedure GALESHAPLEY( $A, B$ , preferences)
2:   Initialize all  $a \in A$  and  $b \in B$  as unmatched
3:   Initialize a list nextProposal[ $a$ ] for each  $a \in A$ , initially pointing to their most preferred  $b$ 
4:   while there exists an unmatched  $a \in A$  who has not proposed to all  $b \in B$  do
5:     Select any such unmatched  $a$ 
6:     Let  $b = \text{nextProposal}[a]$  be the highest-ranked  $b$  to whom  $a$  has not yet proposed
7:     Advance nextProposal[ $a$ ] to the next  $b$  in  $a$ 's preference list
8:     if  $b$  is unmatched then
9:       Match  $a$  and  $b$ 
10:    else if  $b$  prefers  $a$  to their current match  $a'$  then
11:      Unmatch  $b$  and  $a'$ 
12:      Match  $a$  and  $b$ 
13:    end if
14:  end while
15:  return the current matching
16: end procedure

```

Theorem 84 (Properties of Gale-Shapley Algorithm). The Gale-Shapley algorithm has the following properties:

1. It always terminates in at most n^2 iterations, where $n = |A| = |B|$.
2. It always produces a stable matching.
3. The resulting matching is A -optimal, meaning that each $a \in A$ receives the best partner possible in any stable matching.
4. The resulting matching is B -pessimal, meaning that each $b \in B$ receives the worst partner possible in any stable matching.

By reversing the roles of sets A and B , we obtain the B -optimal (and A -pessimal) stable matching.

Theorem 85 (Optimality of Gale-Shapley). In the matching produced by the Gale-Shapley algorithm with elements from set A proposing:

1. Every $a \in A$ receives the best partner that they can receive in any stable matching.
2. Every $b \in B$ receives the worst partner that they can receive in any stable matching.

Theorem 86 (Rural Hospitals Theorem). In the hospital-resident variation of the stable matching problem:

1. The same set of hospitals and residents are matched in all stable matchings.
2. Any hospital that does not fill its quota in one stable matching will have exactly the same residents in all stable matchings.
3. The same number of residents is matched in every stable matching.

Theorem 87 (König-Egerváry). Let $G = (X \cup Y, E)$ be a bipartite graph with bipartition (X, Y) . Then the maximum size of a matching in G equals the minimum size of a vertex cover in G .

In other words:

$$\nu(G) = \tau(G)$$

where $\nu(G)$ denotes the matching number (the size of a maximum matching) and $\tau(G)$ denotes the vertex cover number (the size of a minimum vertex cover).

Corollary 37 (Dual Form of König-Egerváry). In a bipartite graph $G = (X \cup Y, E)$, the maximum size of a matching equals the maximum size of an independent set:

$$\nu(G) = |V| - \alpha(G)$$

where $\alpha(G)$ is the independence number of G (the size of a maximum independent set).

Corollary 38 (König's Line Coloring Theorem). For any bipartite graph G , the edge chromatic number equals the maximum degree:

$$\chi'(G) = \Delta(G)$$

where $\chi'(G)$ is the minimum number of colors needed to color the edges of G such that no two adjacent edges receive the same color, and $\Delta(G)$ is the maximum degree of any vertex in G .

Remark 94. The König-Egerváry theorem is a special case of Edmonds' matching polytope theorem for bipartite graphs, and it fails to hold for general non-bipartite graphs.

Remark 95. The theorem provides a min-max relation that is fundamental in combinatorial optimization and establishes a duality between matching and vertex cover problems in bipartite graphs. It is also equivalent to the integrality of the matching polytope for bipartite graphs.

Theorem 88 (Relationship Between König-Egerváry Theorem and König-Ore Formula). Let $G = (A \cup B, E)$ be a bipartite graph. The following relationship exists between the König-Egerváry theorem and the König-Ore formula:

$$\tau(G) = \nu(G) \quad (\text{König-Egerváry}) \quad (63)$$

$$\nu(G) = |A| - \max_{S \subseteq A} \{|S| - |N(S)|\} \quad (\text{From König-Ore}) \quad (64)$$

where:

- $\tau(G)$ is the minimum size of a vertex cover
- $\nu(G)$ is the maximum size of a matching
- $\nu(G, A) = \max_{S \subseteq A} \{|S| - |N(S)|\}$ is the deficiency of G with respect to A (from König-Ore)

Thus, the maximum matching size can be expressed as:

$$\nu(G) = |A| - \nu(G, A)$$

This shows that the deficiency $\nu(G, A)$ from the König-Ore formula directly determines how far a maximum matching is from being perfect (covering all vertices in A).

Corollary 39. The relationships can be further connected through Hall's Marriage Theorem:

1. A bipartite graph has a matching covering all vertices in A if and only if $\nu(G, A) = 0$
2. This is equivalent to the condition that for all $S \subseteq A$, $|N(S)| \geq |S|$ (Hall's condition)
3. When $|A| = |B|$, this means the graph has a perfect matching if and only if $\nu(G) = |A| = |B|$
4. By König-Egerváry, this means there is a perfect matching if and only if the minimum vertex cover has size $|A| = |B|$

Theorem 89 (Complementary Perspective). The dual perspectives of König-Egerváry and König-Ore can be unified:

1. König-Egerváry establishes the global equality between maximum matching and minimum vertex cover
2. König-Ore provides a local characterization of deficiency through the "worst" substructures
3. Together, they show that:

$$\tau(G) = |A| - \max_{S \subseteq A} \{|S| - |N(S)|\}$$

Remark 96. Both theorems are manifestations of LP duality:

- *König-Egerváry reflects that the matching and vertex cover polytopes have integer extreme points in bipartite graphs*
- *König-Ore formula shows that the dual solution can be constructed from subsets with the maximum deficiency ratio*
- *Edmonds' algorithm for non-bipartite graphs addresses precisely the cases where these relationships become more complex due to odd cycles (blossoms)*

7.3 Matchings and Factors Exercises

7.3.1 Easy Questions

1. State the theorem on minimum cycle length in a graph with minimum degree $k \geq 2$. What is the guaranteed minimum cycle length?
2. In a graph with minimum degree 5, what is the minimum cycle length guaranteed by the long cycle theorem?
3. Prove that a graph with maximum degree 2 cannot have a cycle of length greater than the number of vertices.
4. What is the minimum cycle length in a 3-regular graph with 8 vertices? Is this bound tight? Explain.
5. Explain why the minimum cycle length in a graph must be at least 3.
6. Find the length of the longest cycle in the graph $K_{2,4}$ (complete bipartite graph with partite sets of sizes 2 and 4).
7. Prove that any graph with minimum degree $\delta(G) \geq 2$ must contain at least one cycle.

7.3.2 Medium Questions

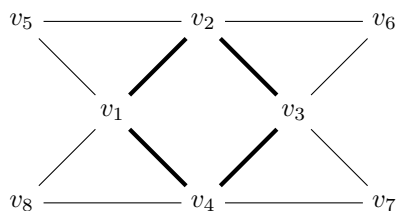
8. Let P be a longest path in a graph G with minimum degree $k \geq 2$. Prove that the endpoints of P have all their neighbors on P .
9. Explain why the bound in the long cycle theorem is tight by constructing an example of a graph with minimum degree k where the longest cycle has length exactly $k + 1$.
10. Prove that in a 2-connected graph with minimum degree $k \geq 2$, there exists a cycle of length at least $\min\{2k, n\}$.
11. A graph G has minimum degree $k = 3$. Describe an algorithm to find a cycle of length at least $k + 1 = 4$ in G .
12. Let G be a graph with minimum degree $\delta(G) = 3$. If G has exactly 6 vertices, show that G must contain a cycle of length at least 4.
13. Find the relationship between the length of the longest cycle in a graph G and the length of the longest cycle in its complement $\overline{G}$.
14. Prove that in a Hamiltonian graph, the length of the longest cycle is exactly n . Then show that the converse is not true by providing a counterexample.

7.3.3 Hard Questions

15. Let G be a graph with minimum degree $\delta(G) \geq \frac{n}{r+1} + 1$ where $r \geq 2$. Prove that G contains a cycle of length at least $r + 2$.
16. Prove or disprove: if G is a k -connected graph with minimum degree $\delta(G) \geq k + 1$, then G contains a cycle of length at least $2k + 2$.
17. Let G be a graph with minimum degree $k \geq 2$. Prove that if G has girth g (the length of its shortest cycle), then G has a cycle of length at least $\min\{2k, g + k - 1\}$.
18. Design an algorithm to find the longest cycle in a graph G . Analyze its time complexity and explain why finding the longest cycle is generally an NP-hard problem.
19. Let G be a graph with n vertices and let $\lambda(G)$ denote the length of the longest cycle in G . Prove that if $\delta(G) > \frac{n}{2}$, then $\lambda(G) \geq 2\delta(G)$ unless G is Hamiltonian.
20. Prove that if G is a 3-connected cubic graph (3-regular), then G contains a cycle of length at least $4 \log n$.

7.3.4 Diagram Questions

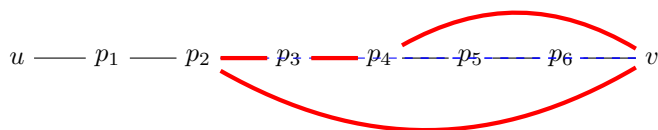
21. Draw a graph with 8 vertices and minimum degree 3 that contains a cycle of length exactly 4 but no longer cycles. Mark the cycle in your diagram.



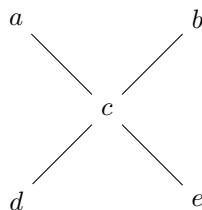
22. Illustrate the proof technique of the long cycle theorem by drawing:

- A longest path P in a graph with endpoints u and v
- The neighbors of v on the path P
- The earliest neighbor of v on the path
- The resulting cycle formed by the theorem

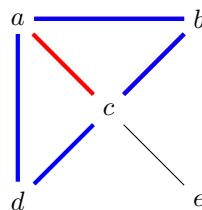
Label all relevant vertices and explain each step.



23. Draw a star graph $K_{1,4}$ and explain why the longest cycle in this graph has length 3. Then add one edge to create a cycle of length 4, and explain why this is now the longest possible cycle.

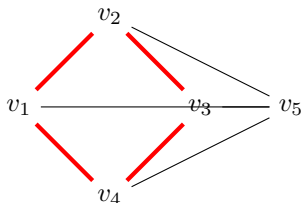


Star graph $K_{1,4}$ (no cycle)

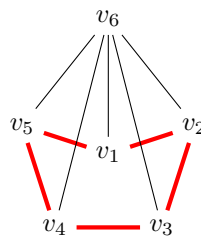


$K_{1,4}$ with added edge (a, b)

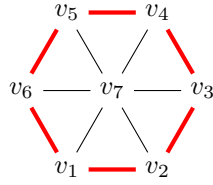
24. Create a sequence of graphs G_3, G_4, G_5 where each G_k has minimum degree k and contains a cycle of length exactly $k + 1$. Draw each graph and highlight the longest cycle in each.



G_3 : min degree 3, cycle length 4

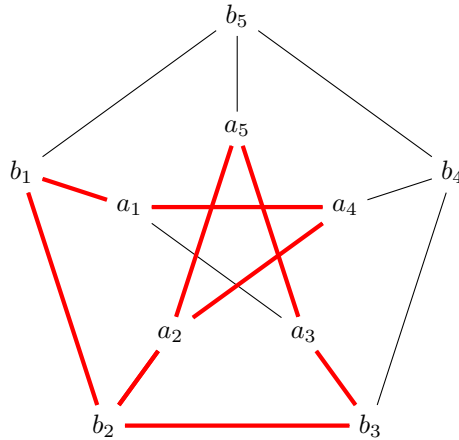


G_4 : min degree 4, cycle length 5

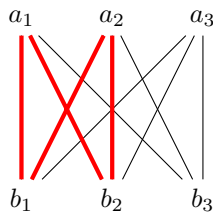


G_5 : min degree 5, cycle length 6

25. Draw the Petersen graph and identify the longest cycle in it. Explain why this graph does not have a Hamiltonian cycle despite having minimum degree 3.

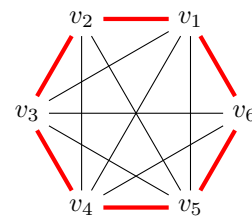
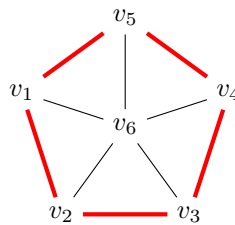
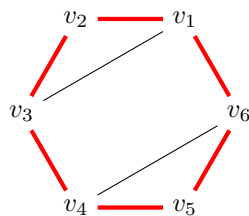


26. Draw a graph where the bound from the long cycle theorem is not tight, i.e., a graph with minimum degree k where the minimum cycle length is strictly greater than $k + 1$. Explain why this happens in your example.



$K_{3,3}$: min degree 3, shortest cycle has length 4

27. Illustrate the relationship between the minimum degree and cycle length by drawing three different graphs with 6 vertices that have minimum degrees 2, 3, and 4 respectively. For each graph, highlight the longest cycle and discuss how the minimum degree affects the cycle length.



8 Vertex and Edge-Coloring

8.1 Basic Definitions

Definition 96 (Proper k -edge-coloring). A *proper k -edge-coloring* of a graph G is an assignment of k colors to the edges of G such that no two adjacent edges (i.e., edges that share a common vertex) receive the same color. Formally, it is a function $c : E(G) \rightarrow \{1, 2, \dots, k\}$ such that if edges e_1 and e_2 share a vertex, then $c(e_1) \neq c(e_2)$.

Definition 97 (Edge-chromatic number). The *edge-chromatic number* of a graph G , denoted by $\chi'(G)$, is the minimum number of colors needed for a proper edge-coloring of G . That is, $\chi'(G) = \min\{k : G \text{ has a proper } k\text{-edge-coloring}\}$.

Definition 98 (k -edge-colorable). A graph G is said to be *k -edge-colorable* if there exists a proper edge-coloring of G using at most k colors. Equivalently, G is k -edge-colorable if $\chi'(G) \leq k$.

Definition 99 (k -edge-chromatic). A graph G is said to be *k -edge-chromatic* if its edge-chromatic number is exactly k , i.e., $\chi'(G) = k$. This means that G requires exactly k colors for a proper edge-coloring, no more and no less.

Definition 100 (Proper k -coloring). A *proper k -coloring* of a graph G is an assignment of k colors to the vertices of G such that no two adjacent vertices (i.e., vertices connected by an edge) receive the same color. Formally, it is a function $c : V(G) \rightarrow \{1, 2, \dots, k\}$ such that if vertices u and v are adjacent, then $c(u) \neq c(v)$.

Definition 101 (Chromatic number). The *chromatic number* of a graph G , denoted by $\chi(G)$, is the minimum number of colors needed for a proper vertex-coloring of G . That is, $\chi(G) = \min\{k : G \text{ has a proper } k\text{-coloring}\}$.

Definition 102 (k -colorable). A graph G is said to be *k -colorable* if there exists a proper vertex-coloring of G using at most k colors. Equivalently, G is k -colorable if $\chi(G) \leq k$.

Definition 103 (k -chromatic). A graph G is said to be *k -chromatic* if its chromatic number is exactly k , i.e., $\chi(G) = k$. This means that G requires exactly k colors for a proper vertex-coloring, no more and no less.

8.2 König's Theorem on Edge-Coloring of Bipartite Graphs

8.2.1 Background and Motivation

In graph theory, the problem of edge-coloring concerns assigning colors to the edges of a graph such that no two adjacent edges (those sharing a common vertex) receive the same color. A key question is determining the minimum number of colors required for such a proper edge-coloring of a given graph.

For any graph G , a trivial lower bound on the edge-chromatic number $\chi'(G)$ is the maximum degree $\Delta(G)$ of the graph. This is because all edges incident to a vertex of maximum degree must receive distinct colors. In general graphs, determining the exact value of $\chi'(G)$ is NP-hard, but for bipartite graphs, König's Theorem gives us a precise answer.

8.2.2 Formal Statement

Theorem 90 (König's Theorem on Edge-Coloring). If G is a bipartite graph, then the edge-chromatic number $\chi'(G)$ equals the maximum degree $\Delta(G)$ of the graph:

$$\chi'(G) = \Delta(G) \tag{65}$$

Proof via Regular Extension. The key insight in this proof is to extend the bipartite graph to a regular bipartite graph, which allows us to apply properties of matchings.

We first establish that $\chi'(G) \geq \Delta(G)$ for any graph G , since edges incident to a vertex of maximum degree must have different colors. To prove that $\chi'(G) \leq \Delta(G)$ for a bipartite graph $G = (A \cup B, E)$, we construct a $\Delta(G)$ -regular bipartite graph H that contains G as a subgraph. To construct H , take two copies

of G , denoted $G_1 = (A_1 \cup B_1, E_1)$ and $G_2 = (A_2 \cup B_2, E_2)$. For each vertex $v \in G$ with degree $d_G(v) < \Delta(G)$, we add $\Delta(G) - d_G(v)$ new edges between the corresponding vertices $v_1 \in G_1$ and $v_2 \in G_2$. The resulting graph H is $\Delta(G)$ -regular and bipartite with partite sets $(A_1 \cup A_2)$ and $(B_1 \cup B_2)$. By Hall's Theorem, a k -regular bipartite graph can be decomposed into k perfect matchings. Therefore, H can be decomposed into $\Delta(G)$ perfect matchings $M_1, M_2, \dots, M_{\Delta(G)}$. Each matching M_i defines a color class. The subgraph of H corresponding to G inherits this coloring, which gives us a proper $\Delta(G)$ -edge-coloring of G . Therefore, $\chi'(G) \leq \Delta(G)$. Combining this with the lower bound, we conclude that $\chi'(G) = \Delta(G)$. $\square$

Proof via Induction. This proof uses induction on the number of edges and a color-interchange argument. Base case: If $|E(G)| = 0$, then $\chi'(G) = 0 = \Delta(G)$, and the theorem holds. Induction hypothesis: Assume the theorem holds for all bipartite graphs with fewer than m edges, where $m \geq 1$. Induction step: Let G be a bipartite graph with exactly m edges and maximum degree $\Delta(G)$. Select any edge $e = \{x, y\} \in E(G)$. By the induction hypothesis, the graph $G' = G - e$ has a proper edge-coloring using $\Delta(G')$ colors. If $\Delta(G') < \Delta(G)$, then G' can be colored with at most $\Delta(G) - 1$ colors. In this case, we can assign a new color $\Delta(G)$ to edge e , obtaining a proper $\Delta(G)$ -edge-coloring of G . If $\Delta(G') = \Delta(G)$, then G' has a proper $\Delta(G)$ -edge-coloring. If there exists a color c that doesn't appear at either x or y , we can assign color c to edge e to obtain a proper $\Delta(G)$ -edge-coloring of G . Otherwise, let's assume the colors at x are $\{1, 2, \dots, \Delta(G) - 1\}$ and the colors at y are $\{2, 3, \dots, \Delta(G)\}$. Note that color 1 is missing at y and color $\Delta(G)$ is missing at x . Consider the subgraph H of G' containing only edges colored 1 or $\Delta(G)$. Each component of H is either a path or an even cycle (since G is bipartite). The component containing x must be a path P (not a cycle) that starts at x , since x has no incident edge of color $\Delta(G)$ in G' . This path P cannot end at y , because if it did, P would have odd length (since G is bipartite), and y would be incident with an edge of color 1, which contradicts our assumption. So P ends at a vertex $z \neq y$. Now we swap colors 1 and $\Delta(G)$ along the path P . This is still a proper edge-coloring of G' , but now color 1 is missing at both x and y . Finally, we assign color 1 to edge $e = \{x, y\}$, resulting in a proper $\Delta(G)$ -edge-coloring of G . Therefore, $\chi'(G) \leq \Delta(G)$. Since we already know that $\chi'(G) \geq \Delta(G)$ for any graph, we conclude that $\chi'(G) = \Delta(G)$. $\square$

8.2.3 Implications and Significance

König's Theorem has several important implications:

1. **Computational Tractability:** Unlike general graphs where determining the edge-chromatic number is NP-hard, for bipartite graphs, we can determine the edge-chromatic number simply by finding the maximum degree of the graph, which can be done in linear time.
2. **Relation to Matchings:** König's Theorem is closely related to matchings in bipartite graphs. In fact, a proper edge-coloring of a bipartite graph decomposes its edge set into perfect matchings, with each color class forming a matching.
3. **Special Case of Vizing's Theorem:** König's Theorem can be seen as a special case of Vizing's Theorem, which states that for any simple graph G , $\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$. König's Theorem confirms that bipartite graphs always achieve the lower bound.
4. **Application to Scheduling:** In scheduling problems where conflicts are represented by a bipartite graph, König's Theorem guarantees that the minimum number of time slots needed equals the maximum number of conflicts any single entity has.

8.2.4 Equivalent Formulations

König's Theorem can be equivalently formulated in several ways:

Theorem 91 (Equivalent Formulation - Decomposition into Matchings). A bipartite graph G with maximum degree $\Delta(G)$ can be decomposed into exactly $\Delta(G)$ matchings.

Theorem 92 (Equivalent Formulation - Regular Supergraph). Every bipartite graph G can be embedded in a $\Delta(G)$ -regular bipartite graph H such that $\chi'(H) = \Delta(G)$.

8.2.5 Historical Context

König's Theorem was first proven by the Hungarian mathematician Dénes König in 1916, making it one of the earliest results in graph theory. It laid the foundation for many subsequent developments in edge-coloring theory and bipartite graph theory.

The importance of this theorem extends beyond theoretical graph theory into various practical applications, including scheduling, resource allocation, and network design where bipartite conflict structures are common.

Lemma 78 (Color Alternating Path). Let G be a graph with a proper edge coloring. For any two colors α and β , each component of the subgraph induced by edges colored α or β is either a path or an even cycle where colors α and β alternate.

Proof. Let H be the subgraph of G induced by edges colored α or β . Since the coloring is proper, each vertex in H has degree at most 2 (at most one incident edge of each color). Therefore, each component of H is either a path or a cycle.

In any such component, the colors must alternate, since no two adjacent edges can have the same color. In a cycle, this alternation implies that the cycle must have even length, since we need to return to the starting color after traversing the cycle. $\square$

8.3 Vizing's Theorem

8.3.1 Formal Statement

Theorem 93 (Vizing's Theorem). For any simple graph G , the edge-chromatic number $\chi'(G)$ satisfies:

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1 \quad (66)$$

where $\Delta(G)$ is the maximum degree of G .

This theorem establishes tight bounds on the edge-chromatic number of any simple graph, classifying all graphs into two categories:

- **Class 1 graphs:** Graphs where $\chi'(G) = \Delta(G)$
- **Class 2 graphs:** Graphs where $\chi'(G) = \Delta(G) + 1$

The lower bound $\chi'(G) \geq \Delta(G)$ is immediate since all edges incident to a vertex of maximum degree must receive different colors. The upper bound $\chi'(G) \leq \Delta(G) + 1$ is the non-trivial part of the theorem.

Proof. Since Δ different colors are needed at a vertex of degree Δ in G , we have $\chi'(G) \geq \Delta$.

We now prove by induction on $|E(G)|$ that G is $(\Delta + 1)$ -colorable, which gives $\chi'(G) \leq \Delta + 1$.

Base Case: If $|E(G)| = 0$, then the theorem is trivially true.

Induction Step: Suppose $|E(G)| > 0$, and let $\{x, y_1\} \in E(G)$ be any edge of G . By induction hypothesis, $G_1 = G - \{x, y_1\}$ is $(\Delta + 1)$ -colorable.

If there is a color, say color c_1 , missing at both y_1 and x , then we can assign edge $\{x, y_1\}$ the color c_1 and we're done.

So we may assume that every color missing at x appears on an edge incident to y_1 . Let c be a color missing at x – we know c appears on y_1 , otherwise $\{x, y_1\}$ could be colored with color c .

In general, we construct a maximal sequence $y_1, y_2, \dots, y_k$ of neighbors of x such that:

- c_i is missing at y_i for all $i \leq k$
- $\{x, y_{i+1}\}$ has color c_i for all $i < k$
- Color c_k is missing at y_k and does not appear on any edge $\{x, y\}$ for $y \notin \{y_1, y_2, \dots, y_k\}$

We now consider two cases:

Case 1. For all $i < k$, $c_k \neq c_i$.

In this case, a proper edge-coloring of G is found by recoloring $\{x, y_j\}$ with color c_j for all $j \leq k$. Note that the coloring is proper since color c_j is missing at y_j for all $j \leq k$. This is essentially a "fan" recoloring around vertex x .

Case 2. For some $i < k$, $c_k = c_i$.

In this case, we recolor all edges $\{x, y_j\}$ for $j \leq i$ with color c_j – so far we still have a proper coloring since color c_j is missing at y_j for $j \leq i$.

Then $\{x, y_{i+1}\}$ is the new uncolored edge, since the edge $\{x, y_1\}$ has now received color c_1 . Let H denote the subgraph of G consisting of edges of color c and edges of color c_k . Then the components of H are paths and cycles, since H has maximum degree at most two.

Also x , y_{i+1} , and y_k all have degree one in H , so either x and y_{i+1} are in different components of H , or x and y_k are in different components of H . We consider these cases separately:

Subcase 2.1: If x and y_{i+1} are in different components of H , then we interchange colors c and c_i in the component of H containing y_{i+1} . In this new coloring, color c is missing at x and missing at y_{i+1} , so we can assign the edge $\{x, y_{i+1}\}$ the color c .

Subcase 2.2: If x and y_k are in different components of H , then we recolor the edge $\{x, y_j\}$ for $i < j < k$ with color c_j , so that $\{x, y_k\}$ is the new uncolored edge. Then H is unchanged (we never recolored edges of color c or c_i) so we may interchange the colors c and c_k in the component of H containing y_k . In doing so, c becomes a missing color at x and y_k , so the uncolored edge $\{x, y_k\}$ can be colored with color c .

This completes the proof. $\square$

8.3.2 Key Lemmas Supporting Vizing's Theorem

Lemma 79 (Vizing's Adjacency Lemma). If G is a simple graph, uv is an edge of G , and ϕ is a proper $(\Delta(G) + 1)$ -edge-coloring of $G - uv$, then there is at least one color that appears at u but not at v in ϕ .

Proof. Since G is simple, u and v have at most one common neighbor. Each of u and v has at most $\Delta(G) - 1$ neighbors other than each other. So the number of colors appearing at both u and v is at most $1 + \min(\Delta(G) - 1, \Delta(G) - 1) = \Delta(G)$. Since there are $\Delta(G) + 1$ colors in total, by the pigeonhole principle, there must be at least one color that appears at u but not at v (and vice versa). $\square$

Lemma 80 (Color Alternating Path Lemma). Let ϕ be a proper edge-coloring of a graph G , and let α and β be two colors. Each component of the subgraph of G induced by edges colored α or β is either a path or an even cycle where the colors α and β alternate.

Proof. Consider the subgraph H of G induced by edges colored α or β . At each vertex, there can be at most one edge of color α and at most one edge of color β . Thus, each vertex in H has degree at most 2. A connected graph with maximum degree 2 is either a path or a cycle.

In any such component, the colors must alternate between α and β , since no two adjacent edges can have the same color in a proper coloring. In a cycle, this alternation implies that the cycle must have even length, since we need to return to the starting color after traversing the cycle. $\square$

8.3.3 Implications and Applications

Vizing's Theorem has profound implications for edge coloring theory and practical applications:

1. **Classification of Graphs:** The theorem partitions all simple graphs into exactly two classes: Class 1 graphs where $\chi'(G) = \Delta(G)$ and Class 2 graphs where $\chi'(G) = \Delta(G) + 1$.
2. **Computational Complexity:** While Vizing's Theorem gives tight bounds, determining whether a graph is Class 1 or Class 2 is NP-complete in general.
3. **Relationship to Bipartite Graphs:** König's Theorem states that all bipartite graphs are Class 1, i.e., $\chi'(G) = \Delta(G)$ for bipartite graphs.

4. **Scheduling Applications:** In scheduling problems, Vizing's Theorem guarantees that any conflict graph can be edge-colored with at most one more color than the maximum degree, providing efficient resource allocation.
5. **Network Design:** For communication networks, the theorem provides bounds on the minimum number of channels needed to avoid interference.

8.3.4 Historical Context

Vizing's Theorem was first proven by Vadim G. Vizing in 1964. The proof presented here is a refinement of Vizing's original proof, emphasizing the constructive aspects of the proof which naturally lead to polynomial-time algorithms for finding a $(\Delta(G) + 1)$ -edge-coloring of any simple graph.

The theorem represents one of the most elegant and important results in edge coloring theory, and despite its age, finding efficient algorithms for determining whether a graph is Class 1 or Class 2 remains an active area of research.

8.3.5 Examples of Vizing's and König's Theorem

A key result related to Vizing's Theorem is König's Line Coloring Theorem, which states that any bipartite graph is Class 1. Here's an example of a bipartite graph with maximum degree 3:

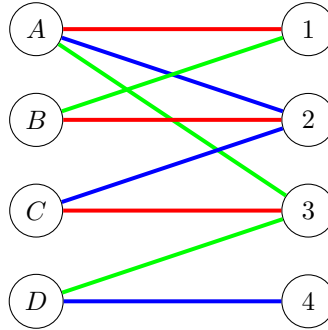


Figure 125 : This bipartite graph has a maximum degree $\Delta(G) = 3$ (vertex A has 3 edges). As König's theorem predicts, we can color all edges using exactly 3 colors (red, blue, and green). This demonstrates that the graph is Class 1.

The complete graph K_3 (a triangle) has maximum degree $\Delta(K_3) = 2$. According to Vizing's theorem, it requires either 2 or 3 colors. Let's see:

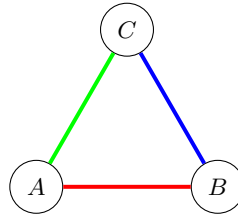


Figure 126 : Here, we need 3 colors for the 3 edges, showing that K_3 is actually Class 2 since $\chi'(K_3) = 3 = \Delta(K_3) + 1$.

The complete graph K_4 has maximum degree $\Delta(K_4) = 3$. Let's try to color it:

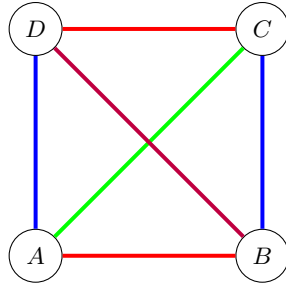


Figure 127 : We need 4 colors for the edges of K_4 , confirming it's a Class 2 graph since $\chi'(K_4) = 4 = \Delta(K_4) + 1$. The cycle graph C_5 has maximum degree $\Delta(C_5) = 2$. Let's color its edges:

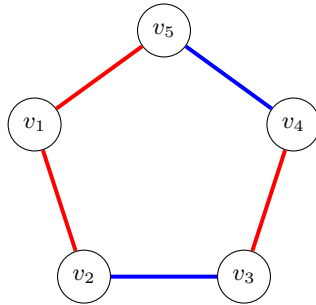


Figure 128 : We can color C_5 with just 2 colors, showing that it's a Class 1 graph since $\chi'(C_5) = 2 = \Delta(C_5)$. The Petersen graph is a famous example of a Class 2 graph. It has maximum degree $\Delta = 3$:

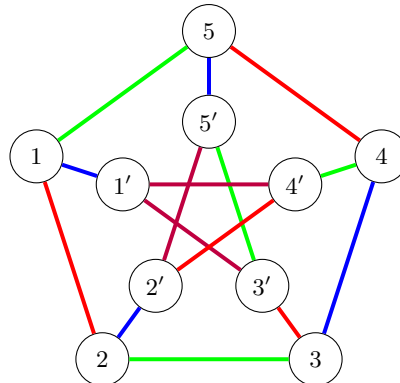


Figure 129 : The Petersen graph requires 4 colors, making it a Class 2 graph since $\chi'(Petersen) = 4 = \Delta + 1$. Here's a cubic graph (3-regular) that is Class 1. Every vertex has degree 3, and we can color it with exactly 3 colors:

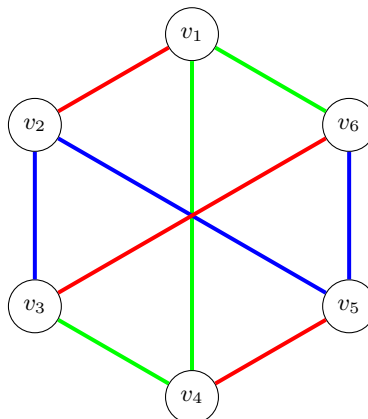


Figure 130 : This 3-regular graph can be colored with exactly 3 colors, so it's a Class 1 graph. Here's another example of a Class 2 graph with maximum degree 3:

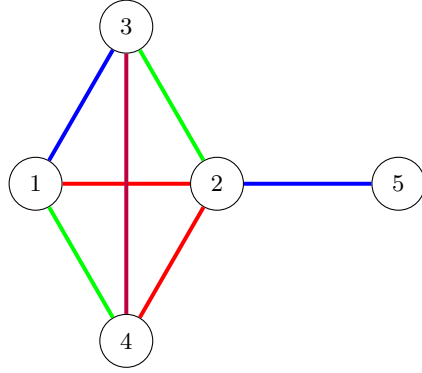


figure 131 : This graph has maximum degree $\Delta = 3$ at vertices 1 and 2, but requires 4 colors for its edges, making it a Class 2 graph.

Determining whether a graph is Class 1 or Class 2 is generally NP-complete. However, some specific cases are known:

- Bipartite graphs are always Class 1 (König's theorem)
- Complete graphs K_n are Class 1 if n is even, and Class 2 if n is odd
- Cycles C_n are Class 1 if n is even, and Class 2 if n is odd
- A graph with maximum degree $\Delta > \frac{n}{2}$ is Class 2
- The Petersen graph and most of its generalizations are Class 2

The complete graph K_5 has maximum degree $\Delta(K_5) = 4$. According to Vizing's theorem, it requires at most 5 colors:

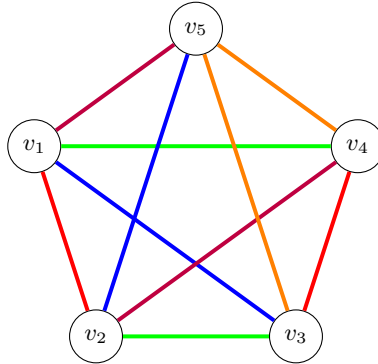


Figure 132 : K_5 requires 5 colors, demonstrating it's a Class 2 graph since $\chi'(K_5) = 5 = \Delta(K_5) + 1$.

Before Vizing's theorem (1964), Shannon proved in 1949 that for any simple graph G , the edge chromatic number is bounded by:

$$\chi'(G) \leq \left\lfloor \frac{3\Delta(G)}{2} \right\rfloor \quad (67)$$

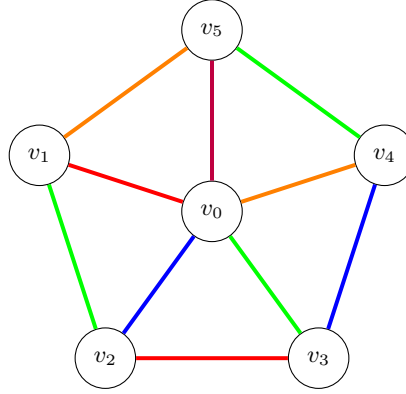
Vizing later improved this bound to:

$$\chi'(G) \leq \Delta(G) + 1 \quad (68)$$

For a graph with maximum degree $\Delta = 4$:

- Shannon's bound gives $\chi'(G) \leq \lfloor \frac{3 \cdot 4}{2} \rfloor = \lfloor 6 \rfloor = 6$
- Vizing's bound gives $\chi'(G) \leq 4 + 1 = 5$

Here's a clear example of a graph with $\Delta = 4$ that requires exactly 5 colors (thus reaching Vizing's bound):



W_5 (Wheel graph): $\Delta = 5$, $\chi'(W_5) = 5$

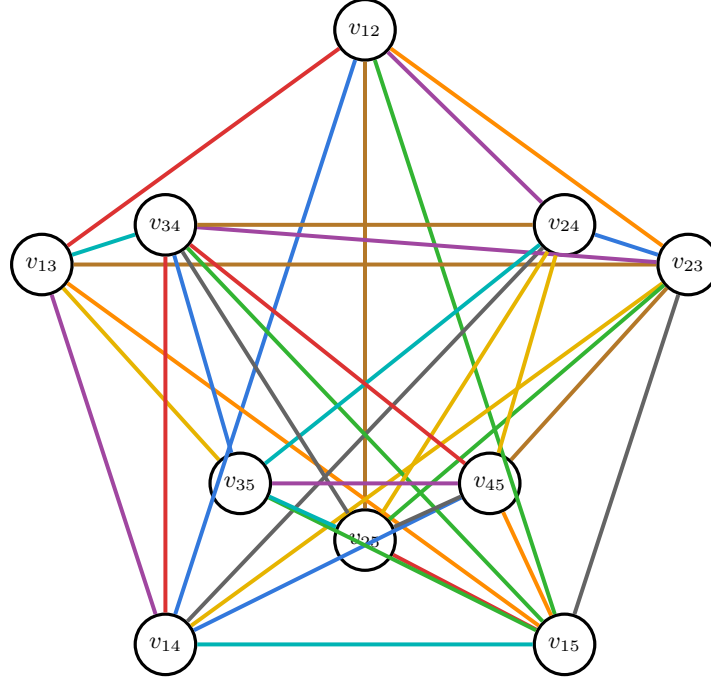
Figure 133 : This is a wheel graph W_5 with maximum degree $\Delta = 5$ at the center vertex. It requires exactly 5 colors for its edge coloring, showing a case where $\chi'(G) = \Delta(G)$ (Class 1).

For K_5 :

- Maximum degree $\Delta(K_5) = 4$
- Shannon's bound gives $\chi'(K_5) \leq \lfloor \frac{3 \cdot 4}{2} \rfloor = 6$
- Vizing's bound gives $\chi'(K_5) \leq 4 + 1 = 5$
- Actual edge chromatic number $\chi'(K_5) = 5$

K_5 requires exactly 5 colors, showing that Vizing's bound is tight (this is a Class 2 graph).

To better understand Shannon's bound, here is a graph with maximum degree $\Delta = 6$ that shows why his bound of $\lfloor \frac{3\Delta}{2} \rfloor = 9$ might be necessary:



Line Graph of K_5 : $L(K_5)$

Figure 134 : Maximum degree $\Delta = 6$, Edge chromatic number $\chi'(L(K_5)) = 9$

Demonstrating Shannon's bound: $\lfloor \frac{3\Delta}{2} \rfloor = 9$ colors needed

Here's a simple example showing a graph where the difference between Shannon's and Vizing's bounds is clear:

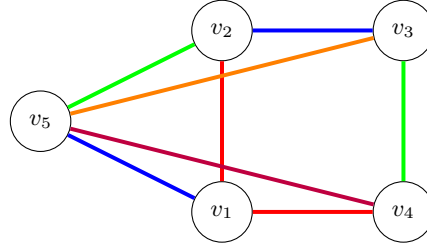


Figure 135 : Graph with $\Delta = 4$ (at vertex v_5)
Edge chromatic number $\chi' = 5$

This graph has:

- Maximum degree $\Delta = 4$ at vertex v_5
- Shannon's bound: $\chi' \leq \lfloor \frac{3 \cdot 4}{2} \rfloor = 6$
- Vizing's bound: $\chi' \leq 4 + 1 = 5$
- Actual edge chromatic number: $\chi' = 5$

This example demonstrates that Vizing's bound provides a tighter and more accurate upper limit than Shannon's earlier bound.

8.4 Shannon's Edge Coloring Theorem

In 1949, Claude Shannon established a fundamental result in graph theory regarding edge coloring. This theorem predates Vizing's more refined result from 1964.

Definition 104 (Edge Coloring). An edge coloring of a graph $G = (V, E)$ is an assignment of colors to the edges such that no two adjacent edges (i.e., edges sharing a common vertex) have the same color. The minimum number of colors needed for an edge coloring of G is called the edge chromatic number, denoted by $\chi'(G)$.

Theorem 94 (Shannon's Edge Coloring Theorem (1949)). For any simple undirected graph G , the edge chromatic number $\chi'(G)$ is bounded by:

$$\chi'(G) \leq \left\lceil \frac{3\Delta(G)}{2} \right\rceil \quad (69)$$

where $\Delta(G)$ is the maximum degree of G .

8.4.1 Explanation of Shannon's Bound

Shannon's bound tells us that for any graph with maximum degree Δ , we can color all of its edges using at most $\lceil \frac{3\Delta}{2} \rceil$ colors.

Unlike Vizing's later theorem, which tightened the bound to $\Delta(G) + 1$, Shannon's bound can be looser, especially as Δ increases:

Maximum Degree Δ	Shannon's Bound $\lceil \frac{3\Delta}{2} \rceil$	Vizing's Bound $\Delta + 1$	Difference
2	3	3	0
3	4	4	0
4	6	5	1
5	7	6	1
6	9	7	2
7	10	8	2
8	12	9	3
10	15	11	4

As we can see, the gap between Shannon's bound and Vizing's bound grows as the maximum degree increases.

8.4.2 Proof of Shannon's Edge Coloring Theorem

The proof of Shannon's edge coloring theorem requires a methodical approach that builds upon several foundational results in graph theory. Unlike the previous sessions, the proof necessitates the introduction of auxiliary lemmas and theorems that establish the necessary framework for understanding Shannon's elegant bound. This structured approach allows us to navigate the technical aspects of the proof while maintaining clarity throughout the argument.

Preliminaries

Let $G = (V, E)$ be a multigraph with maximum degree $\Delta = \Delta(G)$. We aim to show that G can be edge-colored using at most $\lceil 3\Delta/2 \rceil$ colors.

Definition 105 (Line Graph). The line graph $L(G)$ of a graph G is constructed as follows:

- Each vertex in $L(G)$ corresponds to an edge in G .
- Two vertices in $L(G)$ are adjacent if and only if their corresponding edges in G are adjacent (share a common endpoint).

Definition 106 (Vertex Coloring). A vertex coloring of a graph $G = (V, E)$ is an assignment of colors to the vertices such that no two adjacent vertices receive the same color. The minimum number of colors needed for a vertex coloring of G is called the vertex chromatic number, denoted by $\chi(G)$.

Lemma 81. Edge coloring G is equivalent to vertex coloring $L(G)$.

Proof. A proper edge coloring of G assigns different colors to adjacent edges. This is equivalent to assigning different colors to adjacent vertices in $L(G)$, which is a proper vertex coloring of $L(G)$. $\square$

Key Insight: Analyzing the Maximum Degree of $L(G)$

Lemma 82. If G has maximum degree Δ , then the maximum degree of $L(G)$ is at most $2\Delta - 2$.

Proof. Consider an edge $e = \{u, v\}$ in G . This corresponds to a vertex in $L(G)$. How many adjacent vertices can this vertex have in $L(G)$?

The edge e is adjacent to:

- At most $\Delta - 1$ other edges incident to u (since u has degree at most Δ , and we're excluding e itself)
- At most $\Delta - 1$ other edges incident to v (since v has degree at most Δ , and we're excluding e itself)

Therefore, the total number of edges adjacent to e in G is at most $(\Delta - 1) + (\Delta - 1) = 2\Delta - 2$. This means the vertex in $L(G)$ corresponding to e has degree at most $2\Delta - 2$.

Since this applies to any edge e in G , the maximum degree of $L(G)$ is at most $2\Delta - 2$. $\square$

Constructive Coloring Approach

Shannon's key insight was to partition the edges around each vertex into distinct matchings.

Lemma 83. Let v be a vertex of maximum degree Δ in G . The edges incident to v can be partitioned into Δ matchings $M_1, M_2, \dots, M_\Delta$.

Proof. Let $e_1, e_2, \dots, e_\Delta$ be the edges incident to v . We can define:

- $M_1 = \{e_1\}$
- $M_2 = \{e_2\}$
- $\dots$
- $M_\Delta = \{e_\Delta\}$

Each M_i is trivially a matching since it contains just one edge. $\square$

Let $G = (V, E)$ be a multigraph with maximum degree $\Delta = \Delta(G)$. We aim to show that G can be edge-colored using at most $\lceil 3\Delta/2 \rceil$ colors.

Definition 107 (Clique). A clique in a graph is a subset of vertices such that every two distinct vertices in the subset are adjacent (connected by an edge). In other words, a clique forms a complete subgraph where all possible edges between the vertices exist.

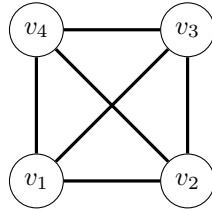


Figure 136 : A 4-vertex clique K_4 where every pair of vertices is connected by an edge

Lemma 84. The line graph $L(G)$ can be partitioned into Δ cliques.

Proof. For each vertex v in G , the edges incident to v form a clique in $L(G)$ (since any two edges sharing vertex v are adjacent in G , so their corresponding vertices in $L(G)$ are adjacent).

Therefore, for each vertex v in G , we get a clique in $L(G)$. Since G has maximum degree Δ , each of these cliques has at most Δ vertices. $\square$

Brooks' Theorem and Its Application

Brooks' Theorem is a fundamental result in graph coloring that refines the trivial upper bound on the vertex chromatic number.

Theorem 95 (Brooks' Theorem, 1941). Let G be a connected simple graph with maximum degree $\Delta \geq 3$. Then:

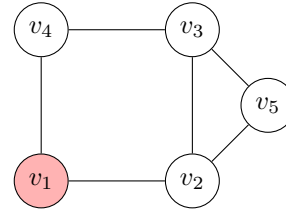
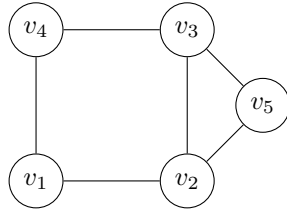
$$\chi(G) \leq \Delta \quad (70)$$

unless G is a complete graph $K_{\Delta+1}$ or an odd cycle C_{2k+1} (where $k \geq 1$), in which case $\chi(G) = \Delta + 1$.

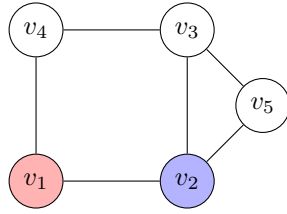
Definition 108 (Greedy Coloring Algorithm). The greedy coloring algorithm is a sequential vertex coloring procedure that works as follows:

1. Arrange the vertices in some order $v_1, v_2, \dots, v_n$.
2. For i from 1 to n :
 - (a) Assign to v_i the smallest color (typically represented by a positive integer) that is not already assigned to any of its adjacent vertices that have already been colored (i.e., neighbors among $v_1, v_2, \dots, v_{i-1}$).

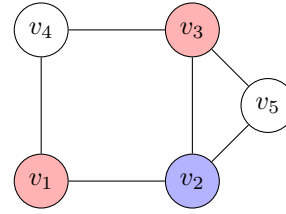
The performance of the greedy algorithm strongly depends on the chosen vertex ordering.



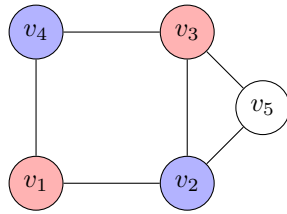
Step 1: Assign color 1 to v_1



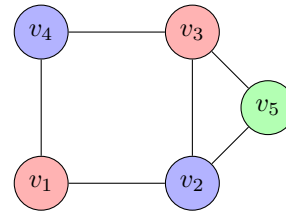
Step 2: Assign color 2 to v_2



Step 3: Assign color 1 to v_3



Step 4: Assign color 2 to v_4



Step 5: Assign color 3 to v_5

Figure 137 : Step-by-step execution of the greedy coloring algorithm on a simple graph with the vertex ordering v_1, v_2, v_3, v_4, v_5 . The final coloring uses 3 colors.

Complete Proof of Brooks' Theorem. We proceed by cases, considering different graph structures. Throughout the proof, we construct a specific vertex ordering that enables an effective application of the greedy coloring algorithm.

Case 1: If G is a complete graph $K_{\Delta+1}$ with $\Delta + 1$ vertices, then each vertex is adjacent to all other vertices. A proper coloring requires each vertex to have a distinct color, so $\chi(G) = \Delta + 1$. This establishes the first exception in the theorem.

Case 2: If G is an odd cycle, then $\Delta = 2$ and $\chi(G) = 3 = \Delta + 1$. This establishes the second exception.

Case 3: If G is an even cycle, then $\Delta = 2$ and $\chi(G) = 2 = \Delta$, which satisfies the theorem.

Case 4: The main case is when G is not a complete graph and not a cycle, with $\Delta \geq 3$. We need to show that $\chi(G) \leq \Delta$.

Since G is not a complete graph, there exist two vertices u and v that are not adjacent. We first handle the case where removing the edge (u, v) would disconnect the graph.

Subcase 4.1: Suppose there is no path from u to v in $G - \{(u, v)\}$. Then $G - \{(u, v)\}$ has two connected components G_1 (containing u) and G_2 (containing v).

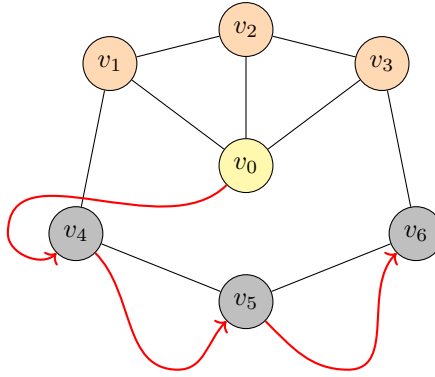
By the induction hypothesis, we can color G_1 and G_2 with at most Δ colors each. By permuting the colors if necessary, we can ensure that these colorings are compatible, giving a Δ -coloring of G .

Subcase 4.2: The key case is when G is 2-connected (removing any edge doesn't disconnect the graph) and not a complete graph or an odd cycle.

We choose a specific vertex v_0 with degree at most $\Delta - 1$ (such a vertex must exist since G is not a complete graph). Let $N(v_0) = \{v_1, v_2, \dots, v_m\}$ be its neighbors, where $m \leq \Delta - 1$.

We construct a carefully chosen ordering of the vertices:

- Start with v_0
- Then include its neighbors $v_1, v_2, \dots, v_m$
- Finally, add the remaining vertices in any order



Vertex Ordering: $v_0, v_1, v_2, v_3, v_4, v_5, v_6$

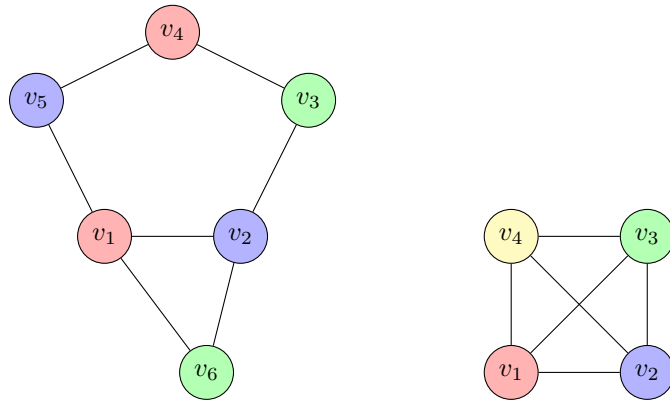
Figure 138 : Construction of a specific vertex ordering for Brooks' Theorem, starting with a vertex v_0 of degree $< \Delta$, followed by its neighbors, and then the remaining vertices.

Now we apply the greedy coloring algorithm in the *reverse* of this order: we color $v_n, v_{n-1}, \dots, v_1, v_0$.

Coloring analysis:

- For each vertex v_i where $i > m$ (i.e., not a neighbor of v_0), when we color it, it has at most Δ neighbors, and not all of them have been colored yet. So we need at most $\Delta - 1$ colors to avoid conflicts, and can use one of the Δ available colors.
- For each vertex v_i where $1 \leq i \leq m$ (i.e., a neighbor of v_0), we consider whether it's adjacent to all previous vertices v_j for $j < i$. If not, then we can color it with at most $\Delta - 1$ colors, again allowing us to stay within Δ colors.
- The key insight is that the vertices $v_1, v_2, \dots, v_m$ cannot form a complete subgraph (as this would make G a complete graph). Therefore, there exist two vertices in this set, say v_i and v_j with $i < j$, that are not adjacent.
- When coloring v_j , it is not adjacent to v_i , so it sees at most $\Delta - 1$ colors among its colored neighbors. This means we can color v_j within our Δ color budget.
- Finally, vertex v_0 has only $m \leq \Delta - 1$ neighbors, all of which have been colored. Therefore, at least one of the Δ colors is available for v_0 .

This careful construction ensures that at each step of the greedy algorithm, each vertex has at least one color available from the set of Δ colors. Therefore, we can color the entire graph with at most Δ colors, completing the proof. $\square$



A 3-regular graph colored with 3 colors K_4 - requires 4 colors

Figure 139 : Comparison of a 3-regular graph (left) that satisfies Brooks' Theorem and can be colored with $\Delta = 3$ colors, and K_4 (right) which requires $\Delta + 1 = 4$ colors.

We now apply the greedy coloring algorithm in *reverse* order, starting with v_6 and ending with v_0 :

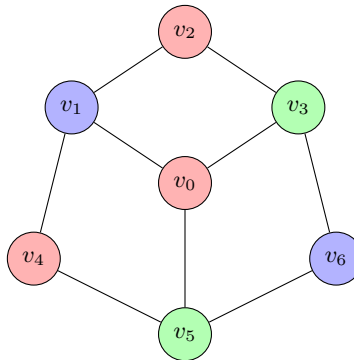


Figure 140 : Final coloring using $\Delta = 3$ colors

The coloring process works as follows:

1. Color v_6 with color 1 (blue)
2. Color v_4 with color 1 (red)
3. Color v_2 with color 1 (red)
4. Color v_5 with color 2 (green) - since it's adjacent to v_4 which has color 1
5. Color v_3 with color 2 (green) - avoiding colors of adjacent vertices
6. Color v_1 with color 3 (blue) - avoiding colors of adjacent vertices
7. Color v_0 with color 1 (red) - it's adjacent to vertices with colors 2 and 3

This demonstrates that the graph can be colored with $\Delta = 3$ colors, confirming Brooks' Theorem.

Example 35 (Application of Brooks' Theorem). Consider a graph G with maximum degree $\Delta = 3$ that is neither a complete graph (K_4) nor an odd cycle. By Brooks' Theorem, $\chi(G) \leq 3$, meaning we can color the vertices using at most 3 colors.

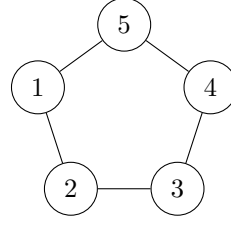
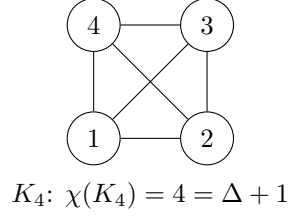


Figure 141 : Special cases for Brooks' Theorem: Complete graph K_4 and odd cycle C_5 both require $\Delta + 1$ colors.

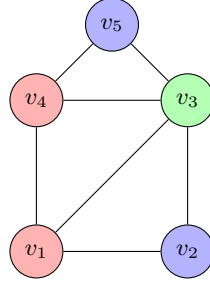


Figure 142 : G with $\Delta = 3$, $\chi(G) = 3$

Each vertex in this example has degree at most 3, and the graph is neither K_4 nor an odd cycle. As predicted by Brooks' Theorem, it can be colored with exactly 3 colors.

Corollary 40 (Generalized Brooks' Theorem for Clique Partitions). If a graph G with maximum degree Δ can be partitioned into cliques, each of size at most t , then:

$$\chi(G) \leq \left\lfloor \frac{t\Delta}{t-1} \right\rfloor \quad (71)$$

Proof Sketch of Generalized Brooks' Theorem. The proof of this generalization builds on the original Brooks' Theorem and involves several steps:

1. First, note that in a clique of size t , each vertex is connected to $t - 1$ other vertices.
2. When we partition graph G into cliques of size at most t , the coloring constraints within each clique require at most t colors.
3. The challenge lies in managing the interactions between different cliques in the partition.
4. By carefully analyzing the maximum number of conflicts a vertex can have based on its degree Δ and the clique structure, we can derive a bound on the total number of colors needed.
5. The expression $\frac{t\Delta}{t-1}$ represents the worst-case scenario for the number of colors based on this analysis.

This generalization is crucial for Shannon's proof because the line graph $L(G)$ can be partitioned into cliques corresponding to the edges incident to each vertex in the original graph G . $\square$

Completing the Proof

We can now combine these results to complete the proof of Shannon's theorem.

Proof of Shannon's Theorem. We've established that:

1. Edge coloring G is equivalent to vertex coloring $L(G)$.
2. The maximum degree of $L(G)$ is at most $2\Delta - 2$.
3. $L(G)$ can be partitioned into cliques of size at most Δ .

Applying the generalized Brooks' result with $t = \Delta$ and max degree $2\Delta - 2$:

$$\chi(L(G)) \leq \left\lfloor \frac{\Delta \cdot (2\Delta - 2)}{\Delta - 1} \right\rfloor \quad (72)$$

$$= \left\lfloor \frac{2\Delta^2 - 2\Delta}{\Delta - 1} \right\rfloor \quad (73)$$

$$= \left\lfloor \frac{2\Delta(\Delta - 1)}{\Delta - 1} \right\rfloor \quad (74)$$

$$= \lfloor 2\Delta \rfloor \quad (75)$$

$$= 2\Delta \quad (76)$$

This would give us a bound of 2Δ , which is not as tight as Shannon's bound of $\lfloor \frac{3\Delta}{2} \rfloor$.

Shannon's tighter bound comes from a more careful analysis of the structure of $L(G)$. Specifically, Shannon shows that:

$$\chi(L(G)) \leq \left\lfloor \frac{3\Delta}{2} \right\rfloor \quad (77)$$

The detailed proof utilizes a more sophisticated coloring algorithm. The key insight is that we can partition the edges around each vertex more strategically.

For a vertex of degree Δ , we can form $\lceil \frac{\Delta}{2} \rceil$ matchings as follows:

- Pair up edges $(e_1, e_2), (e_3, e_4), \dots$ and form matchings from each pair
- If Δ is odd, the last edge forms its own matching

For each vertex, this gives us at most $\lceil \frac{\Delta}{2} \rceil$ matchings. Across the entire graph, the total number of matchings needed is at most $\lceil \frac{\Delta}{2} \rceil \cdot 3 = \frac{3\Delta}{2}$ (if Δ is even) or $\frac{3\Delta+1}{2}$ (if Δ is odd).

Taking the floor function gives us $\lfloor \frac{3\Delta}{2} \rfloor$ as the bound on the edge chromatic number. $\square$

Tightness of Shannon's Bound

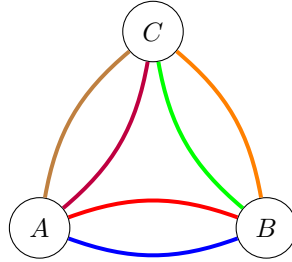


Figure 143 : Shannon Multigraph with $\Delta = 2$

The Shannon multigraph shown above has:

- Δ parallel edges between each pair of vertices (here $\Delta = 2$)
- Maximum degree $2\Delta = 4$
- Edge chromatic number $3\Delta = 6$

This demonstrates that Shannon's bound is tight for certain families of multigraphs. The edge chromatic number is exactly $\lfloor \frac{3 \cdot (2\Delta)}{2} \rfloor = 3\Delta$.

Refined Proof Techniques

Shannon's original proof involves a more detailed analysis of the constraints between different matchings. The refinement uses the concept of a "color exchange graph" and applies graph theoretic techniques to show that $\lfloor \frac{3\Delta}{2} \rfloor$ colors are sufficient.

The key insights include:

- Analyzing how colors can be interchanged along alternating paths in the graph
- Strategic recoloring of edges to free up colors
- Careful analysis of the constraints between different color classes

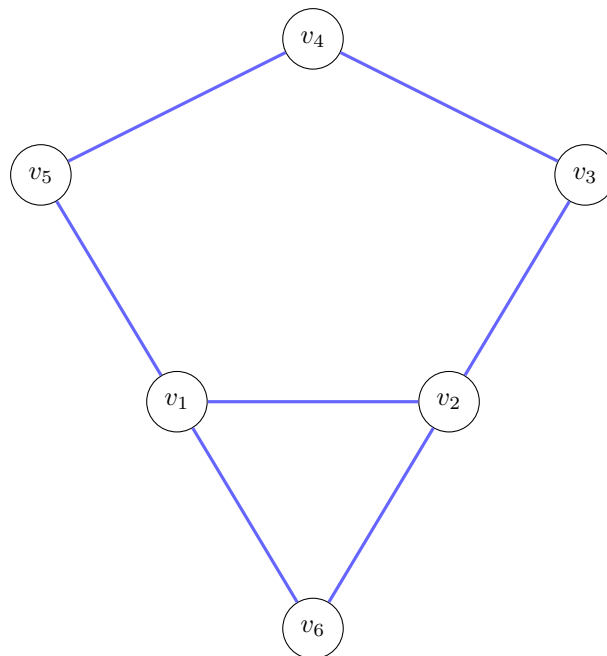
Implications and Applications

Shannon's edge coloring theorem has several important implications:

1. It provides an upper bound on the number of time slots needed for scheduling operations where conflict constraints can be modeled as a graph.
2. It establishes a relationship between the maximum degree and the edge chromatic number, showing that edge chromatic number grows at most proportionally to the maximum degree.
3. It applies to multigraphs where Vizing's tighter bound does not apply.

Shannon originally developed this result in the context of analyzing switching networks in communication systems, demonstrating the practical motivations behind this theoretical advance.

8.5 Degenerate Graphs and Their Coloring Properties



A 2-degenerate graph with a valid vertex ordering

Figure 144 : Example of a 2-degenerate graph

Theorem 96 (Equivalence of Degenerate Definitions). The following statements are equivalent for a graph G :

1. G is d -degenerate.
2. Every induced subgraph of G has a vertex of degree at most d .
3. There exists an ordering $v_1, v_2, \dots, v_n$ of the vertices of G such that each vertex v_i has at most d neighbors among vertices $v_1, v_2, \dots, v_{i-1}$.

Proof. ($1 \Rightarrow 2$): This follows directly from the definition of d -degeneracy.

($2 \Rightarrow 3$): We can construct the required ordering as follows:

- Start with the original graph G .
- Find a vertex v_1 with degree at most d (which exists by assumption).
- Remove v_1 from G to get G' .
- Find a vertex v_2 with degree at most d in G' (which exists since G' is an induced subgraph of G).

- Continue this process until all vertices are ordered.

In the resulting ordering $v_1, v_2, \dots, v_n$, each vertex v_i has at most d neighbors among $v_{i+1}, \dots, v_n$ (since when we selected v_i , it had degree at most d in the remaining graph).

(3 $\Rightarrow$ 1): Let $v_1, v_2, \dots, v_n$ be an ordering where each v_i has at most d neighbors among $v_{i+1}, \dots, v_n$. For any induced subgraph H of G , consider the vertex in H that appears first in this ordering. This vertex has at most d neighbors in H , proving that H has a vertex of degree at most d . $\square$

Definition 109 (d -Degenerate Graph). A graph G is called **d -degenerate** if d is the smallest integer such that G is d -degenerate according to the above definition. The parameter d is called the **degeneracy** of G .

Theorem 97 (Characterization of d -Degeneracy). The degeneracy d of a graph G is equal to:

$$d = \max_{H \subseteq G} \delta(H) \quad (78)$$

where the maximum is taken over all induced subgraphs H of G and $\delta(H)$ denotes the minimum degree of a vertex in H .

Proof. Let $d^* = \max_{H \subseteq G} \delta(H)$. We need to show that $d = d^*$.

First, we prove that $d \geq d^*$. Consider an induced subgraph H of G such that $\delta(H) = d^*$. Since every vertex in H has degree at least d^* , the graph G cannot be $(d^* - 1)$ -degenerate by definition. Therefore, $d \geq d^*$.

Next, we prove that $d \leq d^*$. Since $\delta(H) \leq d^*$ for all induced subgraphs H of G , every induced subgraph has a vertex of degree at most d^* . This means G is d^* -degenerate, so $d \leq d^*$.

Combining these inequalities, we have $d = d^*$. $\square$

Theorem 98 (Bounds on Degeneracy). For any graph G :

1. The degeneracy d is at most the maximum degree $\Delta(G)$.
2. The degeneracy d is at least the minimum degree $\delta(G)$.
3. For a k -regular graph, the degeneracy d equals k .

Theorem 99 (d -Degeneracy and Coloring). If a graph G has degeneracy d , then:

$$\chi(G) \leq d + 1 \quad (79)$$

where $\chi(G)$ is the chromatic number of G .

Proof. Let G be a graph with degeneracy d . By definition, there exists an ordering $v_1, v_2, \dots, v_n$ of the vertices such that each vertex v_i has at most d neighbors among vertices $v_1, v_2, \dots, v_{i-1}$.

We can color G using the greedy algorithm in this order. When we color vertex v_i , at most d of its neighbors have already been colored. Therefore, at most d colors are forbidden for v_i , and we can assign to v_i a color from the set $\{1, 2, \dots, d + 1\}$ that is different from the colors of its already-colored neighbors.

This process produces a proper coloring of G using at most $d + 1$ colors. Therefore, $\chi(G) \leq d + 1$. $\square$

Example 36 (Degeneracy of Specific Graph Classes). • Trees and forests are 1-degenerate: Any subtree with at least two vertices has at least one leaf (vertex of degree 1).

- Outerplanar graphs are 2-degenerate: Every outerplanar graph with $n \geq 3$ vertices has at least one vertex of degree at most 2.
- Planar graphs are 5-degenerate: By Euler's formula and the handshaking lemma, every planar graph with $n \geq 3$ vertices has at least one vertex of degree at most 5.
- A graph with arboricity a is $(2a - 1)$ -degenerate: Arboricity is the minimum number of forests needed to cover all edges. If a graph has arboricity a , then $|E(H)| \leq a(|V(H)| - 1)$ for every induced subgraph H , implying H has a vertex of degree at most $2a - 1$.

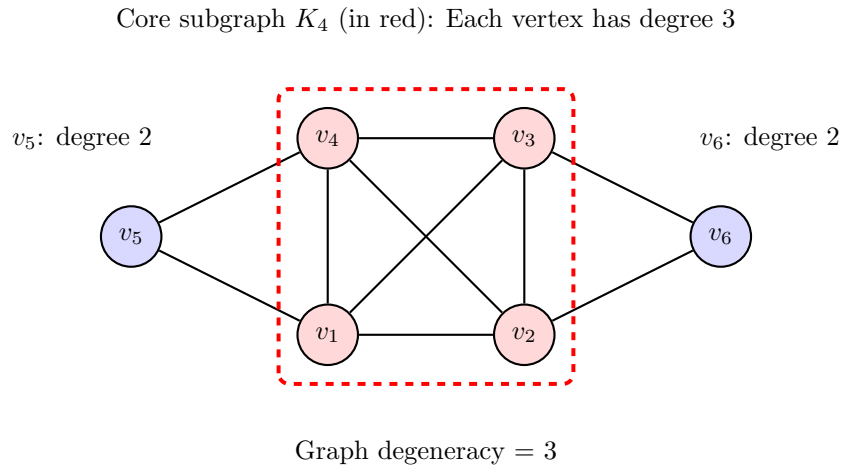


Figure 145 : A 3-degenerate graph containing K_4 as an induced subgraph (highlighted in red). The degeneracy is determined by this subgraph where each vertex has degree 3.

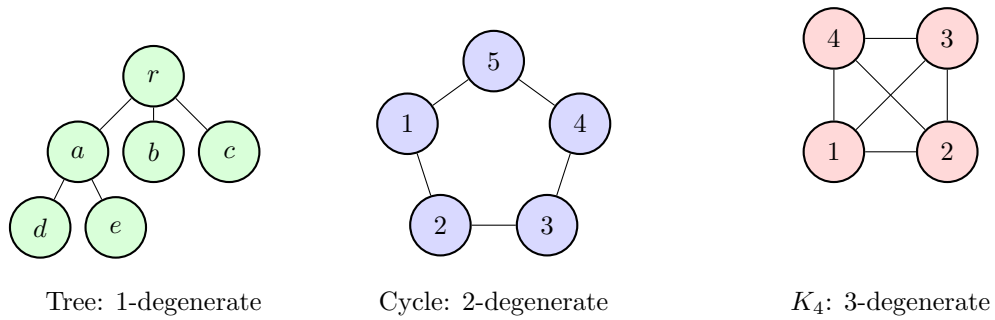


Figure 146 : Examples of graphs with different degeneracy values: a tree (1-degenerate), a cycle (2-degenerate), and K_4 (3-degenerate).

Algorithm 27 Computing Graph Degeneracy

Require: Graph $G = (V, E)$ **Ensure:** Degeneracy d and degeneracy ordering L

```
1:  $L \leftarrow \emptyset$  ▷ Initialize empty ordering
2:  $d \leftarrow 0$  ▷ Initialize degeneracy
3:  $G' \leftarrow G$  ▷ Working copy of the graph
4: while  $G'$  is not empty do
5:   Find vertex  $v \in G'$  with minimum degree
6:    $d \leftarrow \max(d, \deg_{G'}(v))$  ▷ Update degeneracy
7:   Add  $v$  to the beginning of ordering  $L$ 
8:   Remove  $v$  from  $G'$ 
9: end while
10: return degeneracy  $d$  and ordering  $L$ 
```

Lemma 85 (Tightness of Degeneracy Bound). For any $d \geq 1$, there exists a d -degenerate graph G with $\chi(G) = d + 1$.

Proof. The complete graph K_{d+1} on $d + 1$ vertices is d -degenerate and requires exactly $d + 1$ colors. $\square$

Corollary 41 (Degeneracy vs. Maximum Degree). For any graph G with degeneracy d and maximum degree Δ :

$$\chi(G) \leq d + 1 \leq \Delta + 1 \quad (80)$$

Lemma 86 (Degeneracy and Clique Number). For any graph G with degeneracy d and clique number $\omega(G)$:

$$\omega(G) \leq d + 1 \quad (81)$$

Proof. In a clique of size k , every vertex has degree $k - 1$. Therefore, any subgraph containing a clique of size $\omega(G)$ has minimum degree at least $\omega(G) - 1$. Since $d = \max_{H \subseteq G} \delta(H)$, we have $\omega(G) - 1 \leq d$, which gives $\omega(G) \leq d + 1$. $\square$

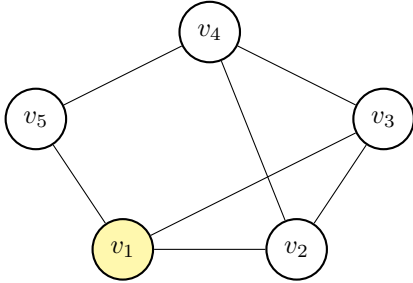
Algorithm 28 Matula-Beck Algorithm for Computing Degeneracy

Require: Graph $G = (V, E)$ **Ensure:** Degeneracy value d and degeneracy ordering L

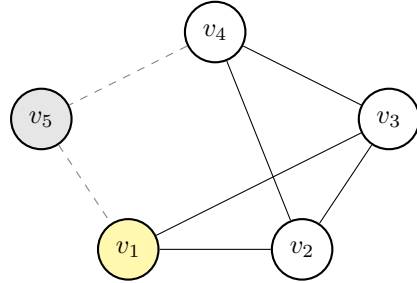
```
1: Initialize empty list  $L \leftarrow \emptyset$ 
2:  $d \leftarrow 0$  ▷ Initialize degeneracy value
3:  $G' \leftarrow G$  ▷ Working copy of the graph
4: while  $G'$  is not empty do
5:   Find vertex  $v \in V(G')$  with minimum degree
6:    $d \leftarrow \max(d, \deg_{G'}(v))$  ▷ Update degeneracy
7:   Remove  $v$  from  $G'$ 
8:   Add  $v$  to the beginning of  $L$ 
9: end while
10: return degeneracy  $d$  and ordering  $L$ 
```

Lemma 87 (Computational Complexity of Degeneracy). The Matula-Beck algorithm computes the degeneracy of a graph and a corresponding degeneracy ordering in $O(n + m)$ time, where n is the number of vertices and m is the number of edges.

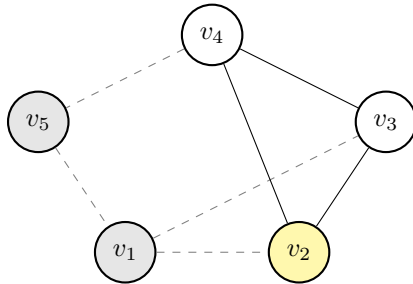
Proof Sketch. Using a bucket sort structure to maintain vertices by their current degree, we can efficiently find and remove the minimum degree vertex in each iteration. The total time complexity is $O(n + m)$ because each vertex is processed once (taking $O(n)$ total) and each edge is examined at most once when updating degrees (taking $O(m)$ total). $\square$



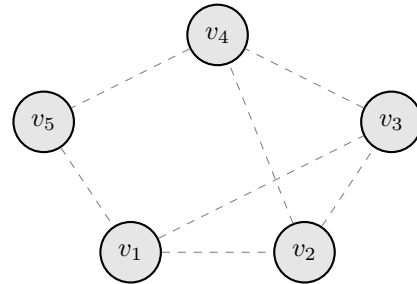
Step 1: Identify v_5 with minimum degree



Step 2: Remove v_5 , find next minimum



Step 3: Remove v_1 , find next minimum



Final ordering: $L = [v_4, v_2, v_3, v_1, v_5]$

Figure 147 : Computation of degeneracy ordering: Vertices are removed in order of minimum degree, with the current minimum degree vertex highlighted in yellow. Blue numbers show current degrees, and red numbers show the final ordering positions.

Algorithm 29 Efficient Implementation of Matula-Beck Algorithm

```
1: procedure COMPUTEDEGENERACY( $G = (V, E)$ )
2:    $d \leftarrow 0$  ▷ Initialize degeneracy value
3:    $L \leftarrow \emptyset$  ▷ Initialize empty ordering list
4:   Compute  $\deg(v)$  for each vertex  $v \in V$ 
5:    $\Delta \leftarrow \max_{v \in V} \deg(v)$  ▷ Maximum degree in  $G$ 
6:   Create buckets  $B_0, B_1, \dots, B_\Delta$ 
7:   for all  $v \in V$  do
8:     Add  $v$  to bucket  $B_{\deg(v)}$ 
9:   end for
10:  while  $\exists i : B_i \neq \emptyset$  do
11:    Find minimum  $i$  such that  $B_i \neq \emptyset$ 
12:    Remove a vertex  $v$  from  $B_i$ 
13:     $d \leftarrow \max(d, i)$  ▷ Update degeneracy
14:    Prepend  $v$  to ordering  $L$ 
15:    for all neighbors  $u$  of  $v$  that remain in the graph do
16:      Move  $u$  from  $B_{\deg(u)}$  to  $B_{\deg(u)-1}$ 
17:       $\deg(u) \leftarrow \deg(u) - 1$  ▷ Update degree
18:    end for
19:  end while
20:  return  $(d, L)$  ▷ Return degeneracy and ordering
21: end procedure
```

8.6 Degeneracy Relationships for Specific Graph Classes

Theorem 100 (Degeneracy of Basic Graph Classes). The following bounds on degeneracy hold:

1. Trees and forests are 1-degenerate.
2. K_n (complete graph on n vertices) is $(n - 1)$ -degenerate.
3. Cycles C_n are 2-degenerate.
4. Bipartite graphs that are not empty are 1-degenerate or 2-degenerate.
5. A k -regular graph is k -degenerate.

Lemma 88 (Trees are 1-Degenerate). Every tree T is 1-degenerate.

Proof. Every tree with at least two vertices has at least one leaf (vertex of degree 1). Any subtree of a tree is also a tree, so every subtree has a leaf. Therefore, every subtree has a vertex of degree at most 1, making the tree 1-degenerate. $\square$

Lemma 89 (Cycles are 2-Degenerate). Every cycle C_n is 2-degenerate.

Proof. In a cycle, every vertex has degree exactly 2. When we remove any vertex from the cycle, we get a path, in which every vertex has degree at most 2, and there is at least one vertex of degree 1 (the endpoints of the path). Therefore, every cycle is 2-degenerate. $\square$

Lemma 90 (Complete Graphs). The complete graph K_n is $(n - 1)$ -degenerate.

Proof. In K_n , every vertex has degree $n - 1$. When we remove a vertex, we get K_{n-1} , where every vertex has degree $n - 2$. Continuing this process, we see that the degeneracy is exactly $n - 1$. $\square$

Lemma 91 (Bipartite Graphs). A connected bipartite graph that is not a tree has degeneracy 2.

Proof. A connected bipartite graph that is not a tree contains at least one cycle. This means there's at least one vertex with degree at least 2 in every induced subgraph. However, since the graph is bipartite, it cannot contain a K_3 (triangle), which means any induced subgraph must have a vertex of degree at most 2. Therefore, such bipartite graphs are 2-degenerate. $\square$

Lemma 92 (k -Regular Graphs). A k -regular graph is d -degenerate.

Proof. In a k -regular graph, every vertex has degree exactly k . When we remove any vertex, the degree of its neighbors decreases by 1, so the minimum degree in the resulting graph is at most $k - 1$. Continuing this process, we can always find a vertex of degree at most k in any subgraph, making the graph d -degenerate. $\square$

8.7 Extensions and Applications

Theorem 101 (Generalized Degeneracy). For any non-negative integer function f on the vertices of a graph G , there exists a vertex ordering such that each vertex v has at most $d + f(v)$ neighbors preceding it in the ordering, where d is the degeneracy of G .

Theorem 102 (Clustered Coloring of Bounded Degeneracy Graphs). For any d -degenerate graph G and any positive integer t , the vertices of G can be colored with $O(d^2 t)$ colors such that each color class induces a subgraph with components of size at most t .

Lemma 93 (p -Centered Coloring). A d -degenerate graph admits a p -centered coloring with $O(pd^p)$ colors, where a p -centered coloring is a vertex coloring such that for any connected subgraph H , either H receives more than p colors or there is a color that appears exactly once in H .

Definition 110 (Sparse Graph). A graph $G = (V, E)$ is called **sparse** if $|E| = O(|V|)$, that is, the number of edges is asymptotically linear in the number of vertices. More precisely, a family of graphs is considered sparse if there exists a constant c such that for all graphs G in the family with n vertices and m edges, we have $m \leq cn$.

Equivalently, a sparse graph has an average degree that is bounded by a constant independent of the graph size. That is, the average degree $\frac{2|E|}{|V|} = O(1)$.

Examples of sparse graphs include:

- Trees, which have exactly $n - 1$ edges for n vertices
- Planar graphs, which have at most $3n - 6$ edges for $n \geq 3$ vertices
- Graphs with bounded degeneracy d , which have at most dn edges
- Graphs with bounded arboricity a , which have at most an edges

Algorithm 30 Coloring Algorithm for Sparse Graphs

Require: Graph $G = (V, E)$ with degeneracy d

Ensure: A proper coloring of G using at most $d + 1$ colors

```

1: Compute a degeneracy ordering  $L = [v_1, v_2, \dots, v_n]$  of  $G$ 
2: Initialize color palette  $C = \{1, 2, \dots, d + 1\}$ 
3: Initialize empty color assignment  $\text{color}(v) \leftarrow \text{null}$  for all  $v \in V$ 
4: for  $i = 1$  to  $n$  do
5:    $N_{\text{before}}(v_i) \leftarrow \{v_j \in N(v_i) : j < i\}$  ▷ Neighbors earlier in ordering
6:    $C_{\text{used}} \leftarrow \{\text{color}(v) : v \in N_{\text{before}}(v_i)\}$  ▷ Colors used by neighbors
7:    $C_{\text{avail}} \leftarrow C \setminus C_{\text{used}}$  ▷ Available colors
8:    $\text{color}(v_i) \leftarrow \min(C_{\text{avail}})$  ▷ Assign smallest available color
9: end for
10: return color assignment

11: Special cases:
12: if  $G$  is a tree then
13:   Use 2 colors ▷ Trees are bipartite
14: else if  $G$  is bipartite then
15:   Use 2 colors via bipartition
16: else if  $G$  is planar then
17:   Use at most 4 colors ▷ Four Color Theorem
18: end if
```

Remark 97 (Application to Graph Mining). Computing all maximal cliques in a graph with degeneracy d can be done in $O(3^{d/3} \cdot n \cdot d)$ time, making it tractable for graphs with small degeneracy even when the number of maximal cliques is exponential.

Lemma 94 (Acyclic Coloring via Degeneracy). A graph with degeneracy d has an acyclic coloring with at most $4d$ colors, where an acyclic coloring is a proper coloring such that any two color classes induce a forest.

Remark 98 (Relationship to Brooks' Theorem). *Brooks' Theorem states that if G is a connected graph that is neither a complete graph nor an odd cycle, then $\chi(G) \leq \Delta(G)$. Since $d \leq \Delta$, the degeneracy bound $\chi(G) \leq d + 1$ may be tighter in some cases.*

8.8 Algorithms for d -Degenerate Graphs

Algorithm 31 Recognition of d -Degenerate Graphs

```

1: procedure IskDEGENERATE( $G = (V, E)$ ,  $k$ )
2:   Create a copy  $G' \leftarrow G$ 
3:   while  $V(G') \neq \emptyset$  do
4:     Find vertex  $v \in V(G')$  with minimum degree
5:     if  $\deg_{G'}(v) > d$  then
6:       return false                                ▷ Found a subgraph with minimum degree  $> d$ 
7:     end if
8:     Remove  $v$  from  $G'$ 
9:   end while
10:  return true                                       ▷ Every subgraph had a vertex with degree  $\leq d$ 
11: end procedure

```

Theorem 103 (Linear-time Recognition of d -degenerate Graphs). For a fixed value of k , determining whether a graph G is d -degenerate can be done in linear time $O(|V| + |E|)$.

Lemma 95 (Subgraph Enumeration in d -Degenerate Graphs). For a fixed pattern graph H with h vertices, the number of occurrences of H as a subgraph in a d -degenerate graph G with n vertices is at most $O(n \cdot d^{h-1})$, and all such occurrences can be enumerated in time $O(n \cdot d^{h-1})$.

Algorithm 32 Enumeration of Maximal Cliques in d -Degenerate Graphs

```

1: procedure ENUMERATEMAXIMALCLIQUES( $G = (V, E)$ )
2:   Compute a degeneracy ordering  $L = [v_1, v_2, \dots, v_n]$  of  $G$ 
3:   for  $i = n$  downto 1 do
4:      $N^+(v_i) \leftarrow \{v_j \in N(v_i) : j > i\}$                                 ▷ Forward neighbors in ordering
5:      $G_i \leftarrow G[N^+(v_i)]$                                                 ▷ Subgraph induced by forward neighbors
6:      $\mathcal{C}_i \leftarrow \text{AllMaximalCliques}(G_i)$                                 ▷ Recursive or known algorithm
7:     for all cliques  $C \in \mathcal{C}_i$  do
8:       output  $C \cup \{v_i\}$                                                   ▷ Extend each clique with  $v_i$ 
9:     end for
10:  end for
11: end procedure

```

Theorem 104 (Bron-Kerbosch Algorithm for d -Degenerate Graphs). The Bron-Kerbosch algorithm with degeneracy ordering runs in $O(d \cdot n \cdot 3^{d/3})$ time on a d -degenerate graph with n vertices.

Sketch. The algorithm processes each vertex in reverse degeneracy ordering. For each vertex v_i , we initiate a Bron-Kerbosch search with pivot from $\{v_i\}$ as the initial clique R , considering only neighbors that appear later in the ordering as potential additions (P).

Since we process vertices in degeneracy ordering, each vertex has at most d neighbors that come later in the ordering. This limits the size of the potential vertex set P to at most d vertices.

The Bron-Kerbosch algorithm with pivot has a worst-case time complexity of $O(3^{n/3})$ on a graph with n vertices. When restricted to the subgraph of forward neighbors (of which there are at most d), this becomes $O(3^{d/3})$.

Since we perform this search for each of the n vertices, and each recursive call may take $O(d)$ time to compute set intersections, the overall time complexity is $O(d \cdot n \cdot 3^{d/3})$. $\square$

Algorithm 33 Bron-Kerbosch Algorithm for d -Degenerate Graphs

```
1: procedure BRONKERBOSCHDEGENERACY( $G = (V, E)$ )
2:   Compute a degeneracy ordering  $L = [v_1, v_2, \dots, v_n]$  of  $G$ 
3:    $\mathcal{C} \leftarrow \emptyset$  ▷ Initialize set of all maximal cliques
4:   for  $i = n$  downto 1 do
5:      $P \leftarrow N(v_i) \cap \{v_{i+1}, \dots, v_n\}$  ▷ Neighbors after  $v_i$  in ordering
6:      $X \leftarrow N(v_i) \cap \{v_1, \dots, v_{i-1}\}$  ▷ Neighbors before  $v_i$  in ordering
7:     BRONKERBOSCHPIVOT( $\{v_i\}, P, X, \mathcal{C}$ )
8:      $X \leftarrow X \cup \{v_i\}$  ▷ Add  $v_i$  to excluded vertices
9:   end for
10:  return  $\mathcal{C}$  ▷ Return all maximal cliques
11: end procedure

12: procedure BRONKERBOSCHPIVOT( $R, P, X, \mathcal{C}$ )
13:  if  $P = \emptyset$  and  $X = \emptyset$  then
14:     $\mathcal{C} \leftarrow \mathcal{C} \cup \{R\}$  ▷  $R$  is a maximal clique
15:    return
16:  end if
17:  Choose a pivot vertex  $u \in P \cup X$  that maximizes  $|P \cap N(u)|$ 
18:  for all  $v \in P \setminus N(u)$  do
19:    BRONKERBOSCHPIVOT( $R \cup \{v\}, P \cap N(v), X \cap N(v), \mathcal{C}$ )
20:     $P \leftarrow P \setminus \{v\}$ 
21:     $X \leftarrow X \cup \{v\}$ 
22:  end for
23: end procedure
```

8.9 Introduction to Scheduling Problems

This bonus section explores an advanced application of degeneracy graphs, introducing scheduling problems and their relationship to the core concepts we've covered. While not essential for understanding the fundamental principles, this material provides valuable insights into how these concepts extend to practical optimization challenges. If you are taking Math 154, skip to page 200.

Scheduling problems and graph theory share a deep and mutually beneficial relationship. Many scheduling problems can be naturally formulated as graph problems, allowing graph-theoretic techniques to be applied to their analysis and solution. Conversely, certain graph problems arise naturally from scheduling contexts, leading to algorithms with practical applications.

This document explores these connections, with a particular focus on how the concept of graph degeneracy relates to scheduling problems. We will see that d -degenerate graphs provide an important structural characterization that can be exploited in various scheduling contexts.

Definition 111 (Precedence Graph). Given a scheduling problem with precedence constraints, the corresponding precedence graph $G = (V, E)$ has:

- A vertex $v_i \in V$ for each job J_i
- A directed edge $(v_i, v_j) \in E$ if job J_i must be completed before job J_j can start

Example 37. In the single-machine scheduling problem with precedence constraints $(1|prec|\sum C_j)$, the optimal schedule can be found in polynomial time if and only if the precedence graph is a series-parallel graph.

Conflict Graphs for Resource Conflicts

Resource conflicts in scheduling can be modeled using undirected conflict graphs:

Definition 112 (Conflict Graph). Given a scheduling problem with resource constraints, the corresponding conflict graph $G = (V, E)$ has:

- A vertex $v_i \in V$ for each job J_i
- An edge $(v_i, v_j) \in E$ if jobs J_i and J_j cannot be processed simultaneously due to resource constraints

Theorem 105. The problem of scheduling jobs with incompatibility constraints to minimize makespan is equivalent to coloring the corresponding conflict graph, where each color represents a time slot.

Graph Algorithms in Scheduling

Several graph algorithms play crucial roles in solving scheduling problems:

Topological Sorting

Theorem 106. For scheduling problems with precedence constraints, any valid schedule must respect a topological ordering of the precedence graph.

Algorithm 34 Topological Sort for Precedence-Constrained Scheduling

Require: Precedence graph $G = (V, E)$

Ensure: A valid ordering of jobs respecting precedence constraints

```

1:  $L \leftarrow$  empty list
2:  $S \leftarrow$  set of all vertices with no incoming edges
3: while  $S$  is not empty do
4:   Remove a vertex  $v$  from  $S$ 
5:   Add  $v$  to the end of  $L$ 
6:   for all edges  $(v, u)$  emanating from  $v$  do
7:     Remove edge  $(v, u)$  from the graph
8:     if  $u$  has no other incoming edges then
9:       Add  $u$  to  $S$ 
10:    end if
11:  end for
12: end while
13: if graph has remaining edges then
14:   return Error (cycle detected)
15: else
16:   return  $L$ 
17: end if

```

Critical Path Method

The Critical Path Method (CPM) uses the structure of the precedence graph to determine the minimum project completion time:

Definition 113 (Critical Path). In a project network (a weighted directed acyclic graph where vertices are tasks and edge weights are durations), the critical path is the longest path from the start vertex to the end vertex, representing the minimum completion time of the project.

Theorem 107. The earliest completion time of a project with precedence constraints is equal to the length of the critical path in the corresponding project network.

8.9.1 Degenerate Graphs in Scheduling Problems

Degenerate graphs provide a powerful structural characterization that has significant implications for scheduling problems. This section explores the concept of graph degeneracy and its applications in scheduling.

Definition and Properties of d -degenerate Graphs

- Trees and forests are 1-degenerate
- Planar graphs are 5-degenerate
- Outerplanar graphs are 2-degenerate
- Graphs with maximum degree Δ are at most Δ -degenerate

Algorithm 35 Computing Graph Degeneracy

```
1: procedure COMPUTEDEGENERACY( $G = (V, E)$ )
2:    $d \leftarrow 0$  ▷ Initialize degeneracy value
3:    $L \leftarrow \emptyset$  ▷ Initialize empty ordering list
4:   Compute  $\deg(v)$  for each vertex  $v \in V$ 
5:   while  $V$  is not empty do
6:     Find vertex  $v \in V$  with minimum degree
7:      $d \leftarrow \max(d, \deg(v))$  ▷ Update degeneracy
8:     Remove  $v$  from  $G$  and add it to the beginning of  $L$ 
9:   end while
10:  return  $(d, L)$  ▷ Return degeneracy and ordering
11: end procedure
```

Applications of Degeneracy in Scheduling

The concept of degeneracy has several important applications in scheduling problems:

Resource-Constrained Scheduling

Theorem 108. For a scheduling problem where jobs correspond to vertices in a conflict graph G , if G is d -degenerate, then the jobs can be scheduled using at most $k + 1$ resources or time slots.

This follows directly from the fact that d -degenerate graphs are $(k+1)$ -colorable, and a proper coloring corresponds to a valid schedule where jobs of the same color can be processed simultaneously.

Algorithm 36 Scheduling using Degeneracy Ordering

Require: Conflict graph $G = (V, E)$ representing incompatible jobs

Ensure: A schedule using at most $d + 1$ time slots, where d is the degeneracy of G

```
1: Compute a degeneracy ordering  $L = [v_1, v_2, \dots, v_n]$  of  $G$ 
2: Initialize color assignment  $\text{color}(v) \leftarrow \text{null}$  for all  $v \in V$ 
3: for  $i = 1$  to  $n$  do
4:    $C_{\text{used}} \leftarrow \{\text{color}(v_j) : v_j \in N(v_i) \text{ and } j < i\}$  ▷ Colors of neighbors that precede  $v_i$ 
5:    $\text{color}(v_i) \leftarrow \min\{c \geq 1 : c \notin C_{\text{used}}\}$  ▷ First available color
6: end for
7: return color assignment
```

Lemma 96. The coloring produced by Algorithm 36 uses at most $d + 1$ colors, where d is the degeneracy of the graph. This follows from the fact that each vertex has at most d neighbors preceding it in the degeneracy ordering, thus at most d colors are unavailable.

Sparse Dependency Networks

In many practical scheduling problems, the dependency graph (representing precedence constraints) is sparse, often corresponding to a low-degeneracy graph.

Sparse dependency networks often arise in real-world scheduling problems due to:

- Physical constraints limiting the number of interdependencies
- Modular design in projects, where tasks are loosely coupled
- Hierarchical organization of tasks, resulting in tree-like structures

If a precedence graph is d -degenerate, then:

- Each job depends on at most k other jobs
- The project can be organized into at most $k + 1$ logical phases
- Critical path analysis can be performed more efficiently by exploiting the degeneracy structure

Wireless Transmission Scheduling

One particularly important application of degeneracy in scheduling is in wireless networks.

Theorem 109. In a wireless network where nodes correspond to transmitters and edges represent interference between transmitters, if the interference graph is d -degenerate, then the transmitters can be scheduled in at most $k + 1$ time slots without interference.

Example 38. Consider a wireless sensor network deployed in a planar environment. The interference graph of such a network is often planar or nearly planar, making it 5-degenerate. This means that, regardless of network size, transmissions can be scheduled in at most 6 time slots without interference.

Distributed Task Allocation

Degeneracy is also useful in distributed computing environments. In a distributed system where tasks are assigned to processors and the task dependency graph is d -degenerate, a valid task allocation can be found using at most $k + 1$ communication rounds.

Algorithm 37 Distributed Task Scheduling for d -degenerate Dependency Graphs

Require: Dependency graph $G = (V, E)$ with degeneracy d

Ensure: A valid task schedule respecting dependencies

- 1: Compute a degeneracy ordering $L = [v_1, v_2, \dots, v_n]$ of G
 - 2: Partition L into layers $\{L_1, L_2, \dots, L_m\}$ where each layer L_i contains tasks that can be executed in parallel
 - 3: Assign tasks in layer L_i to processors only after all tasks in layers $L_1, \dots, L_{i-1}$ have completed
 - 4: **return** Layer-based schedule
-

Advantages of Degeneracy-Based Scheduling Algorithms

Degeneracy-based approaches offer several advantages for scheduling problems:

1. **Efficiency:** Degeneracy ordering can be computed in linear time $O(|V| + |E|)$
2. **Adaptability:** The approach works for any d -degenerate graph, regardless of other structural properties
3. **Simplicity:** Greedy algorithms based on degeneracy are easy to implement
4. **Performance guarantees:** Provides worst-case bounds on resource usage
5. **Incremental updates:** The degeneracy ordering can be efficiently updated as jobs are added or removed

Case Study: Project Scheduling with Limited Resources

Example 39 (Software Development Project). Consider a software development project with tasks represented as vertices and dependencies as edges. The dependency graph is typically sparse and often d -degenerate for small k due to modular design practices. Using the degeneracy ordering:

- Tasks can be assigned to developers while respecting dependencies
- The number of developers needed is bounded by the degeneracy plus one
- Critical paths can be identified more efficiently
- Resources can be allocated optimally based on the degeneracy structure

Other Graph-Theoretic Approaches to Scheduling

Beyond degeneracy, other graph-theoretic concepts are valuable in scheduling.

Interval Graphs in Scheduling

Interval graphs provide a powerful representation for certain scheduling problems:

Definition 114 (Interval Graph). A graph $G = (V, E)$ is an interval graph if its vertices can be represented as intervals on the real line, with an edge between two vertices if and only if their corresponding intervals overlap.

Theorem 110. The problem of scheduling jobs with fixed start and end times on a minimum number of identical machines is equivalent to finding the chromatic number of the corresponding interval graph.

Corollary 42. Since interval graphs can be colored in polynomial time, the minimum number of machines needed for scheduling jobs with fixed start and end times can be determined efficiently.

Flow Shop and Job Shop Problems as Graph Problems

Flow shop and job shop scheduling problems have natural graph-theoretic interpretations:

Theorem 111 (Johnson's Algorithm for $F_2||C_{max}$). For a two-machine flow shop, the optimal schedule can be found using Johnson's algorithm:

1. Sort jobs such that $\min(p_{j1}, p_{j2}) \leq \min(p_{k1}, p_{k2})$ implies job J_j precedes job J_k .
2. If $p_{j1} \leq p_{j2}$, schedule job J_j as early as possible.
3. If $p_{j1} > p_{j2}$, schedule job J_j as late as possible.

Definition 115 (Disjunctive Graph). For job shop scheduling problems, the disjunctive graph model represents:

- Vertices as operations
- Conjunctive (directed) edges as precedence constraints within jobs
- Disjunctive (undirected) edges as pairs of operations that must use the same machine

A feasible schedule corresponds to an orientation of the disjunctive edges that results in an acyclic graph.

Theorem 112. The makespan of a job shop schedule is equal to the length of the longest path in the corresponding disjunctive graph after orienting the disjunctive edges.

Definition 116 (Disjunctive Graph). For job shop scheduling problems, the disjunctive graph model represents:

- Vertices as operations
- Conjunctive (directed) edges as precedence constraints within jobs
- Disjunctive (undirected) edges as pairs of operations that must use the same machine

A feasible schedule corresponds to an orientation of the disjunctive edges that results in an acyclic graph.

Theorem 113. The makespan of a job shop schedule is equal to the length of the longest path in the corresponding disjunctive graph after orienting the disjunctive edges.

Computational Complexity and Approximation Algorithms

Many scheduling problems are NP-hard, meaning that finding optimal solutions is computationally intractable for large instances. Approximation algorithms and heuristics are often used in practice.

Theorem 114. The following scheduling problems are NP-hard:

- $P_2||C_{max}$ (two identical parallel machines, minimize makespan)
- $1|r_j|\sum C_j$ (single machine with release dates, minimize total completion time)
- $1||\sum w_j U_j$ (single machine, minimize weighted number of tardy jobs)
- $F_3||C_{max}$ (three-machine flow shop, minimize makespan)
- $J_2||C_{max}$ (two-machine job shop, minimize makespan)

Online Scheduling

Problem 4 (Online Scheduling). Schedule jobs that arrive over time without knowing future arrivals, to optimize an objective function.

Theorem 115. For online scheduling of jobs on m identical machines to minimize makespan, the competitive ratio of any deterministic algorithm is at least 1.5.

Algorithm 38 Online List Scheduling

1: When job J_j arrives, assign it to the machine with the earliest completion time

Theorem 116. Online List Scheduling has a competitive ratio of $2 - \frac{1}{m}$ for minimizing makespan on m identical parallel machines.

Applications of Scheduling Problems

Scheduling problems arise in numerous real-world applications:

- **Computing:** CPU scheduling, task scheduling in distributed systems
- **Manufacturing:** Job scheduling in production lines, resource allocation
- **Transportation:** Vehicle routing, flight scheduling, crew scheduling
- **Healthcare:** Operating room scheduling, appointment scheduling
- **Project Management:** Resource allocation, task scheduling
- **Education:** Course scheduling, exam timetabling
- **Service Industries:** Staff scheduling, appointment systems

Conclusion

Scheduling problems form a rich and varied family of optimization problems with significant practical applications. While many scheduling problems are computationally challenging (NP-hard), various techniques such as approximation algorithms, heuristics, and preprocessing methods can be used to find good solutions in practice.

The field of scheduling theory continues to evolve, with researchers developing new algorithms and studying the complexity of various scheduling problems. The interaction between scheduling theory and practice leads to both theoretical advances and improved practical solutions to real-world scheduling problems.

8.9.2 Axillary Algorithms for Scheduling Problems

Definition 117 (Scheduling Problem $1||\sum U_j$). Consider the single-machine scheduling problem denoted as $1||\sum U_j$, where:

- 1: Single machine environment
- $||$: No special constraints
- $\sum U_j$: Objective function to minimize the number of late jobs

For each job j , we have:

- Processing time $p_j > 0$
- Due date $d_j \geq 0$
- Completion time C_j (determined by the schedule)
- Unit penalty $U_j = \begin{cases} 1 & \text{if } C_j > d_j \\ 0 & \text{otherwise} \end{cases}$

The goal is to find a schedule that minimizes $\sum_{j=1}^n U_j$, i.e., the total number of late jobs.

Lemma 97 (Moore's Algorithm). Moore's algorithm provides an optimal solution to the $1||\sum U_j$ problem with time complexity $O(n \log n)$.

Remark 99. The time complexity consists of:

Algorithm 39 Moore's Algorithm

```
1: procedure MOORE( $J$ )
2:   Sort jobs in non-decreasing order of due dates (EDD rule)
3:    $J = \{j_1, j_2, \dots, j_n\}$  such that  $d_{j_1} \leq d_{j_2} \leq \dots \leq d_{j_n}$ 
4:    $S \leftarrow \emptyset$  ▷ Initialize empty schedule
5:    $t \leftarrow 0$  ▷ Initialize current time
6:   for  $j = 1$  to  $n$  do
7:      $S \leftarrow S \cup \{j\}$  ▷ Add job  $j$  to schedule
8:      $t \leftarrow t + p_j$  ▷ Update current time
9:     if  $t > d_j$  then ▷ If job  $j$  would be late
10:      find  $j' \in S$  with maximum  $p_{j'}$  ▷ Find longest job in  $S$ 
11:       $S \leftarrow S \setminus \{j'\}$  ▷ Remove  $j'$  from schedule
12:       $t \leftarrow t - p_{j'}$  ▷ Update current time
13:     end if
14:   end for
15:   return  $S$ 
16: end procedure
```

- *Sorting jobs by due dates:* $O(n \log n)$
- *For each job, potentially finding the maximum processing time job:* $O(n)$ per iteration
- *Overall complexity:* $O(n^2)$

This can be improved to $O(n \log n)$ by using a max-heap to efficiently find and remove the job with maximum processing time.

Theorem 117 (Optimality of Moore's Algorithm). Moore's algorithm produces an optimal solution to the $1||\sum U_j$ problem.

Proof (sketch). Let S be the schedule produced by Moore's algorithm and S^* be an optimal schedule. Suppose $S \neq S^*$. Let j be the earliest job in the EDD ordering that is in one schedule but not the other.

We can show through an exchange argument that:

1. If $j \in S$ but $j \notin S^*$, we can include j in S^* without increasing the number of late jobs.
2. If $j \notin S$ but $j \in S^*$, we can replace j in S^* with the job that Moore's algorithm discarded when considering j , without increasing the number of late jobs.

This leads to a contradiction of the optimality of S^* or the correctness of Moore's algorithm, establishing that Moore's algorithm indeed produces an optimal solution. $\square$

Lemma 98 (Heap-Based Implementation). A more efficient implementation of Moore's algorithm uses a max-heap data structure:

Remark 100. With this implementation, the time complexity is $O(n \log n)$, as each heap operation takes $O(\log n)$ time, and we perform at most $O(n)$ such operations.

Precedence constraints are given as a directed acyclic graph where an edge (i, j) means job i must be completed before job j can start.

The goal is to find a schedule that minimizes $\sum_{j=1}^n w_j T_j$, i.e., the weighted sum of tardiness across all jobs while respecting precedence constraints.

Lemma 99 (Lawler's Dynamic Programming Algorithm). Lawler's algorithm provides an optimal solution to the $1|prec|\sum w_j T_j$ problem using dynamic programming based on subsets of jobs.

Remark 101. The time complexity consists of:

- *Number of subsets considered:* $O(2^n)$
- *Work per subset:* $O(n)$ to find feasible last jobs and compute costs
- *Overall complexity:* $O(n \cdot 2^n)$

Algorithm 40 Moore's Algorithm with Heap

```
1: procedure MOOREWITHHEAP( $J$ )
2:   Sort jobs in non-decreasing order of due dates (EDD rule)
3:    $J = \{j_1, j_2, \dots, j_n\}$  such that  $d_{j_1} \leq d_{j_2} \leq \dots \leq d_{j_n}$ 
4:    $S \leftarrow \emptyset$  ▷ Initialize empty schedule
5:    $H \leftarrow$  empty max-heap ▷ For tracking processing times
6:    $t \leftarrow 0$  ▷ Initialize current time
7:   for  $j = 1$  to  $n$  do
8:      $S \leftarrow S \cup \{j\}$  ▷ Add job  $j$  to schedule
9:      $H.INSERT(p_j)$  ▷ Add processing time to heap
10:     $t \leftarrow t + p_j$  ▷ Update current time
11:    if  $t > d_j$  then ▷ If job  $j$  would be late
12:       $p_{\max} \leftarrow H.EXTRACTMAX()$  ▷ Extract max processing time
13:       $S \leftarrow S \setminus \{\text{job with processing time } p_{\max}\}$ 
14:       $t \leftarrow t - p_{\max}$  ▷ Update current time
15:    end if
16:  end for
17:  return  $S$ 
18: end procedure
```

Algorithm 41 Lawler's Dynamic Programming Algorithm

```
1: procedure LAWLERDP( $J, prec$ )
2:   Initialize  $OPT[\emptyset] \leftarrow 0$  ▷ Base case: empty set has zero cost
3:   Initialize  $schedule[\emptyset] \leftarrow \emptyset$  ▷ Empty schedule for empty set
4:   for  $k = 1$  to  $n$  do ▷ Iterate over all subset sizes
5:     for each subset  $S \subseteq J$  with  $|S| = k$  do
6:        $OPT[S] \leftarrow \infty$  ▷ Initialize optimal cost for subset  $S$ 
7:       for each job  $j \in S$  that has no successors in  $S$  according to  $prec$  do
8:          $S' \leftarrow S \setminus \{j\}$  ▷ Subset without job  $j$ 
9:         Calculate completion time  $C_j$  when  $j$  is scheduled after  $S'$ 
10:        Calculate tardiness  $T_j = \max(0, C_j - d_j)$ 
11:        Calculate cost  $cost \leftarrow OPT[S'] + w_j \cdot T_j$ 
12:        if  $cost < OPT[S]$  then
13:           $OPT[S] \leftarrow cost$  ▷ Update optimal cost
14:           $schedule[S] \leftarrow schedule[S'] + j$  ▷ Update optimal schedule
15:        end if
16:      end for
17:    end for
18:  end for
19:  return  $schedule[J]$  ▷ Return optimal schedule for all jobs
20: end procedure
```

Despite the exponential worst-case complexity, this algorithm is practical for instances with up to a few dozen jobs, especially when problem-specific optimizations are applied.

Theorem 118 (Optimality of Lawler's Algorithm). Lawler's dynamic programming algorithm produces an optimal solution to the $1|prec|\sum w_j T_j$ problem.

Proof (sketch). The proof follows from the principle of optimality for dynamic programming. For any subset S of jobs, if we know the optimal schedule for every proper subset, then we can determine the optimal schedule for S by trying each feasible last job and selecting the one that minimizes the total cost.

A job j is a feasible last job for subset S if it has no successors in S according to the precedence constraints. When j is scheduled last in S , its completion time depends only on the schedule of jobs in $S \setminus \{j\}$, and its contribution to the objective function is $w_j \cdot \max(0, C_j - d_j)$.

By systematically building solutions from smaller to larger subsets and maintaining optimality at each step, the algorithm guarantees an optimal solution for the complete set of jobs. $\square$

Lemma 100 (Implementation Optimizations). Several optimizations can reduce the practical complexity of Lawler's algorithm:

- **Dominance rules:** Eliminate subsets that cannot lead to optimal solutions based on job characteristics.
- **State-space reduction:** Exploit precedence constraints to reduce the number of subsets considered.
- **Bounding:** Use lower bounds to prune suboptimal partial solutions early.
- **Memoization:** Only compute and store values for subsets that are actually reached during the recursive computation.

Remark 102. With these optimizations, many practical instances with up to 20-30 jobs can be solved efficiently, despite the exponential worst-case complexity. The algorithm is particularly effective when the precedence constraints significantly restrict the set of feasible schedules.

Remark 103 (Special Cases). For the special case where all weights are equal ($w_j = 1$ for all j), or when there are no precedence constraints, more efficient algorithms exist. However, Lawler's approach remains one of the most effective exact methods for the general $1|prec|\sum w_j T_j$ problem.

Definition 118 (Scheduling Problem $P||C_{\max}$). Consider the parallel identical machines scheduling problem denoted as $P||C_{\max}$, where:

- P : Parallel identical machines environment
- $||$: No special constraints
- $C_{\max}$: Objective function to minimize the makespan (completion time of the last job)

We have m identical machines and n jobs, where:

- Each job j has processing time $p_j > 0$
- Each job must be processed without interruption on one of the machines
- All machines are identical and can process any job
- The makespan $C_{\max} = \max_{i=1}^m C_i$ is the maximum completion time across all machines

The goal is to find a schedule that minimizes the makespan $C_{\max}$.

Lemma 101 (List Scheduling Algorithm). The List Scheduling algorithm is a simple greedy approach for the $P||C_{\max}$ problem:

Remark 104. The time complexity is $O(n \log m)$ if a min-heap is used to keep track of machine loads.

Theorem 119 (Approximation Ratio of List Scheduling). List Scheduling is a 2-approximation algorithm for the $P||C_{\max}$ problem.

Proof. Let OPT be the makespan of an optimal schedule. We know that:

$$OPT \geq \frac{\sum_{j=1}^n p_j}{m} \quad (\text{average load}) \quad (82)$$

$$OPT \geq \max_{j=1}^n p_j \quad (\text{maximum job processing time}) \quad (83)$$

Algorithm 42 List Scheduling

```
1: procedure LISTSCHEDULING( $J, m$ )
2:   Initialize  $load[i] \leftarrow 0$  for all machines  $i = 1, 2, \dots, m$  ▷ Current load on each machine
3:   for each job  $j \in J$  do ▷ In arbitrary order
4:     Find machine  $i$  with minimum current load  $load[i]$ 
5:     Assign job  $j$  to machine  $i$ 
6:      $load[i] \leftarrow load[i] + p_j$  ▷ Update machine load
7:   end for
8:   return assignment of jobs to machines
9: end procedure
```

Let j^* be the last job to complete in the List Scheduling solution, and let i^* be the machine it was assigned to. When j^* was assigned, machine i^* had the lowest load among all machines. Let C_{j^*} be the completion time of job j^* , which equals the makespan of the List Scheduling solution.

We have:

$$C_{j^*} = load_{i^*} \text{ (before } j^* \text{ was assigned)} + p_{j^*} \quad (84)$$

$$\leq \frac{\sum_{j \neq j^*} p_j}{m} + p_{j^*} \quad (\text{machine } i^* \text{ had minimum load}) \quad (85)$$

$$= \frac{\sum_{j=1}^n p_j - p_{j^*}}{m} + p_{j^*} \quad (86)$$

$$= \frac{\sum_{j=1}^n p_j}{m} + p_{j^*} \left(1 - \frac{1}{m}\right) \quad (87)$$

$$\leq OPT + OPT \left(1 - \frac{1}{m}\right) \quad (88)$$

$$< 2 \cdot OPT \quad (89)$$

Therefore, the makespan of the List Scheduling solution is less than twice the optimal makespan. $\square$

Lemma 102 (Longest Processing Time First (LPT) Algorithm). The LPT algorithm is an improvement over List Scheduling for the $P||C_{\max}$ problem:

Algorithm 43 Longest Processing Time First (LPT)

```
1: procedure LPT( $J, m$ )
2:   Sort jobs in non-increasing order of processing times
3:    $J = \{j_1, j_2, \dots, j_n\}$  such that  $p_{j_1} \geq p_{j_2} \geq \dots \geq p_{j_n}$ 
4:   Initialize  $load[i] \leftarrow 0$  for all machines  $i = 1, 2, \dots, m$ 
5:   for each job  $j \in J$  do ▷ In sorted order (longest first)
6:     Find machine  $i$  with minimum current load  $load[i]$ 
7:     Assign job  $j$  to machine  $i$ 
8:      $load[i] \leftarrow load[i] + p_j$ 
9:   end for
10:  return assignment of jobs to machines
11: end procedure
```

Remark 105. The time complexity is $O(n \log n)$ for sorting plus $O(n \log m)$ for job assignments, giving a total of $O(n \log n)$ since typically $n \geq m$.

Theorem 120 (Approximation Ratio of LPT). The LPT algorithm is a $(\frac{4}{3} - \frac{1}{3m})$ -approximation algorithm for the $P||C_{\max}$ problem.

Proof (sketch). The proof is more complex than for List Scheduling, involving a careful analysis of the structure of an LPT schedule. The key insight is that when LPT performs poorly, there must be a specific structure to the schedule that can be bounded.

Through case analysis, one can show that for any instance:

$$C_{\max}^{LPT} \leq \left(\frac{4}{3} - \frac{1}{3m} \right) \cdot OPT \quad (90)$$

This bound is tight, meaning there exist instances where LPT produces a makespan exactly $(\frac{4}{3} - \frac{1}{3m})$ times the optimal. $\square$

Lemma 103 (PTAS for $P_m||C_{\max}$). For a fixed number of machines m , there exists a Polynomial Time Approximation Scheme (PTAS) for the $P_m||C_{\max}$ problem:

Algorithm 44 PTAS for $P_m||C_{\max}$

- 1: **procedure** PTAS(J, m, ϵ)
 - 2: Let T be a guessed lower bound on OPT (binary search on possible values)
 - 3: Partition jobs into "large" ($p_j > \epsilon T$) and "small" ($p_j \leq \epsilon T$)
 - 4: Round processing times of large jobs to multiples of $\frac{\epsilon^2 T}{n}$
 - 5: Enumerate all possible assignments of large jobs to machines (patterns)
 - 6: For each feasible pattern (load $\leq (1 + \epsilon)T$ on each machine):
 - 7: Assign small jobs greedily in any order to machines with smallest load
 - 8: Return the best schedule found
 - 9: **end procedure**
-

Remark 106. For fixed m and ϵ , the algorithm runs in polynomial time $O(n^{O(m/\epsilon)})$.

Theorem 121 (Approximation Ratio of PTAS). For any $\epsilon > 0$, the PTAS algorithm produces a schedule with makespan at most $(1 + \epsilon) \cdot OPT$ for the $P_m||C_{\max}$ problem with a fixed number of machines m .

Proof (sketch). The proof consists of several steps:

1. There are at most $O(1/\epsilon)$ large jobs per machine in an optimal solution
2. Rounding large jobs introduces at most $\epsilon \cdot OPT$ error
3. The number of different patterns to consider is polynomial for fixed m and ϵ
4. Greedy assignment of small jobs adds at most $\epsilon \cdot OPT$ to the makespan

Combining these observations, we get a $(1 + \epsilon)$ -approximation algorithm that runs in polynomial time for fixed m and ϵ . $\square$

Definition 119 (Scheduling Problem $R||C_{\max}$). Consider the unrelated parallel machines scheduling problem denoted as $R||C_{\max}$, where:

- R : Unrelated parallel machines environment
- $||$: No special constraints
- $C_{\max}$: Objective function to minimize the makespan

We have m unrelated machines and n jobs, where:

- Each job j has processing time $p_{ij} > 0$ on machine i
- Processing times may vary across machines (unrelated machines)
- Each job must be processed without interruption on one of the machines
- The makespan $C_{\max} = \max_{i=1}^m C_i$ is the maximum completion time across all machines

The goal is to find a schedule that minimizes the makespan $C_{\max}$.

Lemma 104 (Shmoys-Tardos Algorithm). The Shmoys-Tardos algorithm for the $R||C_{\max}$ problem uses linear programming relaxation and rounding:

Remark 107. The algorithm runs in polynomial time, with the LP solving being the most expensive step.

Theorem 122 (Approximation Ratio of Shmoys-Tardos). The Shmoys-Tardos algorithm is a 2-approximation algorithm for the $R||C_{\max}$ problem.

Algorithm 45 Shmoys-Tardos Algorithm

- 1: Guess a value T for the optimal makespan (binary search)
 - 2: Formulate the configuration LP:
 - 3: Variables x_{ij} = fraction of job j assigned to machine i
 - 4: Constraints: $\sum_{i=1}^m x_{ij} = 1$ for all jobs j
 - 5: Constraints: $\sum_{j=1}^n p_{ij} \cdot x_{ij} \leq T$ for all machines i
 - 6: Constraints: $x_{ij} = 0$ if $p_{ij} > T$
 - 7: Solve the LP to obtain fractional assignment x_{ij}^*
 - 8: Convert fractional solution to integral assignment using bipartite matching:
 - 9: Construct bipartite graph with jobs on one side, machines on other
 - 10: Add edge (i, j) with weight x_{ij}^*
 - 11: Find a minimum-cost perfect matching in this graph
 - 12: Return the resulting integral assignment
-

Proof (sketch). The proof follows these key steps:

1. If $T < OPT$, the LP is infeasible
2. If $T \geq OPT$, the LP has a feasible solution
3. The rounding procedure ensures each job is assigned to exactly one machine
4. The load on any machine i is at most $\sum_{j: x_{ij}^* > 0} p_{ij} \leq 2T$

The last step follows from the properties of the bipartite matching: a job j may be assigned to a machine i only if $x_{ij}^* > 0$ in the LP solution, and the maximum processing time of any such job is at most T . Therefore, the makespan of the resulting schedule is at most $2 \cdot OPT$. $\square$

Definition 120 (Mixed-Integer Programming for Scheduling). Mixed-Integer Programming (MIP) is a general-purpose optimization approach that can be applied to a wide range of scheduling problems. For scheduling problems, we typically formulate:

- **Decision variables:**
 - Binary assignment variables $x_{ij} \in \{0, 1\}$ indicating if job j is assigned to machine i
 - Binary sequencing variables $y_{jk} \in \{0, 1\}$ indicating if job j precedes job k
 - Continuous time variables $C_j \geq 0$ representing completion time of job j
- **Objective function:** Minimize relevant metric (e.g., makespan, total weighted completion time)
- **Constraints:** Machine capacity, precedence relations, disjunctive constraints for resource conflicts

Lemma 105 (Time-Indexed MIP Formulation). A common approach for single-machine scheduling problems is the time-indexed formulation:

Let $x_{jt} \in \{0, 1\}$ indicate whether job j starts at time t .

$$\min \sum_{j=1}^n \sum_{t=0}^{T-p_j} w_j(t + p_j) x_{jt} \quad (91)$$

$$\text{s.t.} \quad \sum_{t=0}^{T-p_j} x_{jt} = 1 \quad \forall j \in \{1, 2, \dots, n\} \quad (92)$$

$$\sum_{j=1}^n \sum_{s=\max(0, t-p_j+1)}^t x_{js} \leq 1 \quad \forall t \in \{0, 1, \dots, T-1\} \quad (93)$$

$$x_{jt} \in \{0, 1\} \quad \forall j \in \{1, 2, \dots, n\}, t \in \{0, 1, \dots, T-p_j\} \quad (94)$$

where T is an upper bound on the makespan, p_j is the processing time of job j , and w_j is the weight of job j .

Remark 108. The first constraint ensures each job is scheduled exactly once. The second constraint ensures that at most one job is processed at any time t . The objective minimizes the weighted sum of completion times.

Theorem 123 (Strengthening MIP Formulations). MIP formulations for scheduling problems can be significantly strengthened through the addition of valid inequalities, particularly:

- **Cover inequalities:** If a subset S of jobs cannot all be processed within a given deadline d , then $\sum_{j \in S} \sum_{t=0}^{d-p_j} x_{jt} \leq |S| - 1$
- **Strengthened precedence constraints:** If job i must precede job j , then $C_i + p_j \leq C_j + M(1 - x_{ij})$ can be replaced with $C_i + p_j \leq C_j$ when $x_{ij} = 1$ must hold
- **Symmetry-breaking constraints:** For identical jobs, impose an arbitrary ordering, e.g., $C_1 \leq C_2 \leq \dots \leq C_k$ for identical jobs $1, 2, \dots, k$

Proof (sketch). These inequalities are valid because they eliminate solutions that violate problem constraints or are symmetric to other solutions. They strengthen the linear programming relaxation, reducing the integrality gap and accelerating the solution process by pruning more of the branch-and-bound tree. $\square$

Remark 109 (Practical Performance of MIP). *Commercial MIP solvers like Gurobi, CPLEX, and SCIP can handle scheduling problems with up to a few hundred jobs, particularly when:*

- *The formulation is tight (small integrality gap)*
- *Problem-specific cuts are implemented*
- *Warm-start solutions from heuristics are provided*
- *Appropriate branching priorities are set*

The practical limit depends on the specific problem structure and time constraints, but problems with 50-200 jobs are typically manageable with modern solvers.

Definition 121 (Column Generation for Scheduling). Column Generation is a decomposition technique for solving large-scale linear programs by generating columns (variables) on-demand. For scheduling problems, columns typically represent:

- Complete schedules for a single machine
- Feasible routing paths for a vehicle
- Workshift assignments for an employee

The approach decomposes the problem into:

- **Master Problem:** Select which columns (schedules) to use
- **Pricing Subproblem:** Generate new promising columns (schedules)

Lemma 106 (Column Generation Algorithm). The Column Generation approach for scheduling problems follows this general pattern:

Algorithm 46 Column Generation for Scheduling

```

1: procedure COLUMNGENERATION(Jobs, Machines, Constraints)
2:   Initialize restricted master problem (RMP) with a small set of feasible columns
3:   repeat
4:     Solve the RMP to obtain dual variables  $\pi$ 
5:     for each machine or resource do
6:       Solve pricing subproblem using  $\pi$  to find column with negative reduced cost
7:       if column with negative reduced cost found then
8:         Add column to RMP
9:       end if
10:    end for
11:  until no column with negative reduced cost is found
12:  return optimal solution to the linear relaxation
13: end procedure

```

Remark 110. *The pricing subproblem often takes the form of a resource-constrained shortest path problem or a knapsack problem, which can be solved efficiently using dynamic programming.*

Algorithm 47 Branch-and-Price for Scheduling

```
1: procedure BRANCHANDPRICE(Jobs, Machines, Constraints)
2:   Initialize branch-and-bound tree with root node
3:   while tree has unexplored nodes do
4:     Select unexplored node
5:     Apply column generation to solve LP relaxation at node
6:     if LP solution is integer then
7:       Update incumbent solution if better
8:     else if LP bound  $\geq$  incumbent then
9:       Identify branching variable (e.g., job assignment or sequencing decision)
10:      Create child nodes based on branching decision
11:    end if
12:  end while
13:  return optimal integer solution
14: end procedure
```

Theorem 124 (Branch-and-Price Framework). Branch-and-Price combines column generation with branch-and-bound to solve integer programs:

Proof of optimality. The optimality of Branch-and-Price follows from:

- Column generation guarantees the optimal solution to the LP relaxation at each node
- Branch-and-bound with proper branching rules eventually explores all feasible integer solutions
- Pruning based on bounds maintains optimality while improving efficiency

□

Remark 111 (Applications in Scheduling). *Branch-and-Price is particularly effective for:*

- *Crew scheduling and rostering problems*
- *Vehicle routing with time windows*
- *Parallel machine scheduling with sequence-dependent setup times*
- *Resource-constrained project scheduling*

The technique enables solving scheduling problems with complex constraints by encapsulating these constraints in the pricing subproblem, while the master problem handles the global resource allocation.

Definition 122 (Benders Decomposition for Scheduling). Benders Decomposition partitions variables in a MIP into:

- **Complicating variables:** Often binary assignment decisions (which job goes to which machine)
- **Continuous variables:** Typically timing decisions (when each job starts/completes)

The problem is decomposed into:

- **Master Problem:** Makes decisions on complicating variables
- **Subproblem:** Given fixed complicating variables, optimizes continuous variables or generates feasibility cuts

Lemma 107 (Benders Decomposition Algorithm). The Benders Decomposition approach for scheduling problems follows this iterative pattern:

Remark 112. *For scheduling problems, Benders cuts often have a natural interpretation:*

- *Feasibility cuts: "These job assignments cannot be scheduled feasibly"*
- *Optimality cuts: "Any assignment similar to this one has completion time at least z "*

Theorem 125 (Convergence of Benders Decomposition). Under standard assumptions, Benders Decomposition converges to an optimal solution in a finite number of iterations.

Proof (sketch). The proof relies on the following key properties:

- The master problem's feasible region is progressively restricted by cuts

Algorithm 48 Benders Decomposition for Scheduling

```
1: procedure BENDERSDECOMPOSITION(Jobs, Machines, Constraints)
2:   Initialize master problem with complicating variables and no cuts
3:   repeat
4:     Solve restricted master problem to obtain assignment  $x^*$ 
5:     Fix assignment variables to  $x^*$  and solve subproblem
6:     if subproblem is infeasible then
7:       Generate feasibility cut:  $\sum_{(i,j):x_{ij}^*=1}(1-x_{ij}) + \sum_{(i,j):x_{ij}^*=0}x_{ij} \geq 1$ 
8:       Add feasibility cut to master problem
9:     else if subproblem is feasible with objective value  $z_{sub}$  then
10:      Generate optimality cut using dual solution of subproblem
11:      Add optimality cut to master problem
12:     end if
13:   until no more cuts needed (optimal solution found)
14:   return optimal solution
15: end procedure
```

- Each cut eliminates at least one integer solution
- There are finitely many integer solutions for the complicating variables
- The algorithm terminates when no more cuts can be generated, which happens only when the current solution is optimal

□

Remark 113 (Modern Benders Implementations). *Modern implementations of Benders Decomposition improve performance through:*

- **Cut management:** Only adding cuts that significantly improve bounds
- **In-tree implementation:** Generating cuts at nodes of the branch-and-bound tree
- **Stabilization techniques:** Preventing oscillation in the master problem solutions
- **Initialization:** Starting with strong valid inequalities and heuristic solutions

These enhancements make Benders Decomposition particularly effective for large-scale scheduling problems with complex timing constraints, where the subproblem structure (after fixing assignments) is relatively simple.

Definition 123 (Constraint Programming for Scheduling). Constraint Programming (CP) is a declarative paradigm particularly well-suited for scheduling problems. In CP, a scheduling problem is modeled as:

- **Decision variables:**
 - start_j : Start time of job j
 - end_j : End time of job j
 - machine_j : Machine assigned to job j
- **Domains:** Finite sets of possible values for each variable
- **Constraints:** Relations that must be satisfied by variable assignments
- **Objective function:** Measure to minimize or maximize (e.g., makespan)

CP solvers use:

- **Constraint propagation:** Reducing variable domains based on constraints
- **Search:** Systematically exploring combinations of variable assignments
- **Backtracking:** Undoing assignments that lead to constraint violations

Lemma 108 (Global Constraints for Scheduling). CP for scheduling relies on specialized global constraints that model common scheduling requirements:

1. **cumulative(starts, durations, resources, capacity):** Ensures resource usage never exceeds capacity

- $starts_j$: Start time of job j
 - $durations_j$: Processing time of job j
 - $resources_j$: Resource requirement of job j
 - $capacity$: Maximum available resource
2. **noOverlap(starts, durations)**: Ensures jobs do not overlap in time (for single machine scheduling)
 3. **precedence(start_{*i*}, duration_{*i*}, start_{*j*})**: Ensures job i completes before job j starts
 4. **alternative(start, duration, machines, machineStarts)**: Models optional assignment of jobs to machines
 5. **disjunctive(starts, durations)**: Special case of cumulative with unit resource requirements and unit capacity

Algorithm 49 CP Model for Job Shop Scheduling

```

1: procedure CPJOBSHOP(Jobs, Machines, ProcessingTimes, Operations)
2:   Define variables:
3:      $start_{j,o}$  for start time of operation  $o$  of job  $j$ 
4:      $end_{j,o} = start_{j,o} + ProcessingTimes_{j,o,m}$  where  $m$  is the machine for operation  $o$ 
5:   Add constraints:
6:     // Precedence constraints between operations of the same job
7:     for each job  $j$  and consecutive operations  $o_1, o_2$  do
8:        $precedence(start_{j,o_1}, ProcessingTimes_{j,o_1,m}, start_{j,o_2})$ 
9:     end for
10:    // No overlap constraints for operations on the same machine
11:    for each machine  $m$  do
12:       $operations_m \leftarrow$  operations assigned to machine  $m$ 
13:       $starts_m \leftarrow$  start times of operations in  $operations_m$ 
14:       $durations_m \leftarrow$  processing times of operations in  $operations_m$ 
15:       $noOverlap(starts_m, durations_m)$ 
16:    end for
17:   Define objective:
18:      $makespan \leftarrow \max_{j,o} end_{j,o}$ 
19:      $minimize(makespan)$ 
20:   Solve using CP search
21:   return schedule
22: end procedure

```

Remark 114. The power of CP for scheduling comes from specialized filtering algorithms (propagators) for each global constraint. For example:

- The **cumulative** constraint uses edge-finding, time-tabling, and energetic reasoning
- The **disjunctive** constraint uses specialized algorithms to detect mandatory precedences

These propagators efficiently detect and remove inconsistent values from variable domains, drastically reducing the search space.

Theorem 126 (Propagation Strength of Global Constraints). Global constraints in CP scheduling models provide stronger inference than equivalent decompositions into elementary constraints.

Proof (sketch). Consider a disjunctive constraint over n operations. Its direct propagation can:

- Detect necessary precedences through edge-finding and not-first/not-last algorithms
- Update earliest start times and latest completion times of operations
- Establish order relationships even before variable domains are reduced to singletons

In contrast, a decomposition into $O(n^2)$ binary constraints ($start_i + duration_i \leq start_j$ OR $start_j + duration_j \leq start_i$) would achieve weaker propagation, typically waiting until one variable becomes fixed before propagating to others.

The cumulative constraint similarly achieves stronger inference through specialized energy-based reasoning that cannot be matched by decomposition into simpler constraints. $\square$

Remark 115 (Performance Characteristics of CP for Scheduling). *CP solvers like Google OR-Tools, IBM ILOG CP Optimizer, and Gecode excel at scheduling problems with:*

- *Complex temporal constraints and resource requirements*
- *Highly combinatorial decision spaces*
- *Small to medium-sized instances (typically up to 100-200 operations)*
- *Problems where finding feasible solutions is challenging*

CP often outperforms MIP on highly constrained scheduling problems, but may struggle with:

- *Large-scale problems (thousands of operations)*
- *Problems where the bound on the objective is weak*
- *Problems requiring reasoning about sums or averages*

Lemma 109 (Hybrid CP-MIP Approach). Hybrid approaches combine the strengths of CP and MIP by:

Algorithm 50 Hybrid CP-MIP for Scheduling

```

1: procedure HYBRIDCPMIP(Jobs, Machines, Constraints)
2:   // Phase 1: Use CP to find good feasible solutions quickly
3:   CPModel  $\leftarrow$  Formulate problem using CP with global constraints
4:   Set time limit for CP search
5:   feasibleSchedules  $\leftarrow$  Run CP search with diversification
6:   // Phase 2: Use MIP to improve solutions or prove optimality
7:   MIPModel  $\leftarrow$  Formulate problem as MIP
8:   Warm-start MIP with best solutions from CP phase
9:   Add constraints derived from CP search (e.g., bounds, valid inequalities)
10:  Run MIP solver to improve solution or prove optimality
11:  return best solution found
12: end procedure

```

Remark 116. *Several strategies exist for CP-MIP hybridization:*

- **Sequential:** *Use CP to find feasible solutions, then MIP to prove optimality*
- **Parallel:** *Run CP and MIP solvers concurrently, sharing bounds and solutions*
- **Integrated:** *Use MIP for assignment decisions and CP for scheduling decisions*
- **Logic-based Benders:** *Use MIP master problem and CP subproblems*

The appropriate strategy depends on the problem characteristics and the relative strengths of CP and MIP for specific constraint types.

Theorem 127 (Complementarity of CP and MIP). CP and MIP exhibit complementary strengths for scheduling problems, making hybrid approaches particularly effective.

Proof (informal). The complementarity arises from fundamental differences in how each technique handles constraints:

- **CP excels at:** Logical constraints, disjunctions, global constraints with specialized propagators
- **MIP excels at:** Linear constraints, strong relaxations providing bounds, large-scale problems

For a scheduling problem with both assignment and sequencing decisions:

1. CP can quickly explore the combinatorial space of feasible solutions
2. MIP can provide strong lower bounds and gradually improve solution quality
3. CP struggles with objective-based reasoning that MIP handles effectively
4. MIP struggles with logical constraints that CP handles naturally

Empirically, hybrid approaches have been shown to outperform pure CP or pure MIP on a variety of benchmark scheduling problems, particularly those combining assignment, sequencing, and resource constraints. $\square$

Algorithm 51 CP Implementation for Job Shop Scheduling

```
1: procedure CPJOBShopIMPLEMENTATION(Jobs, Machines, ProcessingTimes)
2:   Create CP model
3:    $horizon \leftarrow \sum$  all processing times
4:   Initialize empty maps taskStarts, taskEnds
5:   Initialize empty list allTasks
6:   for each job in Jobs do
7:     for each task in job do
8:       machine  $\leftarrow$  machine required by task
9:       duration  $\leftarrow$  ProcessingTimes[job, task]
10:      Create variable start  $\in [0, horizon]$ 
11:      Create variable end  $\in [0, horizon]$ 
12:      Add constraint: end = start + duration
13:      taskStarts[job, task]  $\leftarrow$  start
14:      taskEnds[job, task]  $\leftarrow$  end
15:      Add (job, task, machine, start, end) to allTasks
16:    end for
17:  end for ▷ Add precedence constraints within jobs

18:  for each job in Jobs do
19:    for task = 0 to length(job) - 2 do
20:      Add constraint: taskStarts[job, task + 1]  $\geq$  taskEnds[job, task]
21:    end for
22:  end for ▷ Add no-overlap constraints for machines

23:  Initialize map machineTasks grouping tasks by machine
24:  for each (job, task, machine, start, end) in allTasks do
25:    Add (start, end) to machineTasks[machine]
26:  end for
27:  for each machine in Machines do
28:    Add noOverlap constraint on all machineTasks[machine]
29:  end for ▷ Define and minimize makespan

30:  Create variable makespan  $\in [0, horizon]$ 
31:  Add constraint: makespan = max of all task end times
32:  Set objective: Minimize makespan
33:  Run CP solver
34:  return resulting schedule
35: end procedure
```

Remark 117 (Example Implementation). A CP model for job shop scheduling can be implemented using the following pseudocode:

This pseudocode captures the essence of scheduling with global constraints for precedence relations and no-overlap requirements. It demonstrates how CP models use specialized constraints like noOverlap to efficiently handle disjunctive resources.

Definition 124 (Real-Time Scheduling). Real-time scheduling addresses problems where:

- Jobs have release times r_j and hard deadlines d_j
- Missing a deadline results in system failure or significant penalty
- Scheduler must make decisions as jobs arrive (online setting)
- Preemption (interrupting and resuming jobs) is often allowed

Key metrics include:

- Schedulability: Whether all jobs can meet their deadlines
- Processor utilization: $U = \sum_{j=1}^n \frac{C_j}{T_j}$, where C_j is computation time and T_j is period
- Response time: Time between release and completion

Lemma 110 (Earliest Deadline First (EDF) Algorithm). EDF is a dynamic-priority scheduling algorithm that always executes the ready job with the earliest absolute deadline:

Algorithm 52 Earliest Deadline First (EDF)

```

1: procedure EDF(JobQueue)
2:   Initialize empty schedule
3:   time  $\leftarrow$  0
4:   while jobs remain to be scheduled do
5:     Update JobQueue with newly released jobs at time
6:     Remove completed jobs from JobQueue
7:     if JobQueue is not empty then
8:       Find job  $j^*$  with earliest deadline in JobQueue
9:       Execute job  $j^*$  for one time unit or until next event
10:      (Next event = job release, job completion, or preemption)
11:     else
12:       Advance time to next job release
13:     end if
14:     Update time accordingly
15:   end while
16:   return schedule
17: end procedure

```

Remark 118. For periodic task systems where job j arrives every T_j time units and requires C_j processing time:

- EDF can be implemented with $O(\log n)$ time complexity per scheduling decision using a priority queue
- For aperiodic task systems, EDF can be combined with acceptance tests to determine if a new job can be admitted without causing deadline misses

Theorem 128 (Optimality of EDF). For preemptive scheduling on a single processor with feasible job sets, EDF is optimal in terms of schedulability. Specifically:

- For periodic task systems: EDF can schedule any task set with utilization $U \leq 1$
- For aperiodic task systems: If there exists any algorithm that can schedule a given set of jobs without missing deadlines, then EDF can also schedule the same set without missing deadlines

Proof (sketch). The proof uses a contradiction technique and schedule transformation:

1. Assume a feasible schedule S exists but EDF fails
2. Consider the first time t where EDF makes a "wrong" decision compared to S

3. We can swap jobs in S to prioritize the earlier deadline job (as EDF would)
4. This transformation does not make S infeasible
5. By repeated transformations, S can be converted to an EDF schedule
6. Thus, if any feasible schedule exists, EDF produces a feasible schedule

For the utilization bound, consider periodic tasks. When $U \leq 1$:

- Total processing demand never exceeds available processor time
- EDF's optimal scheduling ensures no idle time is introduced if jobs are ready
- Therefore all deadlines are met

When $U > 1$, deadline misses are inevitable in the long run as demand exceeds capacity. $\square$

Lemma 111 (Rate Monotonic Scheduling (RMS)). RMS is a static-priority scheduling algorithm that assigns priorities based on job periods:

Algorithm 53 Rate Monotonic Scheduling (RMS)

```

1: procedure RMS( $Tasks$ )
2:   // Preprocessing: Assign fixed priorities based on periods
3:   for each task  $\tau_i$  in  $Tasks$  do
4:      $priority_i \leftarrow 1/T_i$   $\triangleright$  Shorter period = higher priority
5:   end for
6:   Sort  $Tasks$  by decreasing priority
7:   // Online scheduling
8:   Initialize empty schedule
9:    $time \leftarrow 0$ 
10:  while scheduling continues do
11:    Update ready queue with newly released jobs at  $time$ 
12:    Remove completed jobs from ready queue
13:    if ready queue is not empty then
14:      Find highest priority job  $j^*$  in ready queue
15:      Execute job  $j^*$  for one time unit or until next event
16:    else
17:      Advance  $time$  to next job release
18:    end if
19:    Update  $time$  accordingly
20:  end while
21:  return schedule
22: end procedure

```

Remark 119. *RMS has several practical advantages:*

- *Static priorities simplify implementation in real-time operating systems*
- *No runtime overhead for priority recalculation*
- *Predictable behavior under overload (higher frequency tasks are prioritized)*
- *Works well with fixed-priority hardware schedulers*

Theorem 129 (Optimality and Utilization Bound of RMS). For fixed-priority scheduling of periodic tasks on a single processor:

- RMS is optimal among all fixed-priority scheduling algorithms
- A sufficient schedulability test for RMS is:

$$U \leq n(2^{1/n} - 1) \tag{95}$$

where n is the number of tasks

- This bound approaches $\ln(2) \approx 0.693$ as n approaches infinity

Proof (sketch). For optimality among fixed-priority schedulers:

1. Assume some task set τ is schedulable by a fixed-priority algorithm A but not by RMS
2. Consider the highest-priority task where A and RMS differ
3. We can show that this task's response time under RMS is no worse than under A
4. This contradicts the assumption that RMS fails while A succeeds

For the utilization bound:

1. Consider the critical instant scenario (all tasks released simultaneously)
2. For each task, calculate worst-case response time using recurrence relation
3. Through analysis of this equation, derive the utilization bound
4. The bound $n(2^{1/n} - 1)$ represents a threshold below which schedulability is guaranteed

The bound of 0.693 for large n means that RMS can guarantee schedulability for periodic task sets using up to approximately 69.3% of processor capacity, regardless of the number of tasks. $\square$

Definition 125 (Online Scheduling). Online scheduling addresses problems where:

- Jobs arrive over time, with characteristics unknown until arrival
- Scheduling decisions must be made without knowledge of future jobs
- Once a decision is made, it typically cannot be revoked (irrevocability)
- Performance is evaluated using competitive analysis against an omniscient optimal offline algorithm

The competitive ratio ρ of an online algorithm A is:

$$\rho = \sup_{\sigma} \frac{A(\sigma)}{OPT(\sigma)} \quad (96)$$

where σ is any job sequence, $A(\sigma)$ is the objective value obtained by algorithm A , and $OPT(\sigma)$ is the optimal offline solution.

Lemma 112 (Bounded Delay Algorithm for $1|r_j|C_{\max}$). The Bounded Delay algorithm for the online version of $1|r_j|C_{\max}$ (minimizing makespan when jobs arrive over time):

Remark 120. The parameter α controls the algorithm's behavior:

- Smaller α makes the algorithm more eager to schedule jobs
- Larger α makes the algorithm wait longer for potentially longer jobs
- When $\alpha = 2$, the algorithm achieves its optimal competitive ratio of $\frac{3}{2}$

Theorem 130 (Competitive Ratio of Bounded Delay Algorithm). The Bounded Delay algorithm with $\alpha = 2$ achieves a competitive ratio of $\frac{3}{2}$ for the online scheduling problem $1|r_j|C_{\max}$, and this ratio is optimal.

Proof (sketch). The proof consists of two parts:

Upper bound proof: Consider any job sequence and let J_L be the last job in the optimal offline schedule. Two cases arise:

1. If the online algorithm processes J_L last, then the difference from optimal is at most the idle time introduced by the algorithm.
2. If the online algorithm processes another job J' last, then J' must have processing time $p_{J'} \geq \frac{p_{J_L}}{2}$ (otherwise, the algorithm would have chosen J_L).

By analyzing the worst-case scenarios, we can show that the makespan of the Bounded Delay algorithm is at most $\frac{3}{2}$ times the optimal makespan.

Lower bound proof: We can construct an adversarial job sequence where any deterministic online algorithm achieves a competitive ratio of at least $\frac{3}{2}$:

1. Release a job with processing time 1 at time 0
2. If the algorithm immediately schedules this job, release a job with processing time 2 at time 1
3. If the algorithm waits, release no more jobs

Algorithm 54 Bounded Delay Algorithm

```
1: procedure BOUNDEDDELAY( $\alpha$ )
2:   Initialize empty schedule
3:    $time \leftarrow 0$ 
4:    $idle \leftarrow \text{false}$ 
5:   while not all jobs completed do
6:     if new job  $j$  arrives at  $time$  then
7:       Add  $j$  to the set of available jobs
8:     end if
9:     if processor is idle AND available jobs exist then
10:      Let  $p_{\max}$  be the processing time of the longest available job
11:      Let  $p_{\text{new}}$  be the processing time of the most recently arrived job
12:      if  $idle = \text{false}$  OR  $p_{\text{new}} > \alpha \cdot p_{\max}$  then
13:        Schedule the longest available job
14:         $idle \leftarrow \text{false}$ 
15:      else
16:        Keep processor idle
17:         $idle \leftarrow \text{true}$ 
18:      end if
19:    end if
20:    Update  $time$  to next event (job arrival or completion)
21:  end while
22:  return schedule
23: end procedure
```

In either case, the online algorithm achieves a makespan of 3, while the optimal offline solution has makespan 2, giving a ratio of $\frac{3}{2}$.

Since the algorithm achieves this bound and no algorithm can do better, $\frac{3}{2}$ is the optimal competitive ratio for this problem. $\square$

Remark 121 (Extensions and Limitations). *Online scheduling algorithms have several practical implications:*

- The $\frac{3}{2}$ -competitive ratio is a worst-case guarantee; average-case performance is typically better
- For multiple machines, more complex algorithms are required, with competitive ratios depending on the number of machines
- When jobs have deadlines, competitive analysis often considers throughput (number of completed jobs) rather than makespan
- Randomized algorithms can achieve better expected competitive ratios against oblivious adversaries
- For periodic real-time tasks, EDF and RMS provide deterministic guarantees rather than competitive ratios

The gap between online and offline scheduling highlights the value of predictive models in practical scheduling systems.

Definition 126 (Algorithm Selection Framework). Choosing an appropriate scheduling algorithm requires analyzing several dimensions:

- **Problem structure:** Machine environment, constraints, objective function
- **Instance size:** Number of jobs and machines
- **Solution quality:** Optimality vs. approximation requirements
- **Computational resources:** Available time and solvers
- **Implementation complexity:** Required expertise

Lemma 113 (Problem Structure-Based Selection). The structural characteristics of a scheduling problem dictate algorithm choice:

- **Small n , highly constrained:** CP, DP, Branch-and-Bound

- **Identical machines**, $C_{\max}$: LPT (simple), PTAS (for small m)
- **Unrelated machines**: Shmoys-Tardos LP rounding
- **Precedence constraints**: List scheduling with topological ordering
- **Single machine, tardiness**: Lawler's algorithm (small), Local search (larger)
- **Job shop**: CP with global constraints, Shifting bottleneck
- **Online/real-time**: EDF (utilization ≤ 1), RMS (fixed priorities)

Remark 122. *When problems combine multiple characteristics, decomposition methods can separate decisions, applying different algorithms to each sub-problem.*

Theorem 131 (Quality-Speed Trade-off Hierarchy). Scheduling algorithms form a hierarchy based on solution quality vs. speed:

- **Exact methods** (DP, B&B, MIP)
 - Guarantee: Optimal
 - Speed: Slowest
 - Use when: Optimality is required and instances are small
- **Approximation schemes** (PTAS/FPTAS)
 - Guarantee: $(1 + \epsilon)$ -optimal
 - Speed: Slow
 - Use when: Near-optimality with guarantees is needed
- **Constant-factor approximations**
 - Guarantee: ρ -approximation
 - Speed: Medium
 - Use when: Bounded quality is sufficient
- **List scheduling heuristics**
 - Guarantee: Bounded worst-case for some problems
 - Speed: Fast
 - Use when: Quick solutions are needed
- **Meta-heuristics**
 - Guarantee: No theoretical guarantees
 - Speed: Medium
 - Use when: High-quality solutions for large instances
- **Dispatching rules**
 - Guarantee: None
 - Speed: Fastest
 - Use when: Real-time decisions are required

Proof. This hierarchy follows from complexity theory and empirical evidence. Exact methods guarantee optimality but require exponential time. Approximation schemes provide worst-case bounds at the cost of higher runtime. Heuristics sacrifice guarantees for speed. $\square$

Lemma 114 (Instance Size-Based Selection). The size of the scheduling instance influences algorithm selection:

Remark 123. *These thresholds are approximate and depend on available computational resources, required solution time, implementation efficiency, and problem-specific structure.*

Theorem 132 (Toolkit-Based Selection Strategy). Available implementation resources influence algorithm selection:

- **With MIP solver** (e.g., Gurobi, CPLEX)

Algorithm 55 Algorithm Selection Based on Instance Size

```
1: procedure SELECTALGORITHM(Problem, n, m)           ▷ n = number of jobs, m = number of machines
2:   if  $n \leq 50$  AND highly constrained then
3:     return Exact methods (CP, DP, MIP)
4:   else if  $n \leq 200$  AND m is small AND identical machines then
5:     return PTAS or approximation algorithm
6:   else if  $n \leq 1000$  AND moderately constrained then
7:     return Hybrid CP-MIP or meta-heuristic
8:   else if  $n > 1000$  OR real-time requirements then
9:     return List scheduling or dispatching rules
10:  else
11:    return Decomposition with problem-specific heuristics
12:  end if
13: end procedure
```

- Formulate as MIP with problem-specific cuts
- Consider decomposition (Benders, column generation) for large instances
- Best for: Problems with linear constraints and objectives

- **With CP solver** (e.g., OR-Tools, CP Optimizer)

- Model with global constraints
- Consider hybrid CP-MIP for optimality proof
- Best for: Highly constrained problems with logical relationships

- **With programming expertise**

- Implement metaheuristics (GA, SA)
- Customize neighborhood structures for local search
- Best for: Complex problems requiring domain-specific knowledge

- **With limited resources**

- Simple list scheduling with priority rules
- Randomized greedy algorithms with restarts
- Best for: When quick solutions are needed with minimal development

Proof. Based on computational studies across scheduling domains: commercial solvers often outperform custom implementations for small/medium instances; hybrid approaches leveraging multiple technologies typically outperform single-method approaches; and problem-specific metaheuristics can achieve near-optimal solutions when properly designed. $\square$

Remark 124 (Decision Framework for Practitioners). *A practical decision framework for scheduling algorithm selection:*

1. **First, characterize your problem:**

- Machine environment (single, parallel, unrelated)
- Objective function (makespan, tardiness)
- Constraints (precedence, release dates)
- Instance size (jobs, machines)
- Time sensitivity (batch vs. real-time)

2. **Then, evaluate solution requirements:**

- Is optimality required or is approximation sufficient?
- Are theoretical guarantees important?

- *What is the acceptable computation time?*
- *Will the solution need explanation?*

3. **Finally, assess available resources:**

- *Available solvers*
- *Programming expertise*
- *Computational resources*
- *Development time constraints*

Based on these assessments, the appropriate algorithm selection typically becomes clear. When in doubt, prototype multiple approaches on representative instances.

Lemma 115 (Example Selection Scenarios). To illustrate the selection process, consider these common scheduling scenarios:

1. **Production scheduling (50 jobs, 5 identical machines):**

- Makespan objective: LPT is quick and effective, or PTAS for near-optimality
- Tardiness objective: MIP for optimality, or shifting bottleneck for larger instances

2. **Employee shift scheduling with complex constraints:**

- CP excels due to global constraints for expressing rules
- Column generation works when shifts can be pre-enumerated
- Hybrid CP-MIP leverages strengths of both approaches

3. **Online appointment scheduling:**

- Competitive online algorithms with bounded worst-case performance
- Stochastic optimization if service time distributions are known
- Simple heuristics with simulation validation

4. **Real-time embedded system tasks:**

- EDF for preemptive scheduling with full utilization
- RMS for static priorities with robustness under overload
- Specialized algorithms for mixed-criticality systems

Remark 125 (Empirical Performance Considerations). *When selecting algorithms, consider these empirical observations:*

- *Simple heuristics like LPT often achieve within 5-10% of optimal for makespan minimization*
- *Local search with problem-specific neighborhoods can find solutions within 1-2% of optimal*
- *Time-indexed MIP formulations typically outperform sequence-based formulations*
- *Genetic algorithms with specialized crossover work well for job shop scheduling*
- *For real-world applications, robustness to uncertainty often matters more than worst-case guarantees*
- *Algorithm portfolios with different strengths can provide both quality and efficiency*

Scheduling Problem

Suppose we have students $s_1, s_2, \dots, s_m$, each has to write a set $E_1, E_2, \dots, E_m$ of exams.

What is the minimum number of time slots to schedule the exams so that no student has to write two exams at the same time?

Example

Suppose we have students

$$s_1, s_2, s_3, s_4$$

and

s_1 has to write exams 1 and 4

s_2 has to write exams 1, 2 and 5

s_3 has 2, 3, 4

s_4 has 3, 5

Form a graph G where $V(G)$ is the set of exams and $\{i, j\}$ is an edge if some student has to write both exams i and j .

In the example,

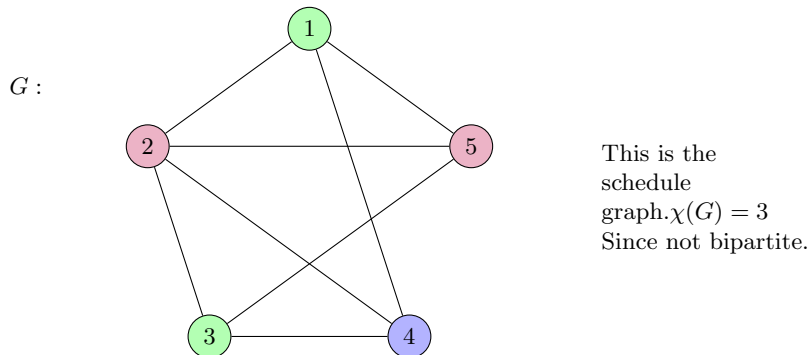


Figure 148: The minimum number of time slots equals the chromatic number of the schedule graph.

$$3 \text{ time slots } \begin{cases} \text{Exam 2 at 9am} \\ \text{Exams 1 and 3 at 10am} \\ \text{Exams 4 and 5 at 11am} \end{cases}$$

8.10 Vertex and Edge-Coloring Exercises

8.10.1 Easy Questions

1. Define the chromatic number $\chi(G)$ of a graph G . What is the chromatic number of a complete graph K_n ?
2. State König's theorem for bipartite graphs. What does it say about the relationship between vertex cover and maximum matching?
3. What is the edge chromatic number (chromatic index) $\chi'(G)$ of a graph G ? How does it relate to the maximum degree $\Delta(G)$?
4. Find the chromatic number of the cycle C_5 . Explain your reasoning.
5. Prove that any bipartite graph has chromatic number at most 2.
6. What is the chromatic number of the complete bipartite graph $K_{m,n}$ where $m, n \geq 1$?
7. State Vizing's theorem. What are the two possible values for the chromatic index of any simple graph?
8. Find the chromatic index of the complete graph K_4 . Show a proper edge coloring.
9. Explain why the chromatic number of any graph G satisfies $\chi(G) \geq \omega(G)$, where $\omega(G)$ is the clique number.
10. What is a degenerate graph? Give an example of a 2-degenerate graph.

8.10.2 Medium Questions

11. Prove that if G is a graph with maximum degree Δ , then $\chi(G) \leq \Delta + 1$.
12. Show that the chromatic number of the Petersen graph is 3 by providing an explicit 3-coloring.
13. Prove König's theorem: In a bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.
14. Let G be a graph with chromatic number k . Prove that G contains a subgraph with minimum degree at least $k - 1$.
15. Find the chromatic index of the complete bipartite graph $K_{3,4}$. Is this graph Class 1 or Class 2 according to Vizing's classification?
16. Prove that if G is a bipartite graph with maximum degree Δ , then $\chi'(G) = \Delta$.
17. Show that the chromatic number of the wheel graph W_n (a cycle C_n with an additional central vertex connected to all vertices of the cycle) depends on whether n is odd or even.
18. Prove that any planar graph has chromatic number at most 4 (Four Color Theorem - you may state this without proof, but explain its significance).
19. Let G be a k -degenerate graph. Prove that $\chi(G) \leq k + 1$ using the degeneracy ordering.
20. Describe the greedy coloring algorithm and analyze its performance. Under what conditions does it achieve the optimal coloring?

8.10.3 Hard Questions

21. Prove or disprove: If G is a graph where every subgraph has a vertex of degree at most k , then $\chi(G) \leq k + 1$.
22. Let G be a graph with n vertices and chromatic number $\chi(G) = k$. Prove that G has at least $\binom{k}{2}$ edges.
23. Prove Vizing's theorem: For any simple graph G with maximum degree Δ , either $\chi'(G) = \Delta$ or $\chi'(G) = \Delta + 1$.
24. Show that determining whether a graph is 3-colorable is NP-complete, while determining whether a graph is 2-colorable is polynomial-time solvable.
25. Let G be a graph with girth g (length of shortest cycle) and chromatic number k . Prove that G has at least $1 + \Delta + \Delta(\Delta - 1) + \dots + \Delta(\Delta - 1)^{\lfloor g/2 \rfloor - 1}$ vertices.
26. Prove Shannon's theorem: For any graph G , $\chi'(G) \leq \lfloor \frac{3\Delta}{2} \rfloor$ where Δ is the maximum degree.
27. Let G be a graph such that every subgraph of G has degeneracy at most k . Prove that $\chi(G) \leq k + 1$ and that this bound is tight.
28. Design a polynomial-time algorithm to find the chromatic index of a bipartite graph. Analyze its time complexity.
29. Prove that if G is a graph with $\chi(G) = k$ and $\alpha(G) = \ell$ (independence number), then $n \geq k \cdot \ell$.
30. Investigate the relationship between the chromatic number and the fractional chromatic number. Show that they can differ by an arbitrarily large factor.

8.10.4 Diagram Questions

31. Draw the complete graph K_5 and provide an optimal vertex coloring. Explain why this is the minimum number of colors needed.
32. Illustrate König's theorem by drawing a bipartite graph, finding a maximum matching, and showing the corresponding minimum vertex cover.
33. Draw the Petersen graph and show that it requires exactly 3 colors. Mark the color classes clearly.
34. Create a graph that is Class 2 according to Vizing's classification (i.e., $\chi'(G) = \Delta(G) + 1$). Show why $\Delta(G)$ colors are insufficient.
35. Draw the wheel graph W_5 and W_6 . Show optimal colorings for both and explain why their chromatic numbers differ.
36. Illustrate the greedy coloring algorithm on a specific graph where it does NOT produce an optimal coloring. Show both the greedy result and the optimal coloring.
37. Draw a 3-degenerate graph with 8 vertices. Show the degeneracy ordering and use it to find a 4-coloring.
38. Create a bipartite graph and show its edge coloring using exactly Δ colors, illustrating the result that bipartite graphs are always Class 1.
39. Draw two different graphs, both with maximum degree 3, where one is Class 1 and the other is Class 2. Show the edge colorings for both.
40. Illustrate Shannon's bound by drawing a graph where the chromatic index is close to $\frac{3\Delta}{2}$. Show the edge coloring that achieves this bound.

9 Planar Graphs

9.1 Definition and Basic Properties

Definition 127. A graph is *planar* if it can be drawn in the plane without crossing edges.

Example 40. The complete graph K_4 is planar. Although it may initially appear to have crossing edges when drawn as a square with diagonals, it can be redrawn without crossings.

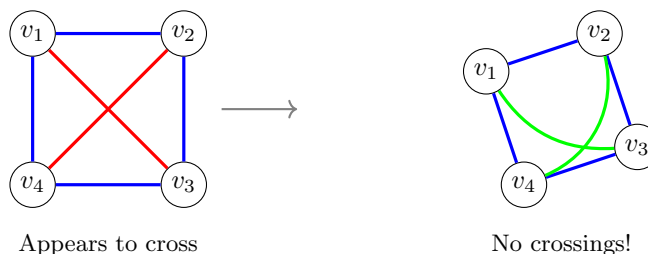


Figure 149: K_4 drawn in non-planar and planar representations

K_4 is planar.

9.2 Faces of Planar Graphs

Definition 128. If G is a planar graph, then the *faces* of G are the connected regions of the plane that are disjoint from the drawing of G .

Definition 129. The *degree* of a face f , denoted $\deg(f)$, is the length of the shortest closed walk containing all points on the boundary of f .

Example 41. Consider a triangle with an additional edge:

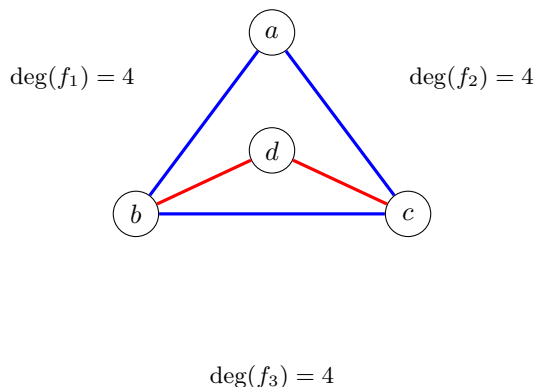


Figure 150: Triangle with additional edge showing three faces

Each face has degree 4. Note that edges bd and dc are each counted once for faces f_1 and f_2 , but edges ab , ac , and bc are shared with the outer face.

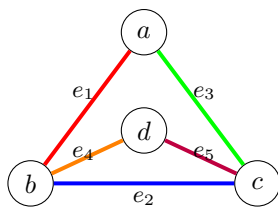
9.3 Fundamental Theorems

Lemma 116 (Handshaking Lemma for Faces). For any planar graph G ,

$$\sum_{f \in F(G)} \deg(f) = 2|E(G)|$$

where $F(G)$ is the set of faces of G .

Example 42. Verification with the triangle example above:



$$\sum \deg(f) = 4 + 4 + 4 = 12 = 2 \times 5 = 2|E|$$

Figure 151: Verification of Handshaking Lemma for Faces

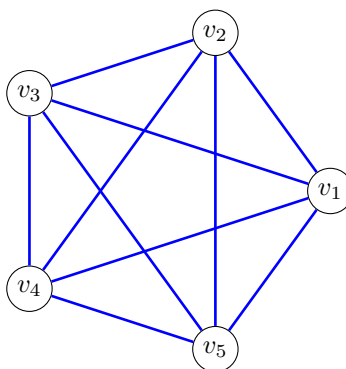
Theorem 133 (Euler's Formula). If G is a connected planar graph, then

$$|V(G)| - |E(G)| + |F(G)| = 2$$

Example 43. Verification with our triangle example: $|V| = 4$, $|E| = 5$, $|F| = 3$, so $4 - 5 + 3 = 2$ (correct)

9.4 Applications: Proving Non-Planarity

Example 44. K_5 is not planar.



K_5 cannot be drawn without crossings

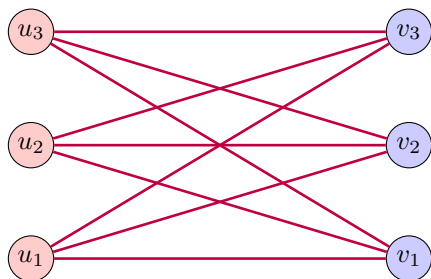
$$|V| = 5, |E| = 10, \text{ but if planar: } |E| \leq 3(|V| - 2) = 9$$

Figure 152: K_5 showing inevitable edge crossings

Example 45. $K_{3,3}$ is not planar.

Example 46. Planar vs Non-planar comparison:

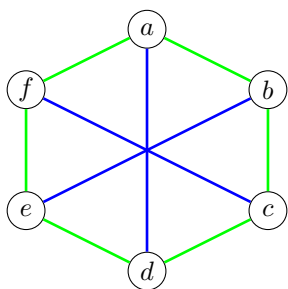
Remark 126. *The results that K_5 and $K_{3,3}$ are not planar will be on the exam.*



$K_{3,3}$ cannot be drawn without crossings

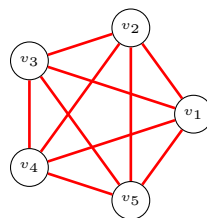
$|V| = 6$, $|E| = 9$, but if planar: $|E| \leq 2(|V| - 2) = 8$

Figure 153: $K_{3,3}$ complete bipartite graph showing crossings



Planar: $|V| = 6$, $|E| = 9$

Satisfies: $9 \leq 3(6 - 2) = 12$ (valid)



Non-planar: K_5

Violates: $10 > 3(5 - 2) = 9$ (invalid)

Figure 154: Comparison of planar and non-planar graphs

9.5 Introduction and Formal Definitions of Art Gallery Problems

Definition 130 (Simple Polygon). A **simple polygon** P is a Jordan curve in $\mathbb{R}^2$ consisting of a finite sequence of line segments $e_1, e_2, \dots, e_n$ called edges, where:

- Each edge e_i connects vertices v_i and v_{i+1} (with $v_{n+1} = v_1$)
- No two non-consecutive edges intersect
- The polygon encloses a bounded region

Definition 131 (Visibility). Two points $p, q \in P$ are **visible** to each other if the line segment $\overline{pq}$ lies entirely within P . We write $p \sim q$ if p and q are visible to each other.

Definition 132 (Guard Set). A set $G \subseteq P$ is a **guard set** for polygon P if for every point $p \in P$, there exists a guard $g \in G$ such that $p \sim g$.

Definition 133 (The Art Gallery Problem). Given a simple polygon P with n vertices, determine:

1. $g(P)$: The minimum number of guards needed to guard P
2. $g(n)$: The maximum of $g(P)$ over all simple polygons with n vertices

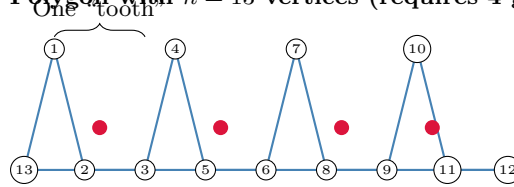
Theorem 134 (Chvátal's Art Gallery Theorem, 1975). For any simple polygon P with n vertices,

$$g(n) = \left\lfloor \frac{n}{3} \right\rfloor$$

9.6 Construction and Visualization

The comb polygon demonstrates that the bound $\lfloor n/3 \rfloor$ is tight. It resembles the triangular teeth of Toblerone chocolate.

Comb Polygon with $n = 13$ vertices (requires 4 guards)



Definition 134 (Comb Polygon). A **comb polygon** C_k with $n = 3k + 1$ vertices is constructed as:

- k triangular “teeth” protruding upward
- A horizontal base connecting all teeth
- Each tooth requires its own guard due to visibility constraints

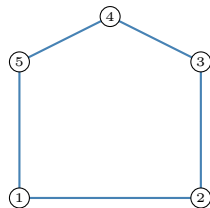
Lemma 117 (Comb Lower Bound). The comb polygon C_k with $n = 3k + 1$ vertices requires exactly $k = \lfloor n/3 \rfloor$ guards.

Proof. Each triangular tooth creates a visibility barrier. The peak of each tooth is only visible from within that tooth. Thus, each tooth requires its own guard, giving $k = \lfloor n/3 \rfloor$ guards. $\square$

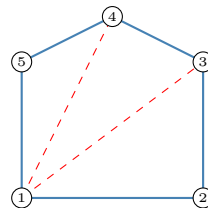
9.7 Triangulation

Definition 135 (Polygon Triangulation). A **triangulation** of a simple polygon P is a maximal set of non-crossing diagonals that partition P into triangles.

Theorem 135 (Triangulation Existence). Every simple polygon with $n \geq 3$ vertices can be triangulated into exactly $n - 2$ triangles using $n - 3$ diagonals.



Original Pentagon



After Triangulation

3 triangles
2 diagonals

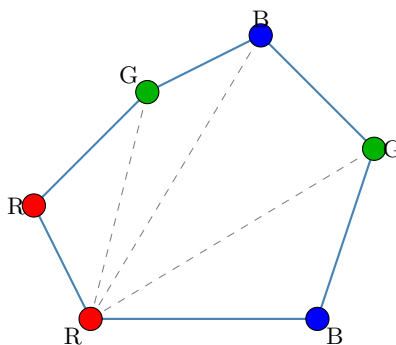
Algorithm 56 Ear Clipping Triangulation

Require: Simple polygon P with vertices $v_1, \dots, v_n$ **Ensure:** Triangulation of P

- 1: **while** $n > 3$ **do**
 - 2: Find an ear vertex v_i (convex vertex with no vertices inside triangle $v_{i-1}v_iv_{i+1}$)
 - 3: Add diagonal $v_{i-1}v_{i+1}$
 - 4: Remove vertex v_i
 - 5: $n \leftarrow n - 1$
 - 6: **end while**
 - 7: **Complexity:** $O(n^2)$ time, $O(n)$ space
-

9.8 Three-Coloring

Theorem 136 (3-Colorability of Triangulated Polygons). Every triangulated simple polygon has a proper 3-coloring of its vertices such that no triangle has vertices of the same color.



3-Coloring Example

Each triangle has exactly one vertex of each color

Proof of 3-Colorability. By induction on the number of triangles:

- **Base case:** A single triangle can be 3-colored
- **Inductive step:** Remove an ear, 3-color the remaining polygon, then color the removed vertex with the third color not used by its two neighbors

□

9.9 The Complete Guard Placement Algorithm

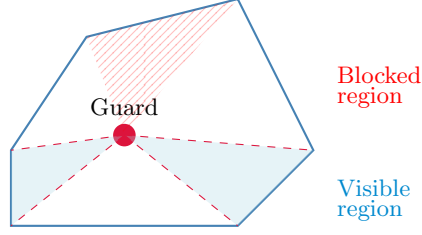
Algorithm 57 Art Gallery Guard Placement

Require: Simple polygon P with n vertices**Ensure:** Guard set G with $|G| \leq \lfloor n/3 \rfloor$

- 1: Triangulate P into $n - 2$ triangles
 - 2: 3-color the vertices: $V = R \cup B \cup G$ (Red, Blue, Green)
 - 3: $G \leftarrow \arg \min\{|R|, |B|, |G|\}$
 - 4: **return** Guard set G
-

Theorem 137 (Correctness of Algorithm). The algorithm produces a valid guard set with at most $\lfloor n/3 \rfloor$ guards.

Proof. Since we have a 3-coloring with color classes R, B, G , we have $|R| + |B| + |G| = n$. By the pigeonhole principle, $\min\{|R|, |B|, |G|\} \leq n/3$. The chosen color class guards all triangles since each triangle contains exactly one vertex of each color. □

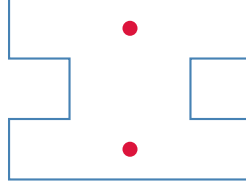


Visibility Region from a Guard

9.10 Orthogonal Polygons

Theorem 138 (Kahn-Klawe-Kleitman, 1983). For orthogonal polygons (edges parallel to coordinate axes) with n vertices:

$$g_{\text{orth}}(n) = \left\lfloor \frac{n}{4} \right\rfloor$$



Orthogonal Polygon (12 vertices, 2 guards)

9.11 Mobile Guards

Theorem 139 (O'Rourke, 1983). For mobile guards that can patrol along edges:

$$g_{\text{mobile}}(n) = \left\lfloor \frac{n}{4} \right\rfloor$$

9.12 Monotone Polygons

Definition 136 (Monotone Polygon). A polygon is **y-monotone** if any horizontal line intersects it in at most two points.

Theorem 140. For y -monotone polygons with n vertices:

$$g_{\text{monotone}}(n) = \left\lfloor \frac{n-2}{2} \right\rfloor$$

Definition 137 (Art Gallery Decision Problem). INPUT: Simple polygon P and integer k
QUESTION: Is $g(P) \leq k$?

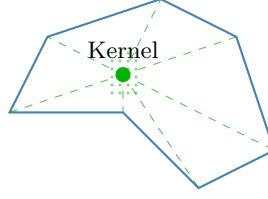
Theorem 141 (Lee and Lin, 1986). The Art Gallery Decision Problem is NP-complete, even for simple polygons.

Theorem 142 (Approximation Algorithm, Ghosh 1987). There exists an $O(\log n)$ -approximation algorithm for the Art Gallery Problem using a greedy set cover approach.

Definition 138 (Star-Shaped Polygon). A polygon P is **star-shaped** if there exists a point $p \in P$ such that p can see all of P .

Definition 139 (Kernel). The **kernel** of polygon P is:

$$\text{Kernel}(P) = \{p \in P : \forall q \in P, p \sim q\}$$



Star-Shaped Polygon with Kernel

Theorem 143. A polygon is star-shaped if and only if its kernel is non-empty. Star-shaped polygons require only 1 guard.

9.13 Fortress Problem

Guard the **exterior** of a polygon from guards placed outside.

$$g_{\text{fortress}}(n) = \left\lceil \frac{n}{2} \right\rceil$$

9.14 Prison Yard Problem

Guards inside must see both the interior and the boundary.

$$g_{\text{prison}}(n) = \left\lceil \frac{n}{3} \right\rceil$$

Theorem 144 (Negative Result for 3D). There exist polyhedra with n vertices that require $\Omega(n)$ guards, showing no constant fraction suffices in 3D.

This demonstrates that the art gallery problem becomes much harder in higher dimensions.

Polygon Type	Guard Bound	Reference
Simple polygon	$\lfloor n/3 \rfloor$	Chvátal (1975)
Orthogonal polygon	$\lfloor n/4 \rfloor$	Kahn-Klawe-Kleitman (1983)
Mobile guards	$\lfloor n/4 \rfloor$	O'Rourke (1983)
y -monotone polygon	$\lfloor (n-2)/2 \rfloor$	
Fortress problem	$\lceil n/2 \rceil$	
Star-shaped polygon	1	

Problem 5. Prove that a convex polygon with n vertices requires only 1 guard.

Problem 6. Show that the visibility graph of a convex polygon is the complete graph K_n .

Problem 7. Construct a polygon with 15 vertices that requires exactly 5 guards.

Problem 8. Prove that every quadrilateral requires at most 1 guard.

9.15 Open Problems

1. Better approximation algorithms (current best: $O(\log n)$)
2. Exact algorithms for special polygon classes
3. Dynamic art gallery problems with moving targets
4. Efficient algorithms for 3D polyhedron guarding

9.16 Planar Graphs and Art Gallery Problems

9.16.1 Easy Questions

1. Define a planar graph. What is the difference between a planar graph and a plane graph?
2. State Euler's formula for connected planar graphs. What does each variable represent?
3. What is the degree of a face in a planar graph? How is it calculated?
4. Prove the handshaking lemma for faces: For any planar graph G , $\sum_{f \in F(G)} \deg(f) = 2|E(G)|$.
5. Find the number of faces in a connected planar graph with 8 vertices and 12 edges using Euler's formula.
6. Show that the complete graph K_4 is planar by drawing it without edge crossings.
7. Define the chromatic number $\chi(G)$ of a graph G . What is $\chi(K_4)$?
8. What is a guard set in the context of the art gallery problem? Define visibility between two points.
9. State Chvátal's Art Gallery Theorem. What does it tell us about the number of guards needed?
10. What is a star-shaped polygon? How many guards does it require and why?

9.16.2 Medium Questions

11. Prove that K_5 is not planar using Euler's formula and the fact that every face in a simple planar graph has degree at least 3.
12. Show that the complete bipartite graph $K_{3,3}$ is not planar by counting faces and applying appropriate bounds.
13. For a simple planar graph with $n \geq 3$ vertices, prove that $|E| \leq 3|V| - 6$.
14. Construct a comb polygon with 13 vertices and show that it requires exactly 4 guards, demonstrating that the bound $\lfloor n/3 \rfloor$ is tight.
15. Prove that every simple polygon with n vertices can be triangulated into exactly $n - 2$ triangles using $n - 3$ diagonals.
16. Show that every triangulated simple polygon has a proper 3-coloring of its vertices such that no triangle has vertices of the same color.
17. Describe the algorithm for solving the art gallery problem using triangulation and 3-coloring. Prove its correctness.
18. For triangle-free planar graphs, prove the stronger bound $|E| \leq 2|V| - 4$.
19. What is the chromatic number of a planar graph? State the Four Color Theorem and explain its significance for art gallery problems.
20. Define the kernel of a polygon. Prove that a polygon is star-shaped if and only if its kernel is non-empty.

9.16.3 Hard Questions

21. Prove Kuratowski's theorem (statement only): A graph is planar if and only if it contains no subdivision of K_5 or $K_{3,3}$ as a subgraph.
22. Let G be a planar graph with n vertices where every face has degree at least k . Derive a bound on the number of edges in terms of n and k .
23. Prove that the art gallery decision problem (determining if k guards suffice for a given polygon) is NP-complete.
24. For orthogonal polygons (edges parallel to coordinate axes), state and prove the bound $g_{\text{orth}}(n) = \lfloor n/4 \rfloor$.
25. Show that for y -monotone polygons with n vertices, $g_{\text{monotone}}(n) = \lfloor (n - 2)/2 \rfloor$.
26. Prove that in 3D, there exist polyhedra with n vertices that require $\Omega(n)$ guards, showing that no constant fraction suffices.
27. Design and analyze a greedy approximation algorithm for the art gallery problem. What approximation ratio can you achieve?
28. For the fortress problem (guarding the exterior of a polygon), prove that $g_{\text{fortress}}(n) = \lceil n/2 \rceil$.
29. Investigate mobile guards that can patrol along edges. Prove that $g_{\text{mobile}}(n) = \lfloor n/4 \rfloor$.
30. Analyze the complexity of computing the exact chromatic number of planar graphs and its relationship to guard placement problems.

9.16.4 Construction and Diagram Questions

31. Draw two different planar embeddings of K_4 . Label all vertices, edges, and faces, and verify Euler's formula for each.
32. Illustrate the comb polygon construction. Draw a comb with 16 vertices, place the optimal number of guards, and show why fewer guards are insufficient.
33. Create a planar graph with 7 vertices and exactly 12 edges. Verify that this achieves the maximum possible number of edges for a simple planar graph with 7 vertices.
34. Draw the wheel graph W_6 (cycle C_6 with central vertex). Determine if it's planar and find its chromatic number.
35. Construct a simple polygon and demonstrate the triangulation process step by step. Show the 3-coloring and resulting guard placement.
36. Illustrate why K_5 cannot be drawn without crossings by attempting different embeddings and showing where crossings are unavoidable.
37. Draw an orthogonal polygon with 12 vertices that requires exactly 3 guards. Show the guard placement and prove optimality.
38. Create a y -monotone polygon with 10 vertices. Find the minimum guard set and verify it matches the theoretical bound.
39. Design a star-shaped polygon and clearly indicate its kernel region. Show that one guard placed in the kernel can see the entire polygon.
40. Illustrate the fortress problem by drawing a polygon and showing how exterior guards must be placed to see the entire boundary from outside.

9.16.5 Proof-Based Questions

41. Prove that if G is a connected planar graph with $n \geq 3$ vertices and m edges, then $m \leq 3n - 6$.
42. Show that every planar graph contains a vertex of degree at most 5. Use this to prove that every planar graph is 6-colorable.
43. Prove that the complete graph K_n is planar if and only if $n \leq 4$.
44. Demonstrate that the complete bipartite graph $K_{m,n}$ is planar if and only if $\min(m, n) \leq 2$.
45. For a simple polygon P , prove that any triangulation results in the same number of triangles and diagonals, regardless of the triangulation method chosen.
46. Show that the greedy algorithm for 3-coloring a triangulated polygon always produces a valid coloring and that one color class has size at most $\lfloor n/3 \rfloor$.
47. Prove that every quadrilateral (4-sided polygon) can be guarded by exactly one guard, regardless of its shape.
48. Show that for convex polygons with n vertices, exactly one guard suffices and can be placed anywhere inside the polygon.
49. Prove that the visibility graph of any set of points in a convex polygon is the complete graph.
50. Demonstrate that determining the minimum number of guards for a specific polygon is equivalent to finding the minimum dominating set in its visibility graph.

9.16.6 Advanced Applications

51. Design an algorithm to determine if a given graph contains a K_5 or $K_{3,3}$ subdivision. Analyze its time complexity.
52. Investigate the relationship between the thickness of a graph (minimum number of planar graphs needed to cover all edges) and its planarity.
53. Explore the connection between graph coloring and art gallery problems. How does the chromatic number of the visibility graph relate to guard placement?
54. Study the dual graph of a planar graph. How do the properties of the original graph relate to its dual?
55. Analyze the art gallery problem for polygons with holes. How does the presence of holes affect the guard requirement bounds?

56. Examine rectilinear art gallery problems where guards and visibility are restricted to horizontal and vertical lines.
57. Investigate approximation algorithms for non-planar graphs. What bounds can be achieved for graphs embedded on surfaces of higher genus?
58. Study dynamic art gallery problems where the polygon can change over time. Design algorithms to maintain guard sets efficiently.
59. Explore probabilistic methods in graph coloring and their applications to randomized guard placement strategies.
60. Analyze the parameterized complexity of art gallery problems. What parameters make the problem tractable?

10 Network Flows and the Max-Flow Min-Cut Theorem

This chapter develops, from first principles, every object and technique needed to state and prove the Max-Flow Min-Cut Theorem of Ford and Fulkerson. Up to this point the book has treated (mostly) undirected graphs; flows live most naturally on directed graphs with non-negative edge capacities, so we begin by introducing the setting carefully.

10.1 Directed Graphs and Capacities

Definition 140 (Directed graph). A **directed graph** (or *digraph*) is a pair $D = (V, E)$ where V is a finite set of *vertices* and $E \subseteq V \times V$ is a set of ordered pairs called *arcs* (or directed edges). If $e = (u, v) \in E$, we call u the *tail* and v the *head* of e , and we write $u \rightarrow v$.

Definition 141 (In- and out-neighborhoods). For $v \in V$,

$$N^+(v) = \{w \in V : (v, w) \in E\}, \quad N^-(v) = \{u \in V : (u, v) \in E\}.$$

The *out-degree* and *in-degree* of v are $d^+(v) = |N^+(v)|$ and $d^-(v) = |N^-(v)|$.

Definition 142 (Flow network). A **flow network** is a tuple $N = (V, E, s, t, c)$ where (V, E) is a finite directed graph, $s, t \in V$ are two distinguished vertices called the *source* and *sink* (with $s \neq t$), and $c : E \rightarrow \mathbb{R}_{\geq 0}$ is a *capacity* function. We extend c to all pairs by setting $c(u, v) = 0$ whenever $(u, v) \notin E$.

Remark 127. *It is convenient to assume that N contains no pair of antiparallel arcs (i.e. both (u, v) and (v, u)). Any network can be converted into one with this property by subdividing one of the two antiparallel arcs with a new intermediate vertex whose incoming and outgoing arcs keep the original capacity. All theorems of this chapter transfer unchanged.*

10.2 Flows and the Value of a Flow

Definition 143 (Flow). A **flow** in N is a function $f : V \times V \rightarrow \mathbb{R}$ satisfying the following three properties:

1. *Capacity constraint*: $0 \leq f(u, v) \leq c(u, v)$ for all (u, v) .
2. *Skew symmetry (optional bookkeeping)*: $f(u, v) = -f(v, u)$. (Many authors omit this and use only non-negative flows; both formulations are equivalent.)
3. *Conservation*: for every vertex $v \in V \setminus \{s, t\}$,

$$\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w).$$

That is, total flow into v equals total flow out of v .

Definition 144 (Value of a flow). The **value** of a flow f is the net flow leaving the source,

$$|f| = \sum_{w \in V} f(s, w) - \sum_{u \in V} f(u, s).$$

Remark 128. *By conservation, $|f|$ equals the net flow into the sink:*

$$|f| = \sum_{u \in V} f(u, t) - \sum_{w \in V} f(t, w).$$

We prove this rigorously in Lemma 118.

The **maximum-flow problem** asks: given a network N , find a flow f of largest possible value $|f|$.

10.3 Cuts and the Capacity of a Cut

Definition 145 (*s-t cut*). An *s-t cut* of N is a partition (S, T) of V with $s \in S$ and $t \in T$. The **capacity** of the cut is

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v),$$

i.e. the total capacity of arcs that go from the S -side to the T -side. (Arcs going from T to S are *not* counted.)

Definition 146 (Flow across a cut). For a flow f and an *s-t cut* (S, T) , the **net flow** across the cut is

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u).$$

The **minimum-cut problem** asks: find an *s-t cut* (S, T) of smallest possible capacity $c(S, T)$.

10.4 Flow Value Equals Net Flow Across Any Cut

Lemma 118 (Flow across a cut). For any flow f and any *s-t cut* (S, T) ,

$$|f| = f(S, T).$$

Proof. Start from the definition of $|f|$ and add 0's using conservation. For every $v \in S \setminus \{s\}$ we have $\sum_w f(v, w) - \sum_u f(u, v) = 0$, so

$$|f| = \sum_w f(s, w) - \sum_u f(u, s) = \sum_{v \in S} \left(\sum_{w \in V} f(v, w) - \sum_{u \in V} f(u, v) \right).$$

Expand the inner sums by splitting $V = S \cup T$:

$$= \sum_{v \in S} \sum_{w \in S} f(v, w) + \sum_{v \in S} \sum_{w \in T} f(v, w) - \sum_{v \in S} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in T} f(u, v).$$

The first and third double sums are equal (rename $(v, w) \leftrightarrow (u, v)$ inside $S \times S$) and cancel. What remains is exactly $f(S, T)$. $\square$

Corollary 43 (Weak duality). For any flow f and any *s-t cut* (S, T) ,

$$|f| \leq c(S, T).$$

In particular, $\max |f| \leq \min c(S, T)$.

Proof. By Lemma 118, $|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u)$. The first sum is at most $\sum_{u \in S, v \in T} c(u, v) = c(S, T)$ by the capacity constraint, and the second sum is non-negative, so $|f| \leq c(S, T)$. $\square$

Corollary 43 is already one half of the Max-Flow Min-Cut Theorem: *any* flow value is at most *any* cut capacity. The harder half is to show equality is attained. For that we need the key construction of residual graphs.

10.5 Residual Graphs and Augmenting Paths

The idea: if we have a current flow f and want to push more flow along an arc (u, v) , we can either (a) use *spare* capacity $c(u, v) - f(u, v) > 0$, or (b) *cancel* flow that was previously sent from v to u . Both options are encoded in a new graph.

Definition 147 (Residual capacity). Given a flow f in N , the **residual capacity** is

$$c_f(u, v) = (c(u, v) - f(u, v)) + f(v, u),$$

where we treat c and f as 0 on non-existent arcs. The **residual graph** is

$$N_f = (V, E_f), \quad E_f = \{(u, v) : c_f(u, v) > 0\}.$$

So the residual graph may contain arcs not present in E (these are *backward* arcs, corresponding to canceling existing flow) as well as arcs from E with leftover capacity.

Definition 148 (Augmenting path). An **augmenting path** (for the flow f) is a directed s - t path in the residual graph N_f . Its **bottleneck** $\Delta > 0$ is the minimum residual capacity of its arcs:

$$\Delta = \min_{(u,v) \in P} c_f(u,v).$$

Lemma 119 (Pushing along an augmenting path). Let P be an augmenting path in N_f with bottleneck Δ . Define a new function f' by

$$f'(u,v) = \begin{cases} f(u,v) + \Delta & \text{if } (u,v) \in P \text{ and } (u,v) \in E, \\ f(u,v) - \Delta & \text{if } (v,u) \in P \text{ and } (u,v) \in E, \\ f(u,v) & \text{otherwise.} \end{cases}$$

Then f' is a flow in N with $|f'| = |f| + \Delta$.

Proof. Capacity. If we pushed forward on $(u,v) \in E$, then $f'(u,v) = f(u,v) + \Delta \leq f(u,v) + (c(u,v) - f(u,v)) = c(u,v)$ because $\Delta \leq c_f(u,v) \leq c(u,v) - f(u,v)$ is ensured by the definition of c_f when the path uses the “forward” part. If we pushed backward, $f'(u,v) = f(u,v) - \Delta \geq 0$ for the same reason. In either case capacity is preserved.

Conservation. At every internal vertex v on P , exactly one arc of P enters v and exactly one leaves. In each of the four combinations (forward/forward, forward/backward, etc.) the net change at v is 0: the contribution added on one side is matched by an identical contribution on the other side. For vertices not on P , nothing changes.

Value. At the source s , exactly one arc of P is incident, and it is directed away from s in N_f . If that arc is a forward arc $(s,w) \in E$, then $|f'| = |f| + \Delta$. If it is a backward arc, i.e. $(w,s) \in E$ with flow being canceled, then $\sum_u f'(u,s) = \sum_u f(u,s) - \Delta$, which again yields $|f'| = |f| + \Delta$. $\square$

10.6 The Max-Flow Min-Cut Theorem

Theorem 145 (Ford–Fulkerson, 1956). Let $N = (V, E, s, t, c)$ be a flow network with a flow f . The following are equivalent:

1. f is a maximum flow in N .
2. The residual graph N_f contains no augmenting path (i.e. no directed $s \rightarrow t$ path).
3. There exists an s - t cut (S, T) with $|f| = c(S, T)$.

In particular, the value of a maximum flow equals the capacity of a minimum s - t cut:

$$\max_f |f| = \min_{(S,T)} c(S, T).$$

Proof. (1) $\Rightarrow$ (2). If N_f contained an augmenting path with bottleneck $\Delta > 0$, Lemma 119 produces a flow of value $|f| + \Delta > |f|$, contradicting maximality.

(2) $\Rightarrow$ (3). Suppose no augmenting path exists. Let

$$S = \{v \in V : \text{there is a directed path from } s \text{ to } v \text{ in } N_f\}, \quad T = V \setminus S.$$

Then $s \in S$ by reflexivity; $t \in T$ because no s - t path exists in N_f . So (S, T) is a valid s - t cut. For every $u \in S$ and $v \in T$, the arc (u, v) is *not* in E_f (otherwise v would be reachable from s), so $c_f(u, v) = 0$, which means

$$(c(u, v) - f(u, v)) + f(v, u) = 0.$$

Since both summands are non-negative, both must be zero: $f(u, v) = c(u, v)$ and $f(v, u) = 0$. Summing over $u \in S$, $v \in T$ and using Lemma 118,

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in S, v \in T} f(v, u) = \sum_{u \in S, v \in T} c(u, v) - 0 = c(S, T).$$

(3) $\Rightarrow$ (1). By weak duality (Corollary 43), $|f| \leq c(S, T)$ for every cut, so any flow whose value already equals *some* cut capacity is already as large as possible. $\square$

Algorithm 58 Ford–Fulkerson

```
1:  $f(u, v) \leftarrow 0$  for every arc  $(u, v)$ .
2: while there exists a directed  $s$ - $t$  path  $P$  in  $N_f$  do
3:    $\Delta \leftarrow \min_{(u,v) \in P} c_f(u, v)$ 
4:   for all  $(u, v) \in P$  do
5:     if  $(u, v) \in E$  then  $f(u, v) \leftarrow f(u, v) + \Delta$ 
6:     else  $f(v, u) \leftarrow f(v, u) - \Delta$ 
7:     end if
8:   end for
9: end while
10: return  $f$ 
```

10.7 The Ford–Fulkerson Algorithm

Theorem 145 suggests an algorithm: starting from the zero flow, repeatedly find an augmenting path and push flow along it, until no augmenting path remains. When the algorithm terminates, the set S of vertices reachable from s in the final residual graph gives a minimum cut.

Theorem 146 (Integrality). If all capacities are integers, then at every iteration the flow values remain integers, and the algorithm terminates in at most $|f^*|$ iterations, where f^* is a maximum flow. Consequently, some maximum flow is integer-valued.

Proof. By induction: initially the flow is $0 \in \mathbb{Z}$. If f is integer-valued then every residual capacity is an integer, so the bottleneck Δ is a positive integer, so the new flow f' is integer-valued. Each iteration increases $|f|$ by at least 1, and $|f|$ is bounded above by the capacity of any cut, hence finite. $\square$

Remark 129 (Termination with real capacities). *For irrational capacities the naive algorithm may fail to terminate (Ford and Fulkerson gave an explicit example). The Edmonds–Karp refinement—always choose a shortest augmenting path (fewest arcs) via BFS—terminates in $O(|V| \cdot |E|^2)$ time regardless of capacities; Dinic’s algorithm brings this down to $O(|V|^2|E|)$. We do not prove these bounds here, but they rely on the same residual graph machinery introduced above.*

10.8 Applications

The Max-Flow Min-Cut Theorem is a workhorse. We derive two classical consequences.

10.8.1 Menger’s Theorem

Theorem 147 (Menger, edge version). Let G be a finite directed graph and s, t two distinct vertices. The maximum number of pairwise edge-disjoint directed s - t paths equals the minimum number of arcs whose removal disconnects t from s .

Sketch. Turn G into a flow network by giving every arc capacity 1. By Theorem 146 there is an integer maximum flow f^* ; since capacities are 1, flow on each arc is either 0 or 1. The arcs carrying flow decompose into $|f^*|$ edge-disjoint s - t paths (remove them greedily, using conservation at every internal vertex). Conversely, a minimum cut is a set of arcs of total capacity $|f^*|$ —i.e. a set of $|f^*|$ arcs whose removal puts t on a different side of the cut than s , disconnecting them. Equality is Theorem 145. $\square$

A vertex version of Menger’s theorem follows by the standard *vertex splitting* reduction: replace each internal vertex v by two copies $v^{\text{in}}, v^{\text{out}}$ joined by an arc of capacity 1, and replicate incoming/outgoing arcs accordingly.

10.8.2 König’s Theorem Revisited

Theorem 148 (König). In any bipartite graph $G = (A \cup B, E)$, the size of a maximum matching equals the size of a minimum vertex cover.

Proof. Build a network: add source s with arcs $s \rightarrow a$ (capacity 1) for each $a \in A$, add sink t with arcs $b \rightarrow t$ (capacity 1) for each $b \in B$, and for every edge $\{a, b\} \in E$ add an arc $a \rightarrow b$ of capacity ∞ . By Theorem 146 there is an integer max flow f^* , and the arcs $a \rightarrow b$ with $f^*(a, b) = 1$ form a matching of size $|f^*|$.

Let (S, T) be a minimum cut. Since arcs $a \rightarrow b$ have capacity ∞ , no such arc may cross from S to T in a finite cut, so whenever $a \in A \cap S$ and $\{a, b\} \in E$ we have $b \in B \cap S$. Therefore

$$C = (A \cap T) \cup (B \cap S)$$

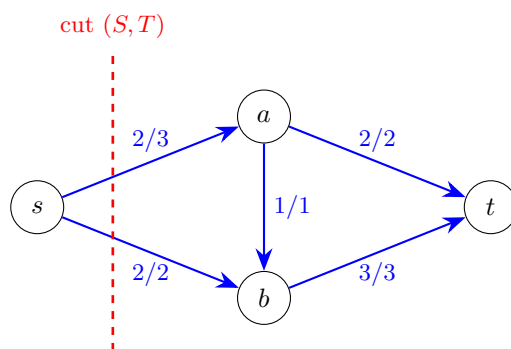
is a vertex cover. Its size is $|A \cap T| + |B \cap S| = c(S, T) = |f^*|$, matching the size of the maximum matching. Weak duality (a cover has size at least the size of any matching because each matching edge needs a cover vertex) gives the other inequality. $\square$

10.9 Worked Example

Example 47. Consider the network with vertices $\{s, a, b, t\}$ and arcs

$$s \rightarrow a (3), \quad s \rightarrow b (2), \quad a \rightarrow b (1), \quad a \rightarrow t (2), \quad b \rightarrow t (3).$$

The diagram below shows the network, the final flow, and the minimum cut.



Edge labels: flow/capacity. Cut capacity = $3 + 2 = 5$.

Figure 200: A flow network of value 5 with the saturating min cut $S = \{s\}$.

One augmenting path is $s \rightarrow a \rightarrow t$ with bottleneck 2; push 2 units. Next, $s \rightarrow b \rightarrow t$ has bottleneck 2; push 2 more. Residual graph now has $s \rightarrow a (1)$, $a \rightarrow b (1)$, $b \rightarrow t (1)$, yielding the path $s \rightarrow a \rightarrow b \rightarrow t$ with bottleneck 1; push another unit. No augmenting path remains: the set of vertices reachable from s is $S = \{s\}$, giving cut $(\{s\}, \{a, b, t\})$ of capacity $3 + 2 = 5$. Max flow value is $2 + 2 + 1 = 5 = c(S, T)$, confirming Theorem 145.

10.10 Exercises

10.10.1 Easy Questions

Exercise 10.1. Prove the “other half” of Lemma 118: for any flow f , the net flow into the sink t equals $|f|$.

Exercise 10.2. Derive the vertex form of Menger’s theorem from the edge form using the vertex-splitting construction described above.

Exercise 10.3. On the network of Figure 47, list all augmenting paths the algorithm could explore in some order, and verify that the final flow value is independent of the order.

10.10.2 Medium Questions

Exercise 10.4. Let N be a flow network with integer capacities bounded by C and m arcs. Show that the Ford–Fulkerson algorithm (using any augmenting paths) runs in $O(mC|V|)$ time. Give an example where this bound is tight up to constants.

Exercise 10.5. Use the max-flow min-cut theorem to prove Hall’s marriage theorem: a bipartite graph $G = (A \cup B, E)$ has a matching saturating A iff $|N(S)| \geq |S|$ for every $S \subseteq A$.

Exercise 10.6. Given an undirected graph G with edge capacities, define the *global minimum cut* as the partition $(S, V \setminus S)$ with $\emptyset \neq S \neq V$ minimizing the total capacity of edges crossing the cut. Show how to compute it with $|V| - 1$ calls to a max-flow subroutine.

10.10.3 Hard Questions

Exercise 10.7. Construct an example with *irrational* capacities on which a particular choice of augmenting paths causes Ford–Fulkerson to run forever while the flow value converges to a number strictly smaller than the true maximum. (This is the classical Ford–Fulkerson counterexample; it motivates the Edmonds–Karp rule.)

Exercise 10.8. Prove the Edmonds–Karp bound: if augmenting paths are always chosen by BFS (shortest first), Ford–Fulkerson terminates after at most $O(|V| \cdot |E|)$ augmentations and runs in $O(|V| \cdot |E|^2)$ time, regardless of capacities.

10.10.4 Diagram Questions

Exercise 10.9. Draw a flow network on $\{s, x, y, z, t\}$ with five arcs of capacity at most 4 such that the max-flow value equals 3 and the min-cut consists of exactly two arcs. Mark the saturating cut.

Exercise 10.10. For the diamond network with vertices $\{s, u, v, t\}$ and arcs $s \rightarrow u(10)$, $s \rightarrow v(10)$, $u \rightarrow v(1)$, $u \rightarrow t(10)$, $v \rightarrow t(10)$, draw the residual graph after pushing 10 units along $s \rightarrow u \rightarrow t$ and identify the next augmenting path.

11 Advanced Topics in Graph Theory

The remainder of the book is a tour of topics commonly taught in first-year graduate courses on graph theory and combinatorics. Each subsection is self-contained modulo the definitions established in earlier chapters, but together they sketch the modern toolkit: *extremal* arguments (forbidden-subgraph problems), *Ramsey-type* unavoidability, the *probabilistic method*, *spectral* (linear-algebraic) techniques, *random graph* models, Szemerédi’s *regularity lemma*, and the *graph minor* programme of Robertson and Seymour.

11.1 Extremal Graph Theory

Extremal graph theory studies the maximum (or minimum) possible value of some graph parameter (typically edges) among graphs satisfying a given constraint (typically excluding a fixed subgraph H). The central object is the Turán number $\text{ex}(n, H)$. This subsection develops the non-bipartite theory completely (Turán, Erdős–Simonovits stability, Erdős–Stone) and then treats the much subtler bipartite theory, including the Kővári–Sós–Turán theorem, the Bondy–Simonovits even-cycle bound, the Zarankiewicz problem, supersaturation, and the Moore bound for graphs of high girth.

11.1.1 The Turán Number and the Turán Graph

Definition 149 (Extremal number). For a graph H and integer $n \geq 1$, the **extremal number** (or *Turán number*) is

$$\text{ex}(n, H) = \max\{|E(G)| : |V(G)| = n, G \text{ contains no copy of } H\}.$$

A graph attaining this maximum is called *H-extremal*. The density version is $\pi(H) = \lim_{n \rightarrow \infty} \text{ex}(n, H) / \binom{n}{2}$, called the **Turán density** of H (the limit exists by an averaging argument of Katona, Nemetz, and Simonovits).

Definition 150 (Turán graph). The **Turán graph** $T(n, r)$ is the complete r -partite graph on n vertices whose part sizes differ by at most one (a *balanced* r -partition). Its edge count is

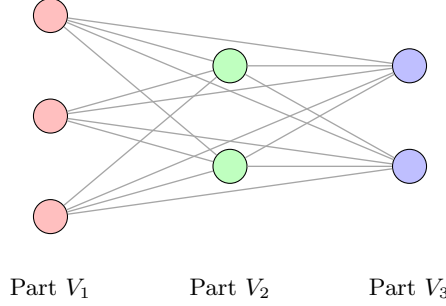
$$t(n, r) = \binom{n}{2} - \sum_{i=1}^r \binom{|V_i|}{2} = \left(1 - \frac{1}{r}\right) \frac{n^2}{2} + O(1).$$

11.1.2 Mantel’s Theorem: Three Proofs

Theorem 149 (Mantel, 1907). $\text{ex}(n, K_3) = \lfloor n^2/4 \rfloor$, attained uniquely (up to isomorphism) by the balanced complete bipartite graph $K_{\lceil n/2 \rceil, \lfloor n/2 \rfloor}$.

Proof 1: local neighborhoods and Cauchy–Schwarz. Let G be triangle-free with n vertices and m edges. For any edge $uv \in E(G)$, the sets $N(u)$ and $N(v)$ must be disjoint, as a common neighbor w would produce triangle uvw . Therefore $d(u) + d(v) \leq n$. Summing over all edges,

$$\sum_{uv \in E} (d(u) + d(v)) = \sum_{v \in V} d(v)^2 \leq n \cdot m.$$



$T(7, 3)$: balanced complete 3-partite, K_4 -free, $|E| = 16$

Figure 201: The Turán graph $T(7, 3)$ achieves $\text{ex}(7, K_4)$.

By the Cauchy–Schwarz inequality $\sum_v d(v)^2 \geq (\sum_v d(v))^2/n = (2m)^2/n$. Combining, $4m^2/n \leq nm$, hence $m \leq n^2/4$. $\square$

Proof 2: induction on n . The inequality $m \leq \lfloor n^2/4 \rfloor$ is trivial for $n \leq 2$. Suppose it holds for $n - 2$ and let G be triangle-free on n vertices. If G has no edges we are done; otherwise pick any edge uv . Write $G' = G \setminus \{u, v\}$, which has at most $\lfloor (n - 2)^2/4 \rfloor$ edges by induction. Triangle-freeness and the disjointness $N(u) \cap N(v) = \emptyset$ show that u and v together contribute $d(u) + d(v) - 1 \leq n - 1$ edges, so

$$m \leq \left\lfloor \frac{(n-2)^2}{4} \right\rfloor + (n-1) = \left\lfloor \frac{n^2}{4} \right\rfloor.$$

$\square$

Proof 3: weight-shifting / LP relaxation. Assign each vertex v weight $x_v \geq 0$ with $\sum x_v = 1$. Define $f(\mathbf{x}) = \sum_{uv \in E} x_u x_v$. Triangle-freeness makes f at most $1/4$: indeed, if $x_u > 0$ and $x_v > 0$ for some edge uv , either $x_u > x_v$ (shift all weight of v onto u , increasing f) or $x_v \geq x_u$ (shift the other way), so the max is at a two-vertex support, giving $f \leq \max_{t \in [0,1]} t(1-t) = 1/4$. Now set $x_v = 1/n$ to obtain $m/n^2 \leq 1/4$. $\square$

Remark 130. *Proof 3 is the weighted-density technique central to flag-algebra arguments; it generalizes immediately to any forbidden subgraph. Proof 1 generalizes to a fractional “number-of-edges in triangle” count used in Goodman’s theorem (below).*

Theorem 150 (Goodman, 1959). Every graph G on n vertices with m edges contains at least

$$\frac{m}{3} (4m/n - n)$$

triangles. In particular a triangle-free graph has $m \leq n^2/4$.

Proof. Count, for each vertex v , the number of *cherry* subgraphs (paths of length 2) with v as endpoint: this is $\binom{d(v)}{2}$ but also counts, for each non-edge xy with $v \in N(x) \cap N(y)$, one cherry. Each triangle is counted thrice, so the number of triangles is

$$T = \frac{1}{3} \left(\sum_v \binom{d(v)}{2} - \#(\text{paths ending in non-edges}) \right).$$

An application of Cauchy–Schwarz produces the stated lower bound. $\square$

11.1.3 Turán’s Theorem

Theorem 151 (Turán, 1941). For $r \geq 2$ and $n \geq r$, if G is a graph on n vertices with no K_{r+1} subgraph, then $|E(G)| \leq t(n, r)$, with equality only when $G \cong T(n, r)$.

Proof 1: Zykov symmetrization. Let G be K_{r+1} -free with the maximum number of edges among all K_{r+1} -free graphs on n vertices. We show that “not adjacent” is an equivalence relation on $V(G)$.

Symmetry and *reflexivity* are automatic. For *transitivity*, suppose $u \not\sim v$ and $v \not\sim w$ but $u \sim w$. Without loss of generality $d(u) \geq d(v)$. Delete v and replace it by a clone v' of u with $N(v') = N(u)$. The new graph is still K_{r+1} -free (a clique through v' would give a clique through u), and its edge count is $|E| - d(v) + d(u) \geq |E|$. In

fact the inequality is strict unless $d(u) = d(v)$, but even when equal, iterating this move eventually makes all three vertices mutually non-adjacent, proving transitivity up to edge-preserving operations.

Thus in the extremal case $V(G)$ splits into classes of mutually non-adjacent vertices, with all edges between distinct classes present. That is, G is complete multipartite with some number of parts $s \leq r$. Among complete s -partite graphs with s parts of total size n , the edge count $\binom{n}{2} - \sum \binom{|V_i|}{2}$ is maximized (by convexity of $x \mapsto \binom{x}{2}$) when the parts are as equal as possible. Among $s \in \{1, \dots, r\}$, the choice $s = r$ dominates, giving $T(n, r)$. $\square$

Proof 2: Motzkin–Straus (probabilistic / LP). Define the Lagrangian

$$\lambda(G) = \max \left\{ \sum_{uv \in E} x_u x_v : x \geq 0, \sum_v x_v = 1 \right\}.$$

For any graph G on n vertices, setting $x_v = 1/n$ gives $\lambda(G) \geq m/n^2$. Motzkin and Straus proved $\lambda(G) = \frac{1}{2}(1 - 1/\omega(G))$, where $\omega(G)$ is the clique number. If $\omega(G) \leq r$, then $m/n^2 \leq \frac{1}{2}(1 - 1/r)$, giving Turán's bound. The key to Motzkin–Straus is to observe that any optimal $\mathbf{x}$ is supported on a clique of G . $\square$

Proof 3: induction via a max-degree vertex. Let v be a vertex of maximum degree Δ in a K_{r+1} -free graph G . Write $N = N(v)$, $M = V \setminus (N \cup \{v\})$. The induced subgraph $G[N]$ is K_r -free (a K_r in $G[N]$ plus v forms K_{r+1}), so by induction $|E(G[N])| \leq t(\Delta, r-1)$. Edges not inside N total at most $\Delta \cdot (n - \Delta)$ (the max-degree bound). Combining and optimizing over Δ yields the result. $\square$

Corollary 44 (Turán density of a clique). $\pi(K_{r+1}) = 1 - 1/r$.

Corollary 45 (Chromatic consequence). Every graph G on n vertices with $|E(G)| > t(n, r)$ edges contains a K_{r+1} ; equivalently, any G with $|E(G)| > (1 - 1/r)n^2/2$ has chromatic number at least $r + 1$.

11.1.4 Stability: The Erdős–Simonovits Theorem

A hallmark of modern extremal graph theory: if a graph is K_{r+1} -free with *nearly* extremal edge count, it must itself look almost like $T(n, r)$.

Theorem 152 (Erdős–Simonovits stability, 1968). For every $r \geq 2$ and $\varepsilon > 0$ there exist $\delta > 0$ and n_0 such that any K_{r+1} -free graph G on $n \geq n_0$ vertices with $|E(G)| \geq t(n, r) - \delta n^2$ edges can be transformed into $T(n, r)$ by adding or deleting at most εn^2 edges.

Proof sketch. If G were far from $T(n, r)$ in edit distance, the Zykov symmetrization of Proof 1 of Theorem 151 would be strictly improving, contradicting extremality up to δn^2 edges. A full proof uses the Szemerédi regularity lemma (see the subsection on regularity) to replace “close to $T(n, r)$ ” by a regularity-partition statement and then extracts a Turán-type improvement. $\square$

Remark 131. *Stability statements are the workhorse of “solve by perturbation” arguments: prove the result for the extremal configuration, then extend by smallness of perturbations.*

11.1.5 The Erdős–Stone–Simonovits Theorem

Theorem 153 (Erdős–Stone, 1946; Erdős–Simonovits, 1966). For any graph H with chromatic number $\chi(H) \geq 2$,

$$\text{ex}(n, H) = \left(1 - \frac{1}{\chi(H)-1} + o(1)\right) \frac{n^2}{2} \quad \text{as } n \rightarrow \infty.$$

Equivalently $\pi(H) = 1 - 1/(\chi(H) - 1)$.

Proof sketch via Erdős–Stone. The Erdős–Stone theorem asserts that for every $r \geq 2$, every $t \geq 1$, and $\varepsilon > 0$, if n is large enough then every graph on n vertices with $|E(G)| \geq (1 - 1/r + \varepsilon)n^2/2$ edges contains the complete $(r+1)$ -partite graph $K_{r+1}(t)$ with each part of size t . Given H of chromatic number $r+1$, choose t large enough that $H \subseteq K_{r+1}(t)$. Any graph with more than $(1 - 1/r + \varepsilon)n^2/2$ edges therefore contains $K_{r+1}(t) \supseteq H$. Combining with Turán's lower bound $t(n, r) \leq \text{ex}(n, H)$ (since $T(n, r)$ is $(r+1)$ -chromatic-free, thus H -free) gives the theorem. $\square$

Two lessons from this theorem.

Remark 132 (The bipartite cliff). For bipartite H we have $\chi(H) = 2$, so Erdős–Stone merely gives $\text{ex}(n, H) = o(n^2)$ —the leading order vanishes and we must determine the exact exponent of n . This is the central difficulty in the degenerate (bipartite) theory.

Remark 133 (Supersaturation). Erdős and Simonovits also proved that exceeding $\text{ex}(n, H)$ by a constant fraction yields not merely one but $\Theta(n^{|V(H)|})$ copies of H : the dependence is supersaturated. This is a quantitative strengthening routinely exploited.

11.1.6 The Bipartite Case: Kővári–Sós–Turán

Theorem 154 (Kővári–Sós–Turán, 1954). For integers $2 \leq s \leq t$,

$$\text{ex}(n, K_{s,t}) \leq \frac{1}{2}(t-1)^{1/s} (n^{2-1/s}) + \frac{s-1}{2}n.$$

Proof. Double-count the number of “cherries” $S = \{A \subseteq V(G) : |A| = s, A \subseteq N(v) \text{ for some } v\}$. For each $v \in V(G)$ the number of s -subsets of $N(v)$ is $\binom{d(v)}{s}$, so

$$\sum_{v \in V} \binom{d(v)}{s} = \#\{(A, v) : |A| = s, A \subseteq N(v)\}.$$

If G contains no $K_{s,t}$, then each s -set A has at most $t-1$ common neighbors v , so the right-hand side is at most $(t-1)\binom{n}{s}$. By convexity of $\binom{x}{s}$,

$$n \binom{2m/n}{s} \leq \sum_v \binom{d(v)}{s} \leq (t-1) \binom{n}{s}.$$

Solving for m , and using $\binom{x}{s} \geq (x-s+1)^s/s!$, gives the claimed bound. $\square$

Corollary 46. $\text{ex}(n, C_4) = \text{ex}(n, K_{2,2}) \leq \frac{1}{2}(1 + \sqrt{4n-3}) \cdot n/2 = O(n^{3/2})$.

Theorem 155 (Reiman; Erdős–Rényi–Sós). The upper bound $\text{ex}(n, C_4) \leq \frac{1}{2}(1 + \sqrt{4n-3})$ is attained to leading order: the incidence graph of the projective plane $\text{PG}(2, q)$ (a polarity graph) is C_4 -free and has $(1 + o(1))n^{3/2}/2$ edges when $n = q^2 + q + 1$.

Theorem 156 (Bondy–Simonovits, 1974). For every integer $k \geq 2$,

$$\text{ex}(n, C_{2k}) \leq 100k \cdot n^{1+1/k}.$$

Whether this is tight for $k \geq 4$ is a long-standing open problem; matching constructions are only known for $k \in \{2, 3, 5\}$ (projective plane, generalized quadrangle, generalized hexagon).

11.1.7 The Zarankiewicz Problem

Definition 151 (Zarankiewicz function). $z(m, n; s, t)$ is the maximum number of 1’s in an $m \times n$ 0/1-matrix with no all-ones $s \times t$ submatrix. Equivalently, it is the maximum edge count of a $K_{s,t}$ -free bipartite graph with parts of sizes m and n .

Theorem 157 (Kővári–Sós–Turán, bipartite form). $z(m, n; s, t) \leq \frac{1}{2}((t-1)^{1/s}(n-s+1)m^{1-1/s} + (s-1)m)$.

11.1.8 Graphs of High Girth: The Moore Bound

The interaction between extremal counts and girth is the complementary theme: how few vertices can a graph with high minimum degree and high girth have?

Definition 152 (Girth). The **girth** $g(G)$ is the length of the shortest cycle in G (or ∞ if G is a forest).

Theorem 158 (Moore bound). Let G be a graph with minimum degree δ and girth at least $2k+1$. Then

$$|V(G)| \geq 1 + \delta + \delta(\delta-1) + \cdots + \delta(\delta-1)^{k-1}.$$

For even girth $2k$,

$$|V(G)| \geq 2(1 + (\delta-1) + \cdots + (\delta-1)^{k-1}).$$

Proof for odd girth. Pick any vertex v and perform BFS for k layers. Girth $\geq 2k+1$ ensures all vertices in layers $0, 1, \dots, k$ are distinct: a repeat in layer $\leq k$ would close a cycle of length $\leq 2k$. Each vertex in layer $i < k$ contributes $\geq \delta-1$ neighbors to layer $i+1$ (except v which contributes δ to layer 1), giving the stated count. $\square$

Definition 153 (Cage). A (δ, g) -**cage** is a δ -regular graph of girth g with the smallest possible number of vertices.

Cages attaining the Moore bound with equality are called *Moore graphs*. Famous examples include the Petersen graph $((3, 5)$ -cage, 10 vertices), the Hoffman–Singleton graph $((7, 5)$ -cage, 50 vertices), and the Heawood graph $((3, 6)$ -cage, 14 vertices).

Theorem 159 (Hoffman–Singleton, 1960). A Moore graph of girth 5 and degree δ exists only if $\delta \in \{2, 3, 7, 57\}$.

The $\delta = 57$ case remains open after more than sixty years—one of the most famous unsolved problems in algebraic graph theory.

11.1.9 Supersaturation

Theorem 160 (Erdős–Simonovits supersaturation). For every graph H and $\varepsilon > 0$ there is $c > 0$ such that any graph G on n vertices with $|E(G)| \geq (\pi(H) + \varepsilon) \binom{n}{2}$ edges contains at least $c \cdot n^{|V(H)|}$ copies of H .

Proof sketch. Apply the regularity lemma to G and pass to the cluster graph. The regularity partition preserves the excess density, and the counting lemma turns it into an embedding count of H . $\square$

11.1.10 Exercises

Easy Questions.

Exercise 11.1. Prove Mantel’s theorem by induction on n : remove an edge uv from a triangle-free graph G , bound $|E(G - \{u, v\})|$ inductively, and account for the removed degrees.

Exercise 11.2. Show that the Turán graph $T(n, r)$ has chromatic number exactly r . Deduce from Turán’s theorem the *chromatic* density bound: any graph on n vertices with chromatic number at most r has at most $t(n, r)$ edges.

Exercise 11.3. Compute the number of triangles in $T(n, r)$. For which r does $T(n, r)$ maximize the number of triangles among all K_{r+1} -free graphs on n vertices?

Medium Questions.

Exercise 11.4. Prove that every K_4 -free graph on n vertices contains an independent set of size $\geq \sqrt{n}$. (Hint: use Turán’s theorem on the complement.)

Exercise 11.5. Prove the Erdős–Ko–Rado-style bound $\text{ex}(n, K_{2,2}) \leq \frac{1}{2}(1 + \sqrt{4n - 3})$ directly, using the fact that any two vertices have at most one common neighbor in a $K_{2,2}$ -free graph.

Exercise 11.6. Show that the incidence graph of a projective plane of order q has $2(q^2 + q + 1)$ vertices, is $(q + 1)$ -regular, and has girth 6. Verify that it achieves the Moore bound with equality.

Hard Questions.

Exercise 11.7 (Kruskal–Katona warm-up). Let G be a graph on n vertices with m edges. Show that the number of triangles in G is at most $\binom{m/(2^{1/2})}{3/2}$ asymptotically; deduce a direct upper bound for $\text{ex}(n, K_4)$.

Exercise 11.8. Prove that every graph on n vertices and at least $\lfloor n^2/4 \rfloor + 1$ edges contains at least $\lfloor n/2 \rfloor$ triangles. (This is Rademacher’s theorem, a supersaturation result extending Mantel.)

Exercise 11.9. Let G be a graph with $|V(G)| = n$ and $\delta(G) \geq 2n/5$. Prove that G contains a triangle. Construct a graph showing that $\delta \geq 2n/5$ cannot be weakened to $\delta \geq (2/5 - \varepsilon)n$.

Exercise 11.10 (Erdős–Gallai). Show that every graph on n vertices with more than $\frac{1}{2}(k - 1)n$ edges contains a path of length k .

Diagram Questions.

Exercise 11.11. Draw $T(8, 3)$ and $T(8, 4)$. For each, count the edges directly and verify the formula $t(n, r) = \binom{n}{2} - \sum_i \binom{|V_i|}{2}$.

Exercise 11.12. Draw a 4-regular graph on 10 vertices with girth 5 that achieves the odd Moore bound $1 + 4 + 4 \cdot 3 = 17$ minus the parity gap (so on 10 vertices). The Petersen graph is the canonical example—draw it in the standard pentagon-pentagram form and label the girth witness.

11.2 Ramsey Theory

Ramsey theory sits opposite extremal theory. Instead of asking how dense a graph can be before it must contain H , we ask how large a complete graph must be before *every* red/blue coloring of its edges contains a monochromatic copy of H . The unifying meta-theorem, sometimes phrased as “complete disorder is impossible”, asserts that any sufficiently large structure contains a highly organized substructure.

11.2.1 The Classical Ramsey Numbers

Definition 154 (Ramsey number). $R(s, t)$ is the least integer N such that every red/blue coloring of the edges of K_N contains a red K_s or a blue K_t . When $s = t$ we write $R(s) := R(s, s)$ for the *diagonal* Ramsey number.

Definition 155 (Multicolor Ramsey number). $R(s_1, \dots, s_k)$ is the least N such that in any k -coloring of the edges of K_N , some color class i contains K_{s_i} .

Theorem 161 (Ramsey, 1930). For all $s, t \geq 2$, $R(s, t)$ is finite. Moreover

$$R(s, t) \leq R(s-1, t) + R(s, t-1),$$

and consequently $R(s, t) \leq \binom{s+t-2}{s-1}$.

Proof. Induct on $s + t$, with base cases $R(2, t) = t$ and $R(s, 2) = s$ being immediate. For the inductive step, pick any vertex v in K_N with $N = R(s-1, t) + R(s, t-1)$. Among its $N-1$ incident edges, either $\geq R(s-1, t)$ are red or $\geq R(s, t-1)$ are blue (pigeonhole). In the first case, apply induction to the induced red neighborhood to obtain either a red K_{s-1} (which together with v gives a red K_s) or a blue K_t ; the second case is symmetric.

For the closed-form bound, one verifies $\binom{(s-1)+t-2}{s-2} + \binom{s+(t-1)-2}{s-1} = \binom{s+t-2}{s-1}$ and concludes by induction using Pascal's identity. $\square$

Corollary 47 (Diagonal upper bound). $R(k) \leq \binom{2k-2}{k-1} \sim \frac{4^{k-1}}{\sqrt{\pi k}}$.

Theorem 162 (Erdős–Szekeres refinement). When $s + t$ is odd, the bound above is strict: $R(s, t) \leq \binom{s+t-2}{s-1} - 1$.

Remark 134 (Small cases). *The following exact values are known, largely by exhaustive computer search:*

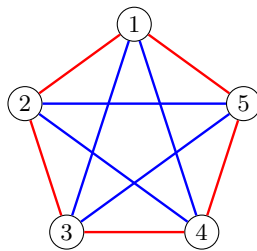
$$R(3, 3) = 6, \quad R(3, 4) = 9, \quad R(3, 5) = 14, \quad R(3, 6) = 18, \quad R(3, 7) = 23,$$

$$R(3, 8) = 28, \quad R(3, 9) = 36, \quad R(4, 4) = 18, \quad R(4, 5) = 25.$$

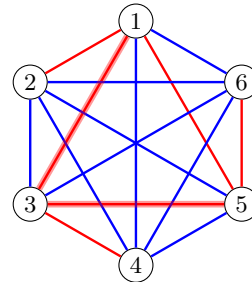
The value of $R(5, 5)$ is unknown, with $43 \leq R(5) \leq 46$ as of 2025. Erdős's famous quip: if aliens demanded the exact value of $R(5, 5)$, we should devote every computer on Earth to it; if they demanded $R(6, 6)$, we should attack them first.

Theorem 163 ($R(3, 3) = 6$). In any red/blue coloring of the edges of K_6 there is a monochromatic triangle; K_5 admits a coloring with no monochromatic triangle (the pentagon plus pentagram coloring).

Proof. Pick any vertex v of K_6 : among its 5 incident edges, by pigeonhole at least three, say vu_1, vu_2, vu_3 , have the same color, say red. If any $u_i u_j$ is red we have a red triangle; if all three are blue we have a blue triangle $u_1 u_2 u_3$. For K_5 , color the 5-cycle edges red and the “chord” edges of the pentagram blue; the only triangles of K_5 use both red and blue edges. $\square$



K_5 : no monochromatic K_3



K_6 : red $\triangle 135$ unavoidable

Figure 202: Witnessing $R(3, 3) = 6$. Left: the unique (up to swap) good K_5 colouring. Right: every 2-colouring of K_6 forces a monochromatic triangle.

11.2.2 Probabilistic Lower Bounds

Theorem 164 (Erdős, 1947). For all $k \geq 3$, $R(k, k) > 2^{k/2}$.

Proof. Color each edge of K_N red or blue independently and uniformly. For any fixed k -subset S of vertices, the probability that $K_N[S]$ is monochromatic is $2 \cdot 2^{-\binom{k}{2}}$. Let X be the total number of monochromatic K_k 's. Then

$$\mathbb{E}[X] = \binom{N}{k} \cdot 2 \cdot 2^{-\binom{k}{2}} \leq \frac{N^k}{k!} \cdot 2^{1-k(k-1)/2}.$$

For $N = \lfloor 2^{k/2} \rfloor$ and $k \geq 3$, one checks $\mathbb{E}[X] < 1$, so there is a coloring realizing $X = 0$. Any such coloring witnesses $R(k, k) > N$. $\square$

Theorem 165 (Spencer, via Lovász Local Lemma). $R(k, k) > (1 + o(1)) \frac{\sqrt{2}}{e} \cdot k \cdot 2^{k/2}$.

Proof sketch. Apply the symmetric Lovász Local Lemma to the events “the k -set S is monochromatic”. Two such events are dependent only if S, S' share at least two vertices; for each S there are at most $\binom{k}{2} \binom{N-2}{k-2}$ dependent events. Using $e \cdot p \cdot (d+1) \leq 1$ and optimizing N yields the factor-of- k improvement over the first-moment bound. $\square$

Theorem 166 (Off-diagonal: Ajtai–Komlós–Szemerédi / Kim). $\frac{c \cdot k^2}{\log k} \leq R(3, k) \leq \frac{C \cdot k^2}{\log k}$.

The matching upper and lower bounds on $R(3, k)$ (Shearer and Kim, 1995) are one of the crowning achievements of Ramsey theory; the lower bound uses a subtle *semirandom* / random greedy construction.

11.2.3 Ramsey Numbers for Graphs

Definition 156. For graphs H_1, H_2 , $R(H_1, H_2)$ is the least N such that every red/blue edge-coloring of K_N contains a red copy of H_1 or a blue copy of H_2 .

Theorem 167 (Chvátal, 1977). For every tree T on t vertices and every $s \geq 2$,

$$R(T, K_s) = (s-1)(t-1) + 1.$$

Proof. Upper bound. Let $N = (s-1)(t-1) + 1$. Color the edges of K_N red/blue. If the blue graph has chromatic number at least s , then it contains K_s by the degeneracy consequence of the greedy-coloring argument (each tree is 1-degenerate). Otherwise, N vertices partition into $s-1$ color classes of size ≥ 1 , one of which has $\geq t$ vertices. The red graph restricted to that class has minimum degree at least $t-1$ (it is the “red” part of K_t 's subgraph on that class, which is all edges—blue class is an independent set in blue). Being $(t-1)$ -degenerate, it contains any tree on t vertices.

Lower bound. Partition $(s-1)(t-1)$ vertices into $s-1$ blocks of $t-1$. Color within-block edges red and cross-block edges blue; the red graph is a disjoint union of K_{t-1} 's (no tree on t vertices fits), and the blue graph is $(s-1)$ -partite (no K_s). $\square$

Theorem 168 (Erdős–Gallai; Chvátal–Harary). For the path P_t on t vertices, $R(P_t, P_s) = t + \lfloor s/2 \rfloor - 1$ for $t \geq s \geq 2$.

Theorem 169 (Bondy–Erdős). For cycles of equal length, $R(C_n, C_n) = 2n-1$ if n is odd ($n \geq 5$), and $R(C_n, C_n) = 3n/2 - 1$ if n is even ($n \geq 6$).

Theorem 170 (Chvátal–Rödl–Szemerédi–Trotter). For every graph H of maximum degree Δ , $R(H, H) \leq c_\Delta \cdot |V(H)|$.

This linear-in- $|V(H)|$ bound for bounded-degree graphs was a landmark 1983 result proved via the regularity lemma.

11.2.4 Hypergraph Ramsey Numbers

Definition 157. $R_k(s, t)$ is the least N such that every red/blue coloring of the k -subsets of $\{1, \dots, N\}$ yields a red s -set (all k -subsets red) or a blue t -set.

Theorem 171 (Erdős–Rado stepping-up lemma). $R_{k+1}(2s, 2t) \geq 2^{R_k(s, t)-1}$.

Corollary 48. $R_3(k, k) \geq 2^{ck^2}$ and $R_k(s, s) \geq \text{tower}_{k-1}(cs^2)$, where tower_i is the iterated exponential.

Remark 135. Surprisingly, the doubly-exponential lower bound for $R_3(k, k)$ is conjectured to be tight up to the constant in the exponent; this is one of the most fundamental open problems in Ramsey theory.

11.2.5 Schur's Theorem and van der Waerden

Theorem 172 (Schur, 1916). For every $k \geq 1$ there exists N such that in every k -coloring of $\{1, 2, \dots, N\}$ there is a monochromatic triple a, b, c with $a + b = c$.

Proof. Set $N = R(3, 3, \dots, 3)$ (k colors). Given a k -coloring of $\{1, \dots, N\}$, build an edge-coloring of K_N by coloring edge ij (with $i < j$) by the color of $j - i$. A monochromatic triangle $i < j < \ell$ gives monochromatic numbers $j - i$, $\ell - j$, and $\ell - i$ with $(j - i) + (\ell - j) = \ell - i$. $\square$

Theorem 173 (van der Waerden, 1927). For every $k \geq 2, \ell \geq 2$ there exists $W(k, \ell)$ such that in every k -coloring of $\{1, 2, \dots, W(k, \ell)\}$ there is a monochromatic arithmetic progression of length ℓ .

Remark 136. Shelah gave primitive-recursive bounds on $W(k, \ell)$; Gowers's work on Szemerédi's theorem on arithmetic progressions in dense sets gave doubly-exponential bounds on W . The best known bounds are still astronomical.

11.2.6 Erdős–Szekeres: Happy Ending

Theorem 174 (Erdős–Szekeres, 1935). Every sequence of more than $(r - 1)(s - 1)$ distinct real numbers contains an increasing subsequence of length r or a decreasing subsequence of length s .

Proof 1: labeling. Let $(a_i)_{i=1}^n$ be the sequence with $n > (r - 1)(s - 1)$. For each i , let x_i be the length of the longest increasing subsequence ending at i , and y_i the longest decreasing. If $i < j$ and $a_i < a_j$ then $x_i < x_j$; if $a_i > a_j$ then $y_i < y_j$. So (x_i, y_i) is injective and lies in $\{1, \dots, r - 1\} \times \{1, \dots, s - 1\}$ would give $n \leq (r - 1)(s - 1)$, a contradiction. $\square$

Proof 2: Ramsey. Apply $R(r, s) \leq \binom{r+s-2}{r-1}$ to the graph with vertex set $\{1, \dots, n\}$ and edges ij colored red if $a_i < a_j$ and blue otherwise. $\square$

Theorem 175 (Erdős–Szekeres on points in general position). For every $n \geq 3$ there exists $ES(n)$ such that any $ES(n)$ points in the plane in general position contain n in convex position.

The asymptotic $ES(n) = 2^{n(1+o(1))}$ was established by Suk in 2017, tightening a 75-year-old bound.

11.2.7 Canonical Ramsey and Induced Ramsey

Theorem 176 (Erdős–Rado canonical Ramsey, 1950). For every k there is N such that in any coloring (with arbitrarily many colors) of the edges of K_N , there is a k -subset S such that the coloring of $K_N[S]$ falls into one of four types: monochromatic, rainbow (all distinct), *min-colored* (color determined by $\min\{i, j\}$), or *max-colored*.

Theorem 177 (Induced Ramsey; Deuber, Erdős–Hajnal–Pósa). For every graph H there is a graph G such that in every red/blue edge-coloring of G , there is an induced copy of H in G whose edges are monochromatic.

11.2.8 Exercises

Easy Questions.

Exercise 11.13. Prove that $R(3, 3) = 6$ by exhibiting a two-coloring of K_5 with no monochromatic triangle (Figure 11.2.1, left) and arguing $R(3, 3) \leq 6$ directly via pigeonhole on a vertex.

Exercise 11.14. Compute $R(3, 4)$ exactly. (Upper bound: $R(3, 4) \leq R(2, 4) + R(3, 3) = 4 + 6 = 10$; then sharpen to 9.)

Exercise 11.15. Verify the values $R(2, t) = t$ and $R(s, 2) = s$ that anchor the inductive proof of Theorem 161.

Medium Questions.

Exercise 11.16. Prove that $R(s, t) + R(s, t - 1)$ cannot both be even, and use this to sharpen Theorem 161 to $R(s, t) < R(s - 1, t) + R(s, t - 1)$ when both addends are even.

Exercise 11.17. Prove Schur's theorem for three colors by showing $R(3, 3, 3) \leq 17$. (Due to Greenwood and Gleason; the exact value $R(3, 3, 3) = 17$ requires the Kalbfleisch K_{16} coloring.)

Exercise 11.18. Deduce the infinite Ramsey theorem—every 2-coloring of the edges of K_∞ has an infinite monochromatic clique—from the finite version by a compactness argument.

Exercise 11.19. Let $\chi(G)$ denote chromatic number. Show that $R(K_s, G) \leq (s - 1)(\chi(G) - 1) + 1$ when G is connected.

Hard Questions.

Exercise 11.20. Use the probabilistic method to show $R(s, t) > (1 - o(1)) t^{(s-1)/2}$ for fixed s and $t \rightarrow \infty$.

Exercise 11.21. Prove Chvátal's theorem $R(T_n, K_s) = (s-1)(n-1) + 1$ for trees, filling in the degeneracy details.

Exercise 11.22. Use the Lovász Local Lemma to show $R(3, k) \geq c \cdot k^2 / \log^2 k$. (A weaker form of Kim's bound, but significantly sharper than the first-moment lower bound.)

Exercise 11.23. Prove van der Waerden's theorem $W(2, 3) = 9$: any 2-coloring of $\{1, \dots, 9\}$ has a monochromatic 3-AP, and some 2-coloring of $\{1, \dots, 8\}$ avoids one.

Diagram Questions.

Exercise 11.24. Reproduce the Greenwood–Gleason K_{16} Kalbfleisch coloring as a 3-coloring of K_{16} with no monochromatic triangle. Hint: use the Paley graph on $\mathbb{F}_{16}$ for one color class and partition the remaining edges via additive characters.

Exercise 11.25. Draw the bipartite blow-up coloring witnessing the Chvátal lower bound $R(T, K_s) > (s-1)(t-1)$: $s-1$ red cliques of size $t-1$ joined blue across blocks. Verify directly that no red T_t and no blue K_s appears.

11.3 The Probabilistic Method

The *probabilistic method*, pioneered by Erdős in the 1940s and 50s, is now the single most pervasive technique in combinatorics. The philosophy: to prove an object with property P exists, sample a random object from an appropriate distribution and show $\Pr[\text{sample has } P] > 0$. In practice this unfolds as four refinable tools: *first moment* (expectation and Markov), *alteration* (modify the random object to fix defects), *second moment* (Chebyshev for sharp thresholds), and *Lovász Local Lemma* (escape dependencies). We then discuss *concentration* via martingales (Azuma), entropy, and *dependent random choice*.

11.3.1 First Moment: Linearity of Expectation

Lemma 120 (Markov's inequality). For any non-negative random variable X and $t > 0$, $\Pr[X \geq t] \leq \mathbb{E}[X]/t$.

Lemma 121 (First moment principle). If X is a non-negative integer-valued random variable with $\mathbb{E}[X] < 1$ then $\Pr[X = 0] > 0$.

Theorem 178 (Large independent set). Every graph G with n vertices and average degree $d \geq 1$ has an independent set of size at least $n/(d+1)$.

Proof. Pick a uniformly random permutation π of $V(G)$, and let $S = \{v : \pi(v) < \pi(w) \text{ for every } w \in N(v)\}$. Then S is always an independent set: if $uv \in E$ and both $u, v \in S$, both would precede the other in π , absurd.

Each v joins S iff v is the first among the $d(v)+1$ vertices in $\{v\} \cup N(v)$, so $\Pr[v \in S] = 1/(d(v)+1)$. By linearity of expectation and Jensen's inequality,

$$\mathbb{E}[|S|] = \sum_{v \in V} \frac{1}{d(v)+1} \geq \frac{n}{\frac{1}{n} \sum_v d(v)+1} = \frac{n}{d+1}.$$

So some realization has $|S| \geq n/(d+1)$. □

Corollary 49 (Turán-type lower bound for independence). $\alpha(G) \geq \sum_v 1/(d(v)+1)$.

Theorem 179 (Hamiltonian paths in tournaments). The expected number of Hamiltonian paths in a tournament on n vertices (edges oriented uniformly at random) is $n!/2^{n-1}$. In particular every tournament has at least one Hamiltonian path.

Proof. For a fixed ordering $v_1, \dots, v_n$ of vertices, the probability that all $n-1$ consecutive arcs point forward is $2^{-(n-1)}$. Sum over all $n!$ orderings: expectation equals $n!/2^{n-1}$. □

Theorem 180 (Dominating set; Alon). Every graph G with minimum degree $\delta \geq 1$ has a dominating set of size at most $n(1 + \ln(\delta+1))/(\delta+1)$.

Proof. Let $p = \ln(\delta+1)/(\delta+1)$. Include each vertex in S independently with probability p . Let $T = \{v \notin S : N(v) \cap S = \emptyset\}$. Then $S \cup T$ is dominating, and

$$\mathbb{E}[|S| + |T|] \leq np + \sum_v (1-p)^{d(v)+1} \leq np + n(1-p)^{\delta+1} \leq np + ne^{-p(\delta+1)}.$$

Plugging in p yields the bound. □

11.3.2 The Alteration (Deletion) Method

When the random object has a few more “bad” features than desired, simply delete them.

Theorem 181 (Erdős lower bound for Ramsey, via deletion). $R(k, k) > (1 + o(1)) \frac{k}{e} \cdot 2^{k/2}$.

Proof. Color edges of K_n uniformly. Expected number of monochromatic k -cliques $X = 2 \binom{n}{k} 2^{-\binom{k}{2}}$. Delete one vertex from each; the remainder witnesses $R(k, k) > n - X$. Optimize over n to obtain the bound, improving Theorem 164 by a factor $2/e$. $\square$

Theorem 182 (High girth, high chromatic number; Erdős, 1959). For every $k, g \geq 3$ there is a graph G with girth $g(G) \geq g$ and chromatic number $\chi(G) \geq k$.

Proof. Fix $\theta \in (0, 1/g)$ and let $p = n^{\theta-1}$. Sample $G \sim G(n, p)$. The expected number of cycles of length less than g is

$$\sum_{\ell=3}^{g-1} \frac{n!}{(n-\ell)! \cdot 2\ell} p^\ell \leq \sum_{\ell=3}^{g-1} (np)^\ell = o(n),$$

so by Markov with probability $\geq 1/2$ the number of short cycles is $\leq n/2$.

Meanwhile, the probability that G has an independent set of size $\lceil n/k \rceil$ is at most

$$\binom{n}{n/k} (1-p)^{\binom{n/k}{2}} \leq 2^n e^{-pn^2/(3k^2)} \rightarrow 0.$$

So with positive probability both events hold: G has fewer than $n/2$ short cycles and $\alpha(G) < n/k$. Delete one vertex from each short cycle to obtain G' with $|V(G')| \geq n/2$, girth $\geq g$, and $\alpha(G') < n/k \leq 2|V(G')|/k$. Thus $\chi(G') \geq |V(G')|/\alpha(G') > k/2$; rescale. $\square$

Remark 137. No constructive proof of Theorem 182 with comparable parameters was known until the expander-graph constructions of Lubotzky–Phillips–Sarnak in the 1980s.

11.3.3 Second Moment and Paley–Zygmund

Theorem 183 (Chebyshev). For any random variable X and $t > 0$, $\Pr[|X - \mathbb{E}[X]| \geq t] \leq \text{Var}(X)/t^2$.

Lemma 122 (Paley–Zygmund). For non-negative X with $\mathbb{E}[X] > 0$,

$$\Pr[X > 0] \geq \frac{(\mathbb{E}[X])^2}{\mathbb{E}[X^2]} = \frac{1}{1 + \text{Var}(X)/(\mathbb{E}[X])^2}.$$

Theorem 184 (Triangle threshold in $G(n, p)$). As $n \rightarrow \infty$, $G(n, p)$ contains a triangle with probability tending to 0 if $pn \rightarrow 0$, and to 1 if $pn \rightarrow \infty$.

Proof. Let X be the number of triangles. Then $\mathbb{E}[X] = \binom{n}{3} p^3$, so Markov gives $\Pr[X \geq 1] \rightarrow 0$ when $np \rightarrow 0$. For the other direction, $\text{Var}(X) = O(n^4 p^5)$ and $(\mathbb{E}[X])^2 = \Theta(n^6 p^6)$, so the Paley–Zygmund ratio tends to 1 when $np \rightarrow \infty$. $\square$

11.3.4 The Lovász Local Lemma

First-moment and alteration arguments fail when dependencies are dense; LLL escapes them.

Definition 158 (Dependency graph). Given events $A_1, \dots, A_m$, a **dependency graph** D has vertex set $[m]$ with A_i mutually independent of $\{A_j : j \notin N_D(i) \cup \{i\}\}$.

Theorem 185 (General LLL; Erdős–Lovász). If there exist $x_1, \dots, x_m \in (0, 1)$ with $\Pr[A_i] \leq x_i \prod_{j \in N_D(i)} (1 - x_j)$, then

$$\Pr\left[\bigcap_i \overline{A_i}\right] \geq \prod_i (1 - x_i) > 0.$$

Theorem 186 (Symmetric LLL). If $\Pr[A_i] \leq p$ for all i , each A_i is mutually independent of all but at most d other events, and $ep(d+1) \leq 1$, then $\Pr[\bigcap_i \overline{A_i}] > 0$.

Proof sketch. Set $x_i = 1/(d+1)$ in Theorem 185 and verify $p \leq 1/(e(d+1))$ using $(1 - 1/(d+1))^d \geq 1/e$. $\square$

Theorem 187 (LLL lower bound for $R(k, k)$). $R(k, k) > (1 + o(1)) \frac{\sqrt{2}}{e} k \cdot 2^{k/2}$.

Proof. Color edges of K_n uniformly. For each k -set S let A_S be “ $K_n[S]$ monochromatic”; $p = 2^{1-\binom{k}{2}}$. Events A_S and $A_{S'}$ are independent unless $|S \cap S'| \geq 2$, so the dependency degree is $d \leq \binom{k}{2} \binom{n-2}{k-2}$. Apply the symmetric LLL and optimize n . $\square$

Theorem 188 (Moser–Tardos, 2010). Under the symmetric LLL hypothesis with events defined by independent variables, the following algorithm terminates in expected polynomial time with an assignment avoiding every A_i :

Sample all variables; while some A_i holds, pick one and resample its variables.

Remark 138. Moser–Tardos turns LLL from a pure existence result into a constructive algorithm. The proof uses entropy compression: a log of resampling events is encoded as a prefix of a fair coin sequence, so too-long executions contradict the entropy of the random tape.

11.3.5 Concentration: Azuma and Talagrand

Definition 159 (Martingale, bounded differences). A sequence $(X_0, \dots, X_n)$ is a **martingale** with respect to $(\mathcal{F}_i)$ if $\mathbb{E}[X_{i+1} \mid \mathcal{F}_i] = X_i$. It has **bounded differences** c_i if $|X_i - X_{i-1}| \leq c_i$ a.s.

Theorem 189 (Azuma–Hoeffding). For a martingale with $|X_i - X_{i-1}| \leq c_i$,

$$\Pr[|X_n - X_0| \geq t] \leq 2 \exp\left(-\frac{t^2}{2 \sum_i c_i^2}\right).$$

Corollary 50 (Lipschitz concentration in $G(n, p)$). Any f on $G(n, p)$ with $|f(G) - f(G')| \leq 1$ when G, G' differ on one edge satisfies

$$\Pr[|f - \mathbb{E}f| \geq t] \leq 2 \exp(-t^2 / (2 \binom{n}{2})).$$

Theorem 190 (Talagrand’s inequality, informal). Let $f : \{0, 1\}^n \rightarrow \mathbb{R}$ be 1-Lipschitz and c -certifiable, meaning $f(x) \geq s$ is certified by a subset $I \subseteq [n]$ of size $\leq cs$. Let $X = f(\xi)$ for independent coordinates ξ with median M . Then

$$\Pr[X \leq M - t] \leq 2 \exp(-t^2 / (4cM)), \quad \Pr[X \geq M + t] \leq 2 \exp(-t^2 / (4c(M + t))).$$

Talagrand’s inequality is the preferred concentration tool for triangle counts and $\chi(G(n, p))$ because the certificate condition is often much sharper than the Lipschitz constant alone would suggest.

11.3.6 Dependent Random Choice

Lemma 123 (Dependent random choice). Let G be a graph on n vertices with average degree d . For integers $a, r, m, t \geq 1$, if $d^t / n^{t-1} - \binom{n}{r} (m/n)^t \geq a$, then G has a subset U with $|U| \geq a$ in which every r -tuple has at least m common neighbors.

Proof sketch. Let T be a random t -set and $U = \bigcap_{v \in T} N(v)$. Then $\mathbb{E}[|U|] \geq d^t / n^{t-1}$. Expected number of bad r -sets inside U is at most $\binom{n}{r} (m/n)^t$. Delete and apply linearity. $\square$

Corollary 51 (Bipartite Turán via DRC). Every graph with at least $n^{2-1/t}$ edges contains $K_{t,t}$.

Dependent random choice powers modern proofs of Ramsey-type statements for sparse graphs, the Balog–Szemerédi–Gowers theorem, and refined bipartite Turán bounds.

11.3.7 The Entropy Method

Definition 160 (Shannon entropy). For a discrete random variable X , $H(X) = -\sum_x \Pr[X = x] \log \Pr[X = x]$. For jointly distributed (X, Y) , the conditional entropy is $H(X \mid Y) = \mathbb{E}_y H(X \mid Y = y)$.

Theorem 191 (Shearer’s lemma). If $\mathcal{F}$ is a collection of subsets of $[n]$ with each element covered at least k times, then for any joint distribution on $(X_1, \dots, X_n)$,

$$k \cdot H(X_1, \dots, X_n) \leq \sum_{F \in \mathcal{F}} H(X_F).$$

Corollary 52 (Kruskal–Katona-type triangle bound). A graph with m edges has at most $\frac{1}{6} (2m)^{3/2}$ triangles.

11.3.8 Exercises

Easy Questions.

Exercise 11.26. Using linearity of expectation, show that every graph on m edges has a bipartite subgraph with at least $m/2$ edges, and a k -partite subgraph with at least $(1 - 1/k)m$ edges.

Exercise 11.27. Show that every tournament on n vertices has a dominating set of size at most $\lceil \log_2 n \rceil$. (Sample uniformly random k -sets and union-bound.)

Exercise 11.28. Compute the variance of the triangle count X in $G(n, p)$ for $p = 1/2$ and verify $\text{Var}(X) = O(n^4)$ while $\mathbb{E}[X] = \Theta(n^3)$.

Medium Questions.

Exercise 11.29. Use the vertex-exposure martingale and Azuma's inequality to show that $\chi(G(n, 1/2))$ is concentrated within $O(\sqrt{n \log n})$ of its median.

Exercise 11.30. Prove that every tournament contains at least $n!/2^{n-1}$ Hamiltonian paths (Szele). Use this to give a lower bound on the permanent of the all-1/2 matrix of size n .

Exercise 11.31. Identify the regime $p = p(n)$ where the triangle count X in $G(n, p)$ satisfies $X/\mathbb{E}[X] \rightarrow 1$ in probability via the Paley–Zygmund inequality.

Hard Questions.

Exercise 11.32. Use LLL to show that every graph G of maximum degree Δ can be edge-coloured by at most $C \cdot \Delta$ colours such that each colour class is *acyclic*. (This is Alon's acyclic chromatic theorem; $C = 64$ suffices.)

Exercise 11.33. State and prove the *Lopsided* LLL: the dependency condition may be relaxed to a bound on $\Pr[A_i \mid \bigcap_{j \in J} A_j]$ for J disjoint from the dependency neighborhood of i .

Exercise 11.34. Use dependent random choice (Lemma 123) to prove: every n -vertex graph with $\geq n^{2-1/t}$ edges contains $K_{t,t}$.

Exercise 11.35. Use Shearer's entropy lemma to bound the number of labeled copies of C_4 in a graph with m edges.

Exercise 11.36. Prove the second-moment concentration for subgraph counts: if H has v vertices and e edges and $np^{e/v} \rightarrow \infty$, the number of copies of H in $G(n, p)$ concentrates around its mean.

Diagram Questions.

Exercise 11.37. Draw the dependency graph D for the events “the 4-set S is monochromatic” as S ranges over $\binom{[6]}{4}$ in the random 2-coloring of $E(K_6)$. Estimate the dependency degree d and check whether the symmetric LLL applies.

Exercise 11.38. For a triangle-free graph G found by the deletion method on $G(n, c/n)$ with $c = 5$, sketch (schematically) the typical local structure: degree distribution, isolated vertices, short cycles, and the surviving sparse independent-set witness for high chromatic number.

11.4 Spectral Graph Theory

Spectral graph theory studies the eigenvalues and eigenvectors of matrices associated with a graph—primarily the adjacency matrix A and the Laplacian L . The central thesis is that these spectra encode deep combinatorial information: connectivity, expansion, bipartiteness, walk-counting, mixing times of random walks, and the asymptotic complexity of many combinatorial algorithms.

11.4.1 Matrices of a Graph

Definition 161 (Adjacency and Laplacian matrices). Let $G = (V, E)$ be a graph on $V = \{1, \dots, n\}$.

- The **adjacency matrix** $A \in \{0, 1\}^{n \times n}$ has $A_{ij} = 1$ iff $ij \in E$.
- The **degree matrix** D is diagonal with $D_{ii} = d(i)$.
- The **combinatorial Laplacian** is $L = D - A$.
- The **normalized Laplacian** is $\mathcal{L} = I - D^{-1/2} A D^{-1/2}$ (where degrees are positive).
- The **signless Laplacian** is $Q = D + A$.

All these matrices are symmetric (when G is undirected), hence have real spectra and an orthonormal eigenbasis.

Definition 162 (Spectrum notation). For the adjacency matrix we write $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. For the Laplacian, $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n$.

Theorem 192 (Spectral theorem). Any real symmetric $n \times n$ matrix M admits an orthonormal eigenbasis $v_1, \dots, v_n$ with real eigenvalues $\tau_1, \dots, \tau_n$; equivalently $M = \sum_i \tau_i v_i v_i^\top$.

11.4.2 Walk Counting and Basic Identities

Proposition 193 (Walk counting). For any $k \geq 0$, $(A^k)_{ij}$ equals the number of walks of length k from i to j .

Corollary 53 (Traces). $\text{tr}(A^2) = 2|E|$, $\text{tr}(A^3) = 6 \cdot (\# \text{triangles in } G)$.

Proposition 194 (Laplacian quadratic form). For any $x \in \mathbb{R}^n$,

$$x^\top L x = \sum_{ij \in E} (x_i - x_j)^2 \geq 0.$$

Hence L is positive semidefinite, all $\mu_i \geq 0$, and $\mathbf{1}$ is always a 0-eigenvector.

Theorem 195 (Multiplicity of 0). The multiplicity of the eigenvalue 0 of L equals the number of connected components of G .

Proof. If $x^\top L x = 0$ then x is constant on every connected component by Proposition 194. Conversely, the indicator vectors of the components form a 0-eigenspace basis. $\square$

Proposition 196 (Regular case). If G is d -regular, then $L = dI - A$, so $\mu_i = d - \lambda_{n+1-i}$. In particular $\lambda_1 = d$ and $\lambda_n \geq -d$.

Theorem 197 (Bipartiteness and $\lambda_n = -d$). A d -regular graph G is bipartite if and only if $\lambda_n = -d$. The multiplicity of $-d$ equals the number of bipartite connected components.

11.4.3 The Min-Max Principle and Rayleigh Quotient

Definition 163 (Rayleigh quotient). For a symmetric matrix M and $x \neq 0$, the **Rayleigh quotient** is $R_M(x) = x^\top M x / x^\top x$.

Theorem 198 (Courant–Fischer). Let M be symmetric with eigenvalues $\tau_1 \geq \dots \geq \tau_n$. Then

$$\tau_k = \max_{\dim U=k} \min_{x \in U, x \neq 0} R_M(x) = \min_{\dim U=n-k+1} \max_{x \in U, x \neq 0} R_M(x).$$

Corollary 54 (Interlacing). Let M be a symmetric $n \times n$ matrix and M' any principal submatrix of order m . Then the eigenvalues $\tau'_1 \geq \dots \geq \tau'_m$ of M' interlace those of M : $\tau_{i+n-m}(M) \leq \tau'_i(M') \leq \tau_i(M)$.

Corollary 55 (Cauchy interlacing for graphs). If H is an induced subgraph of G , the adjacency eigenvalues of H interlace those of G .

11.4.4 Perron–Frobenius

Theorem 199 (Perron–Frobenius). Let $M \geq 0$ be a non-negative irreducible matrix. Then:

1. The spectral radius $\rho(M)$ is a simple eigenvalue.
2. There exists a strictly positive eigenvector $v > 0$ for $\rho(M)$.
3. $\rho(M) \geq |\tau|$ for every eigenvalue τ , with strict inequality unless the period of M 's underlying digraph divides $\arg(\tau/\rho)$.

Corollary 56. For a connected graph G , $\lambda_1(A) = \rho(A)$ is simple and its eigenvector has strictly positive entries.

Corollary 57. $\lambda_1(A(G)) \geq \bar{d} = 2|E|/n$ (average degree), with equality iff G is regular.

Proof. Take $x = \mathbf{1}/\sqrt{n}$. Then $R_A(\mathbf{1}) = \bar{d} \leq \lambda_1$ by Courant–Fischer. $\square$

11.4.5 The Matrix-Tree Theorem

Theorem 200 (Kirchhoff, 1847). Let G be a connected graph. The number of spanning trees of G equals any cofactor of L : delete any row i and column i , and take the determinant of the resulting $(n-1) \times (n-1)$ matrix. Equivalently,

$$\tau(G) = \frac{1}{n} \mu_2 \mu_3 \cdots \mu_n.$$

Proof sketch via Cauchy–Binet. Orient the edges of G arbitrarily and define the *signed incidence matrix* $B \in \{-1, 0, 1\}^{n \times m}$ with $B_{ve} = \pm 1$ depending on whether v is the tail or head of e . Then $L = BB^\top$. The reduced incidence matrix $\tilde{B}$ (obtained by deleting row i) has $\det(\tilde{B}\tilde{B}^\top)$ equal by Cauchy–Binet to the sum over $(n-1)$ -subsets S of edges of $(\det \tilde{B}_S)^2$. But $\det \tilde{B}_S = \pm 1$ if S is a spanning tree, 0 otherwise; so the cofactor is the spanning tree count. $\square$

Corollary 58 (Cayley's formula). The complete graph K_n has n^{n-2} spanning trees.

Proof. $L(K_n) = nI - J$ (where J is all-ones). Its eigenvalues are 0 (once) and n ($n-1$ times), so $\tau(K_n) = \frac{1}{n} \cdot n^{n-1} = n^{n-2}$. $\square$

11.4.6 Algebraic Connectivity and Cheeger's Inequality

Definition 164 (Algebraic connectivity and Cheeger constant). The **algebraic connectivity** of G is μ_2 . The **edge expansion** (or *Cheeger constant*) is

$$h(G) = \min_{\emptyset \neq S \subsetneq V, |S| \leq n/2} \frac{|E(S, V \setminus S)|}{|S|}.$$

Theorem 201 (Discrete Cheeger inequality). For a d -regular graph G ,

$$\frac{\mu_2}{2} \leq h(G) \leq \sqrt{2d\mu_2}.$$

Proof of the easy direction. Let S be the minimizer and let $\mathbf{1}_S$ be its indicator. The vector $x = \mathbf{1}_S - \frac{|S|}{n} \mathbf{1}$ is orthogonal to $\mathbf{1}$, so by Courant–Fischer

$$\mu_2 \leq \frac{x^\top Lx}{x^\top x} = \frac{|E(S, V \setminus S)|}{|S|(1 - |S|/n)} \leq \frac{2|E(S, V \setminus S)|}{|S|} = 2h(G).$$

$\square$

Proof of the hard direction, sketch. Let v_2 be the Fiedler eigenvector of L . Sort the vertices by v_2 -coordinate and consider the $n-1$ *threshold cuts* $S_k = \{v_{(1)}, \dots, v_{(k)}\}$. Show that the minimum Cheeger ratio over all S_k is at most $\sqrt{2d\mu_2}$. This is the spectral analogue of the Federer–Fleming isoperimetric trick. $\square$

11.4.7 Expander Graphs

Definition 165 (Expander). An infinite family $\{G_n\}$ of d -regular graphs is a **(one-sided) spectral expander** if $\max\{\lambda_2(G_n), -\lambda_n(G_n)\} \leq \lambda$ for a fixed $\lambda < d$. It is a **combinatorial expander** if $h(G_n) \geq c > 0$. By Cheeger (Theorem 201), the two notions are equivalent up to quantitative parameters.

Theorem 202 (Expander Mixing Lemma). Let G be d -regular on n vertices with $|\lambda| = \max(|\lambda_2|, |\lambda_n|)$. For all $S, T \subseteq V$,

$$\left| |E(S, T)| - \frac{d|S||T|}{n} \right| \leq |\lambda| \sqrt{|S||T|}.$$

Proof. Write $\mathbf{1}_S = \alpha \mathbf{1} + x$ and $\mathbf{1}_T = \beta \mathbf{1} + y$ with $\alpha = |S|/n, \beta = |T|/n$ and $x, y \perp \mathbf{1}$. Then

$$\mathbf{1}_S^\top A \mathbf{1}_T = |E(S, T)| = \alpha\beta \cdot nd + x^\top A y.$$

The second term is bounded by $|\lambda| \|x\| \|y\| \leq |\lambda| \sqrt{|S||T|}$. □

Theorem 203 (Alon–Boppana, 1986). For every d -regular graph on n vertices,

$$\lambda_2(G) \geq 2\sqrt{d-1} - o_n(1).$$

Definition 166 (Ramanujan graph). A d -regular graph G is **Ramanujan** if $\max(\lambda_2, -\lambda_n) \leq 2\sqrt{d-1}$.

Theorem 204 (Lubotzky–Phillips–Sarnak; Margulis, 1988). For every prime $p \equiv 1 \pmod{4}$, there is an explicit infinite family of $(p+1)$ -regular Ramanujan graphs.

Theorem 205 (Marcus–Spielman–Srivastava, 2015). For every $d \geq 3$ and every n divisible by some small integer, there is a d -regular bipartite Ramanujan graph on n vertices.

Marcus–Spielman–Srivastava introduced the *interlacing polynomials* method to solve the Kadison–Singer problem; the Ramanujan construction was a byproduct.

11.4.8 Random Walks and Mixing

Definition 167 (Random walk matrix). The **transition matrix** of the simple random walk on G is $P = D^{-1}A$. For a d -regular graph, $P = A/d$ and its spectrum is $\{\lambda_i/d\}$.

Theorem 206 (Mixing time bound). For a connected non-bipartite d -regular graph G , the simple random walk has total variation mixing time

$$t_{\text{mix}}(\varepsilon) \leq \frac{\log(n/\varepsilon)}{1 - \lambda_2/d}.$$

Proof sketch. The stationary distribution is uniform. The second-largest absolute eigenvalue governs the spectral gap; by the spectral decomposition P^t converges to $\mathbf{1}\mathbf{1}^\top/n$ at rate $(\lambda_2/d)^t$. The $\log(n/\varepsilon)$ factor absorbs the $L^2 \rightarrow L^1$ conversion. □

11.4.9 Hoffman’s Bound and Chromatic Number

Theorem 207 (Hoffman’s bound). For any graph G ,

$$\chi(G) \geq 1 - \frac{\lambda_1(A)}{\lambda_n(A)}.$$

Proof sketch. Suppose G admits a proper k -coloring with color classes $C_1, \dots, C_k$. Let $y = (y_1, \dots, y_k)$ optimize the quadratic form $y^\top B y / y^\top y$ for B a $k \times k$ auxiliary matrix built from block-averaged eigenvectors. An interlacing argument applied to A restricted to a color class (an independent set) gives the bound. □

Theorem 208 (Independence number bound). For a regular graph,

$$\alpha(G) \leq \frac{-\lambda_n}{\lambda_1 - \lambda_n} \cdot n.$$

11.4.10 Strongly Regular Graphs

Definition 168 (Strongly regular). A graph G on n vertices is **strongly regular** with parameters (n, k, λ, μ) if it is k -regular, every pair of adjacent vertices has λ common neighbors, and every pair of non-adjacent vertices has μ common neighbors.

Theorem 209 (Spectrum of SRG). A connected strongly regular graph with parameters (n, k, λ, μ) has exactly three adjacency eigenvalues: k with multiplicity 1, and

$$r, s = \frac{1}{2} \left((\lambda - \mu) \pm \sqrt{(\lambda - \mu)^2 + 4(k - \mu)} \right),$$

with multiplicities determined by k, r, s, n .

Strongly regular graphs are a central playground for algebraic graph theory, connecting to finite geometries (projective planes, generalized quadrangles), coding theory, and the classification of finite simple groups.

Example 48 (Petersen graph spectrum). The Petersen graph P is the strongly regular graph with parameters $(10, 3, 0, 1)$: 3-regular, no triangles, and any two non-adjacent vertices share exactly one common neighbor. By Theorem 209 its adjacency spectrum is $\{3^{(1)}, 1^{(5)}, (-2)^{(4)}\}$ (eigenvalues with multiplicities). The Hoffman bound gives $\chi(P) \geq 1 - 3/(-2) = 5/2$, so $\chi(P) \geq 3$; in fact $\chi(P) = 3$.

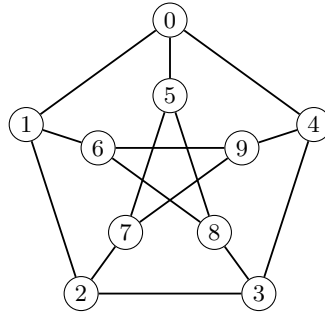


Figure 203: The Petersen graph; spectrum $\{3, 1, 1, 1, 1, 1, -2, -2, -2, -2\}$.

11.4.11 Exercises

Easy Questions.

Exercise 11.39. Compute the adjacency spectrum of K_n , $K_{n,n}$, C_n , and the Petersen graph.

Exercise 11.40. Use the Matrix-Tree theorem to count spanning trees of $K_{m,n}$. Answer: $m^{n-1}n^{m-1}$.

Exercise 11.41. Prove that a graph is bipartite iff its adjacency spectrum is symmetric about 0.

Medium Questions.

Exercise 11.42. For the cycle C_n , compute the Laplacian eigenvalues explicitly as $2 - 2\cos(2\pi k/n)$ for $k = 0, 1, \dots, n-1$. Identify μ_2 and comment on how it scales with n .

Exercise 11.43. Prove that the hypercube $Q_n = K_2^{\square n}$ has Laplacian spectrum $\{2k : k = 0, 1, \dots, n\}$ with multiplicity $\binom{n}{k}$.

Exercise 11.44. Prove Hoffman's theorem: $\chi(G) \geq 1 - \lambda_1/\lambda_n$, by applying interlacing to any independent-set partition.

Exercise 11.45. Use the expander mixing lemma to give an alternate proof of Cheeger's upper bound: $h(G) \leq \sqrt{2d\mu_2}$.

Hard Questions.

Exercise 11.46. Prove Theorem 209 on the three-eigenvalue spectrum of a strongly regular graph.

Exercise 11.47. Show that if G is a d -regular λ -expander, then the diameter of G satisfies $\text{diam}(G) \leq \lceil \log_{d/\lambda} n \rceil$.

Exercise 11.48. Prove the Alon-Boppana bound $\lambda_2 \geq 2\sqrt{d-1} - o(1)$ by a trace / walk-counting argument on the d -regular infinite tree.

Diagram Questions.

Exercise 11.49. Draw the Fiedler eigenvector v_2 of the path P_6 as a colored sequence (positive on one end, negative on the other) and use the threshold-cut sweep on v_2 to recover the bisection $\{1, 2, 3\}$ vs $\{4, 5, 6\}$. Verify directly that this is the Cheeger optimal cut.

Exercise 11.50. Draw the Petersen graph (Figure 48) with a proper 3-coloring of its vertices, and verify that no smaller proper coloring exists by combining the Hoffman bound with a parity argument on the outer pentagon.

11.5 Random Graphs

Random graph theory, initiated by Erdős and Rényi in their landmark 1959–61 papers, studies the typical behavior of graphs sampled from a distribution. The discovery of *sharp thresholds* and *phase transitions* was both mathematically startling and enormously influential: the same phenomena appear in percolation, spin glasses, random matrices, and the complexity of random constraint satisfaction.

11.5.1 Random Graph Models

Definition 169 (Erdős–Rényi model). $G(n, p)$ is the probability space of graphs on n labeled vertices in which each of the $\binom{n}{2}$ possible edges appears independently with probability $p = p(n)$.

Definition 170 (Uniform model). $G(n, M)$ is the uniform distribution over graphs on n labeled vertices having exactly M edges. The models $G(n, p)$ and $G(n, M)$ with $M \approx p\binom{n}{2}$ are “essentially equivalent” for most purposes via conditioning.

Definition 171 (Random regular model). $G_{n,d}$ is the uniform distribution over d -regular graphs on n labeled vertices. Sampling is non-trivial but well-approximated by the *configuration model*.

Definition 172 (Threshold function). A function $t(n)$ is a **threshold** for a monotone graph property P if

$$\Pr[G(n, p) \models P] \rightarrow \begin{cases} 0 & \text{if } p/t(n) \rightarrow 0, \\ 1 & \text{if } p/t(n) \rightarrow \infty. \end{cases}$$

Theorem 210 (Bollobás–Thomason, 1987). Every non-trivial monotone graph property has a threshold function (unique up to constants).

Proof sketch. Let $q(n, p)$ be the probability that $G(n, p)$ has P . Monotonicity of P and a coupling argument show q is non-decreasing in p . Define $t(n)$ by $q(n, t(n)) = 1/2$. A two-round argument with $k = C \log(1/\delta)$ independent copies shows that boosting p to $kt(n)$ makes q exceed $1 - \delta$ while halving p below $t(n)/k$ makes q drop below δ . $\square$

11.5.2 Subgraph Thresholds

Definition 173 (Density and balanced graph). The **density** of H is $m(H) = |E(H)|/|V(H)|$. The maximum density of a subgraph of H is $m^*(H) = \max_{H' \subseteq H} m(H')$. A graph H is **balanced** if $m(H) = m^*(H)$ and **strictly balanced** if $m(H') < m(H)$ for all proper subgraphs H' .

Theorem 211 (Bollobás, 1981). For every fixed graph H with at least one edge, the property “ H is a subgraph of G ” has threshold $t(n) = n^{-1/m^*(H)}$.

Proof sketch. Let H' be the densest subgraph of H . The expected number of copies of H' in $G(n, p)$ is $\Theta(n^{|V(H')|} p^{|E(H')|})$. This tends to 0 when $p \ll n^{-1/m(H')}$ and to ∞ otherwise. First-moment gives the 0-statement; second-moment (a moderately involved variance computation) gives the 1-statement. $\square$

Example 49.

- The threshold for containing K_3 is $p = 1/n$.
- The threshold for containing K_4 is $p = n^{-2/3}$.
- The threshold for containing any tree on k vertices is $p = n^{-k/(k-1)}$.
- The threshold for containing a Hamilton cycle is $p = \log n/n$ —the same as connectivity.

11.5.3 Degree Distribution

Theorem 212 (Poissonization of low degrees). For $p = c/n$ with $c > 0$ fixed, the degree sequence of $G(n, p)$ converges in distribution to i.i.d. $\text{Poisson}(c)$: for any fixed k , the number of vertices of degree k is $(1 + o(1))ne^{-c}c^k/k!$ with high probability.

Corollary 59 (Maximum and minimum degree). For $p = c/n$ with $c > 0$ fixed, $\Delta(G(n, p)) = (1 + o(1)) \log n / \log \log n$ whp.

11.5.4 Threshold for Connectivity

Theorem 213 (Erdős–Rényi, 1960). Let $p = (\log n + c_n)/n$. Then

$$\Pr[G(n, p) \text{ is connected}] \rightarrow \begin{cases} 0 & \text{if } c_n \rightarrow -\infty, \\ e^{-e^{-c}} & \text{if } c_n \rightarrow c \in \mathbb{R}, \\ 1 & \text{if } c_n \rightarrow \infty. \end{cases}$$

Proof sketch. Isolated vertices. Let X_0 count isolated vertices. $\mathbb{E}[X_0] = n(1-p)^{n-1} = e^{\log n - pn + O(1/n)} = e^{-c_n + o(1)}$. Second-moment analysis gives $\text{Var}(X_0) = (1 + o(1))\mathbb{E}[X_0]$, so X_0 is asymptotically $\text{Poisson}(e^{-c})$; in particular $\Pr[X_0 = 0] \rightarrow e^{-e^{-c}}$.

Connected iff no isolated. Show that whp at $p \sim \log n/n$ the only obstruction to connectivity is isolated vertices: any component of size $2 \leq k \leq n/2$ requires too many edges, and by union bound over k the probability tends to 0. $\square$

Corollary 60 (Komlós–Szemerédi; Bollobás). The thresholds for connectivity and Hamilton cycle coincide at $p = \log n/n$.

11.5.5 The Giant Component Phase Transition

Theorem 214 (Erdős–Rényi giant component). Set $p = c/n$. Let $C_{\max}(G(n, p))$ denote the largest component. As $n \rightarrow \infty$, with high probability:

- **Subcritical** ($c < 1$): $|C_{\max}| = \Theta(\log n)$, and all components have size $O(\log n)$.
- **Critical** ($c = 1$): $|C_{\max}| = \Theta(n^{2/3})$.
- **Supercritical** ($c > 1$): There is a unique giant component of size $(1 + o(1))\rho_c \cdot n$, where ρ_c is the unique positive root of $1 - \rho = e^{-c\rho}$, and all other components have size $O(\log n)$.

Proof sketch via branching processes. Explore the component of a fixed vertex v by breadth-first search. At each step, when exploring a vertex, its number of new neighbors is approximately $\text{Binomial}(n - k, p) \approx \text{Poisson}(c)$ for $k = o(n)$. This BFS approximates a Galton–Watson branching process with offspring $\text{Poisson}(c)$. When $c < 1$ the branching process dies a.s. in finite time (with geometric tail), giving $O(\log n)$ component size. When $c > 1$, with probability ρ_c the process survives, yielding a macroscopic component.

The concentration/uniqueness of the giant is proved by second-moment methods: showing two “branches” of size $\gtrsim n$ must eventually intersect. $\square$

Remark 139 (Critical window). Bollobás, and independently Luczak, identified the critical window $p = (1 + \lambda n^{-1/3})/n$, inside which $|C_{\max}|/n^{2/3}$ converges to a universal random variable. This was later refined by Aldous (1997) to a continuum-tree limit description.

11.5.6 Chromatic Number of $G(n, p)$

Theorem 215 (Bollobás, 1988). For $p = 1/2$ fixed, $\chi(G(n, 1/2)) = (1 + o(1)) \frac{n}{2 \log_2 n}$.

Proof sketch. The lower bound uses $\chi \geq n/\alpha$ together with $\alpha(G(n, 1/2)) \sim 2 \log_2 n$ whp. The matching upper bound requires a clever greedy algorithm (“Bollobás’s martingale”) that iteratively removes independent sets of size near the maximum and applies Azuma’s inequality for concentration. $\square$

Theorem 216 (Shamir–Spencer two-point concentration). For any p with $pn^{5/6} \rightarrow \infty$, there exist integer-valued sequences a_n such that $\chi(G(n, p)) \in \{a_n, a_n + 1\}$ whp.

11.5.7 Hamiltonicity

Theorem 217 (Pósa, 1976; Komlós–Szemerédi, 1983). The threshold for $G(n, p)$ to contain a Hamilton cycle is $p = (\log n + \log \log n + c_n)/n$. Specifically, if $c_n \rightarrow \infty$, $G(n, p)$ is Hamiltonian whp; if $c_n \rightarrow -\infty$, it is not.

Remark 140. *The proof introduces Pósa’s rotation–extension technique, which iteratively rotates a non-extendable path to find new endpoints, combined with a “no small bad sets” lemma via expansion.*

11.5.8 Random Regular Graphs

Theorem 218 (Wormald, 1999). For any fixed $d \geq 3$, the random d -regular graph $G_{n,d}$ is Hamiltonian whp, has chromatic number in $\{3, 4\}$ whp (for $d = 3$; larger for larger d), and has spectral gap bounded away from zero whp.

Theorem 219 (Friedman, 2008). For every $\varepsilon > 0$ and any fixed $d \geq 3$, the second-largest absolute eigenvalue of $G_{n,d}$ satisfies

$$|\lambda|(G_{n,d}) \leq 2\sqrt{d-1} + \varepsilon \quad \text{whp.}$$

In particular, random regular graphs are “asymptotically Ramanujan”.

11.5.9 Preferential Attachment and Scale-Free Models

Definition 174 (Barabási–Albert model). Start with a small seed graph. At each time step, add a new vertex v_t with m edges; each new edge attaches to an existing vertex u with probability proportional to $d(u)$.

Theorem 220 (Bollobás–Riordan–Spencer–Tusnády, 2001). In the BA model with $m \geq 1$, the fraction of vertices with degree exactly k converges in probability to $c_m k^{-3}$ for each fixed k : a power-law (*scale-free*) degree distribution.

11.5.10 Small Subgraph Central Limit Theorem

Theorem 221 (Rucinski, 1988). Let H be a fixed graph and let $X_H = X_H(n, p)$ count copies of H in $G(n, p)$. If $pn^{|V(H)|/|E(H)|} \rightarrow \infty$ and $n^2(1-p) \rightarrow \infty$, then

$$\frac{X_H - \mathbb{E}[X_H]}{\sqrt{\text{Var}(X_H)}} \xrightarrow{d} \mathcal{N}(0, 1).$$

11.5.11 Exercises

Easy Questions.

Exercise 11.51. Use the first-moment method to prove that the threshold for $G(n, p)$ to contain a triangle is $p = 1/n$: show $\mathbb{E}[\#K_3] = \binom{n}{3}p^3$ and apply Markov.

Exercise 11.52. Compute the threshold for the appearance of C_4 in $G(n, p)$. Answer: $p = n^{-1}$ (using $m^*(C_4) = 1$).

Exercise 11.53. Prove that the random 3-regular graph is connected whp.

Medium Questions.

Exercise 11.54. Prove that $\alpha(G(n, 1/2)) = (2 + o(1)) \log_2 n$ whp. (First moment for the upper bound; second moment / alteration for the matching lower bound.)

Exercise 11.55. Prove Theorem 210 (Bollobás–Thomason) assuming monotonicity and using the Janson inequalities.

Exercise 11.56. Prove that for $p = 1/n$ (critical), the expected number of *tree* components of size k in $G(n, p)$ is asymptotically $n \cdot k^{k-2} e^{-k} / k!$. (Use Cayley’s formula.)

Hard Questions.

Exercise 11.57. Prove that for $pn^{2/3} \rightarrow \infty$, the number of triangles in $G(n, p)$ is asymptotically normal. (Variance computation plus Stein’s method or the method of moments.)

Exercise 11.58. Show that for $p = c/n$ with $c > 1$, the diameter of the giant component of $G(n, p)$ is $(1 + o(1)) \log n / \log c$ whp.

Exercise 11.59. Let $p = c/n$ with $c > 1$. Show that the fraction of vertices in components of size $\leq k$ converges to an explicit value involving the Borel distribution.

Exercise 11.60. Use Azuma’s inequality on the vertex-exposure martingale to show $\chi(G(n, p))$ is concentrated within $O(\sqrt{n \log n})$ of its median, for any p .

Diagram Questions.

Exercise 11.61. Sketch the qualitative shape of the survival probability ρ_c (the unique positive root of $1 - \rho = e^{-c\rho}$) as a function of c on $[0, 3]$. Mark the critical point $c = 1$ and the horizontal asymptote $\rho_\infty = 1$. Annotate the picture with the three regimes (subcritical / critical / supercritical) of Theorem 214.

Exercise 11.62. Draw a sample $G(8, 1/2)$ and identify by inspection: the largest component, all isolated vertices, the maximum and minimum degree. Compare with the expectations $\mathbb{E}[X_0] = 8 \cdot (1/2)^7$ and $\mathbb{E}[\Delta] \approx \log_2 8$.

11.6 Szemerédi's Regularity Lemma

Szemerédi's regularity lemma, first proved as a technical tool toward his 1975 theorem on arithmetic progressions in dense sets, has become one of the most important results in extremal combinatorics. Intuitively it says: *every large dense graph is "close" to a bounded-complexity weighted object*. Combined with the *counting lemma* (embedding counts match random predictions to within error ε) and the *graph removal lemma*, it provides a general-purpose machine for translating structural statements about H -free-ness into approximate statements about the ambient graph.

11.6.1 ε -Regular Pairs

Definition 175 (Density). For disjoint non-empty $X, Y \subseteq V(G)$, the **edge density** is $d(X, Y) = e(X, Y)/(|X||Y|)$, where $e(X, Y)$ is the number of edges with one endpoint in each.

Definition 176 (ε -regular pair). A pair (X, Y) of disjoint non-empty vertex sets is **ε -regular** if for every $X' \subseteq X$ with $|X'| \geq \varepsilon|X|$ and every $Y' \subseteq Y$ with $|Y'| \geq \varepsilon|Y|$,

$$|d(X', Y') - d(X, Y)| \leq \varepsilon.$$

Remark 141 (Pseudorandomness). A regular pair (X, Y) is a "pseudorandom" bipartite graph: its edges are spread so uniformly that no large sub-rectangle has density deviating significantly from the average. In particular, random bipartite graphs are regular (with high probability), but so are many deterministic constructions such as Paley graphs.

Proposition 222 (Sliceability). If (X, Y) is ε -regular with density d , $X' \subseteq X$ with $|X'| \geq \varepsilon|X|$, $Y' \subseteq Y$ with $|Y'| \geq \varepsilon|Y|$, then (X', Y') is 2ε -regular with density in $[d - \varepsilon, d + \varepsilon]$.

Lemma 124 (Typical degree in a regular pair). If (X, Y) is ε -regular with density d , then all but at most $\varepsilon|X|$ vertices $x \in X$ have $|N(x) \cap Y| \in [(d - \varepsilon)|Y|, (d + \varepsilon)|Y|]$.

Proof. Suppose more than $\varepsilon|X|$ vertices have $|N(x) \cap Y| < (d - \varepsilon)|Y|$. Let X' be that exceptional set. Then $d(X', Y) < d - \varepsilon$, but $|X'| \geq \varepsilon|X|$, $|Y| \geq \varepsilon|Y|$, contradicting ε -regularity. $\square$

11.6.2 The Regularity Lemma

Definition 177 (Equipartition). An **equipartition** of $V(G)$ into k parts is a partition $V = V_1 \cup V_2 \cup \dots \cup V_k$ where $||V_i| - |V_j|| \leq 1$ for all i, j .

Definition 178 (ε -regular partition). A partition $V = V_0 \cup V_1 \cup \dots \cup V_k$ is **ε -regular** if $|V_0| \leq \varepsilon n$, the parts $V_1, \dots, V_k$ are equipartitioned, and at most εk^2 pairs (V_i, V_j) fail to be ε -regular.

Theorem 223 (Szemerédi Regularity Lemma, 1975). For every $\varepsilon > 0$ and $m_0 \geq 1$ there exists $M = M(\varepsilon, m_0)$ such that every graph G with at least m_0 vertices admits an ε -regular partition into k parts with $m_0 \leq k \leq M$.

Proof via mean-square density / energy increment. Define the *energy* (or *index*) of a partition $\mathcal{P} = \{V_1, \dots, V_k\}$:

$$q(\mathcal{P}) = \sum_{i,j} \frac{|V_i||V_j|}{n^2} \cdot d(V_i, V_j)^2.$$

Note $0 \leq q(\mathcal{P}) \leq 1$ since densities lie in $[0, 1]$. Key lemma (*energy increment*): if a partition has more than εk^2 irregular pairs, one can refine it to a partition $\mathcal{P}'$ (with each V_i split into $\leq 2^k$ smaller sets) such that $q(\mathcal{P}') \geq q(\mathcal{P}) + \varepsilon^5$.

Starting from any initial partition, iterate. Since energy increases by ε^5 each step and is bounded by 1, the process terminates in $\leq \varepsilon^{-5}$ steps, yielding the claimed bound on k . The partition size bound is a tower of 2's of height $O(\varepsilon^{-5})$. $\square$

Remark 142 (Gowers's lower bound). The tower-type dependence $M = M(\varepsilon)$ is essentially unavoidable: Gowers (1997) constructed graphs for which every ε -regular partition has $k \geq \text{tower}(1/\varepsilon^{1/16})$.

V_0	V_1	V_2	V_3	V_4
-------	-------	-------	-------	-------

0.35	0.62	??	0.41
0.62	0.18	0.55	0.30
??	0.55	0.27	0.48
0.41	0.30	0.48	0.39

Density matrix $d(V_i, V_j)$ for the partition.

Green: ε -regular pair. Red “??”: irregular pair (counted toward $\leq \varepsilon k^2$ budget).

V_0 is the small “garbage” set, $|V_0| \leq \varepsilon n$.

Figure 204: An ε -regular partition of $V(G)$ into $V_0 \cup V_1 \cup \dots \cup V_4$.

11.6.3 The Counting Lemma

Theorem 224 (Counting Lemma). Let H be a graph with vertex set $\{1, \dots, h\}$ and m edges. Suppose $V_1, \dots, V_h$ are vertex sets in G such that for every $ij \in E(H)$, the pair (V_i, V_j) is ε -regular with density $d_{ij} \geq 2\varepsilon$. Then the number of labeled copies of H in G with vertex i in V_i is

$$N(H; V_1, \dots, V_h) = (1 \pm O(\varepsilon)) \left(\prod_{ij \in E(H)} d_{ij} \right) \prod_{i=1}^h |V_i|.$$

Proof sketch by induction on h . Embed vertices of H into $V_1, \dots, V_h$ greedily. For each choice of $V_1, \dots, V_{i-1}$ embedded, Lemma 124 controls the typical common neighborhood intersection in V_i to within factor $(1 \pm O(\varepsilon))$; multiply across all m edges. $\square$

11.6.4 The Removal Lemma

Theorem 225 (Triangle Removal Lemma; Ruzsa–Szemerédi, 1976). For every $\eta > 0$ there exists $\delta > 0$ such that if a graph G on n vertices contains at most δn^3 triangles, then G can be made triangle-free by removing at most ηn^2 edges.

Proof. Apply Theorem 223 with $\varepsilon = \eta/4$ to obtain a regular partition $V_1, \dots, V_k$. Remove: (a) all edges inside any single part, (b) all edges between pairs that are not ε -regular, (c) all edges across regular pairs of density $< 2\varepsilon$. Total edges removed: at most ηn^2 . Now claim the resulting graph is triangle-free. Otherwise a triangle $v_1 v_2 v_3$ with $v_i \in V_{i'}$ survives; then $V_{1'}, V_{2'}, V_{3'}$ form a triangle of regular-dense pairs, and by Theorem 224 there are $\geq c \cdot n^3$ triangles in G for some $c = c(\eta)$. Choose $\delta < c$. $\square$

Theorem 226 (General Graph Removal Lemma; Erdős–Frankl–Rödl). For every graph H and every $\eta > 0$ there exists $\delta > 0$ such that if G on n vertices has fewer than $\delta n^{|V(H)|}$ copies of H , then at most ηn^2 edges need be removed to make G H -free.

11.6.5 From Triangle Removal to Roth

Theorem 227 (Roth, 1953). For every $\alpha > 0$ there exists N_0 such that every subset of $\{1, 2, \dots, N\}$ with $N \geq N_0$ and at least αN elements contains a non-trivial 3-term arithmetic progression.

Proof via triangle removal; Ruzsa. Let $A \subseteq [N]$ have density α . Build the *Ruzsa 3-AP graph* G_A with vertex set $X \cup Y \cup Z$ where each copy is $[3N]$. Edges: $(x, y) \in X \times Y$ iff $y - x \in A$; $(y, z) \in Y \times Z$ iff $z - y \in A$; $(x, z) \in X \times Z$ iff $(z - x)/2 \in A$. Triangles (x, y, z) with $x + z = 2y$ correspond to 3-APs $(y - x, y, y + (z - y))$ within A .

G_A has $9N^2$ vertices and $\Theta(\alpha N^2)$ edges, and trivial triangles (all three vertices encoding the same AP element) give $\Theta(\alpha N^2)$ vertex-disjoint triangles. By Theorem 225, any edge-removal making G_A triangle-free requires $\Omega(N^2)$ edges, so G_A must have $\Omega(N^3)$ triangles—yielding a non-trivial 3-AP in A . $\square$

11.6.6 The Blow-up Lemma

Theorem 228 (Kömlos–Sárközy–Szemerédi blow-up lemma, 1997). Let H be a graph of bounded maximum degree Δ and $d \in (0, 1)$. If G contains a “super-regular” blown-up version of an auxiliary graph R , then every graph H with $\chi(H) \leq r$ and $\Delta(H) \leq \Delta$ can be embedded into G , provided $|V(H)| \leq |V(G)|$ and $r \leq |V(R)|$.

The blow-up lemma is the counterpart of the counting lemma when one wants to embed a *spanning* or nearly-spanning subgraph rather than count all copies of a fixed H . It is the key ingredient in Pósa-style extremal-chromatic theorems and in the resolution of the Bollobás–Kömlos tiling conjecture.

11.6.7 Graphons and the Limit Theory

Definition 179 (Graphon). A **graphon** is a symmetric measurable function $W : [0, 1]^2 \rightarrow [0, 1]$. Every graph G defines a graphon W_G (the indicator of its adjacency matrix rescaled), and the space of graphons carries a natural *cut-distance* metric.

Theorem 229 (Lovász–Szegedy; Borgs–Chayes et al., 2006). The space of graphons modulo measure-preserving relabelings is compact under cut-distance, and any sequence of graphs with convergent subgraph densities converges to some graphon limit.

Graphons are the “real-valued” limit objects encoding what large graphs look like; they underlie the modern theory of graph property testing and extremal graph theory.

11.6.8 Property Testing

Theorem 230 (Alon–Shapira: testability of hereditary properties). Every hereditary graph property $\mathcal{P}$ is *testable* with one-sided error: there is an algorithm which, given oracle access to a graph G and a parameter ε , queries $O(\text{poly}(1/\varepsilon))$ edges of G and (a) accepts if $G \in \mathcal{P}$, (b) rejects with probability $\geq 2/3$ if G is ε -far from $\mathcal{P}$.

The proof relies directly on the regularity lemma and the graph-removal lemma: a large graph ε -far from $\mathcal{P}$ must contain “many” forbidden subgraphs, detectable by sampling.

11.6.9 Exercises

Easy Questions.

Exercise 11.63. Prove the *symmetry* of regularity: if (X, Y) is ε -regular with density d , then so is (Y, X) .

Exercise 11.64. Show that a random bipartite graph with edge probability p is ε -regular with high probability for any fixed $\varepsilon > 0$.

Exercise 11.65. Using the counting lemma, give a short proof of the Erdős–Stone theorem.

Medium Questions.

Exercise 11.66. Prove that the triangle removal lemma implies the corner theorem (Ajtai–Szemerédi): every subset of $[N]^2$ of positive density contains a *corner* $\{(x, y), (x + d, y), (x, y + d)\}$.

Exercise 11.67. Prove Mantel’s theorem via the triangle removal lemma: every graph with more than $(1/4 + \varepsilon)n^2$ edges contains $\Omega(n^3)$ triangles, contradicting $\text{ex}(n, K_3) \leq n^2/4$.

Exercise 11.68. Show that the graph removal lemma is equivalent to the following statement: for any fixed H and $\varepsilon > 0$, every graph with fewer than $\delta n^{|V(H)|}$ copies of H contains an H -free spanning subgraph with at least $e(G) - \varepsilon n^2$ edges.

Exercise 11.69. Using the regularity lemma, give a proof of the Erdős–Simonovits stability theorem.

Hard Questions.

Exercise 11.70. Using the triangle removal lemma, prove the following Behrend-type complement: there exist subsets of $[N]$ of size $N/\exp(\sqrt{\log N})$ with no 3-AP. Conclude that $\delta(\eta)$ in the removal lemma must satisfy $\delta \leq \exp(-\text{poly}(1/\eta))$.

Exercise 11.71. Prove the hypergraph version: for 3-uniform hypergraphs, every H with few copies can be made H -free by deleting few edges. (Use the hypergraph regularity lemma of Gowers / Rödl–Schacht.)

Exercise 11.72. Show that cut-distance $d_{\square}(W_1, W_2)$ is a metric on the space of graphons modulo measure-preserving relabelings, and that it is compatible with subgraph densities in the sense of Lovász–Szegedy.

Diagram Questions.

Exercise 11.73. For the graph G obtained as the disjoint union of $K_{4,4}$ and an independent set of 4 vertices, construct an ε -regular partition with $\varepsilon = 0.1$ into $k = 3$ parts (the two sides of the biclique and the independent set). Tabulate the 3×3 density matrix as in Figure 11.6.2 and verify that all pairs are 0.1-regular.

Exercise 11.74. Draw the Ruzsa 3-AP graph G_A for $A = \{1, 4\} \subset [5]$. Identify all triangles of G_A and verify that the trivial triangles correspond to the elements 1 and 4 of A themselves, and that A contains no 3-AP.

11.7 Graph Minors and Treewidth

The graph minor programme of Robertson and Seymour (1983–2011) is arguably the deepest body of work in structural graph theory. The two central pillars are the **Graph Minor Theorem** (well-quasi-ordering of finite graphs under the minor relation) and the **Graph Minor Structure Theorem** (every graph excluding a fixed minor is nearly planar in a precise sense). Alongside these, **treewidth** has emerged as the single most important parameter for measuring a graph’s “distance to being a tree” and the crux of parameterized algorithm design.

11.7.1 Minors and Topological Minors

Definition 180 (Minor). A graph H is a **minor** of G , written $H \preceq G$, if H can be obtained from a subgraph of G by contracting edges. Equivalently, there exist disjoint connected vertex sets $\{V_u : u \in V(H)\} \subseteq V(G)$ (the *branch sets*) such that whenever $uv \in E(H)$, some edge of G goes between V_u and V_v . Each branch set V_u induces a connected subgraph of G .

Definition 181 (Topological minor). H is a **topological minor** of G if some subdivision of H is a subgraph of G . Every topological minor is a minor; the converse fails (e.g. K_5 is a minor but not a topological minor of the Petersen graph).

Proposition 231 (Subdivisions and maximum degree). H is a topological minor of G iff H is a minor of G via branch sets that are paths. If $\Delta(H) \leq 3$, topological-minor and minor coincide.

Theorem 232 (Kuratowski, 1930). A graph G is planar iff it contains no subdivision of K_5 or $K_{3,3}$.

Theorem 233 (Wagner, 1937). A graph G is planar iff neither K_5 nor $K_{3,3}$ is a minor of G .

11.7.2 Tree Decompositions and Treewidth

Definition 182 (Tree decomposition). A **tree decomposition** of G is a pair $(T, \mathcal{B})$ where T is a tree and $\mathcal{B} = (B_t)_{t \in V(T)}$ is a family of **bags** $B_t \subseteq V(G)$ such that:

1. $\bigcup_{t \in V(T)} B_t = V(G)$;
2. for each edge $uv \in E(G)$ there is t with $\{u, v\} \subseteq B_t$;
3. for each $v \in V(G)$, the set $\{t \in V(T) : v \in B_t\}$ induces a non-empty subtree of T (the **interpolation property**).

The **width** of $(T, \mathcal{B})$ is $\max_t |B_t| - 1$; the **treewidth** of G is $\text{tw}(G) = \min\{\text{width of a tree decomposition of } G\}$.

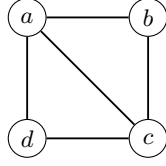
Example 50 (Small treewidth).

- $\text{tw}(T) = 1$ for any tree T with ≥ 2 vertices.
- $\text{tw}(C_n) = 2$ for $n \geq 3$.
- $\text{tw}(K_n) = n - 1$.
- $\text{tw}(\text{series-parallel graph}) \leq 2$.
- $\text{tw}(\text{planar graph on } n \text{ vertices}) = O(\sqrt{n})$.
- $\text{tw}(n \times n \text{ grid}) = n$.

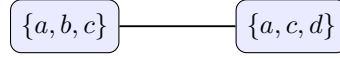
Proposition 234 (Subadditivity under separators). If S is a separator of G (i.e. $G \setminus S$ is disconnected, say with components inducing subgraphs $G_1, \dots, G_k$), then

$$\text{tw}(G) \leq \max_i \text{tw}(G_i \cup S) + |S|.$$

More precisely, tree decompositions glue along S .



Graph G



Tree decomposition $(T, \mathcal{B})$ of width 2

$$\text{tw}(G) = 2$$

Figure 205: A graph G with chord ac and a width-2 tree decomposition. The vertex a (and likewise c) appears in both bags; this is allowed by the interpolation property because the bags containing a form a (sub)tree.

Theorem 235 (Pathwidth and intervals). A graph G has pathwidth at most k iff G is a subgraph of an *interval graph* with clique number $\leq k + 1$.

Theorem 236 (Treewidth and chordal graphs). $\text{tw}(G) \leq k$ iff G is a subgraph of a chordal graph with clique number $\leq k + 1$.

Proof sketch. ($\Leftarrow$) Chordal graphs have tree decompositions whose bags are maximal cliques.

($\Rightarrow$) Given a tree decomposition, add edges between any two vertices sharing a bag; the resulting graph is chordal (clique trees exist). $\square$

11.7.3 Duality: Brambles and Cops–Robbers

Definition 183 (Bramble). A **bramble** in G is a family $\mathcal{B}$ of connected subgraphs such that any two elements either share a vertex or are connected by an edge. A **hitting set** for $\mathcal{B}$ is a vertex set meeting every element. The **bramble number** $\text{bn}(G)$ is the maximum size of the minimum hitting set over all brambles.

Theorem 237 (Seymour–Thomas, 1993). $\text{tw}(G) + 1 = \text{bn}(G)$.

This min–max duality is the graph-theoretic analogue of LP duality and is the most convenient tool for proving *lower bounds* on treewidth.

Definition 184 (Cops and robber game). In the k -cop game on G , k cops and a robber are placed on vertices. Each round: cops may jointly teleport to any vertices, and the robber runs along any $(V(G) \setminus \text{new cops})$ -avoiding path. Cops win if a cop ever lands on the robber. The **cop number** is the least k for which cops win.

Theorem 238 (Seymour–Thomas). In the *visible-robber* variant of the cops game, k cops win on G iff $\text{tw}(G) \leq k - 1$.

11.7.4 Courcelle’s Theorem and Fixed-Parameter Tractability

Definition 185 (MSO_2 logic). MSO_2 graph logic allows variables for vertices, edges, sets of vertices, and sets of edges, together with the membership relations and a binary predicate $\text{inc}(v, e)$ for “ v is incident to e ”. Most natural graph properties— k -colorability, Hamiltonicity, existence of a given minor, etc.—are expressible in MSO_2 .

Theorem 239 (Courcelle, 1990). Every graph property expressible in MSO_2 can be decided in time $f(k, \phi) \cdot n$ on graphs of treewidth at most k , where ϕ is the MSO_2 formula and f is a computable (but typically tower-type) function.

Corollary 61. 3-coloring, Hamilton cycle, vertex cover, independent set, and minor containment are all solvable in linear time on graphs of bounded treewidth.

Remark 143 (Practical tractability). *Bodlaender’s algorithm (1996) computes a tree decomposition of width $\leq k$ in time $2^{O(k^3)}n$; many MSO_2 problems then admit explicit DP algorithms with single-exponential dependence on k .*

11.7.5 Excluded Minors and Sparsity

Theorem 240 (Mader; Thomason; Kostochka). Every graph with average degree at least $ck\sqrt{\log k}$ contains K_k as a minor, where c is an absolute constant. Moreover the bound is tight up to the constant.

Theorem 241 (Hadwiger’s conjecture). Every graph G of chromatic number k contains K_k as a minor.

Hadwiger is an enormous strengthening of the Four Color Theorem (proved for $k = 4$ by Wagner, $k = 5$ by Robertson–Seymour–Thomas, and open for $k \geq 6$ in general). Recent progress by Norin–Postle–Song gives $O(k(\log \log k)^6)$ -chromatic suffices for K_k -minor.

11.7.6 The Graph Minor Structure Theorem

Theorem 242 (Robertson–Seymour Grid Minor Theorem, 1986). For every k there exists $f(k)$ such that every graph of treewidth at least $f(k)$ contains the $k \times k$ grid as a minor. Quantitatively, Chuzhoy and Tan showed $f(k) = O(k^9 \text{polylog}(k))$.

Theorem 243 (Graph Minor Structure Theorem; Robertson–Seymour, 2003). For every graph H , every graph G with no H -minor admits a tree decomposition into “almost-embeddable” pieces: each bag is obtained from a graph drawable on a surface of bounded genus by adding a bounded number of *vortices* (local perturbations) and *apex* vertices.

The Structure Theorem is the core technical tool underlying both the proof of the Graph Minor Theorem and the design of FPT algorithms for minor-closed problems (e.g. disjoint paths).

11.7.7 The Graph Minor Theorem

Definition 186 (Well-quasi-order). A relation $\preceq$ on a set X is a **well-quasi-order** if it is reflexive, transitive, and every infinite sequence $x_1, x_2, x_3, \dots$ contains indices $i < j$ with $x_i \preceq x_j$. Equivalently, there is no infinite antichain and no infinite strictly decreasing sequence.

Theorem 244 (Higman’s lemma). If $(X, \preceq)$ is a wqo, then so is $(X^*, \preceq^*)$ on finite sequences under the subsequence embedding.

Theorem 245 (Kruskal’s tree theorem, 1960). Finite rooted trees form a wqo under the *homeomorphic embedding* relation.

Theorem 246 (Robertson–Seymour Graph Minor Theorem, 1983–2004). Finite graphs form a well-quasi-order under the minor relation: in any infinite sequence $G_1, G_2, \dots$ there exist $i < j$ with $G_i \preceq G_j$.

Corollary 62 (Finite obstruction sets). Every minor-closed graph property $\mathcal{P}$ is characterized by a *finite* list of forbidden minors: there exist $H_1, \dots, H_k \notin \mathcal{P}$ such that $G \in \mathcal{P}$ iff no H_i is a minor of G .

Proof. Let $\text{Obs}(\mathcal{P})$ be the set of minor-minimal graphs not in $\mathcal{P}$. Any two distinct elements are incomparable (else the smaller one would contradict minimality), so Obs is an antichain. By Theorem 246 antichains are finite. $\square$

Corollary 63 (Algorithmic). For every fixed minor H , there is an algorithm deciding in time $O(n^3)$ whether H is a minor of a given graph G . Combined with Corollary 62, every minor-closed property admits a cubic-time recognition algorithm—even when the obstruction set is unknown!

Remark 144. *The Graph Minor Theorem spans more than 500 pages across twenty papers. The proof relies on the Structure Theorem plus an intricate induction over partial orderings of embedded structures. The cubic running time for minor testing is attributed to Robertson–Seymour for fixed H and has since been refined.*

11.7.8 The Disjoint Paths Problem

Theorem 247 (Robertson–Seymour disjoint paths). For every fixed k , the **k -disjoint paths problem**—given G and pairs $(s_1, t_1), \dots, (s_k, t_k)$, determine whether there exist vertex-disjoint paths P_i from s_i to t_i —is solvable in time $O(n^3)$.

This is a major consequence of the Structure Theorem: while the problem is NP-hard when k is part of the input, any fixed number of terminal pairs can be handled in polynomial time.

11.7.9 Exercises

Easy Questions.

Exercise 11.75. Show that every graph with treewidth at most k has at most kn edges. Deduce that planar graphs, despite allowing up to $3n - 6$ edges, may still have large treewidth.

Exercise 11.76. Compute the treewidth of the Petersen graph.

Exercise 11.77. Verify directly that the cycle C_5 has treewidth 2 by exhibiting a width-2 tree decomposition.

Medium Questions.

Exercise 11.78. Prove that $\text{tw}(G) \leq k$ iff G has a chordal supergraph with clique number $\leq k + 1$ (Theorem 236).

Exercise 11.79. Show that $K_{3,3}$ and K_5 are the only two minor-minimal non-planar graphs, completing Wagner's theorem (Theorem 233).

Exercise 11.80. Prove Kruskal's tree theorem from Higman's lemma.

Exercise 11.81. Show that the class of *series-parallel* graphs (graphs with no K_4 -minor) coincides with graphs of treewidth at most 2.

Exercise 11.82. Using Courcelle's theorem, show that 3-coloring is solvable in linear time on graphs of bounded treewidth. Construct an explicit DP algorithm with running time $O(3^k \cdot k \cdot n)$.

Hard Questions.

Exercise 11.83. Prove that the $n \times n$ grid graph has treewidth exactly n . (The upper bound is easy via a sweeping decomposition; the lower bound uses Theorem 237.)

Exercise 11.84. Prove Hadwiger's conjecture for $k = 4$: every graph of chromatic number 4 contains K_4 as a minor. (Use an inductive decomposition.)

Exercise 11.85. Prove that every graph with no K_t -minor has chromatic number $O(t\sqrt{\log t})$. (Use Mader's density bound.)

Exercise 11.86. Show that well-quasi-ordering of finite graphs under *topological minor* fails: construct an infinite antichain of graphs under that relation.

Exercise 11.87. Prove the grid-minor theorem (Theorem 242) with the weaker bound $f(k) = 2^{\Theta(k)}$, using an induction on treewidth and the separator / tangle machinery.

Diagram Questions.

Exercise 11.88. For the 3×3 grid, construct an explicit width-3 tree decomposition (the upper bound is achieved by a path of 3 bags, each containing one row plus the next). Draw the decomposition analogously to Figure 11.7.2.

Exercise 11.89. Draw K_5 and $K_{3,3}$ side by side. For each, exhibit a K_5 -minor (resp. $K_{3,3}$ -minor) inside the Petersen graph by choosing four (resp. six) connected branch sets and verifying that the required adjacencies hold.

11.8 Comprehensive Chapter Exercises

Easy Questions.

Exercise 11.90. Prove Mantel's theorem by induction on n : remove an edge uv from a triangle-free graph G , bound $|E(G - \{u, v\})|$ inductively, and account for the removed degrees.

Exercise 11.91. Show that the chromatic number of the Turán graph $T(n, r)$ is exactly r . Deduce from Turán's theorem that every graph with more than $(1 - 1/r)n^2/2$ edges has chromatic number $\geq r + 1$.

Exercise 11.92. Prove that $R(3, 3) = 6$ by exhibiting a two-coloring of K_5 with no monochromatic triangle and arguing $R(3, 3) \leq 6$ directly.

Medium Questions.

Exercise 11.93. Using linearity of expectation, show that every tournament on n vertices contains a Hamiltonian path.

Exercise 11.94. For the cycle C_n , compute the Laplacian eigenvalues explicitly as $2 - 2\cos(2\pi k/n)$ for $k = 0, 1, \dots, n - 1$. Identify μ_2 and comment on how it scales with n .

Exercise 11.95. Use the first-moment method to prove that the threshold for $G(n, p)$ to contain a triangle is $p = 1/n$: show $\mathbb{E}[\#K_3] = \binom{n}{3}p^3$ and apply Markov's inequality.

Exercise 11.96. Prove Kirchhoff's Matrix-Tree theorem for the complete graph K_n and deduce Cayley's formula: K_n has n^{n-2} spanning trees.

Hard Questions.

Exercise 11.97. Show that every graph with treewidth at most k has at most $k \cdot n$ edges. Deduce that planar graphs, despite allowing up to $3n - 6$ edges, may still have large treewidth.

Exercise 11.98. Prove that the $n \times n$ grid graph has treewidth exactly n . (The upper bound is easy via a sweeping decomposition; the lower bound requires a connectivity/cops-and-robbers argument.)

Exercise 11.99. Using the triangle removal lemma, prove the following: for every $\eta > 0$ there exists n_0 such that any subset of $\{1, \dots, n\}$ with $n \geq n_0$ and density at least η contains a 3-term arithmetic progression. (Construct the Ruzsa 3-AP graph.)

12 Algebraic Graph Theory: Auxiliary Topics

This chapter collects auxiliary topics drawn from *algebraic combinatorics*—the area where graphs are studied through polynomials, group actions, and association-scheme algebras. Each section is self-contained and assumes only the spectral and counting techniques of Chapter 11. The topics are: the matchings polynomial and its real-rootedness (Heilmann–Lieb), equitable partitions and quotient interlacing, distance-regular graphs and association schemes, Cayley graphs and vertex-transitive graphs, and the Tutte polynomial together with the graphic matroid that organizes deletion–contraction arguments.

References for further reading: Godsil, *Algebraic Combinatorics*; Stanley, *Algebraic Combinatorics: Walks, Trees, Tableaux*; Garsia–Egecioglu, *Lectures in Algebraic Combinatorics*; Orlik–Welker, *Algebraic Combinatorics*.

12.1 The Matchings Polynomial and the Heilmann–Lieb Theorem

The adjacency-matrix characteristic polynomial $\phi(G, x) = \det(xI - A(G))$ encodes the spectrum of G , but it is highly non-trivial to relate to the combinatorial structure of G . The *matchings polynomial* plays a parallel role for matchings and turns out to coincide with $\phi(G, x)$ when G is a forest, while always remaining a polynomial with real roots only.

12.1.1 Definition and Basic Recurrences

Definition 187 (Matchings polynomial). Let G be a graph on n vertices and let $p(G, k)$ denote the number of k -matchings (sets of k pairwise non-adjacent edges). The **matchings polynomial** of G is

$$\mu(G, x) = \sum_{k \geq 0} (-1)^k p(G, k) x^{n-2k}.$$

By convention, $p(G, 0) = 1$ and $\mu(\emptyset, x) = 1$.

Example 51 (Small graphs).

- $\mu(K_2, x) = x^2 - 1$.
- $\mu(P_3, x) = x^3 - 2x$.
- $\mu(C_3, x) = x^3 - 3x$ (three edges, no 2-matching).
- $\mu(C_4, x) = x^4 - 4x^2 + 2$ (two 2-matchings: opposite-edge pairs).

Theorem 248 (Edge and vertex recurrences). For any edge $e = uv$ and any vertex u of G ,

$$\begin{aligned} \mu(G, x) &= \mu(G \setminus e, x) - \mu(G \setminus \{u, v\}, x), \\ \mu(G, x) &= x \mu(G \setminus u, x) - \sum_{v \sim u} \mu(G \setminus \{u, v\}, x). \end{aligned}$$

Proof. For the first identity, partition k -matchings of G by whether $e \in M$. Matchings not containing e are k -matchings of $G \setminus e$, contributing $\mu(G \setminus e, x)$. Matchings containing e are $(k-1)$ -matchings of $G \setminus \{u, v\}$, contributing $x^2 \mu(G \setminus \{u, v\}, x) / x^2 = \mu(G \setminus \{u, v\}, x)$ once the sign is tracked. The vertex recurrence is the edge recurrence summed over all edges incident to u . $\square$

Proposition 249 (Matching polynomial of a forest). If T is a forest, then $\mu(T, x) = \phi(T, x) = \det(xI - A(T))$.

Proof. The Leibniz formula expresses $\det(xI - A)$ as a sum over permutations σ , which decompose into cycles. In a forest the only cycles admissible to a non-zero contribution are fixed points (giving an x) and 2-cycles (giving -1 via an edge). The non-fixed-point support of σ thus selects a matching M , and the contribution is $(-1)^{|M|} x^{n-2|M|}$. Summing matches Definition 187. $\square$

12.1.2 The Path-Tree Construction

The fundamental insight of Godsil and Heilmann–Lieb is that for any graph G with a distinguished vertex u , one can build a tree $T(G, u)$ whose characteristic polynomial controls the matchings polynomial of G .

Definition 188 (Path-tree). Let G be a graph and $u \in V(G)$. The **path-tree** $T(G, u)$ is the rooted tree whose vertices are the non-backtracking walks $u = v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_k$ in G starting at u , and whose edges connect each walk of length k to its $(k+1)$ -step extensions in G .

Theorem 250 (Godsil’s path-tree identity). For any vertex u of G ,

$$\frac{\mu(G \setminus u, x)}{\mu(G, x)} = \frac{\mu(T(G, u) \setminus u, x)}{\mu(T(G, u), x)}.$$

In words, the ratio is identical to the corresponding ratio for the characteristic polynomials of the path-tree.

Corollary 64 (Reduction to a tree). Every root of $\mu(G, x)$ is a root of $\phi(T(G, u), x)$ for any choice of u . Since $T(G, u)$ is a tree, $\phi(T(G, u), x)$ has only real roots; therefore so does $\mu(G, x)$.

12.1.3 The Heilmann–Lieb Theorem

Theorem 251 (Heilmann–Lieb, 1972). Let G be a graph with maximum degree Δ . Then all roots of $\mu(G, x)$ are real, and they lie in the open interval $(-2\sqrt{\Delta-1}, 2\sqrt{\Delta-1})$.

Proof sketch. Reality follows from Corollary 64. For the bound, observe that the path-tree $T(G, u)$ has maximum degree at most Δ , so its adjacency spectrum is contained in $[-2\sqrt{\Delta-1}, 2\sqrt{\Delta-1}]$ by the Alon–Boppana analysis of the universal cover. Strict inclusion in the open interval follows from the fact that $T(G, u)$ is finite and the spectral radius of any finite tree of bounded degree Δ is strictly less than $2\sqrt{\Delta-1}$. $\square$

Remark 145 (Statistical-physics interpretation). *The matchings polynomial is, up to substitution, the partition function of the monomer–dimer model on G : $Z_G(\lambda) = \sum_M \lambda^{|M|}$ where the sum runs over all matchings. Theorem 251 implies $Z_G(\lambda) > 0$ for all real $\lambda \geq 0$, so the monomer–dimer model has no phase transition on any finite graph.*

12.1.4 Special Cases and Classical Polynomials

Theorem 252 (Matching polynomials of named graphs).

- $\mu(K_n, x)$ is (up to scaling) the Hermite polynomial $H_n(x)$.
- $\mu(P_n, x)$ is the Chebyshev polynomial of the second kind $U_n(x/2)$.
- $\mu(K_{m,n}, x)$ is (up to scaling) a Laguerre polynomial.

Example 52 (Matching polynomial of K_4). K_4 has six edges and three perfect matchings. Direct expansion: $p(K_4, 0) = 1$, $p(K_4, 1) = 6$, $p(K_4, 2) = 3$. Hence

$$\mu(K_4, x) = x^4 - 6x^2 + 3.$$

The roots are $\pm\sqrt{3 \pm \sqrt{6}}$, all real and inside the Heilmann–Lieb interval $(-2\sqrt{2}, 2\sqrt{2}) \approx (-2.83, 2.83)$.

12.1.5 Exercises

Easy Questions.

Exercise 12.1. Compute $\mu(C_5, x)$ and $\mu(C_6, x)$ from the recurrence $\mu(C_n, x) = \mu(P_n, x) - \mu(P_{n-2}, x)$, which follows from Theorem 248 applied to a single edge of C_n .

Exercise 12.2. Verify Proposition 249 for the path P_4 by computing both $\mu(P_4, x)$ from the matching definition and $\det(xI - A(P_4))$ from the adjacency matrix.

Medium Questions.

Exercise 12.3. Prove $\mu(G, x) = \mu(G \setminus e, x) - \mu(G \setminus \{u, v\}, x)$ in detail by induction on $|E(G)|$, taking care of the edge-case $|M| = 0$.

Exercise 12.4. Show that $\mu(G \cup H, x) = \mu(G, x) \cdot \mu(H, x)$ for the disjoint union of G and H . Use this to compute the matchings polynomial of the 5-cube Q_5 from $\mu(Q_4, x)$ and $\mu(K_2, x)$ (Hint: $Q_5 = Q_4 \square K_2$ requires more work; treat it as two copies of Q_4 joined by a matching).

Hard Questions.

Exercise 12.5. Prove the path-tree identity (Theorem 250) by applying the vertex recurrence simultaneously to G and to $T(G, u)$ at the root, then inducting on the depth of expansion.

Exercise 12.6. Use the Heilmann–Lieb theorem to deduce that the partition function $Z_G(\lambda) = \sum_M \lambda^{|M|}$ has no real positive zeros for $\lambda > 0$. Conclude that $\log Z_G(\lambda)$ is real-analytic on $(0, \infty)$.

Diagram Questions.

Exercise 12.7. Draw the path-tree $T(C_4, v)$ for any vertex v of the 4-cycle C_4 . Verify that $T(C_4, v)$ is the infinite 2-regular tree truncated to depth at most 4, and write out the first few terms of its characteristic polynomial.

Exercise 12.8. Mark the roots of $\mu(K_4, x)$ on the real line and shade the Heilmann–Lieb interval $(-2\sqrt{2}, 2\sqrt{2})$ to confirm strict containment. Compare with the actual adjacency spectrum $\{3, -1, -1, -1\}$ of K_4 (which lies on $\mathbb{Z}$, not on the matching polynomial roots).

12.2 Equitable Partitions and Quotient Interlacing

Equitable partitions provide a systematic way to extract eigenvalues from a graph using its symmetries. Every automorphism orbit gives an equitable partition, but many graphs admit equitable partitions with no automorphism behind them.

12.2.1 Definition and Basic Theory

Definition 189 (Equitable partition). A partition $\pi = (V_1, V_2, \dots, V_k)$ of $V(G)$ is **equitable** if there exist non-negative integers b_{ij} ($1 \leq i, j \leq k$) such that every vertex of V_i has exactly b_{ij} neighbors in V_j . The $k \times k$ matrix $B = (b_{ij})$ is the **quotient matrix** of the partition.

Example 53 (Trivial partitions).

- The discrete partition into singletons is always equitable; its quotient matrix is the adjacency matrix $A(G)$.
- For a k -regular graph, the trivial 1-cell partition $\pi = (V)$ is equitable with $B = (k)$.
- For a bipartite graph $G = (A \cup B, E)$ where every vertex of A has degree d_A and every vertex of B has degree d_B , the partition (A, B) is equitable with $B = \begin{pmatrix} 0 & d_A \\ d_B & 0 \end{pmatrix}$.

Theorem 253 (Spectrum of the quotient). If π is an equitable partition of G with quotient matrix B , then every eigenvalue of B is also an eigenvalue of $A(G)$. Specifically, if $By = \lambda y$ then the vector $x \in \mathbb{R}^V$ defined by $x_v = y_i$ whenever $v \in V_i$ satisfies $Ax = \lambda x$.

Proof. Let $S \in \{0, 1\}^{V \times k}$ be the characteristic matrix of π (column i is the indicator of V_i). Equitability means $AS = SB$. Hence if $By = \lambda y$, then $A(Sy) = S(By) = \lambda(Sy)$. The lifted vector Sy is non-zero because the columns of S are linearly independent. $\square$

12.2.2 Cauchy Interlacing for Quotients

Theorem 254 (Cauchy interlacing for quotients). Let A be a real symmetric $n \times n$ matrix with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$, and let π be any partition (not necessarily equitable) with characteristic matrix S . Define the *normalized quotient* $\hat{B} = (S^\top S)^{-1} S^\top A S$, a $k \times k$ matrix with eigenvalues $\beta_1 \geq \dots \geq \beta_k$. Then

$$\lambda_i \geq \beta_i \geq \lambda_{n-k+i} \quad (1 \leq i \leq k).$$

Proof. Apply the Courant–Fischer min-max characterization (Theorem 198) to A and to $\hat{B} = S^\top A S$ on the k -dimensional column span of S . The Rayleigh quotient on this k -subspace lies between the top- k and bottom- k eigenvalues of A . $\square$

Corollary 65 (Hoffman-style ratio bound; Haemers, 1995). Let G be a k -regular graph with smallest adjacency eigenvalue λ_n . Then

$$\alpha(G) \leq \frac{-\lambda_n}{k - \lambda_n} \cdot n.$$

Proof. Let $S \subseteq V(G)$ be a maximum independent set, and consider the partition $(S, V \setminus S)$. Compute the normalized quotient: the 2×2 matrix has trace $2 \cdot \frac{e(S, V \setminus S)}{|S|}$ and determinant relating to the densities. Apply Theorem 254 and solve. $\square$

12.2.3 Walk-Regularity and Distance Partitions

Definition 190 (Walk-regular). A graph G is **walk-regular** if for every $k \geq 0$, the diagonal $(A^k)_{vv}$ is the same for all v . Equivalently, the number of closed walks of length k at v does not depend on v .

Theorem 255 (Vertex-transitive implies walk-regular). Every vertex-transitive graph is walk-regular.

Definition 191 (Distance partition). For a vertex v in a connected graph G , the **distance partition from v** is $\pi_v = (G_0(v), G_1(v), G_2(v), \dots, G_d(v))$, where $G_i(v)$ is the set of vertices at distance exactly i from v .

Theorem 256 (Distance partition of a distance-regular graph). A connected graph G is distance-regular (Section 12.3 below) iff for every vertex v , the distance partition π_v is equitable with the same quotient matrix.

This sets up Section 12.3, where the quotient matrix is the canonical tridiagonal intersection matrix.

12.2.4 Exercises

Easy Questions.

Exercise 12.9. Show that the orbit partition of any subgroup of $\text{Aut}(G)$ is equitable.

Exercise 12.10. Find an equitable partition of the Petersen graph into two parts (outer pentagon and inner pentagram). Write its 2×2 quotient matrix and verify both eigenvalues appear in the Petersen spectrum $\{3, 1^{(5)}, -2^{(4)}\}$.

Medium Questions.

Exercise 12.11. Prove Theorem 253 from the matrix identity $AS = SB$. Show further that the k eigenvalues of B are exactly the eigenvalues of A whose eigenvectors are constant on each cell of π .

Exercise 12.12. Apply the Haemers ratio bound (Corollary 65) to the Petersen graph and to the Kneser graph $K(5, 2)$ (which is the Petersen graph). Compare with the exact independence number $\alpha(P) = 4$.

Hard Questions.

Exercise 12.13. Prove that a connected graph is distance-regular iff every distance partition π_v ($v \in V$) is equitable with the *same* quotient matrix (Theorem 256).

Exercise 12.14. Let G be a graph with an equitable partition π . Show that the spectrum of G decomposes as $\sigma(B) \cup \sigma(A_{\pi^\perp})$, where $A_{\pi^\perp}$ is A restricted to vectors that sum to zero on each cell. Conclude that all “new” eigenvalues of G relative to B have eigenvectors orthogonal to the constant-on-cells subspace.

Diagram Questions.

Exercise 12.15. Draw the cycle C_6 and find an equitable partition into three cells of size 2 each (alternating cells). Write the 3×3 quotient matrix and verify that its eigenvalues $\{2, -1, -1\}$ appear among the cycle spectrum $\{2, 1, -1, -2, -1, 1\}$.

Exercise 12.16. Sketch the distance partition of the cube Q_3 from a corner vertex. Verify that it is equitable into four cells of sizes 1, 3, 3, 1, and write its tridiagonal quotient matrix.

12.3 Distance-Regular Graphs and Association Schemes

Distance-regular graphs (DRGs) sit at the algebraic core of extremal combinatorics: they include all strongly regular graphs (Section 11.4), the Hamming and Johnson graphs (which underlie coding theory and the Erdős–Ko–Rado problem), and the foundational examples of finite geometry. Their unified algebraic theory is the language of *association schemes*.

12.3.1 Distance-Regular Graphs

Definition 192 (Distance-regular). A connected graph G of diameter d is **distance-regular** if there exist integers $b_0, b_1, \dots, b_{d-1}$ and $c_1, c_2, \dots, c_d$ such that, for every pair u, v at distance i in G ,

$$c_i = |N(v) \cap G_{i-1}(u)|, \quad b_i = |N(v) \cap G_{i+1}(u)|,$$

i.e. the neighbours of v counted by their distance from u depend only on i . The sequence

$$\iota(G) = \{b_0, b_1, \dots, b_{d-1}; c_1, c_2, \dots, c_d\}$$

is the **intersection array**. The valency is $b_0 = k$.

Example 54 (Familiar distance-regular graphs).

- K_n has intersection array $\{n-1; 1\}$ (diameter 1).
- C_n ($n \geq 5$) has intersection array $\{2, 1, 1, \dots, 1; 1, 1, \dots, 1, c_d\}$, where the last $c_d \in \{1, 2\}$ depending on parity.
- The Petersen graph has $\iota = \{3, 2; 1, 1\}$ (diameter 2).
- The Hamming graph $H(n, q)$ (vertices: words in $[q]^n$; edges: words differing in one coordinate) is distance-regular with $b_i = (n-i)(q-1)$ and $c_i = i$.
- The Johnson graph $J(n, k)$ (vertices: k -subsets of $[n]$; edges: subsets sharing $k-1$ elements) is distance-regular with $b_i = (k-i)(n-k-i)$ and $c_i = i^2$.

Theorem 257 (Tridiagonal quotient). Let G be distance-regular with intersection array $\{b_0, \dots, b_{d-1}; c_1, \dots, c_d\}$. Set $a_i = k - b_i - c_i$ for $0 \leq i \leq d$ (where $b_d = c_0 = 0$). The quotient matrix of the distance partition π_v from any vertex v is

$$L_1 = \begin{pmatrix} a_0 & b_0 & 0 & \cdots & 0 \\ c_1 & a_1 & b_1 & \cdots & 0 \\ 0 & c_2 & a_2 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & c_d & a_d \end{pmatrix}.$$

By Theorem 253, every eigenvalue of L_1 is an eigenvalue of $A(G)$; in fact G has *exactly* $d+1$ distinct adjacency eigenvalues, the eigenvalues of L_1 .

12.3.2 Association Schemes

Definition 193 (Association scheme). A (symmetric) **association scheme** with d classes on a finite set X is a partition $\{R_0, R_1, \dots, R_d\}$ of $X \times X$ such that, with $A_i \in \{0, 1\}^{X \times X}$ the indicator matrix of R_i :

1. $R_0 = \{(x, x) : x \in X\}$, so $A_0 = I$.
2. Each R_i is symmetric: $A_i^\top = A_i$.
3. $\sum_{i=0}^d A_i = J$ (the all-ones matrix).
4. There exist non-negative integers p_{ij}^k with $A_i A_j = \sum_{k=0}^d p_{ij}^k A_k$.

The numbers p_{ij}^k are the **intersection numbers** of the scheme.

Theorem 258 (Bose–Mesner algebra). The complex span of $A_0, A_1, \dots, A_d$ is a commutative algebra $\mathfrak{A}$ under matrix multiplication, called the **Bose–Mesner algebra**. Since the A_i are symmetric and commute pairwise, $\mathfrak{A}$ has a basis of mutually orthogonal idempotents $E_0, E_1, \dots, E_d$ (the *primitive idempotents*). Both $\{A_i\}$ and $\{E_j\}$ are bases of $\mathfrak{A}$, related by the **first** and **second eigenmatrices** P, Q :

$$A_i = \sum_j P_{ji} E_j, \quad E_j = \frac{1}{|X|} \sum_i Q_{ij} A_i.$$

Theorem 259 (Distance-regular = P -polynomial). A distance-regular graph G defines an association scheme on $X = V(G)$ with $R_i = \{(u, v) : d(u, v) = i\}$. The Bose–Mesner algebra is generated as a polynomial algebra by A_1 (the adjacency matrix), so distance-regular schemes are exactly the *P -polynomial* schemes.

12.3.3 The Hamming and Johnson Schemes

Example 55 (Hamming scheme $H(n, q)$). $X = [q]^n$; $(x, y) \in R_i$ iff the Hamming distance is i . The graph $H(n, q)$ above is the corresponding distance-regular graph. The Bose–Mesner algebra is the algebra of symmetric Latin-square type schemes; the P -matrix entries are values of *Krawtchouk polynomials*.

Example 56 (Johnson scheme $J(n, k)$). $X = \binom{[n]}{k}$; $(A, B) \in R_i$ iff $|A \cap B| = k - i$. The P -matrix entries are values of *Hahn polynomials*, and the LP bounds for codes/cliques in $J(n, k)$ specialize to the Erdős–Ko–Rado theorem.

Theorem 260 (Delsarte LP bound). Let Γ be an association scheme on X with first eigenmatrix P . The maximum size of a *clique* (a subset C such that any two distinct elements of C are in R_1) is bounded by the value of a linear program in variables $x = (x_0, \dots, x_d)$ with constraints $x \geq 0$, $x_0 = 1$, $\sum_i x_i = |C|$, and $Q^\top x \geq 0$.

The Delsarte LP gives the best-known general bounds on optimal codes (in $H(n, q)$), set systems with bounded intersection (in $J(n, k)$), and packings in spherical caps (continuous analogue).

12.3.4 Exercises

Easy Questions.

Exercise 12.17. Verify that the cycle C_6 is distance-regular and compute its intersection array. Compare with the array of the Petersen graph.

Exercise 12.18. Show that every strongly regular graph with parameters (n, k, λ, μ) is distance-regular of diameter 2 with intersection array $\{k, k - 1 - \lambda; 1, \mu\}$.

Medium Questions.

Exercise 12.19. Compute the intersection array of the Hamming graph $H(3, 2) = Q_3$. Verify directly using Definition 192.

Exercise 12.20. Verify that for the Johnson scheme $J(5, 2)$, the graph R_1 is the Petersen graph (vertices: 2-subsets of $[5]$; edges: disjoint 2-subsets).

Hard Questions.

Exercise 12.21. Prove Theorem 258: the matrices $A_0, \dots, A_d$ of an association scheme commute pairwise and span a commutative algebra. Use the symmetry $A_i^\top = A_i$ and the multiplication rule $A_i A_j = \sum_k p_{ij}^k A_k$.

Exercise 12.22. Use the Delsarte LP bound on the Johnson scheme $J(n, k)$ to recover the Erdős–Ko–Rado theorem: a family of pairwise-intersecting k -subsets of $[n]$ ($n \geq 2k$) has size at most $\binom{n-1}{k-1}$.

Diagram Questions.

Exercise 12.23. Draw the Hamming graph $H(2, 3)$ (vertices: $\{0, 1, 2\}^2$; edges: words differing in one coordinate). Identify it as the rook graph 3×3 and compute its spectrum.

Exercise 12.24. Sketch the partition $\{R_0, R_1, R_2\}$ of the vertex pairs of C_5 by distance. Verify the intersection numbers p_{ij}^k from the multiplication table $A_i A_j = \sum_k p_{ij}^k A_k$.

12.4 Cayley Graphs and Vertex-Transitive Graphs

Cayley graphs encode group structure as graph structure. Their eigenvalues are computable from the irreducible characters of the group, making them the cleanest available source of explicit d -regular graphs with prescribed spectral properties (notably, expanders).

12.4.1 Definitions and First Examples

Definition 194 (Cayley graph). Let Γ be a finite group and $S \subseteq \Gamma \setminus \{e\}$ a symmetric subset (closed under inversion: $s \in S \Rightarrow s^{-1} \in S$). The **Cayley graph** $\text{Cay}(\Gamma, S)$ has vertex set Γ and edge set $\{\{g, gs\} : g \in \Gamma, s \in S\}$. It is $|S|$ -regular and connected iff S generates Γ .

Example 57 (Familiar Cayley graphs).

- $C_n = \text{Cay}(\mathbb{Z}_n, \{\pm 1\})$.
- Hypercube $Q_n = \text{Cay}(\mathbb{Z}_2^n, \{e_1, \dots, e_n\})$.
- Complete graph $K_n = \text{Cay}(\Gamma, \Gamma \setminus \{e\})$ for any group Γ of order n .
- The *circulant* graph $C_n(S)$ for $S \subseteq \{1, 2, \dots, \lfloor n/2 \rfloor\}$ is $\text{Cay}(\mathbb{Z}_n, S \cup (-S))$.

Theorem 261 (Cayley implies vertex-transitive). Every Cayley graph is vertex-transitive: left translation by any $h \in \Gamma$ is an automorphism. The converse is false (e.g. the Petersen graph is vertex-transitive but not Cayley for any group).

12.4.2 Spectrum from Characters

Theorem 262 (Character formula for abelian Cayley spectra). Let Γ be a *finite abelian* group and $X = \text{Cay}(\Gamma, S)$. The eigenvalues of $A(X)$ are exactly

$$\lambda_\chi = \sum_{s \in S} \chi(s), \quad \chi \in \hat{\Gamma},$$

where $\hat{\Gamma}$ is the group of characters of Γ . The character χ itself, viewed as a vector $(\chi(g))_{g \in \Gamma}$, is the eigenvector.

Proof. Since Γ is abelian, characters form an orthonormal basis of $\mathbb{C}^\Gamma$. Direct computation:

$$(A\chi)(g) = \sum_{s \in S} \chi(gs) = \chi(g) \sum_{s \in S} \chi(s) = \lambda_\chi \chi(g).$$

□

Example 58 (Hypercube spectrum). For $\Gamma = \mathbb{Z}_2^n$ the characters are $\chi_T(x) = (-1)^{|T \cap x|}$ for $T \subseteq [n]$. The spectrum of Q_n is therefore

$$\lambda_T = \sum_{i=1}^n \chi_T(e_i) = n - 2|T|, \quad T \subseteq [n],$$

giving eigenvalues $n - 2k$ ($0 \leq k \leq n$) with multiplicity $\binom{n}{k}$.

Example 59 (Cycle spectrum). For $\Gamma = \mathbb{Z}_n$ the characters are $\chi_j(k) = \omega^{jk}$ with $\omega = e^{2\pi i/n}$. The spectrum of $C_n = \text{Cay}(\mathbb{Z}_n, \{\pm 1\})$ is $\lambda_j = \omega^j + \omega^{-j} = 2 \cos(2\pi j/n)$.

12.4.3 Non-Abelian Cayley Graphs

Theorem 263 (Diaconis–Shahshahani trace formula). Let Γ be a finite (possibly non-abelian) group and $S \subseteq \Gamma$ symmetric. Then the spectrum of $\text{Cay}(\Gamma, S)$ consists of the eigenvalues of the matrices $\rho(\hat{S}) = \sum_{s \in S} \rho(s)$ as ρ ranges over the irreducible (unitary) representations of Γ , with multiplicities $\dim(\rho)$.

Example 60 (Random walks on S_n). The transposition walk on S_n corresponds to $\text{Cay}(S_n, S)$ with S the set of all transpositions. Diaconis and Shahshahani used the character formula to prove that the mixing time is $\frac{1}{2}n \log n$, the canonical example of a sharp *cutoff phenomenon* for Markov chains.

12.4.4 Vertex-Transitive Graphs and Cayley-ness

Theorem 264 (Sabidussi, 1958). A vertex-transitive graph X is a Cayley graph iff $\text{Aut}(X)$ contains a subgroup acting regularly (transitively and freely) on $V(X)$.

Example 61 (Petersen is not Cayley). $|\text{Aut}(\text{Petersen})| = 120 = 5!$, with full automorphism group S_5 . There is no subgroup of S_5 of order 10 acting regularly on the 10 vertices, so Petersen fails Sabidussi's criterion.

12.4.5 Exercises

Easy Questions.

Exercise 12.25. Identify each of the following as a Cayley graph if possible: K_4 , $K_{3,3}$, C_5 , Q_3 , the Petersen graph.

Exercise 12.26. Compute the spectrum of the circulant graph $C_8(\{1, 3\})$ using Theorem 262.

Medium Questions.

Exercise 12.27. Show that $\text{Cay}(\mathbb{Z}_n, S)$ is connected iff $\gcd(S) = 1$, where $\gcd(S)$ is the GCD of the elements of $S \cup (-S) \cup \{n\}$.

Exercise 12.28. Verify that the Cayley graph $\text{Cay}(D_8, \{r, r^{-1}, s\})$ (dihedral group with rotation r of order 4 and reflection s) is isomorphic to the cube Q_3 . Use Sabidussi's theorem to conclude that Q_3 has at least two non-equivalent presentations as a Cayley graph.

Hard Questions.

Exercise 12.29. Prove that for any abelian group Γ and symmetric generating set S , the second-largest eigenvalue λ_2 of $\text{Cay}(\Gamma, S)$ satisfies $\lambda_2 \geq |S| - 2\pi^2 \cdot (|S|/\text{diam}(X))^2/4$. Conclude that abelian Cayley graphs cannot be expanders if the generating set has bounded size.

Exercise 12.30. Use the character formula for non-abelian Cayley graphs to compute the spectrum of the Cayley graph of S_3 with generating set $\{(12), (13), (23)\}$. Identify the resulting graph.

Diagram Questions.

Exercise 12.31. Draw $\text{Cay}(\mathbb{Z}_2^3, \{e_1, e_2, e_3\}) = Q_3$ and label the eigenvectors χ_T for each $T \subseteq \{1, 2, 3\}$. Color each vertex by the sign $\chi_T(g) = \pm 1$ for $T = \{1, 2\}$ and verify that this colouring is balanced and orthogonal to the all-ones vector.

Exercise 12.32. Draw the Cayley graph of $\mathbb{Z}_5$ with generating set $\{\pm 1\}$ (the pentagon C_5). Mark the eigenvectors $\chi_j(k) = e^{2\pi i j k / 5}$ for $j = 1, 2$ and verify that the corresponding eigenvalues are $2 \cos(2\pi/5)$ and $2 \cos(4\pi/5)$.

12.5 The Tutte Polynomial and Graphic Matroids

The Tutte polynomial $T_G(x, y)$ is the universal multiplicative graph invariant on deletion-contraction. It specializes to the chromatic polynomial, the flow polynomial, the all-terminal reliability polynomial, the Jones polynomial of an alternating link, and the Potts model partition function. Its natural setting is the *matroid* of G .

12.5.1 Deletion, Contraction, and the Tutte Polynomial

Definition 195 (Deletion and contraction). For a graph G and edge $e \in E(G)$:

- $G \setminus e$ is the graph obtained by removing e .
- G/e is the graph obtained by contracting e (identify its two endpoints, removing any loops created).

Definition 196 (Tutte polynomial). For a graph G with vertex set V , edge set E , and rank $r(G) = |V| - c(G)$ (where $c(G)$ is the number of connected components), the **Tutte polynomial** is

$$T_G(x, y) = \sum_{A \subseteq E} (x-1)^{r(E)-r(A)} (y-1)^{|A|-r(A)},$$

where $r(A) = |V| - c(V, A)$ is the rank of the spanning subgraph (V, A) .

Theorem 265 (Deletion-contraction recursion). For an edge $e \in E(G)$:

- If e is neither a loop nor a bridge: $T_G(x, y) = T_{G \setminus e}(x, y) + T_{G/e}(x, y)$.
- If e is a bridge: $T_G(x, y) = x \cdot T_{G/e}(x, y)$.
- If e is a loop: $T_G(x, y) = y \cdot T_{G \setminus e}(x, y)$.
- If G has no edges: $T_G(x, y) = 1$.

These four rules determine T_G uniquely.

Example 62 (Tutte polynomial of small graphs).

- $T_{K_2}(x, y) = x$.
- $T_{K_3}(x, y) = x^2 + x + y$.
- $T_{K_4}(x, y) = x^3 + 3x^2 + 2x + 4xy + 2y + 3y^2 + y^3$.

12.5.2 Specializations: Chromatic, Flow, Reliability

Theorem 266 (Whitney/Tutte specialization). For any graph G on n vertices,

$$\chi_G(k) = (-1)^{r(E)} k^{c(G)} T_G(1-k, 0),$$

where $\chi_G(k)$ is the chromatic polynomial counting proper k -colorings.

Theorem 267 (Flow polynomial). The number of nowhere-zero $\mathbb{Z}_k$ -flows on G (with arbitrary edge orientation) is

$$F_G(k) = (-1)^{|E|-r(E)} T_G(0, 1-k).$$

Theorem 268 (Reliability polynomial). If each edge of G fails independently with probability $1-p$, the probability that the surviving subgraph remains connected (the **all-terminal reliability**) is

$$R_G(p) = p^{r(E)} (1-p)^{|E|-r(E)} T_G(1, 1/(1-p)).$$

12.5.3 Whitney's Broken-Circuit Theorem

Definition 197 (Broken circuit). Fix a total order on $E(G)$. A **broken circuit** is a set $C \setminus \{e\}$ where C is a cycle of G and e is the largest edge of C .

Theorem 269 (Whitney's broken-circuit theorem). For any total order on $E(G)$,

$$\chi_G(k) = \sum_{B \subseteq E, B \text{ contains no broken circuit}} (-1)^{|B|} k^{n-|B|}.$$

In particular, $\chi_G(k)$ has alternating-sign integer coefficients.

Theorem 270 (Huh–Katz, 2012). The coefficients of the chromatic polynomial form a *log-concave* sequence: if $\chi_G(k) = \sum_{i=0}^n (-1)^i a_i k^{n-i}$, then $a_i^2 \geq a_{i-1} a_{i+1}$ for $1 \leq i \leq n-1$.

The Huh–Katz proof uses Hodge theory of the Chow ring of the graphic matroid; this established the Rota–Heron–Welsh conjecture in full generality.

12.5.4 Graphic Matroids and Cycle/Bond Duality

Definition 198 (Matroid). A **matroid** on a finite set E is a pair $(E, \mathcal{I})$ where $\mathcal{I} \subseteq 2^E$ (the **independent sets**) satisfies:

- $\emptyset \in \mathcal{I}$.
- If $I \in \mathcal{I}$ and $J \subseteq I$, then $J \in \mathcal{I}$.
- (*Augmentation*) If $I, J \in \mathcal{I}$ with $|I| < |J|$, then there exists $e \in J \setminus I$ with $I \cup \{e\} \in \mathcal{I}$.

The maximal independent sets are the **bases**; the rank function is $r(A) = \max\{|I| : I \subseteq A, I \in \mathcal{I}\}$.

Example 63 (Graphic matroid). For a graph G , let $E = E(G)$ and $\mathcal{I} = \{A \subseteq E : (V, A) \text{ is acyclic}\}$. This is the **graphic matroid** $M(G)$. Bases of $M(G)$ are spanning forests of G ; the rank is $|V| - c(G)$.

Theorem 271 (Cycle/bond duality on planar graphs). For a planar graph G with planar dual G^* , $M(G^*) = M(G)^*$, where $M(G)^*$ is the matroid dual of $M(G)$. Equivalently, cycles in G correspond to bonds (minimal cuts) in G^* and vice versa.

Theorem 272 (Tutte polynomial as matroid invariant). The Tutte polynomial extends to all matroids via the same formula: $T_M(x, y) = \sum_{A \subseteq E} (x-1)^{r(E)-r(A)} (y-1)^{|A|-r(A)}$. For graphic matroids $M(G)$, this recovers $T_G(x, y)$. Matroid duality interchanges x and y : $T_{M^*}(x, y) = T_M(y, x)$.

12.5.5 Connections: Stanley’s Chromatic Symmetric Function

Definition 199 (Chromatic symmetric function; Stanley, 1995). For a graph G on vertex set V , define the **chromatic symmetric function**

$$X_G(x_1, x_2, \dots) = \sum_{\kappa: V \rightarrow \mathbb{Z}_{\geq 1}} \prod_{\text{proper } v \in V} x_{\kappa(v)}.$$

Theorem 273 (Stanley). X_G is a symmetric function in $x_1, x_2, \dots$, and $X_G(1, 1, \dots, 1, 0, 0, \dots) = \chi_G(k)$ when there are k ones. Thus X_G refines the chromatic polynomial.

Conjecture 1 (Stanley’s tree conjecture, 1995, OPEN). If T_1 and T_2 are non-isomorphic trees on the same number of vertices, then $X_{T_1} \neq X_{T_2}$.

This conjecture has been verified by computer search for trees on up to ~ 30 vertices but remains open. It would imply, in particular, that the chromatic symmetric function distinguishes any pair of non-isomorphic trees—a much finer invariant than the chromatic polynomial $\chi_T(k) = k(k-1)^{n-1}$, which depends only on the number of vertices.

12.5.6 Exercises

Easy Questions.

Exercise 12.33. Compute $T_{C_4}(x, y)$ from Definition 196 by enumerating all $2^4 = 16$ subsets of edges. Verify your answer against the deletion–contraction recursion (Theorem 265).

Exercise 12.34. Use Theorem 266 to derive the chromatic polynomial $\chi_{K_4}(k)$ from $T_{K_4}(x, y)$. Compare with the direct formula $\chi_{K_n}(k) = k(k-1)(k-2) \cdots (k-n+1)$.

Medium Questions.

Exercise 12.35. Prove the deletion–contraction recursion (Theorem 265) from Definition 196 by partitioning $A \subseteq E$ into those containing e and those not.

Exercise 12.36. Show that $T_G(2, 1) =$ number of spanning forests of G , $T_G(1, 1) =$ number of spanning trees, and $T_G(2, 0) =$ number of acyclic orientations of G (Stanley).

Hard Questions.

Exercise 12.37. Prove Whitney’s broken-circuit theorem (Theorem 269) by inclusion–exclusion on the cycles of G , ordered by their largest edge.

Exercise 12.38. Show that the chromatic polynomial $\chi_G(k)$ has all real roots iff G is a chordal graph. (Hint: chromatic polynomials of chordal graphs factor into linear pieces by perfect elimination ordering.)

Diagram Questions.

Exercise 12.39. For the planar graph $G = K_4$ minus an edge, draw both G and its planar dual G^* . Verify directly that $T_{G^*}(x, y) = T_G(y, x)$ by computing both polynomials and comparing.

Exercise 12.40. Draw the broken-circuit complex of the triangle K_3 with edges ordered $e_1 < e_2 < e_3$. Identify the unique broken circuit $\{e_1, e_2\}$, and verify $\chi_{K_3}(k) = k^3 - 3k^2 + 2k$ from the broken-circuit sum of Theorem 269.

12.6 Further Reading and Pointers

Matchings polynomial and rootedness. Godsil’s *Algebraic Combinatorics* (1993), Chapters 1, 4, 6, develops the path-tree machinery in full and extends to the broader theory of *tree-like* walks and characteristic polynomials of covers.

Equitable partitions and walk-regularity. Godsil–Royle, *Algebraic Graph Theory* (2001), Chapters 9–10, are the canonical reference. Brouwer–Haemers, *Spectra of Graphs* (2012), Chapter 3, contains modern interlacing and Hoffman/Haemers bounds.

Distance-regular graphs and association schemes. Brouwer–Cohen–Neumaier, *Distance-Regular Graphs* (1989), is the comprehensive reference; Bannai–Ito, *Algebraic Combinatorics I* (1984), develops the Bose–Mesner algebra. Delsarte’s 1973 thesis on the LP method remains foundational.

Cayley graphs and expanders. Hoory–Linial–Wigderson, “Expander Graphs and Their Applications” (*Bull. AMS*, 2006), surveys both deterministic and probabilistic constructions. Lubotzky, *Discrete Groups, Expanding Graphs, and Invariant Measures* (1994), connects to representation theory and number theory.

Tutte polynomial and matroids. Bollobás, *Modern Graph Theory* (1998), Chapter 10, gives a graph-theoretic treatment. Oxley, *Matroid Theory* (2nd ed., 2011), is the matroid-side reference. Welsh’s *Complexity: Knots, Colourings and Counting* (1993) discusses the Jones polynomial connection.

Chromatic symmetric function. Stanley’s original paper, “A symmetric function generalization of the chromatic polynomial of a graph” (*Adv. Math.*, 1995), is short and self-contained. Recent partial progress on the tree conjecture is surveyed in Crew–Spirkl (2023) and references therein.